A Tour of C++

- 정리 및 요약

목차

원문 범위와 출처 읽는 법 4
제1장. 기초 6
제2장. 사용자 정의 타입 17
제3장. 모듈화 24
제4장. 클래스 36
제5장. 필수적인 연산 48
제6장. 템플릿 59
제7장. 컨셉과 제네릭 프로그래밍 69
제8장. 라이브러리의 개요 78
제9장. 문자열과 정규 표현식 82
제10장. 입력과 출력 91
제11장. 컨테이너 103
제12장. 알고리즘 113
제13장. 유틸리티 125
제14장. 수치 139
제15장. 동시성 147
제16장. 역사와 호환성 158
원문 기반 복습 질문과 정답 172
제1장. 기초 173
제2장. 사용자 정의 타입 174
제3장. 모듈화 175
제4장. 클래스 176
제5장. 필수적인 연산 177
제6장. 템플릿 178
제7장. 컨셉과 제네릭 프로그래밍 179
제8장. 라이브러리의 개요 180
제9장. 문자열과 정규 표현식 181
제10장. 입력과 출력 182
제11장. 컨테이너 183
제12장. 알고리즘 184
제13장. 유틸리티 185
제14장. 수치 186

제15장. 동시성 187
제16장. 역사와 호환성 188

원문 범위와 출처 읽는 법

제공된 351쪽 스캔 PDF에서 실제로 확인한 1~16장의 내용을 원래 장·절 순서로 재구성한 학습 교재다. 원본 책 전체의 완전한 요약이라고 표시하지 않는다. 원문만을 내용 출처로 삼았으며, 원문 속 인용 자료나 인터넷 자료로 설명을 보충하지 않았다.

기존 상세 내용을 원문 장·절 순서로 유지하면서 항목별 근거 번호, 정의 중심 용어 표, 원문 요약·조언에 근거한 장 요약과 복습 정답을 보강했습니다. 문장 끝의 인쇄 쪽수는 원본 스캔의 쪽수입니다. 별도 근거표에는 원본 PDF 면 번호도 병기했습니다.

같은 [U장-항목] 번호는 세 PDF에서 같은 내용을 가리킵니다. 원문에 없는 예시·해설·실행 결과를 보충하지 않았습니다. 당시의 표준·기능 상태와 원문 내부의 의심 표기는 원문 확인 메모와 함께 보존했습니다.

수록 한계

표지·표제지·판권지·지은이 소개가 제공되지 않았다. 책 제목은 사용자가 확인한 A Tour of C++를 반영했다. 저자명은 원문에서 확인할 수 없어 기재하지 않았다.

인쇄 90·140·288·289·326·346쪽은 제공 파일에 없다. 장 경계에 있는 쪽도 빈 페이지라고 추정하지 않았다.

인쇄 288~289쪽은 13.4 특수 컨테이너 설명 중간에서 누락됐다. 해당 부분의 코드·정의·결과를 임의 복원하지 않았다.

인쇄 371쪽 이후는 제공되지 않았다. 16.4 참고 자료의 나머지, 16.5 조언 본문(목차상 376쪽), 원본 찾아보기(목차상 378쪽)를 읽거나 요약한 것으로 표시하지 않는다.

제공 PDF 1~12쪽은 원문 인쇄 359~370쪽이다. 이 부분을 인쇄 347~358쪽 뒤에 배치해 원래 16장의 흐름을 복원했다.

OCR은 독서 보조로 사용했다. 수록한 대표 코드는 원문 이미지와 대조했으며, 의심되는 원문 표기와 판독 한계는 해당 위치에 표시했다. 나머지 모든 문자에 대해 완전한 전사 정확성을 보증하는 것은 아니다.

원문의 C++11·14·17 관련 설명과 C++20·모듈·컨셉·Ranges TS 등에 대한 당시의 예정·실험적 상태를 그대로 유지했다. 현재 표준이나 최신 자료로 수정하지 않았다.

중요한 원문 코드 중 학습 목적에 맞는 대표 예제를 선별했다. 발췌와 원문의 생략 표시는 구별해 알렸으며, 모든 코드를 전재한 자료는 아니다.

실행 결과가 제시되지 않은 예제에는 [원문에 실행 결과 미제시]를 표시했다. 코드를 실행·수정해 새 결과를 생성하지 않았다.

장별 원문 범위

장	원문 위치
제1장. 기초	(제1장. 기초, 인쇄 p.19-48 / PDF p.18-47)
제2장. 사용자 정의 타입	(제2장. 사용자 정의 타입, 인쇄 p.49-60 / PDF p.48-59)
제3장. 모듈화	(제3장. 모듈화, 인쇄 p.61-89 / PDF p.60-88)
제4장. 클래스	(제4장. 클래스, 인쇄 p.91-118 / PDF p.89-116)
제5장. 필수적인 연산	(제5장. 필수적인 연산, 인쇄 p.119-139 / PDF p.117-137)
제6장. 템플릿	(제6장. 템플릿, 인쇄 p.141-162 / PDF p.138-159)
제7장. 컨셉과 제네릭 프로그래밍	(제7장. 컨셉과 제네릭 프로그래밍, 인쇄 p.163-182 / PDF p.160-179)
제8장. 라이브러리의 개요	(제8장. 라이브러리의 개요, 인쇄 p.183-188 / PDF p.180-185)
제9장. 문자열과 정규 표현식	(제9장. 문자열과 정규 표현식, 인쇄 p.189-208 / PDF p.186-205)
제10장. 입력과 출력	(제10장. 입력과 출력, 인쇄 p.209-232 / PDF p.206-229)
제11장. 컨테이너	(제11장. 컨테이너, 인쇄 p.233-252 / PDF p.230-249)
제12장. 알고리즘	(제12장. 알고리즘, 인쇄 p.253-274 / PDF p.250-271)
제13장. 유틸리티	(제13장. 유틸리티, 인쇄 p.275-287,290-314 / PDF p.272-309)
제14장. 수치	(제14장. 수치, 인쇄 p.315-325 / PDF p.310-320)
제15장. 동시성	(제15장. 동시성, 인쇄 p.327-345 / PDF p.321-339)
제16장. 역사와 호환성	(제16장. 역사와 호환성, 인쇄 p.347-370 / PDF p.1-12,340-351)

제1장. 기초

(제1장. 기초, 인쇄 p.19-48)

학습 목표·범위

타입은 **객체**가 취할 수 있는 값과 연산의 집합을 정의한다. [U01-001] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)

constexpr는 이 값을 컴파일 시간에 평가한다는 의미이며, **constexpr**의 값은 반드시 컴파일 시간에 계산된다. [U01-002] (1.6 상수, 인쇄 p.33)

핵심 개념·동작 절차·원문 예제

1.1 소개

이 장은 C++의 표기법, 메모리 모델과 계산 모델, 코드를 조직해 프로그램을 만드는 기본 방법을 소개한다. [U01-003] (1.1 소개, 인쇄 p.20)

C에서 흔히 볼 수 있는 스타일을 뒷받침하는 기능을 포함하며, 원문은 이 스타일을 절차적 프로그래밍이라고 부른다. [U01-004] (1.1 소개, 인쇄 p.20)

1.2 프로그램

C++는 컴파일되는 언어다. [U01-005] (1.2 프로그램, 인쇄 p.20-22)

컴파일러가 소스 텍스트에서 목적 파일을 만들고, 링커가 목적 파일들을 병합해 실행 가능한 프로그램을 만든다. [U01-006] (1.2 프로그램, 인쇄 p.20-22)

실행 파일은 특정 시스템·하드웨어 조합에 맞게 만들어진다. [U01-007] (1.2 프로그램, 인쇄 p.20-22)

책에서 말하는 이식성은 주로 같은 소스를 여러 시스템에서 컴파일하고 실행할 수 있다는 뜻이다. [U01-008] (1.2 프로그램, 인쇄 p.20-22)

ISO C++는 내장 **타입**·루프 등을 포함하는 핵심 언어 기능과 **컨테이너**·입출력 등을 포함하는 **표준 라이브러리** 구성 요소를 정의한다. [U01-009] (1.2 프로그램, 인쇄 p.20-22)

표준 라이브러리는 보조적인 머신 코드를 제외하면 C++ 자체로 구현할 수 있다. [U01-010] (1.2 프로그램, 인쇄 p.20-22)

C++는 정적 **타입** 언어이므로 컴파일러는 **객체**·값·이름·표현식을 사용하는 곳에서 그 **타입**을 알아야 한다. [U01-011] (1.2 프로그램, 인쇄 p.20-22)

적용할 수 있는 연산은 **객체**의 **타입**으로 정해진다. [U01-012] (1.2 프로그램, 인쇄 p.20-22)

1.2.1 Hello, World!

모든 C++ 프로그램에는 전역 함수 main()이 하나 있어야 하며, main() 호출로 프로그램이 시작된다. [U01-013] (1.2.1 Hello, World!, 인쇄 p.21-23)

중괄호는 함수 몸체를 묶고 //는 줄 끝까지 이어지는 주석이다. [U01-014] (1.2.1 Hello, World!, 인쇄 p.21-23)

main()의 int 반환 값은 시스템에 전달되며, 반환 구문이 없으면 성공을 나타내는 값이 전달된다. [U01-015] (1.2.1 Hello, World!, 인쇄 p.21-23)

원문은 0이 아닌 반환 값을 실패로 설명하면서, 운영체제·실행 환경마다 이 값의 활용 여부는 다르다고 덧붙인다. [U01-016] (1.2.1 Hello, World!, 인쇄 p.21-23)

Hello, World! 출력 (원문 발췌)

```
#include <iostream>

int main()
{
    std::cout << "Hello, World!\n";
}
```

[U01-017] (1.2.1 Hello, World!, 인쇄 p.22)

#include <iostream>은 표준 스트림 입출력 기능의 선언을 포함한다. [U01-018] (1.2.1 Hello, World!, 인쇄 p.21-23)

std::cout은 표준 출력 스트림이며, std::는 **표준 라이브러리 네임스페이스**에서 이름을 찾도록 한정한다. [U01-019] (1.2.1 Hello, World!, 인쇄 p.21-23)

문자열 리터럴은 큰따옴표로 둘러싼 문자들의 열이며 \n은 개행 문자다. [U01-020] (1.2.1 Hello, World!, 인쇄 p.21-23)

실행 가능한 코드는 기본적으로 함수 안에 있고 main()에서 직접 또는 간접 호출된다. [U01-021] (1.2.1 Hello, World!, 인쇄 p.21-23)

1.3 함수

함수 정의는 어떤 작업을 어떻게 수행할지 명시한다. [U01-022] (1.3 함수, 인쇄 p.23-26)

함수를 호출하기 전에 선언해야 하며, 선언에는 이름·반환 **타입**·인자의 개수와 **타입**이 들어간다. [U01-023] (1.3 함수, 인쇄 p.23-26)

반환 **타입**은 이름 앞에, 인자 **타입**들은 이름 뒤의 괄호 안에 둔다. [U01-024] (1.3 함수, 인쇄 p.23-26)

void는 반환 값이 없음을 뜻한다. [U01-025] (1.3 함수, 인쇄 p.23-26)

인자의 **타입**을 검사하고 필요한 암묵적 변환을 수행하므로, 요구 **타입**과 맞지 않는 인자는 컴파일 시점에 거부될 수 있다. [U01-026] (1.3 함수, 인쇄 p.23-26)

함수의 **타입**은 반환 **타입**과 인자 **타입**으로 이뤄진다. [U01-027] (1.3 함수, 인쇄 p.23-26)

멤버 함수의 **타입**에는 클래스 이름도 포함된다. [U01-028] (1.3 함수, 인쇄 p.23-26)

선언에 붙인 인자 이름은 독자를 돕지만, 정의가 아닌 선언에서는 컴파일러가 무시한다. [U01-029] (1.3 함수, 인쇄 p.23-26)

원문은 의미 있는 작업을 짧은 함수로 나누고 이름을 붙이면 의존성을 드러내고 코드의 길이·복잡도에 따른 에러를 줄일 수 있다고 설명한다. [U01-030] (1.3 함수, 인쇄 p.23-26)

인자 타입에 따른 함수 오버로딩 (원문 발췌)

```
void print(int);        // 정수를 인자로 받음
void print(double);     // 부동소수점 수를 인자로 받음
void print(string);     // 문자열을 인자로 받음

void user()
{
    print(42);           // print(int) 호출
    print(9.65);         // print(double) 호출
    print("Barcelona"); // print(string) 호출
}
```

[U01-031] (1.3 함수, 인쇄 p.25)

같은 이름에 인자 **타입**이 다른 함수들을 정의하는 것을 함수 오버로딩이라고 한다. [U01-032] (1.3 함수, 인쇄 p.23-26)

컴파일러는 호출마다 가장 적절한 함수를 선택한다. [U01-033] (1.3 함수, 인쇄 p.23-26)

둘 이상을 호출할 수 있고 어느 쪽도 더 적합하지 않으면 모호한 호출 에러가 발생한다. [U01-034] (1.3 함수, 인쇄 p.23-26)

오버로딩한 함수들은 모두 주어진 인자를 출력하는 print()처럼 같은 의미를 구현해야 한다. [U01-035] (1.3 함수, 인쇄 p.23-26)

1.4 타입과 변수, 산술 연산

선언은 프로그램에 어떤 실체를 드러내고 그 **타입**을 지정한다. [U01-036] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

타입은 **객체**가 취할 수 있는 값과 연산의 집합이다. [U01-037] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

객체는 어떤 **타입**의 값을 갖는 메모리이며, 값은 **타입**에 따라 해석되는 비트의 집합이다. [U01-038] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

변수는 이름이 붙은 **객체**다. [U01-039] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

원문에 제시된 기본 **타입**은 bool, char, int, double, unsigned다. [U01-040] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

기본 **타입**은 하드웨어 기능에 직접 대응하고 고정된 크기를 갖지만, 실제 크기는 구현에 따라 다르다. [U01-041] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

char는 한 문자를 저장하는 크기이며 다른 **타입**의 크기는 char 크기의 배수다. [U01-042] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

sizeof로 크기를 확인한다. [U01-043] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

수 리터럴은 소수점이나 지수가 있으면 부동소수점 수로 인식한다. [U01-044] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

정수의 기본은 10진수이며 0b·0x·0 접두사는 각각 2진·16진·8진을 나타낸다. [U01-045] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

작은따옴표는 긴 리터럴의 자릿수 구분자로 사용할 수 있다. [U01-046] (1.4 타입과 변수, 산술 연산, 인쇄 p.26-27)

1.4.1 산술 연산

기본 **타입**의 적절한 조합에는 +, -, *, /, 정수 나머지 %와 단항 +··를 사용할 수 있다. ==·!=·<·>·<=·>=는 비교를 수행한다. [U01-047] (1.4.1 산술 연산, 인쇄 p.27-29)

비트별 논리 연산 &, |, ^, ~와 논리 연산 &&, ||, !를 구분한다. [U01-048] (1.4.1 산술 연산, 인쇄 p.27-29)

비트별 연산은 각 비트에 적용되며, &&와 ||는 연산 항의 값에 따라 true 또는 false를 반환한다. [U01-049] (1.4.1 산술 연산, 인쇄 p.27-29)

대입과 산술 연산에서는 기본 **타입** 간의 유효한 변환이 수행된다. [U01-050] (1.4.1 산술 연산, 인쇄 p.27-29)

일반 산술 변환은 주어진 연산 항에서 얻을 수 있는 최대 정밀도를 목표로 한다. [U01-051] (1.4.1 산술 연산, 인쇄 p.27-29)

double과 int의 덧셈은 배정도 부동소수점 연산으로 수행되지만, 그 결과를 int에 저장하면 정보가 버려질 수 있다. =는 대입이고 ==는 상등 비교다. +=, -=, *=, /=, %= 및 증가·감소 연산은 자주 쓰는 갱신을 간결하게 표현한다. [U01-052] (1.4.1 산술 연산, 인쇄 p.27-29)

원문은 표현식 평가와 대입 방향을 설명하고 함수 인자의 평가 순서는 정해져 있지 않다고 명시한다. [U01-053] (1.4.1 산술 연산, 인쇄 p.27-29)

1.4.2 초기화

객체는 사용 전에 값을 받아야 한다. = 형식과 중괄호 초깃값 목록 등을 사용할 수 있다. [U01-054] (1.4.2 초기화, 인쇄 p.29-31)

원문은 명명된 **타입**에 일반적인 {} 형식을 주로 사용하도록 권한다. {}는 정보 손실 위험이 있는 축소 변환을 막는다. [U01-055] (1.4.2 초기화, 인쇄 p.29-31)

반면 =를 이용한 기본 **타입** 변환에서는 double→int나 int→char와 같은 암묵적 축소 변환이 허용될 수 있다. [U01-056] (1.4.2 초기화, 인쇄 p.29-31)

초기화와 축소 변환 (원문 발췌)

```
int i1 = 7.8;   // i1은 7이 됨(당황했는가?)
int i2 {7.8};   // 에러: 부동소수점 수를 정수로 변환
```

[U01-057] (1.4.2 초기화, 인쇄 p.30)

상수는 초기화하지 않은 채 존재할 수 없다. [U01-058] (1.4.2 초기화, 인쇄 p.29-31)

변수도 적절한 초깃값을 얻기 전에는 정의하지 않는 것이 원문의 권고다. [U01-059] (1.4.2 초기화, 인쇄 p.29-31)

auto는 초깃값에서 **타입**을 유추해 반복되는 **타입**명을 줄인다. [U01-060] (1.4.2 초기화, 인쇄 p.29-31)

특히 정확한 **타입**명이 길거나 알기 어려운 제네릭 코드에 유용하다. [U01-061] (1.4.2 초기화, 인쇄 p.29-31)

스코프가 커서 독자에게 **타입**을 명확히 보이려 하거나 범위·정밀도를 지정하려는 경우처럼 이유가 있을 때는 구체적인 **타입**을 명시한다. [U01-062] (1.4.2 초기화, 인쇄 p.29-31)

1.5 스코프와 수명

선언은 이름을 스코프에 드러낸다. [U01-063] (1.5 스코프와 수명, 인쇄 p.31-32)

함수·람다 안의 지역 이름은 선언 지점부터 해당 블록 끝까지 유효하다. [U01-064] (1.5 스코프와 수명, 인쇄 p.31-32)

클래스 정의 안에서 함수·람다 밖에 둔 이름, 또는 **enum class**의 이름은 멤버 이름이며 해당 선언의 스코프를 갖는다. [U01-065] (1.5 스코프와 수명, 인쇄 p.31-32)

네임스페이스 안에서 함수·람다·클래스·**enum class** 밖에 정의한 이름은 **네임스페이스** 멤버다. [U01-066] (1.5 스코프와 수명, 인쇄 p.31-32)

어떤 구성 요소에도 속하지 않는 이름은 전역 **네임스페이스**의 전역 이름으로 본다. [U01-067] (1.5 스코프와 수명, 인쇄 p.31-32)

이름의 유효 범위와 **객체**가 살아 있는 기간을 함께 구별한다. [U01-068] (1.5 스코프와 수명, 인쇄 p.31-32)

객체는 사용 전 생성·초기화되어야 하며 일반적으로 포함된 스코프 끝에서 소멸한다. [U01-069] (1.5 스코프와 수명, 인쇄 p.31-32)

네임스페이스 객체는 프로그램 종료 시 소멸하고, 멤버는 자신을 포함하는 **객체**의 소멸 시점에 따른다. [U01-070] (1.5 스코프와 수명, 인쇄 p.31-32)

new로 생성한 **객체**는 delete로 소멸할 때까지 유효하다. [U01-071] (1.5 스코프와 수명, 인쇄 p.31-32)

임시 **객체**나 new로 생성한 **객체**에는 이름이 없을 수도 있다. [U01-072] (1.5 스코프와 수명, 인쇄 p.31-32)

1.6 상수

const는 값을 바꾸지 않겠다는 약속을 나타내며 컴파일러가 이를 강제한다. [U01-073] (1.6 상수, 인쇄 p.32-34)

포인터나 참조로 전달된 데이터를 변경하지 않는다는 사실을 인터페이스에 표시할 수 있다. [U01-074] (1.6 상수, 인쇄 p.32-34)

const 값은 실행 시간에 계산될 수 있다. [U01-075] (1.6 상수, 인쇄 p.32-34)

constexpr는 값을 컴파일 시간에 평가한다는 뜻이며, **constexpr** 상수의 값은 반드시 컴파일 시간에 계산된다. [U01-076] (1.6 상수, 인쇄 p.32-34)

constexpr 함수와 상수 표현식 (원문 발췌)

```
constexpr double square(double x) { return x*x; }

constexpr double max1 = 1.4*square(17);  // OK 1.4*square(17)는 상수 표현식
constexpr double max2 = 1.4*square(var); // 에러: var은 상수 표현식이 아님
const double max3 = 1.4*square(var);     // OK, 실행 시간에 평가할 수 있음
```

[U01-077] (1.6 상수, 인쇄 p.33)

발췌 범위: 같은 페이지의 선행 선언 예제는 생략하고 square와 max1~max3 블록을 발췌했다.

상수 표현식 안에서 호출할 함수도 **constexpr**로 정의해야 한다. [U01-078] (1.6 상수, 인쇄 p.32-34)

constexpr 함수에 상수가 아닌 인자를 주는 것은 허용되지만 그 호출 결과는 상수 표현식이 아니다. [U01-079] (1.6 상수, 인쇄 p.32-34)

따라서 같은 연산을 상수 버전·일반 버전으로 중복 정의할 필요가 없다. [U01-080] (1.6 상수, 인쇄 p.32-34)

원문은 **constexpr** 함수가 단순하고 부수 효과가 없으며 인자로 전달된 정보만 사용해야 한다고 설명한다. [U01-081] (1.6 상수, 인쇄 p.32-34)

지역 **변수**와 루프는 허용되지만 지역이 아닌 **변수**를 변경할 수 없다. [U01-082] (1.6 상수, 인쇄 p.32-34)

지역 변수와 루프를 사용하는 constexpr (원문 발췌)

```
constexpr double nth(double x, int n) // 0<=n이라고 가정
{
    double res = 1;
    int i = 0;
    while (i<n) { // while 루프: 조건이 true일 동안 반복(1.7.1절)
        res*=x;
        ++i;
    }
    return res;
}
```

[U01-083] (1.6 상수, 인쇄 p.34)

원문에서 상수 표현식이 필요한 사례는 배열의 경계, case 라벨, **템플릿** 값 인자, **constexpr**로 선언한 상수다. [U01-084] (1.6 상수, 인쇄 p.32-34)

컴파일 시간 평가는 성능과 관계되며, 상태가 변하지 않는 **객체**라는 불변성 자체도 설계의 고려 사항이다. [U01-085] (1.6 상수, 인쇄 p.32-34)

1.7 포인터, 배열, 참조

배열은 연속된 영역에 할당되는 동일 **타입**의 시퀀스다. [U01-086] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

배열의 아래쪽 경계는 0이고, 크기는 상수 표현식이어야 한다. [U01-087] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

포인터 **변수**는 적절한 **타입 객체**가 있는 주소를 저장한다. [U01-088] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

선언에서 T*는 T를 가리키는 포인터, 표현식의 단항 &는 주소, 접두사 *는 가리키는 **객체**의 내용을 뜻한다. [U01-089] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

일반 for는 초기화·조건 검사·증가를 명시한다. [U01-090] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

구간 for는 시퀀스의 요소들을 처음부터 끝까지 탐색한다. [U01-091] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

auto x는 요소를 복사하고, auto& x는 요소 자체를 참조한다. [U01-092] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

참조는 접근할 때 *를 쓰지 않으며 일단 초기화되면 다른 **객체**를 참조하도록 바꿀 수 없다. [U01-093] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

참조 인자는 원본을 복사하지 않고 변경할 때 사용하며, **const** 참조는 복사 없이 변경도 방지할 때 사용한다. [U01-094] (1.7 포인터, 배열, 참조, 인쇄 p.34-37)

구간 for와 요소 참조 (원문 발췌)

```
void increment()
{
    int v[] = {0,1,2,3,4,5,6,7,8,9};

    for (auto& x : v) // v에 포함된 모든 x에 1을 더함
        ++x;
    // ...
}
```

[U01-095] (1.7 포인터, 배열, 참조, 인쇄 p.36)

1.7.1 널 포인터

유효하게 역참조하려면 포인터가 실제 **객체**를 가리켜야 한다. [U01-096] (1.7.1 널 포인터, 인쇄 p.37-39)

가리킬 **객체**가 없거나 가용한 **객체**가 없음을 나타낼 때 nullptr를 저장한다. [U01-097] (1.7.1 널 포인터, 인쇄 p.37-39)

모든 포인터 **타입**에 같은 nullptr 값을 사용하며, nullptr는 정수가 아니다. [U01-098] (1.7.1 널 포인터, 인쇄 p.37-39)

원문은 0·NULL 대신 nullptr를 사용해 정수와 포인터의 혼동을 줄이라고 권한다. [U01-099] (1.7.1 널 포인터, 인쇄 p.37-39)

널 검사 후 C 스타일 문자열 탐색 (원문 발췌)

```
int count_x(const char* p, char x)
// p[]에 존재하는 x와 동일한 값의 수를 센다.
// p는 0으로 끝나는 char의 배열을 가리킨다고 가정(그렇지 않으면 아무것도 가리키지 않음)
{
    if (p==nullptr)
        return 0;
    int count = 0;
    for (; *p!=0; ++p)
        if (*p==x)
            ++count;
    return count;
}
```

[U01-100] (1.7.1 널 포인터, 인쇄 p.38)

수치 값의 조건 테스트는 0과의 비교, 포인터의 조건 테스트는 nullptr와의 비교에 해당한다. [U01-101] (1.7.1 널 포인터, 인쇄 p.37-39)

while는 조건이 false가 될 때까지 반복한다. [U01-102] (1.7.1 널 포인터, 인쇄 p.37-39)

참조에는 널 참조라는 개념이 없으며 반드시 유효한 **객체**를 참조해야 한다. [U01-103] (1.7.1 널 포인터, 인쇄 p.37-39)

1.8 조건 테스트

if와 switch는 선택을, while와 for는 반복을 표현한다. [U01-104] (1.8 조건 테스트, 인쇄 p.39-42)

입력 연산자 >>는 표준 입력 스트림 **cin**에서 값을 읽으며, 오른쪽 연산 항 **타입**이 받아들일 입력을 결정한다. [U01-105] (1.8 조건 테스트, 인쇄 p.39-42)

변수는 구문을 둘 수 있는 곳에서 선언할 수 있으므로 원문은 실제로 필요한 위치에 정의한다. [U01-106] (1.8 조건 테스트, 인쇄 p.39-42)

switch는 값을 서로 다른 상수 case 라벨들과 비교한다. [U01-107] (1.8 조건 테스트, 인쇄 p.39-42)

어느 라벨과도 맞지 않으면 default가 선택되며, default도 없으면 아무 동작을 하지 않는다. [U01-108] (1.8 조건 테스트, 인쇄 p.39-42)

함수를 반환하지 않고 switch 다음으로 진행하려면 break를 사용한다. [U01-109] (1.8 조건 테스트, 인쇄 p.39-42)

여러 case를 같은 행동에 연결할 수도 있다. [U01-110] (1.8 조건 테스트, 인쇄 p.39-42)

if 초기화 절과 조건 (원문 발췌)

```
void do_something(vector<int>& v)
{
    if (auto n = v.size(); n!=0) {
        // ... n!=0 이면 여기를 실행...
    }
    // ...
}
```

[U01-111] (1.8 조건 테스트, 인쇄 p.42)

if의 초기화 절에서 선언한 이름은 다른 브랜치에서도 유효하다. [U01-112] (1.8 조건 테스트, 인쇄 p.39-42)

for에서처럼 **변수**의 스코프를 제한해 가독성을 높이고 에러를 줄이려는 목적이다. 0 또는 nullptr와 비교할 때는 조건 값을 직접 테스트하는 간단한 형식도 사용할 수 있다. [U01-113] (1.8 조건 테스트, 인쇄 p.39-42)

1.9 하드웨어 대응

C++의 기본 연산은 하드웨어 기능에 직접 대응하며 대부분 하나의 기계어로 처리된다. [U01-114] (1.9 하드웨어 대응, 인쇄 p.42-43)

구현은 메모리를 일련의 위치로 바라보고, **타입**이 있는 **객체**를 그 위치에 저장하며, 포인터로 위치를 가리킨다. [U01-115] (1.9 하드웨어 대응, 인쇄 p.42-43)

포인터는 머신 주소로 표현된다. [U01-116] (1.9 하드웨어 대응, 인쇄 p.42-43)

배열은 메모리상의 연속된 **객체**들을 추상화한 것이다. [U01-117] (1.9 하드웨어 대응, 인쇄 p.42-43)

원문은 이 기본 머신 모델이 수학이 아니라 컴퓨터 하드웨어에 기반한다고 설명한다. [U01-118] (1.9 하드웨어 대응, 인쇄 p.42-43)

1.9.1 대입

내장 **타입**의 대입은 머신의 복사 연산이다. [U01-119] (1.9.1 대입, 인쇄 p.43-45)

x=y 뒤에 x와 y는 같은 값을 갖지만 서로 독립된 **객체**다. [U01-120] (1.9.1 대입, 인쇄 p.43-45)

포인터 대입 p=q도 주소 값을 복사하므로 두 포인터 자체는 독립적이지만 같은 **객체**를 가리키게 된다. [U01-121] (1.9.1 대입, 인쇄 p.43-45)

반면 참조에 대입하면 참조 대상을 바꾸는 대신 이미 참조하고 있는 **객체**에 값을 쓴다. [U01-122] (1.9.1 대입, 인쇄 p.43-45)

원문은 모든 내장 **타입**에서 x=y의 결과로 x==y가 성립하며, 대입·상등 비교를 제공하는 잘 설계된 사용자 정의 **타입**에도 같은 기대를 적용한다. [U01-123] (1.9.1 대입, 인쇄 p.43-45)

1.9.2 초기화

초기화는 초기화되지 않은 메모리를 유효한 **객체**로 만드는 일이고, 대입은 이미 값을 가진 **객체**에 새 값을 쓰는 일이다. [U01-124] (1.9.2 초기화, 인쇄 p.45-46)

초기화되지 않은 **변수**를 읽거나 쓰는 효과는 일반적으로 예측할 수 없다. [U01-125] (1.9.2 초기화, 인쇄 p.45-46)

참조는 초기화 없이 만들 수 없으며, int& r = x도 값 복사가 아니라 r을 x에 바인딩하는 초기화다. [U01-126] (1.9.2 초기화, 인쇄 p.45-46)

string·vector처럼 해제할 **자원**을 소유하는 **타입**에서도 두 동작의 구별이 중요하다. [U01-127] (1.9.2 초기화, 인쇄 p.45-46)

함수 인자 전달과 반환 값의 기본 동작은 초기화와 유사하다. [U01-128] (1.9.2 초기화, 인쇄 p.45-46)

1.10 조언

의미 있는 연산을 짧은 함수로 묶고 한 함수는 하나의 논리적 연산을 수행하게 한다. [U01-129] (1.10 조언, 인쇄 p.46-48)

비슷한 연산은 같은 의미로 오버로딩한다. [U01-130] (1.10 조언, 인쇄 p.46-48)

변수는 반드시 초기화하고, 초깃값이 생길 때까지 선언을 미루며, 스코프를 작게 유지한다. [U01-131] (1.10 조언, 인쇄 p.46-48)

축소 변환과 복잡한 표현식을 피한다. [U01-132] (1.10 조언, 인쇄 p.46-48)

마법의 수 대신 기호 상수를 쓰고 변경 불가한 데이터를 선호한다. [U01-133] (1.10 조언, 인쇄 p.46-48)

컴파일 시간 평가가 필요한 함수에는 **constexpr**를 쓴다. [U01-134] (1.10 조언, 인쇄 p.46-48)

명명된 **타입**은 {} 초기화를 주로 사용하고 auto로 **타입** 이름 반복을 줄인다. [U01-135] (1.10 조언, 인쇄 p.46-48)

선언 하나에는 이름 하나를 둔다. [U01-136] (1.10 조언, 인쇄 p.46-48)

unsigned는 비트 조작에 사용하고 포인터는 단순하게 사용한다. 0·NULL 대신 nullptr를 쓴다. [U01-137] (1.10 조언, 인쇄 p.46-48)

이름의 범위와 용도에 맞게 길이를 정하고 비슷한 이름·모두 대문자인 이름을 피한다. [U01-138] (1.10 조언, 인쇄 p.46-48)

코드가 이미 보여 주는 사실보다 의도를 주석에 쓰고 들여쓰기를 일관되게 한다. [U01-139] (1.10 조언, 인쇄 p.46-48)

주요 비교표

const와 constexpr

구분	const	constexpr
의미 [U01-140] (const와 constexpr, 인쇄 p.32-34)	값을 바꾸지 않는다는 약속 [U01-141] (const와 constexpr, 인쇄 p.32-34)	컴파일 시간에 평가 [U01-142] (const와 constexpr, 인쇄 p.32-34)
계산 시점 [U01-143] (const와 constexpr, 인쇄 p.32-34)	실행 시간 계산 가능 [U01-144] (const와 constexpr, 인쇄 p.32-34)	상수의 값은 컴파일 시간 계산 [U01-145] (const와 constexpr, 인쇄 p.32-34)
함수 인자와의 관계 [U01-146] (const와 constexpr, 인쇄 p.32-34)	**const** 참조로 변경 방지 [U01-147] (const와 constexpr, 인쇄 p.32-34)	비상수 인자를 주면 결과는 상수 표현식이 아님 [U01-148] (const와 constexpr, 인쇄 p.32-34)

포인터와 참조의 대입

구분	포인터	참조
대입 대상 [U01-149] (포인터와 참조의 대입, 인쇄 p.44-46)	저장한 주소 값 [U01-150] (포인터와 참조의 대입, 인쇄 p.44-46)	이미 참조하는 **객체** [U01-151] (포인터와 참조의 대입, 인쇄 p.44-46)
재지정 [U01-152] (포인터와 참조의 대입, 인쇄 p.44-46)	다른 **객체** 주소를 저장 가능 [U01-153] (포인터와 참조의 대입, 인쇄 p.44-46)	초기화 뒤 참조 대상을 바꾸지 못함 [U01-154] (포인터와 참조의 대입, 인쇄 p.44-46)
없음의 표현 [U01-155] (포인터와 참조의 대입, 인쇄 p.44-46)	nullptr 사용 [U01-156] (포인터와 참조의 대입, 인쇄 p.44-46)	유효한 **객체**를 반드시 참조 [U01-157] (포인터와 참조의 대입, 인쇄 p.44-46)

핵심 용어 표

용어	원문 정의·설명
타입	**타입**은 **객체**가 취할 수 있는 값과 연산의 집합을 정의한다. [U01-001] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)
객체	**객체**는 어떤 **타입**의 값을 갖는 메모리다. [U01-158] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)
값	값은 **타입**에 따라 해석되는 비트의 집합이다. [U01-159] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)
변수	**변수**는 명명된 **객체**다. [U01-160] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)
지역 스코프	지역 이름의 유효 스코프는 선언 지점부터 선언을 포함하는 블록의 끝까지다. [U01-161] (1.5 스코프와 수명, 인쇄 p.31)
new로 생성한 객체의 수명	new로 생성한 **객체**는 delete로 소멸할 때까지 유효하다. [U01-162] (1.5 스코프와 수명, 인쇄 p.32)
const	**const**는 이 값을 바꾸지 않는다고 약속하며, **const**의 값은 실행 시간에 계산할 수 있다. [U01-163] (1.6 상수, 인쇄 p.33)
constexpr	**constexpr**는 이 값을 컴파일 시간에 평가한다는 의미이며, **constexpr**의 값은 반드시 컴파일 시간에 계산된다. [U01-002] (1.6 상수, 인쇄 p.33)

장 요약

요약 기준: 1.10 조언

함수 하나는 논리적인 연산 하나를 수행하게 하고 짧게 유지한다. [U01-164] (1.10 조언 [7]~[8], 인쇄 p.47)

변수는 반드시 초기화하고 스코프는 작게 유지한다. [U01-165] (1.10 조언 [24]~[25], 인쇄 p.47)

명명된 **타입**에는 초깃값 목록 {}를 사용하고 auto로 **타입** 이름의 반복을 피한다. [U01-166] (1.10 조언 [22]~[23], 인쇄 p.47)

0이나 NULL 대신 nullptr를 사용한다. [U01-167] (1.10 조언 [29], 인쇄 p.47)

제2장. 사용자 정의 타입

(제2장. 사용자 정의 타입, 인쇄 p.49-60)

학습 목표·범위

클래스는 데이터·함수·**타입**을 멤버로 가질 수 있으며, 인터페이스는 public 멤버로 정의된다. [U02-001] (2.3 클래스, 인쇄 p.53)

enum class는 독립된 **타입**이며 열거자의 스코프를 해당 **enum class**로 제한한다. [U02-002] (2.5 열거형, 인쇄 p.58)

핵심 개념·동작 절차·원문 예제

2.1 소개

기본 **타입**, **const** 제한자, 선언 연산에서 만들 수 있는 **타입**은 내장 **타입**이다. [U02-003] (2.1 소개, 인쇄 p.49-50)

원문은 내장 **타입**이 하드웨어를 직접적·효율적으로 반영하도록 의도적으로 로우레벨에 가깝게 설계되었다고 설명한다. [U02-004] (2.1 소개, 인쇄 p.49-50)

C++는 내장 **타입**과 연산에 추상화 메커니즘을 조합해 프로그래머가 필요한 하이레벨 기능을 만들도록 한다. [U02-005] (2.1 소개, 인쇄 p.49-50)

이렇게 만든 사용자 정의 **타입**은 적절한 표현 방식과 연산을 갖추며, 클래스와 열거형을 예로 들 수 있다. [U02-006] (2.1 소개, 인쇄 p.49-50)

내장 **타입**이나 다른 사용자 정의 **타입**을 기반으로 만들 수 있다. [U02-007] (2.1 소개, 인쇄 p.49-50)

잘 설계한 사용자 정의 **타입**은 사용하기 쉽고 에러 위험이 적으며 효율적일 수 있다. [U02-008] (2.1 소개, 인쇄 p.49-50)

표준 라이브러리도 주로 사용자 정의 **타입**으로 구성된다. [U02-009] (2.1 소개, 인쇄 p.49-50)

2.2 구조체

새로운 **타입**을 만드는 첫 단계는 필요한 구성 요소를 데이터 구조로 조직화하는 것이다. [U02-010] (2.2 구조체, 인쇄 p.50-52)

struct Vector의 첫 버전은 요소의 개수 sz와 요소들을 가리키는 double* elem을 묶는다. [U02-011] (2.2 구조체, 인쇄 p.50-52)

Vector **변수**를 선언하는 것만으로는 elem이 사용할 배열을 가리킨다는 보장이 없으므로 **vector**_init()이 별도로 필요하다. [U02-012] (2.2 구조체, 인쇄 p.50-52)

Vector의 데이터 표현 (원문 발췌)

```
struct Vector {
    int sz;       // 요소의 개수
    double* elem; // 요소를 가리키는 포인터
};
```

[U02-013] (2.2 구조체, 인쇄 p.50)

구조체를 초기화하는 별도 함수 (원문 발췌)

```
void vector_init(Vector& v, int s)
{
    v.elem = new double[s]; // s개의 double로 이뤄진 배열 할당
    v.sz = s;
}
```

[U02-014] (2.2 구조체, 인쇄 p.51)

new는 자유 저장 영역, 동적 메모리 또는 힙이라고 부르는 곳에 메모리를 할당한다. [U02-015] (2.2 구조체, 인쇄 p.50-52)

그 **객체**는 생성된 스코프와 무관하게 delete로 소멸할 때까지 유효하다. [U02-016] (2.2 구조체, 인쇄 p.50-52)

Vector&는 **const**가 아닌 참조이므로 **vector**_init()은 인자로 받은 원본 Vector를 변경할 수 있다. [U02-017] (2.2 구조체, 인쇄 p.50-52)

이름·참조를 통해 **struct** 멤버에 접근할 때는 .을 쓰고, 포인터를 통해서는 ->를 쓴다. [U02-018] (2.2 구조체, 인쇄 p.50-52)

원문의 read_and_sum()은 Vector를 초기화하고 입력을 요소에 저장한 뒤 합을 계산한다. [U02-019] (2.2 구조체, 인쇄 p.50-52)

그러나 사용자는 elem과 sz라는 표현의 세부 사항을 알아야 한다. [U02-020] (2.2 구조체, 인쇄 p.50-52)

원문은 이 Vector를 언어 기능·설계 기법을 설명하기 위한 점진적 예제로 쓰며, 표준 **vector**나 **string**을 재발명하기보다 **표준 라이브러리** 구성 요소를 활용하라고 권한다. [U02-021] (2.2 구조체, 인쇄 p.50-52)

2.3 클래스

사용자 정의 **타입**을 일관되게 사용하고 표현을 나중에 개선하려면, 모두가 쓰는 인터페이스와 외부에서 접근할 수 없는 데이터를 사용하는 구현을 분리한다. [U02-022] (2.3 클래스, 인쇄 p.53-55)

class가 이를 지원한다. [U02-023] (2.3 클래스, 인쇄 p.53-55)

클래스는 데이터·함수·**타입**을 멤버로 가질 수 있고 public 멤버가 인터페이스를 정의한다. [U02-024] (2.3 클래스, 인쇄 p.53-55)

priv**at**e 멤버에 대한 접근은 인터페이스를 통해 이뤄진다. [U02-025] (2.3 클래스, 인쇄 p.53-55)

인터페이스와 표현을 분리한 Vector (원문 발췌)

```
class Vector {
public:
    Vector(int s) :elem{new double[s]}, sz{s} { } // Vector 생성
    double& operator[](int i) { return elem[i]; } // 요소 접근: 첨자 연산
    int size() { return sz; }
private:
    double* elem; // 요소를 가리키는 포인터
    int sz;       // 요소의 개수
};
```

[U02-026] (2.3 클래스, 인쇄 p.53)

Vector와 이름이 같은 멤버 함수는 **생성자**다. [U02-027] (2.3 클래스, 인쇄 p.53-55)

객체를 만들 때 **생성자**가 반드시 사용되므로 별도 초기화 함수 호출을 잊는 실수를 막는다. [U02-028] (2.3 클래스, 인쇄 p.53-55)

Vector(int)는 생성 시 요소 개수를 정수로 주어야 함을 드러낸다. [U02-029] (2.3 클래스, 인쇄 p.53-55)

멤버 초깃값 목록의 elem{new double[s]}는 할당된 배열의 주소로 elem을, sz{s}는 s로 sz를 초기화한다. [U02-030] (2.3 클래스, 인쇄 p.53-55)

operator[]는 적절한 요소의 double&를 반환해 읽기와 쓰기를 제공하고, size()는 요소 개수를 돌려준다. [U02-031] (2.3 클래스, 인쇄 p.53-55)

Vector **객체** 자체는 elem과 sz를 갖는 고정 크기의 **핸들**이며, 실제 요소는 다른 곳에 저장된다. [U02-032] (2.3 클래스, 인쇄 p.53-55)

따라서 **객체**마다 요소 개수가 달라도 **핸들** 자체의 크기는 같다. [U02-033] (2.3 클래스, 인쇄 p.53-55)

원문은 이러한 **핸들** 기법이 개수가 변하는 데이터에 흔히 쓰인다고 설명한다. [U02-034] (2.3 클래스, 인쇄 p.53-55)

이 단계의 Vector에는 아직 에러 처리와 할당한 배열의 해제가 없다. [U02-035] (2.3 클래스, 인쇄 p.53-55)

원문은 이를 각각 3.5절과 4.2.2절에서 보완한다. [U02-036] (2.3 클래스, 인쇄 p.53-55)

struct와 **class**에는 근본적 차이가 없으며, **struct**는 멤버의 기본 접근이 public인 **class**다. [U02-037] (2.3 클래스, 인쇄 p.53-55)

따라서 **struct**에도 **생성자**를 포함한 멤버 함수를 둘 수 있다. [U02-038] (2.3 클래스, 인쇄 p.53-55)

2.4 공용체

union은 모든 멤버를 같은 메모리 주소에 할당하는 구조체이며, 원문은 공용체의 크기를 가장 큰 멤버의 크기로 설명한다. [U02-039] (2.4 공용체, 인쇄 p.55-57)

일반적으로 실제 값을 갖는 멤버는 하나뿐이다. [U02-040] (2.4 공용체, 인쇄 p.55-57)

원문의 Entry 예에서는 Node*와 int가 동시에 사용되지 않아 별도 멤버로 두면 공간이 낭비된다. [U02-041] (2.4 공용체, 인쇄 p.55-57)

둘을 **union** Value에 넣으면 메모리를 절약한다. [U02-042] (2.4 공용체, 인쇄 p.55-57)

Node*와 int의 저장 공간 공유 (원문 발췌)

```
union Value {
    Node* p;
    int i;
};
```

[U02-043] (2.4 공용체, 인쇄 p.56)

언어는 **union**에 현재 어느 종류의 값이 있는지 확인하지 않으므로 프로그래머가 관리해야 한다. [U02-044] (2.4 공용체, 인쇄 p.55-57)

Entry의 Type t가 ptr이면 v.p, num이면 v.i를 쓰는 관계를 지켜야 한다. [U02-045] (2.4 공용체, 인쇄 p.55-57)

타입 필드와 실제 저장된 **타입**의 관계를 직접 관리하면 에러가 발생하기 쉽다. [U02-046] (2.4 공용체, 인쇄 p.55-57)

원문은 공용체와 **타입** 필드를 캡슐화하고 멤버 함수로만 접근하게 하는 태그된 공용체의 추상화를 권한다. [U02-047] (2.4 공용체, 인쇄 p.55-57)

variant로 표현하는 대안 값 (원문 발췌)

```
struct Entry {
    string name;
    variant<Node*,int> v;
};

void f(Entry* pe)
{
    if (holds_alternative<int>(pe->v)) // *pe가 int를 포함하는가?(13.5.1절 참고)
        cout << get<int>(pe->v); // int 가져오기
    // ...
}
```

[U02-048] (2.4 공용체, 인쇄 p.57)

표준 라이브러리 variant는 대안적인 여러 **타입** 중 하나의 값을 제공하며 대부분의 상황에서 **union**을 대체할 수 있다. [U02-049] (2.4 공용체, 인쇄 p.55-57)

variant<Node*,int>는 Node* 또는 int를 저장한다. [U02-050] (2.4 공용체, 인쇄 p.55-57)

holds_alternative<int>()로 int 포함 여부를 확인하고 get<int>()로 값을 가져오는 원문 예는 별도의 **타입** 필드를 수작업으로 관리하는 경우보다 간단하고 안전하다는 설명에 연결된다. [U02-051] (2.4 공용체, 인쇄 p.55-57)

2.5 열거형

열거형은 여러 정수 값의 집합을 명명된 상수들로 표현하는 사용자 정의 **타입**이다. [U02-052] (2.5 열거형, 인쇄 p.57-59)

enum class는 열거자의 스코프를 해당 열거형으로 제한하므로 Color::red와 Traffic_light::red를 혼동하지 않고 같은 red 이름을 사용할 수 있다. [U02-053] (2.5 열거형, 인쇄 p.57-59)

두 열거형은 독립된 **타입**이며 서로의 값이나 정수와 암묵적으로 혼용할 수 없다. [U02-054] (2.5 열거형, 인쇄 p.57-59)

서로 독립된 enum class (원문 발췌)

```
enum class Color { red, blue , green };
enum class Traffic_light { green, yellow, red };

Color col = Color::red;
Traffic_light light = Traffic_light::red;
```

[U02-055] (2.5 열거형, 인쇄 p.57)

원문은 기반 **타입**에서 열거형을 초기화할 필요가 있으면 Color{5}나 Color y {6}처럼 명시적 초기화를 할 수 있다고 설명한다. [U02-056] (2.5 열거형, 인쇄 p.57-59)

기본적으로 **enum class**는 대입·초기화·비교를 정의하며, 사용자 정의 **타입**이므로 연산을 추가할 수 있다. [U02-057] (2.5 열거형, 인쇄 p.57-59)

Traffic_light의 전치 증가 연산은 green→yellow→red→green 순서의 변화를 정의한다. [U02-058] (2.5 열거형, 인쇄 p.57-59)

열거형의 전치 증가 연산 정의 (원문 발췌)

```
Traffic_light& operator++(Traffic_light& t) // 전치 증가: ++
{
    switch (t) {
    case Traffic_light::green: return t=Traffic_light::yellow;
    case Traffic_light::yellow: return t=Traffic_light::red;
    case Traffic_light::red: return t=Traffic_light::green;
    }
}

Traffic_light next = ++light; // 다음 값은 Traffic_light::green
```

[U02-059] (2.5 열거형, 인쇄 p.59)

class 없이 enum을 쓰면 열거자 이름은 enum 이름과 같은 스코프에 속하고, 열거자 값은 정수로 암묵 변환된다. [U02-060] (2.5 열거형, 인쇄 p.57-59)

원문의 enum Color { red, green, blue }; int col = green;에서 col은 1이다. [U02-061] (2.5 열거형, 인쇄 p.57-59)

기본 열거자 값은 0에서 시작해 하나씩 증가한다. [U02-062] (2.5 열거형, 인쇄 p.57-59)

원문은 예측하지 못한 에러를 줄이기 위해 **enum class**를 선호하라고 조언한다. [U02-063] (2.5 열거형, 인쇄 p.57-59)

2.6 조언

내장 **타입**이 너무 로우레벨이면 잘 정의된 사용자 정의 **타입**을 쓰고, 관련 데이터는 **struct**나 **class**로 조직한다. [U02-064] (2.6 조언, 인쇄 p.59-60)

class로 인터페이스와 구현을 분리하고 **생성자**로 초기화를 보장·단순화한다. [U02-065] (2.6 조언, 인쇄 p.59-60)

벌거벗은 **union**을 피하고 공용체와 **타입** 필드를 클래스로 감싼다. [U02-066] (2.6 조언, 인쇄 p.59-60)

명명된 상수 집합에는 열거형을 사용한다. [U02-067] (2.6 조언, 인쇄 p.59-60)

enum보다 **enum class**를 선호하고 안전하고 쉽게 쓸 수 있도록 필요한 연산을 정의한다. [U02-068] (2.6 조언, 인쇄 p.59-60)

주요 비교표

원문에서 비교한 데이터 표현

구분	원문의 핵심 차이
struct / **class** [U02-069] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)	**struct**는 기본 멤버 접근이 public인 **class**이며 근본적 차이는 없다. [U02-070] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)
직접 **union** / **variant** [U02-071] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)	**union**의 저장 **타입**은 프로그래머가 관리한다. **variant**는 대안 **타입** 중 하나를 제공한다. [U02-072] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)
enum / **enum class** [U02-073] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)	enum은 이름이 바깥 스코프에 있고 정수로 암묵 변환된다. **enum class**는 스코프와 **타입**을 제한한다. [U02-074] (원문에서 비교한 데이터 표현, 인쇄 p.53-59)

핵심 용어 표

용어	원문 정의·설명
class	클래스는 데이터·함수·**타입**을 멤버로 가질 수 있으며, 인터페이스는 public 멤버로 정의된다. [U02-001] (2.3 클래스, 인쇄 p.53)
생성자	해당 클래스와 이름이 동일한 멤버 함수를 **생성자**라고 하며, 클래스를 **객체**화할 때 사용한다. [U02-075] (2.3 클래스, 인쇄 p.54)
핸들	원문의 Vector **객체**는 요소 포인터 elem과 요소 개수 sz를 포함하는 **핸들**이며, Vector **객체** 자체의 크기는 항상 같다. [U02-076] (2.3 클래스, 인쇄 p.54)
struct	**struct**는 멤버가 기본적으로 public인 **class**이며, **struct**에도 **생성자**를 포함한 멤버 함수를 정의할 수 있다. [U02-077] (2.3 클래스, 인쇄 p.55)
union	**union**은 모든 멤버가 같은 메모리 주소에 할당되는 **struct**이며, 공용체의 크기는 가장 큰 멤버의 크기와 같다. [U02-078] (2.4 공용체, 인쇄 p.55)
variant	**variant**는 대안적인 여러 **타입**의 값 중 하나를 제공하며, 대부분의 상황에서 공용체를 대체할 수 있다. [U02-079] (2.4 공용체, 인쇄 p.57)
enum class	**enum class**는 독립된 **타입**이며 열거자의 스코프를 해당 **enum class**로 제한한다. [U02-002] (2.5 열거형, 인쇄 p.58)

장 요약

요약 기준: 2.6 조언

관련된 데이터는 **struct**나 **class**로 구조화하고 **class**로 인터페이스와 구현을 분리한다. [U02-080] (2.6 조언 [2]~[3], 인쇄 p.59)

클래스의 초기화를 보장하고 단순화하도록 **생성자**를 정의한다. [U02-081] (2.6 조언 [5], 인쇄 p.60)

벌거벗은 **union**은 피하고 공용체와 **타입** 필드를 클래스로 감싼다. [U02-082] (2.6 조언 [6], 인쇄 p.60)

명명된 상수 집합은 열거형으로 표현하며, 예상하지 못한 에러를 줄이기 위해 **enum class**를 사용한다. [U02-083] (2.6 조언 [7]~[8], 인쇄 p.60)

제3장. 모듈화

(제3장. 모듈화, 인쇄 p.61-89)

학습 목표·범위

네임스페이스는 일부 선언을 함께 묶고 다른 이름과 충돌하지 않게 하는 메커니즘이다. [U03-001] (3.4 네임스페이스, 인쇄 p.68)

어써션은 기대되는 조건을 나타내는 구문이며, st**at**ic_assert는 상수 표현식으로 표현할 수 있는 속성을 확인한다. [U03-002] (3.5.5 정적 어써션, 인쇄 p.80)

핵심 개념·동작 절차·원문 예제

3.1 소개

독립적으로 개발된 함수·**타입**·계층 구조·**템플릿**을 관리하려면 상호작용을 명확히 정의해야 한다. [U03-003] (3.1 소개, 인쇄 p.61-63)

첫 단계는 인터페이스와 구현을 분리하는 것이다. [U03-004] (3.1 소개, 인쇄 p.61-63)

C++는 선언으로 인터페이스를 표현하고 함수·**타입**을 사용하는 데 필요한 사항을 명시한다. [U03-005] (3.1 소개, 인쇄 p.61-63)

함수 몸체인 정의는 다른 곳에 둘 수 있다. [U03-006] (3.1 소개, 인쇄 p.61-63)

클래스 밖의 멤버 정의에서는 Vector::처럼 클래스 이름으로 한정한다. [U03-007] (3.1 소개, 인쇄 p.61-63)

라이브러리도 같은 언어 기능으로 만든 다른 코드이므로 기본 원리는 같다. [U03-008] (3.1 소개, 인쇄 p.61-63)

원문은 구성 요소를 여러 번 선언할 수 있지만 정의는 한 번만 가능하다고 설명한다. [U03-009] (3.1 소개, 인쇄 p.61-63)

3.2 분할 컴파일

사용자 코드는 **타입**·함수의 선언만 있으면 컴파일할 수 있고, 정의는 다른 파일에서 따로 컴파일할 수 있다. [U03-010] (3.2 분할 컴파일, 인쇄 p.63-66)

분할 컴파일은 논리적으로 구별되는 부분을 강제로 나누고 컴파일 시간을 줄이는 데 도움이 된다. [U03-011] (3.2 분할 컴파일, 인쇄 p.63-66)

모듈의 인터페이스 선언은 보통 헤더에 두고, 사용자와 구현 파일이 모두 같은 헤더를 포함해 컴파일러가 일관성을 확인하게 한다. [U03-012] (3.2 분할 컴파일, 인쇄 p.63-66)

분할 컴파일을 위한 헤더 (원문 발췌)

```
// Vector.h:

class Vector {
public:
    Vector(int s);
    double& operator[](int i);
    int size();
private:
    double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
    int sz;
};
```

[U03-013] (3.2 분할 컴파일, 인쇄 p.64)

헤더를 사용하는 독립 사용자 파일 (원문 발췌)

```
// user.cpp:

#include "Vector.h" // Vector의 인터페이스 가져오기
#include <cmath> // sqrt()를 포함하는 표준 라이브러리 수학 함수 인터페이스 가져오기

double sqrt_sum(Vector& v)
{
    double sum = 0;
    for (int i=0; i!=v.size(); ++i)
        sum+=std::sqrt(v[i]); // 제곱근의 합
    return sum;
}
```

[U03-014] (3.2 분할 컴파일, 인쇄 p.64)

Vector.h는 인터페이스를 공유하는 지점이며 user.cpp와 Vector.cpp는 독립적으로 컴파일한다. [U03-015] (3.2 분할 컴파일, 인쇄 p.63-66)

원문은 프로그램을 의존성이 잘 정의된 모듈의 집합으로 생각하고 파일을 통한 물리적 모듈화를 활용하라고 권한다. [U03-016] (3.2 분할 컴파일, 인쇄 p.63-66)

독립적으로 컴파일하는 .cpp 파일과 그 파일이 #include하는 헤더를 포함한 단위를 변환 단위라고 부른다. [U03-017] (3.2 분할 컴파일, 인쇄 p.63-66)

한 프로그램은 수천 개의 변환 단위로 구성될 수 있다. [U03-018] (3.2 분할 컴파일, 인쇄 p.63-66)

3.3 모듈(C++20)

#include는 사용하는 변환 단위마다 헤더 텍스트를 다시 처리한다. [U03-019] (3.3 모듈(C++20), 인쇄 p.66-68)

또한 먼저 포함한 헤더의 선언·매크로가 뒤 헤더의 의미에 영향을 줄 수 있어 포함 순서가 중요해지는 문제가 있다. [U03-020] (3.3 모듈(C++20), 인쇄 p.66-68)

원문 시점의 module은 아직 ISO C++가 아니라 ISO 기술 명세에 포함된 기능이며, 세부 사항이 바뀔 수 있고 제품 수준에서 사용하려면 더 시간이 필요할 수 있다고 설명된다. [U03-021] (3.3 모듈(C++20), 인쇄 p.66-68)

이 요약은 그 상태 설명을 그대로 유지한다. [U03-022] (3.3 모듈(C++20), 인쇄 p.66-68)

모듈 선언과 노출할 클래스 (원문 발췌)

```
// Vector.cpp 파일:
module; // module을 정의

// ... Vector 구현에 필요한 내용 작성 ...

export module Vector; // "Vector" 모듈 정의

export class Vector {
public:
    Vector(int s);
    double& operator[](int i);
    int size();
private:
    double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
    int sz;
};
```

[U03-023] (3.3 모듈(C++20), 인쇄 p.66-67)

발췌 범위: 이 모듈 예제의 앞부분을 발췌했다. 뒤의 멤버 함수 정의·비멤버 size 정의와 사용자 파일은 생략했다.

export는 모듈 사용자에게 보여 줄 인터페이스를 노출하고, 필요한 곳에서 import로 가져온다. [U03-024] (3.3 모듈(C++20), 인쇄 p.66-68)

원문은 import와 #include를 함께 써 기존 코드에서 점진적으로 옮기는 방법을 보인다. [U03-025] (3.3 모듈(C++20), 인쇄 p.66-68)

모듈은 사용하는 모든 변환 단위마다 다시 컴파일하지 않고 한 번 컴파일한다. [U03-026] (3.3 모듈(C++20), 인쇄 p.66-68)

두 모듈의 import 순서는 의미에 영향을 주지 않으며 import는 전이되지 않는다. [U03-027] (3.3 모듈(C++20), 인쇄 p.66-68)

즉 모듈 내부에서 가져온 이름을 그 모듈의 사용자가 암묵적으로 접근·조작할 수 있는 것은 아니다. [U03-028] (3.3 모듈(C++20), 인쇄 p.66-68)

3.4 네임스페이스

네임스페이스는 일부 선언을 묶고 독립적으로 개발한 이름들이 충돌하지 않도록 한다. [U03-029] (3.4 네임스페이스, 인쇄 p.68-70)

다른 **네임스페이스**의 이름은 std::cout 또는 My_code::main처럼 이름을 한정해 접근한다. [U03-030] (3.4 네임스페이스, 인쇄 p.68-70)

프로그램의 실제 main()은 전역 **네임스페이스**에 둔다. [U03-031] (3.4 네임스페이스, 인쇄 p.68-70)

using std::swap과 같은 using 선언은 특정 이름을 현재 스코프에서 선언한 것처럼 사용하게 한다. [U03-032] (3.4 네임스페이스, 인쇄 p.68-70)

using namespace std와 같은 using 지시자는 **네임스페이스**의 모든 이름을 사용할 수 있게 한다. [U03-033] (3.4 네임스페이스, 인쇄 p.68-70)

후자는 이름을 선택적으로 가져오는 것이 아니므로 신중히 사용한다. [U03-034] (3.4 네임스페이스, 인쇄 p.68-70)

원문은 애플리케이션 전반에서 쓰는 기반 라이브러리나 기존 프로그램을 개선하는 과정에서의 사용을 설명하고, 헤더에는 using 지시자를 넣지 말라고 조언한다. [U03-035] (3.4 네임스페이스, 인쇄 p.68-70)

3.5 에러 처리

에러 처리의 가장 중요한 도구는 **타입** 시스템이다. [U03-036] (3.5 에러 처리, 인쇄 p.71)

내장 **타입**과 제어 구문만 사용하기보다 적절한 사용자 정의 **타입**과 알고리즘으로 높은 수준의 개념을 표현하면 실수 가능성을 낮추고 컴파일러가 에러를 찾을 가능성을 높인다. [U03-037] (3.5 에러 처리, 인쇄 p.71)

추상화는 런타임 에러를 탐지하는 지점과 처리하는 지점도 분리한다. [U03-038] (3.5 에러 처리, 인쇄 p.71)

라이브러리를 많이 쓰는 큰 프로그램일수록 일관된 전략이 중요하므로 설계 초기에 에러 처리 방식을 고려한다. [U03-039] (3.5 에러 처리, 인쇄 p.71)

3.5.1 예외

Vector 작성자는 범위 밖 접근 시 사용자가 무엇을 해야 할지 알 수 없고, 사용자는 잘못된 접근을 항상 감지할 수 없다. [U03-040] (3.5.1 예외, 인쇄 p.71-73)

따라서 구현자가 문제를 탐지해 알리고 사용자가 조치를 취하게 한다. [U03-041] (3.5.1 예외, 인쇄 p.71-73)

예제의 oper**at**or[]는 인덱스가 범위 밖이면 out_of_range를 던진다. [U03-042] (3.5.1 예외, 인쇄 p.71-73)

범위 검사와 예외 핸들러 (원문 발췌)

```
double& Vector::operator[](int i)
{
    if (i<0 || size()<=i)
        throw out_of_range{"Vector::operator[]"};
    return elem[i];
}

void f(Vector& v)
{
    // ...
    try { // 여기서 발생한 예외를 아래에 정의한 핸들러가 처리
        v[v.size()] = 7; // v의 끝을 넘어서 접근 시도
    }
    catch (out_of_range& err) { // out_of_range 예외
        // ... 스코프 밖 에러 처리...
        cerr << err.what() << '\n';
    }
    // ...
}
```

[U03-043] (3.5.1 예외, 인쇄 p.72)

throw는 해당 **타입**의 **핸들**러에 제어를 넘긴다. [U03-044] (3.5.1 예외, 인쇄 p.71-73)

구현은 호출 스택을 거슬러 올라가며 필요한 스코프·함수를 종료하고 그 과정에서 **소멸자**를 호출한다. [U03-045] (3.5.1 예외, 인쇄 p.71-73)

try에 처리할 코드를 놓고 c**at**ch에서 예외를 잡는다. [U03-046] (3.5.1 예외, 인쇄 p.71-73)

예외는 복사를 피하려고 참조로 잡으며 wh**at**()은 던질 때 설정한 메시지를 제공한다. [U03-047] (3.5.1 예외, 인쇄 p.71-73)

모든 함수에서 try를 반복하기보다 **생성자**에서 **자원**을 획득하고 **소멸자**에서 해제하는 **RAII**를 이용하는 것이 원문의 권고다. [U03-048] (3.5.1 예외, 인쇄 p.71-73)

예외를 던지지 않는 함수는 **noexcept**로 선언할 수 있다. [U03-049] (3.5.1 예외, 인쇄 p.71-73)

그런 함수에서 예외가 빠져나오면 std::termin**at**e()를 호출해 프로그램을 즉시 종료한다. [U03-050] (3.5.1 예외, 인쇄 p.71-73)

선언만으로 내부 호출이 성공한다고 보장되는 것은 아니라는 점을 원문의 user() 예제가 보여 준다. [U03-051] (3.5.1 예외, 인쇄 p.71-73)

3.5.2 불변 조건

함수의 **사전 조건**은 작동의 기본 가정이다. [U03-052] (3.5.2 불변 조건, 인쇄 p.73-76)

Vector 인덱싱은 인덱스가 [0:size()) 안에 있어야 한다는 전제를 검사한다. [U03-053] (3.5.2 불변 조건, 인쇄 p.73-76)

[a:b)는 a를 포함하고 b는 포함하지 않는 **반개구간**이다. [U03-054] (3.5.2 불변 조건, 인쇄 p.73-76)

클래스 불변 조건은 멤버 함수가 의지할 수 있도록 클래스 수준에서 보장되어야 하는 조건이다. [U03-055] (3.5.2 불변 조건, 인쇄 p.73-76)

Vector에서는 elem이 sz개의 double을 갖는 배열을 가리킨다는 관계가 이에 해당한다. [U03-056] (3.5.2 불변 조건, 인쇄 p.73-76)

생성자에서 유효한 크기 확인 (원문 발췌)

```
Vector::Vector(int s)
{
    if (s<0)
        throw length_error{"Vector 생성자: 크기가 음수임"};
    elem = new double[s];
    sz = s;
}
```

[U03-057] (3.5.2 불변 조건, 인쇄 p.74)

생성자는 불변 조건을 성립시키고, 멤버 함수는 종료할 때 불변 조건을 보존해야 한다. [U03-058] (3.5.2 불변 조건, 인쇄 p.73-76)

s<0이면 length_error를 던지고, new가 메모리를 할당하지 못하면 std::bad_alloc을 던진다. [U03-059] (3.5.2 불변 조건, 인쇄 p.73-76)

사용자 정의 예외 **타입**으로 탐지 지점에서 처리 지점까지 필요한 정보를 전달할 수도 있다. [U03-060] (3.5.2 불변 조건, 인쇄 p.73-76)

함수를 완수하지 못하면 최소한의 지역 정리를 하고 throw;로 다시 던지거나, 복구가 불가능하면 종료하는 예를 원문이 제시한다. [U03-061] (3.5.2 불변 조건, 인쇄 p.73-76)

불변 조건은 무엇을 원하는지 명확하게 하고 더 구체적인 설계를 강제하며, 디버깅·테스팅을 거쳐 올바른 코드를 유지하게 돕는다. [U03-062] (3.5.2 불변 조건, 인쇄 p.73-76)

생성자·**소멸자** 기반 **자원** 관리도 이 개념에 기초한다. [U03-063] (3.5.2 불변 조건, 인쇄 p.73-76)

원문의 **생성자** 조건은 s<0인데 이어지는 설명은 “양수가 아님”이라고 표현한다. [U03-064] (3.5.2 불변 조건, 인쇄 p.73-76)

[원문 확인 필요] 이 요약에서는 코드의 음수 검사 조건을 바꾸지 않았다. [U03-065] (3.5.2 불변 조건, 인쇄 p.73-76)

3.5.3 여러 가지 에러 처리 방식

에러를 지역적으로 처리할 수 없다면 호출자에게 실패를 알려야 한다. [U03-066] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

원문은 예외, 실패를 나타내는 값, 프로그램 종료를 비교한다. [U03-067] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

C++의 예외는 정상적인 값을 대신 반환하는 기법이 아니라 요청받은 작업을 완수하지 못했음을 보고하도록 설계되었고 **생성자**·**소멸자**와 통합된다. [U03-068] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

에러 코드: 실패가 흔하고 예상 가능하며 직접 호출자가 처리할 것으로 기대할 수 있는 경우. [U03-069] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

파일 열기 실패가 예다. [U03-070] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

예외: 에러가 드물어 확인을 잊기 쉬운 경우, 직접 호출자가 처리하기 어려운 경우, 새 에러 종류가 늘 수 있는 경우, **생성자**처럼 반환 경로가 없는 경우, 값과 에러를 함께 반환하면 경로가 복잡한 경우. [U03-071] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

예외가 적합한 다른 경우: 여러 호출을 거슬러 전달해야 하거나 복구가 여러 호출 결과에 의존하는 경우, 콜백에서 실패한 경우, 작업 취소가 필요한 경우. [U03-072] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

종료: 복구할 수 없거나, 심각한 에러를 스레드·프로세스·컴퓨터 재시작으로 처리하도록 설계한 시스템. [U03-073] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

원문은 에러 코드와 예외 중 어느 하나를 완전히 배제하지 말고, 판단이 어려우면 예외를 사용하라고 권한다. [U03-074] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

예외가 항상 느리다는 믿음도 부정하며, 복잡하거나 드문 실패를 처리하고 반복적인 에러 코드 확인을 피할 때 빠를 수도 있다고 설명한다. [U03-075] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

간결한 예외 처리에는 **RAII**가 필수적이다. [U03-076] (3.5.3 여러 가지 에러 처리 방식, 인쇄 p.76-78)

3.5.4 계약

원문 시점에는 **사전 조건**·불변 조건을 선택적으로 실행 시간에 검사하는 일반적인 표준 방법이 없다고 설명한다. [U03-077] (3.5.4 계약, 인쇄 p.79)

C++20에 계약 메커니즘이 제안되어 테스트에서는 엄격히 검사하고 배포에서는 최소 검사하는 용도가 소개되지만, 원문은 당시 임시 방편을 사용해야 한다고 명시한다. [U03-078] (3.5.4 계약, 인쇄 p.79)

예로 RANGE_CHECK 매크로에 따라 범위 검사를 수행하는 코드를 제시한다. [U03-079] (3.5.4 계약, 인쇄 p.79)

표준 라이브러리의 assert()는 주어진 실행 시간 조건을 확인하는 디버그용 매크로다. [U03-080] (3.5.4 계약, 인쇄 p.79)

디버그 모드에서 조건이 성립하지 않으면 종료하며, 디버그 모드가 아니면 검사하지 않는다. [U03-081] (3.5.4 계약, 인쇄 p.79)

원문은 유연성은 적지만 충분한 경우가 많다고 설명한다. [U03-082] (3.5.4 계약, 인쇄 p.79)

3.5.5 정적 어써션

실행 시간 에러는 예외로 보고하되, 컴파일 시간에 찾을 수 있는 에러는 그때 찾는 것이 좋다. [U03-083] (3.5.5 정적 어써션, 인쇄 p.80-81)

기대 조건을 표현하는 구문을 어써션이라고 하며 st**at**ic_assert는 상수 표현식으로 나타낼 수 있는 속성을 검사한다. [U03-084] (3.5.5 정적 어써션, 인쇄 p.80-81)

st**at**ic_assert(A,S)에서 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다. [U03-085] (3.5.5 정적 어써션, 인쇄 p.80-81)

S를 생략하면 기본 메시지를 사용한다. [U03-086] (3.5.5 정적 어써션, 인쇄 p.80-81)

컴파일 시간 속성 검사 (원문 발췌)

```
static_assert(4<=sizeof(int), "정수가 너무 작음"); // 정수 크기 확인
```

[U03-087] (3.5.5 정적 어써션, 인쇄 p.80)

원문의 속력 예제는 함수 인자 speed는 상수 표현식이 아니므로 st**at**ic_assert에 쓸 수 없고 **constexpr** local_max는 사용할 수 있음을 보여 준다. [U03-088] (3.5.5 정적 어써션, 인쇄 p.80-81)

제네릭 코드의 **타입** 파라미터에 대한 가정을 확인하는 용도로도 유용하다. [U03-089] (3.5.5 정적 어써션, 인쇄 p.80-81)

3.6 함수 인자와 반환 값

함수 호출은 프로그램의 두 부분 사이에 정보를 전달하는 주된 방법이다. [U03-090] (3.6 함수 인자와 반환 값, 인쇄 p.81-82)

작업에 필요한 정보는 인자로, 결과는 반환 값으로 전달한다. [U03-091] (3.6 함수 인자와 반환 값, 인쇄 p.81-82)

전역 **변수**는 에러의 주요 원인이므로 피하고, 공유 상태는 클래스 멤버처럼 함께 잘 정의된 추상 개념을 구현하는 함수들로 제한한다. [U03-092] (3.6 함수 인자와 반환 값, 인쇄 p.81-82)

객체의 복사·공유 여부, 변경 허용 여부, 빈 **객체**를 남기는 이동 가능성이 경계 설계의 고려 사항이다. [U03-093] (3.6 함수 인자와 반환 값, 인쇄 p.81-82)

기본은 복사지만 암묵적 이동으로 최적화될 수 있다. [U03-094] (3.6 함수 인자와 반환 값, 인쇄 p.81-82)

3.6.1 인자 전달

값에 의한 전달은 복사본을 사용하고, 참조에 의한 전달은 호출자 환경의 원본 **객체**를 참조한다. [U03-095] (3.6.1 인자 전달, 인쇄 p.82-83)

원문의 test()는 값 인자 v를 수정해도 원본은 그대로이고, 참조 인자 rv를 수정하면 원본이 바뀜을 출력으로 보여 준다. [U03-096] (3.6.1 인자 전달, 인쇄 p.82-83)

작은 **객체**는 값으로, 복사 비용이 큰 **객체**는 참조로 전달한다. “작다”는 기준은 아키텍처에 따라 달라지며, 원문은 대략 포인터 두세 개 크기 이하를 제시한다. [U03-097] (3.6.1 인자 전달, 인쇄 p.82-83)

성능상 참조를 사용하되 변경이 필요 없으면 **const** 참조를 선호한다. [U03-098] (3.6.1 인자 전달, 인쇄 p.82-83)

기본 함수 인자는 자주 쓰는 기본값을 선언에 지정한다. [U03-099] (3.6.1 인자 전달, 인쇄 p.82-83)

원문의 print(int value, int base =10)는 base를 생략하면 10을 사용하며, 기본값을 넘기는 별도 오버로드보다 문법이 간단하다. [U03-100] (3.6.1 인자 전달, 인쇄 p.82-83)

3.6.2 값 반환

반환의 기본 동작도 복사이며 작은 값에는 적합하다. [U03-101] (3.6.2 값 반환, 인쇄 p.84-86)

참조 반환은 함수 호출과 독립적으로 존재하는 **객체**에 호출자가 접근할 때 사용한다. [U03-102] (3.6.2 값 반환, 인쇄 p.84-86)

Vector 요소는 인덱싱 호출 뒤에도 존재하므로 참조로 반환할 수 있다. [U03-103] (3.6.2 값 반환, 인쇄 p.84-86)

반면 지역 **변수**는 함수가 끝나면 사라지므로 그 포인터·참조를 반환하면 안 된다. [U03-104] (3.6.2 값 반환, 인쇄 p.84-86)

큰 M**at**rix 결과도 값을 반환하는 인터페이스를 유지하면서 이동 **생성자**로 효율적으로 전달할 수 있다. [U03-105] (3.6.2 값 반환, 인쇄 p.84-86)

new로 만든 M**at**rix*를 반환하고 사용자가 delete하게 하는 예전 스타일은 직접 메모리 관리와 해제 누락 위험을 키운다. [U03-106] (3.6.2 값 반환, 인쇄 p.84-86)

작업을 완료하지 못하면 예외를 던진다. [U03-107] (3.6.2 값 반환, 인쇄 p.84-86)

auto 반환 **타입** 추론은 함수 **템플릿**·람다에서 유용하지만 구현 변경이 반환 **타입** 변경으로 이어질 수 있어 안정적인 인터페이스를 위해 주의한다. [U03-108] (3.6.2 값 반환, 인쇄 p.84-86)

3.6.3 구조화된 바인딩

함수는 하나의 값만 반환하지만 그 값이 여러 멤버를 가진 **객체**일 수 있다. [U03-109] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

Entry 읽기 예제는 문자열·정수를 하나의 **객체**로 반환한다. [U03-110] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

구조화된 바인딩은 그 **객체**의 멤버에 지역 이름을 부여한다. [U03-111] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

auto [n,v]의 **타입**은 반환 **객체**에서 유추하며, 일반 auto처럼 **const**와 &를 붙일 수 있다. [U03-112] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

여러 멤버를 가진 반환 객체 (원문 발췌)

```
struct Entry {
    string name;
    int value;
};

Entry read_entry(istream& is) // 원시적인 읽기 함수(10.5절에서 더 나은 버전을 볼 수 있음)
{
    string s;
    int i;
    is >> s >> i;
    return {s,i};
}

auto e = read_entry(cin);
cout << "{ " << e.name << " , " << e.value << " }\n";
```

[U03-113] (3.6.3 구조화된 바인딩, 인쇄 p.86)

구조화된 바인딩으로 멤버 사용 (원문 발췌)

```
auto [n,v] = read_entry(is);
cout << "{ " << n << " , " << v << " }\n";
```

[U03-114] (3.6.3 구조화된 바인딩, 인쇄 p.87)

발췌 범위: 앞 페이지의 Entry·read_entry 정의에 이어지는 사용 블록을 발췌했다.

private 데이터가 없는 클래스의 단순한 경우 바인딩 이름의 수는 비정적 데이터 멤버 수와 같아야 하고 이름들은 각 멤버에 대응한다. [U03-115] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

원문은 멤버를 직접 지칭할 때와 **객체** 코드 품질 차이는 없지만 아이디어 표현에 유용하다고 설명한다. [U03-116] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

접근 함수를 가진 complex의 구조화된 바인딩 예도 제시하지만 그 구현 기법은 책의 범위 밖이라고 명시한다. [U03-117] (3.6.3 구조화된 바인딩, 인쇄 p.86-88)

3.7 조언

선언과 정의를 구별하고 인터페이스를 헤더로 표현한다. [U03-118] (3.7 조언, 인쇄 p.88-89)

구현 파일도 해당 헤더를 포함한다. [U03-119] (3.7 조언, 인쇄 p.88-89)

지원되는 경우 모듈을 선호한다. [U03-120] (3.7 조언, 인쇄 p.88-89)

네임스페이스로 논리적 구조를 표현하고 헤더에 using 지시자를 넣지 않는다. [U03-121] (3.7 조언, 인쇄 p.88-89)

설계 초기부터 에러 전략을 정한다. [U03-122] (3.7 조언, 인쇄 p.88-89)

생성자는 불변 조건을 지키거나 예외를 던지며, 에러는 적절한 지점에서 처리한다. [U03-123] (3.7 조언, 인쇄 p.88-89)

모든 함수에서 모든 예외를 잡으려 하지 말고 명시적 try 반복보다 **RAII**를 이용한다. [U03-124] (3.7 조언, 인쇄 p.88-89)

던지지 않는 함수는 **noexcept**로 선언한다. [U03-125] (3.7 조언, 인쇄 p.88-89)

컴파일 시간에 확인할 수 있는 것은 그때 확인한다. [U03-126] (3.7 조언, 인쇄 p.88-89)

작은 것은 값으로, 큰 것은 참조로 전달하고 변경이 필요 없으면 **const** 참조를 선호한다. [U03-127] (3.7 조언, 인쇄 p.88-89)

결과는 함수 반환 값으로 전달하며 반환 **타입** 추론·구조화된 바인딩을 과용하지 않는다. [U03-128] (3.7 조언, 인쇄 p.88-89)

명명된 반환 **타입**은 문서화에 유리할 수 있다. [U03-129] (3.7 조언, 인쇄 p.88-89)

주요 비교표

헤더와 원문 시점의 모듈

기준	#include 헤더	모듈
처리 [U03-130] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	변환 단위마다 헤더 텍스트 처리 [U03-131] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	모듈 한 번 컴파일 [U03-132] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)
순서 [U03-133] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	선언·매크로가 포함 순서에 영향 [U03-134] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	import 순서가 의미에 영향 없음 [U03-135] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)
가시성 [U03-136] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	포함한 선언·매크로가 뒤 코드에 영향 [U03-137] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)	import는 전이되지 않음 [U03-138] (헤더와 원문 시점의 모듈, 인쇄 p.66-68)

함수 경계의 전달 방법

방법	원문의 사용 기준
값 인자 [U03-139] (함수 경계의 전달 방법, 인쇄 p.82-86)	작고 복사 비용이 낮은 **객체** [U03-140] (함수 경계의 전달 방법, 인쇄 p.82-86)
참조 인자 [U03-141] (함수 경계의 전달 방법, 인쇄 p.82-86)	원본 접근 또는 큰 **객체** 복사 회피 [U03-142] (함수 경계의 전달 방법, 인쇄 p.82-86)
const 참조 [U03-143] (함수 경계의 전달 방법, 인쇄 p.82-86)	복사를 피하고 원본을 변경하지 않음 [U03-144] (함수 경계의 전달 방법, 인쇄 p.82-86)
값 반환 [U03-145] (함수 경계의 전달 방법, 인쇄 p.82-86)	결과 전달의 기본 방법, 큰 값은 이동 가능 [U03-146] (함수 경계의 전달 방법, 인쇄 p.82-86)
참조 반환 [U03-147] (함수 경계의 전달 방법, 인쇄 p.82-86)	호출과 독립적으로 존재하는 **객체** 접근 [U03-148] (함수 경계의 전달 방법, 인쇄 p.82-86)

핵심 용어 표

용어	원문 정의·설명
네임스페이스	**네임스페이스**는 일부 선언을 함께 묶고 다른 이름과 충돌하지 않게 하는 메커니즘이다. [U03-001] (3.4 네임스페이스, 인쇄 p.68)
예외 전달	throw는 해당 **타입**의 예외 **핸들**러에 제어권을 넘기며, 필요한 만큼 스코프와 함수를 종료하는 과정에서 **소멸자**를 호출한다. [U03-149] (3.5.1 예외, 인쇄 p.72)
noexcept	절대로 예외를 던지지 않는 함수는 **noexcept**로 선언할 수 있으며, 그래도 예외를 던지면 std::termin**at**e()가 호출된다. [U03-150] (3.5.1 예외, 인쇄 p.73)
사전 조건	**사전 조건**은 함수 작동의 기본적인 가정이다. [U03-151] (3.5.2 불변 조건, 인쇄 p.73)
클래스 불변 조건	클래스 수준에서 보장돼야 할 조건을 **클래스 불변 조건**이라고 한다. [U03-152] (3.5.2 불변 조건, 인쇄 p.74)
어써션·static_assert	어써션은 기대되는 조건을 나타내는 구문이며, st**at**ic_assert는 상수 표현식으로 표현할 수 있는 속성을 확인한다. [U03-002] (3.5.5 정적 어써션, 인쇄 p.80)

장 요약

요약 기준: 3.7 조언

선언과 정의를 구별하고 인터페이스를 표현하기 위해 헤더 파일을 사용한다. [U03-153] (3.7 조언 [1]~[3], 인쇄 p.88)

설계 초기부터 에러 처리 전략을 세우고, **생성자**는 불변 조건을 지키거나 예외를 던진다. [U03-154] (3.7 조언 [14]·[19], 인쇄 p.88-89)

컴파일 시간에 확인할 수 있는 것은 그때 확인하며, 작은 것은 값으로 큰 것은 참조로 전달한다. [U03-155] (3.7 조언 [21]~[22], 인쇄 p.89)

값을 반환할 때는 함수 반환 값을 사용하고 출력 파라미터는 피한다. [U03-156] (3.7 조언 [24], 인쇄 p.89)

원문 확인 메모

모듈·계약의 표준화 상태는 원문 시점의 설명을 유지한다.

제4장. 클래스

(제4장. 클래스, 인쇄 p.91-118)

학습 목표·범위

구체 타입은 **타입** 정의의 일부로 메모리 표현을 포함하며, 내장 **타입**처럼 작동하는 것이 기본 아이디어다. [U04-001] (4.2 구체 타입, 인쇄 p.93)

구현 상속은 기반 클래스의 함수와 데이터 멤버로 파생 클래스의 구현을 단순하게 한다. [U04-002] (4.5.1 계층 구조의 장점, 인쇄 p.112)

핵심 개념·동작 절차·원문 예제

4.1 소개

클래스는 프로그램에서 어떤 개념을 표현하는 사용자 정의 **타입**이다. [U04-003] (4.1 소개, 인쇄 p.91-93)

원문은 유용한 아이디어를 클래스라는 코드로 직접 표현하면 내장 **타입**만으로 구현한 경우보다 이해하고 디버깅하기 쉽다고 설명한다. [U04-004] (4.1 소개, 인쇄 p.91-93)

이 장은 구체 클래스, 추상 클래스, 계층 구조에 속하는 클래스를 다룬다. [U04-005] (4.1 소개, 인쇄 p.91-93)

뒤의 장에서는 생애 주기 연산, **템플릿**, **제네릭 프로그래밍**으로 이어진다. [U04-006] (4.1 소개, 인쇄 p.91-93)

4.2 구체 타입

구체 타입의 기본 생각은 내장 **타입**처럼 작동하도록 하는 것이다. [U04-007] (4.2 구체 타입, 인쇄 p.93-94)

complex는 자신만의 산술 연산을 가진다는 점을 제외하면 int처럼 쓰고, **vector·string**은 개선된 배열처럼 쓴다. [U04-008] (4.2 구체 타입, 인쇄 p.93-94)

타입 정의의 일부로 메모리 표현이 포함되므로, **객체**를 지역 메모리나 다른 **객체** 안에 둘 수 있고, 포인터·참조 없이 직접 접근하고, 완전히 초기화하며, 복사·이동할 수 있다. [U04-009] (4.2 구체 타입, 인쇄 p.93-94)

표현이 private이더라도 각 **객체** 안에 실제로 존재하므로 표현을 바꾸면 다시 컴파일해야 한다. [U04-010] (4.2 구체 타입, 인쇄 p.93-94)

이는 명확성과 효율성을 얻는 대가다. [U04-011] (4.2 구체 타입, 인쇄 p.93-94)

중요한 데이터를 자유 저장 영역에 두고 **객체**에는 접근용 포인터만 두면 유연성을 높일 수 있다. [U04-012] (4.2 구체 타입, 인쇄 p.93-94)

원문은 **vector·string**을 잘 설계된 인터페이스를 가진 **자원 핸들**로 설명한다. [U04-013] (4.2 구체 타입, 인쇄 p.93-94)

4.2.1 산술 타입

원문의 complex는 실수부 re와 허수부 im이라는 두 double을 표현으로 사용한다. [U04-014] (4.2.1 산술 타입, 인쇄 p.94-97)

두 스칼라, 한 스칼라, 인자 없음의 세 **생성자**가 각각 일반 복소수·실수에서의 복소수·{0,0} 기본값을 만든다. [U04-015] (4.2.1 산술 타입, 인쇄 p.94-97)

인자 없이 호출할 수 있는 **생성자**를 기본 **생성자**라고 한다. [U04-016] (4.2.1 산술 타입, 인쇄 p.94-97)

이를 두면 초기화되지 않는 경우를 막을 수 있다. [U04-017] (4.2.1 산술 타입, 인쇄 p.94-97)

구체 산술 타입 complex (원문 발췌)

```
class complex {
    double re, im; // 메모리 표현: double 두 개
public:
    complex(double r, double i) :re{r}, im{i} {} // 두 스칼라를 이용해 complex 생성
    complex(double r) :re{r}, im{0} {} // 스칼라 하나를 이용해 complex 생성
    complex() :re{0}, im{0} {} // 기본 complex: {0,0}

    double real() const { return re; }
    void real(double d) { re=d; }
    double imag() const { return im; }
    void imag(double d) { im=d; }

    complex& operator+=(complex z)
    {
        re+=z.re; // re와 im에 더하기
        im+=z.im;
        return *this; // 결과 반환
    }
    complex& operator-=(complex z)
    {
        re-=z.re;
        im-=z.im;
        return *this;
    }

    complex& operator*=(complex); // 클래스 밖의 어딘가에서 정의
    complex& operator/=(complex); // 클래스 밖의 어딘가에서 정의
};
```

[U04-018] (4.2.1 산술 타입, 인쇄 p.94-95)

클래스 안에 정의한 함수는 기본적으로 인라인이며, 선언 앞의 inline으로 그 의도를 명시할 수도 있다. [U04-019] (4.2.1 산술 타입, 인쇄 p.94-97)

원문은 단순한 연산이 함수 호출 없이 구현될 수 있도록 하여 효율성을 얻는다고 설명한다. [U04-020] (4.2.1 산술 타입, 인쇄 p.94-97)

const 멤버 함수는 호출한 **객체**를 변경하지 않는다. [U04-021] (4.2.1 산술 타입, 인쇄 p.94-97)

const·비**const** **객체** 모두에서 호출할 수 있지만, 비**const** 멤버 함수는 비**const** **객체**에서만 호출한다. [U04-022] (4.2.1 산술 타입, 인쇄 p.94-97)

표현에 직접 접근할 필요가 없는 산술 연산은 클래스 밖에 정의할 수 있다. [U04-023] (4.2.1 산술 타입, 인쇄 p.94-97)

원문의 비멤버 +·-·*·/는 값으로 받은 인자 복사본을 변경해 반환하므로 호출자의 인자를 바꾸지 않는다. [U04-024] (4.2.1 산술 타입, 인쇄 p.94-97)

상등 비교는 실수부·허수부를 모두 비교하고 !=는 !(a==b)로 정의한다. [U04-025] (4.2.1 산술 타입, 인쇄 p.94-97)

연산자는 관례대로 신중하게 사용한다. [U04-026] (4.2.1 산술 타입, 인쇄 p.94-97)

언어가 정한 문법을 바꿀 수 없으므로 단항 /를 새로 만들거나 int의 +를 다른 뜻으로 재정의할 수 없다. [U04-027] (4.2.1 산술 타입, 인쇄 p.94-97)

4.2.2 컨테이너

컨테이너는 요소의 컬렉션을 저장하는 **객체**다. [U04-028] (4.2.2 컨테이너, 인쇄 p.97-99)

앞의 Vector도 **컨테이너**지만 new로 할당한 요소를 해제하지 않았다. [U04-029] (4.2.2 컨테이너, 인쇄 p.97-99)

생성자에서 얻은 **자원**을 해제하는 **소멸자**가 이를 보완한다. [U04-030] (4.2.2 컨테이너, 인쇄 p.97-99)

소멸자의 이름은 클래스 이름 앞에 ~를 붙인다. [U04-031] (4.2.2 컨테이너, 인쇄 p.97-99)

delete는 개별 **객체**를, delete[]는 배열을 해제한다. [U04-032] (4.2.2 컨테이너, 인쇄 p.97-99)

생성자·소멸자로 배열을 관리하는 Vector (원문 발췌)

```
class Vector {
public:
    Vector(int s) :elem{new double[s]}, sz{s} // 생성자: 자원 획득
    {
        for (int i=0; i!=s; ++i) // 요소 초기화
            elem[i]=0;
    }

    ~Vector() { delete[] elem; } // 소멸자 : 자원 해제

    double& operator[](int i);
    int size() const;
private:
    double* elem; // elem은 double요소 sz개를 가리킴
    int sz;
};
```

[U04-033] (4.2.2 컨테이너, 인쇄 p.98)

Vector 사용자는 해제 코드를 직접 쓰지 않아도 된다. [U04-034] (4.2.2 컨테이너, 인쇄 p.97-99)

지역 Vector는 내장 **타입**처럼 명명·스코프·할당·생애 주기 규칙에 따르고 스코프를 벗어나면 소멸된다. [U04-035] (4.2.2 컨테이너, 인쇄 p.97-99)

원문은 예제를 단순화하려고 이 위치에서 에러 처리 코드는 생략했다고 밝힌다. [U04-036] (4.2.2 컨테이너, 인쇄 p.97-99)

따라서 앞 절의 검사가 이 코드에 그대로 들어 있다고 가정하지 않는다. [U04-037] (4.2.2 컨테이너, 인쇄 p.97-99)

생성자에서 **자원**을 획득하고 **소멸자**에서 해제하는 기법은 **RAII**, 즉 **자원** 획득이 곧 초기화다. [U04-038] (4.2.2 컨테이너, 인쇄 p.97-99)

객체가 실제 데이터를 소유하는 **핸들**이 되어 생성·소멸과 **자원** 수명을 결합한다. [U04-039] (4.2.2 컨테이너, 인쇄 p.97-99)

원문은 애플리케이션에 드러난 벌거벗은 new·delete를 피하면 에러와 메모리 누수에 덜 취약한 코드를 만들 수 있다고 설명한다. [U04-040] (4.2.2 컨테이너, 인쇄 p.97-99)

4.2.3 컨테이너 초기화

컨테이너는 요소의 저장을 위해 존재하므로 편리한 입력 방법이 필요하다. [U04-041] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

초깃값 목록 **생성자**는 요소 목록으로 바로 초기화하며, push_back()은 끝에 하나를 추가하고 크기를 1 늘린다. [U04-042] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

요소 개수를 미리 모를 때 push_back()이 유용하다. [U04-043] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

원문의 read()는 입력이 파일 끝이나 형식 에러로 끝날 때까지 읽은 double을 계속 추가한다. [U04-044] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

for를 쓴 이유는 임시 d의 스코프를 루프 안으로 제한하기 위해서다. [U04-045] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

초깃값 목록 생성자 (원문 발췌)

```
Vector::Vector(std::initializer_list<double> lst) // 초깃값 목록 초기화
    :elem{new double[lst.size()]}, sz{static_cast<int>(lst.size())}
{
    copy(lst.begin(),lst.end(),elem); // lst를 elem에 복사(12.6절)
}
```

[U04-046] (4.2.3 컨테이너 초기화, 인쇄 p.101)

std::initializer_**list**는 {} 목록을 받아 초깃값 목록 **생성자**에 전달할 때 사용하는 **표준 라이브러리 타입**이다. [U04-047] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

생성자는 목록 크기만큼 배열을 할당한 후 copy()로 요소를 옮긴다. [U04-048] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

표준 라이브러리의 크기 **타입**은 unsigned 정수여서 이 예제는 st**at**ic_cast<int>로 변환한다. [U04-049] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

st**at**ic_cast는 실제 값을 검사하지 않으므로 올바르게 쓰는 책임은 프로그래머에게 있다. [U04-050] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

원문은 명시적 **타입** 변환을 가능하면 피하고, 검사 없는 캐스트는 시스템의 낮은 수준에 제한하라고 권한다. [U04-051] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

객체를 바이트 시퀀스로 취급하는 reinterpret_cast와 **const**를 제거하는 **const**_cast도 언급한다. [U04-052] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

높은 수준에서는 **타입** 시스템과 잘 설계된 라이브러리로 검사 없는 변환을 피한다. [U04-053] (4.2.3 컨테이너 초기화, 인쇄 p.100-102)

4.3 추상 타입

구체 타입이 정의 안에 표현을 포함하는 것과 달리, **추상 타입**은 사용자에게 상세 구현을 완전히 감춘다. [U04-054] (4.3 추상 타입, 인쇄 p.102-106)

인터페이스와 메모리 표현을 분리하고 포인터·참조로 접근한다. [U04-055] (4.3 추상 타입, 인쇄 p.102-106)

순수 가상 함수를 가진 추상 클래스 자체의 **객체**는 만들 수 없다. [U04-056] (4.3 추상 타입, 인쇄 p.102-106)

순수 가상 인터페이스 Container (원문 발췌)

```
class Container {
public:
    virtual double& operator[](int) = 0; // 순수 가상 함수
    virtual int size() const = 0; // const 멤버 함수(4.2.1절)
    virtual ~Container() {} // 소멸자(4.2.2절)
};
```

[U04-057] (4.3 추상 타입, 인쇄 p.102-103)

구체 구현을 모르는 사용자 함수 (원문 발췌)

```
void use(Container& c)
{
    const int sz = c.size();
    for (int i=0; i!=sz; ++i)
        cout << c[i] << '\n';
}
```

[U04-058] (4.3 추상 타입, 인쇄 p.103)

virtual은 파생 클래스에서 재정의할 수 있는 함수를 표시한다. =0은 순수 가상 함수로서 실제 구현 클래스가 그 연산을 구현해야 함을 나타낸다. [U04-059] (4.3 추상 타입, 인쇄 p.102-106)

Container는 oper**at**or[]와 size()라는 인터페이스만 제공하며, use(Container&)는 구체 구현을 모른 채 그 인터페이스를 호출한다. [U04-060] (4.3 추상 타입, 인쇄 p.102-106)

여러 클래스에 대한 인터페이스를 제공하는 **타입**을 원문은 다형 **타입**이라고 부른다. [U04-061] (4.3 추상 타입, 인쇄 p.102-106)

Vector를 이용한 추상 인터페이스 구현 (원문 발췌)

```
class Vector_container :public Container { // Vector_container가 Container를 구현
public:
    Vector_container(int s) : v(s) { } // 요소들의 벡터
    ~Vector_container() {}

    double& operator[](int i) override { return v[i]; }
    int size() const override { return v.size(); }
private:
    Vector v;
};
```

[U04-062] (4.3 추상 타입, 인쇄 p.104)

:public Container는 Container로부터 파생된 서브**타입**임을 뜻한다. [U04-063] (4.3 추상 타입, 인쇄 p.102-106)

Container는 기반·슈퍼클래스, Vector_container는 파생·서브클래스다. [U04-064] (4.3 추상 타입, 인쇄 p.102-106)

파생 클래스는 기반 클래스 멤버를 상속한다. [U04-065] (4.3 추상 타입, 인쇄 p.102-106)

override는 어떤 가상 함수를 재정의한다는 의도를 명시해 이름이나 **타입**을 잘못 쓴 실수를 컴파일러가 찾게 한다. [U04-066] (4.3 추상 타입, 인쇄 p.102-106)

Container에는 초기화할 데이터가 없어 **생성자**를 따로 두지 않지만 가상 **소멸자**가 있다. [U04-067] (4.3 추상 타입, 인쇄 p.102-106)

인터페이스 포인터로 소멸할 때 구체 구현의 **자원**을 알 수 없으므로 올바른 파생 **소멸자**를 부르기 위해서다. [U04-068] (4.3 추상 타입, 인쇄 p.102-106)

Vector_container 소멸 시 그 멤버 Vector의 **소멸자**도 암묵 호출된다. [U04-069] (4.3 추상 타입, 인쇄 p.102-106)

원문은 **list**<double>을 표현으로 쓰는 List_container도 구현해 같은 use()를 적용한다. [U04-070] (4.3 추상 타입, 인쇄 p.102-106)

list의 인덱싱은 **vector**보다 크게 느려 일반적인 인덱스 **컨테이너** 구현에는 쓰지 않지만 대안 구현을 보이는 예다. [U04-071] (4.3 추상 타입, 인쇄 p.102-106)

구현 변경이나 새 파생 클래스 추가에도 use(Container&)를 다시 컴파일할 필요가 없다는 유연성을 얻으며, 그 대가로 **객체**에는 포인터·참조로 접근한다. [U04-072] (4.3 추상 타입, 인쇄 p.102-106)

4.4 가상 함수

c[i]가 어느 구현을 호출할지는 실행 시간의 실제 **객체**에 달려 있다. [U04-073] (4.4 가상 함수, 인쇄 p.107-108)

따라서 **객체**에는 적절한 함수를 찾을 정보가 필요하다. [U04-074] (4.4 가상 함수, 인쇄 p.107-108)

원문은 일반적인 구현 기법으로 **가상 함수 테이블**(vtbl)을 설명한다. [U04-075] (4.4 가상 함수, 인쇄 p.107-108)

컴파일러는 가상 함수 이름을 함수 포인터 테이블의 인덱스로 변환하고, 가상 함수가 있는 각 클래스는 자기 함수들의 테이블을 갖는다. [U04-076] (4.4 가상 함수, 인쇄 p.107-108)

호출자는 구체 **객체**의 크기나 데이터 모양을 몰라도 테이블 포인터 위치와 함수 인덱스로 접근할 수 있다. [U04-077] (4.4 가상 함수, 인쇄 p.107-108)

원문은 가상 호출을 일반 호출에 가까운 효율로 구현할 수 있다고 설명하고, 추가 저장 공간은 가상 함수가 있는 **객체**당 포인터 하나와 클래스당 테이블 하나라고 제시한다. [U04-078] (4.4 가상 함수, 인쇄 p.107-108)

4.5 클래스 계층 구조

계층 구조는 파생 관계로 클래스 집합을 조직하며 “원은 도형의 일종” 같은 관계를 표현한다. [U04-079] (4.5 클래스 계층 구조, 인쇄 p.108-111)

원문은 Shape 아래 Circle·Triangle, Circle 아래 Smiley를 둔다. [U04-080] (4.5 클래스 계층 구조, 인쇄 p.108-111)

Shape는 중심·이동·그리기·회전과 가상 **소멸자**의 추상 인터페이스를 제공한다. [U04-081] (4.5 클래스 계층 구조, 인쇄 p.108-111)

rotate_all()은 실제 도형 종류를 몰라도 Shape*를 통해 같은 rotate() 호출을 적용한다. [U04-082] (4.5 클래스 계층 구조, 인쇄 p.108-111)

Circle은 중심 Point와 반지름으로 구체 속성을 구현하고, Smiley는 Circle을 기반으로 눈·입을 추가한다. [U04-083] (4.5 클래스 계층 구조, 인쇄 p.108-111)

Smiley::draw()는 Circle::draw()로 얼굴을 그리고 eyes와 mouth의 draw()를 호출한다. [U04-084] (4.5 클래스 계층 구조, 인쇄 p.108-111)

새로운 파생 클래스는 데이터·연산을 추가할 수 있지만 유연성만큼 잘못된 설계의 가능성도 있다. [U04-085] (4.5 클래스 계층 구조, 인쇄 p.108-111)

추상 기반 클래스 포인터로 파생 **객체**를 소멸시키려면 기반 **소멸자**는 가상이어야 한다. [U04-086] (4.5 클래스 계층 구조, 인쇄 p.108-111)

가상 호출이 올바른 파생 **소멸자**를 선택하고, 그 **소멸자**는 멤버·기반 **소멸자**를 암묵 호출한다. [U04-087] (4.5 클래스 계층 구조, 인쇄 p.108-111)

원문의 Smiley **소멸자**는 소유한 눈·입을 해제한 후 기반 소멸로 이어진다. [U04-088] (4.5 클래스 계층 구조, 인쇄 p.108-111)

4.5.1 계층 구조의 장점

인터페이스 상속은 기반 **객체**를 사용하는 곳에서 파생 **객체**를 대신 쓸 수 있게 한다. [U04-089] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

Container·Shape처럼 기반이 파생의 인터페이스가 되며 종종 추상 클래스다. [U04-090] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

구현 상속은 기반의 데이터·함수로 파생 구현을 단순화한다. [U04-091] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

Smiley가 Circle **생성자**와 draw()를 이용하는 것이 예다. [U04-092] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

원문의 read_shape()는 입력으로 도형 종류를 읽고 알맞은 Circle·Triangle·Smiley를 생성해 Shape*로 돌려준다. [U04-093] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

user()는 **vector**<Shape*>에 넣어 종류를 모른 채 그리기·회전을 수행한다. [U04-094] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

이 사용자 코드는 새 도형이 추가되어도 한 번 컴파일해 사용할 수 있다. [U04-095] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

예제에서는 user()만 포인터를 보관하므로 모든 도형을 해제할 책임도 user()에 있다. [U04-096] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

delete와 Shape의 가상 **소멸자**가 최하위 파생 클래스부터 **소멸자**를 호출해 **자원**을 반납한다. [U04-097] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

생성은 기반에서 파생으로, 소멸은 파생에서 기반으로 진행된다. [U04-098] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

원문은 해당 예제가 특히 에러 처리 면에서 단순화되어 있음을 명시한다. [U04-099] (4.5.1 계층 구조의 장점, 인쇄 p.112-114)

4.5.2 계층 구조 탐색

공통 Shape 인터페이스 밖에 있는 Smiley::wink() 같은 기능을 사용하려면 실제 **타입**을 확인할 수 있다. [U04-100] (4.5.2 계층 구조 탐색, 인쇄 p.114-115)

포인터에 대한 dynamic_cast는 기대 **타입** 또는 그 파생 **타입**을 가리키지 않으면 nullptr를 반환하므로 다른 **타입**이 합법적인 대안일 때 사용한다. [U04-101] (4.5.2 계층 구조 탐색, 인쇄 p.114-115)

포인터 dynamic_cast와 대안 처리 (원문 발췌)

```
Shape* ps {read_shape(cin)};

if (Smiley* p = dynamic_cast<Smiley*>(ps)) { // ... ps 가 Smiley를 가리키는가? ...
    // Smiley가 맞음. 적절히 사용.
}
else {
    // ... Smiley가 아님. 다른 작업 시도 ...
}
```

[U04-102] (4.5.2 계층 구조 탐색, 인쇄 p.114)

기대한 **타입**이 아닌 경우를 실패로 취급해야 하면 참조에 dynamic_cast를 사용할 수 있다. [U04-103] (4.5.2 계층 구조 탐색, 인쇄 p.114-115)

이 경우 잘못된 **타입**이면 bad_cast 예외를 던진다. [U04-104] (4.5.2 계층 구조 탐색, 인쇄 p.114-115)

원문은 **타입** 검사를 직접 쓰지 않아도 되는 설계가 더 간단·효율적이라고 권하면서, 기반 인터페이스만 받는 시스템에 **객체**를 보낸 뒤 돌려받을 때처럼 숨겨진 원래 **타입**을 복원해야 할 수도 있다고 설명한다. [U04-105] (4.5.2 계층 구조 탐색, 인쇄 p.114-115)

4.5.3 자원 누수 피하기

원문의 벌거벗은 포인터 예제에는 Smiley가 mouth 삭제를 잊거나, read_shape() 사용자가 반환 포인터 삭제를 잊거나, 포인터 **컨테이너** 소유자가 각 **객체** 삭제를 잊을 위험이 있다. [U04-106] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

new 다음에 예외를 던지거나 조기 return하면 마지막 delete까지 도달하지 못해 누수가 생긴다. [U04-107] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

unique_ptr로 소유권을 표현한 Smiley (원문 발췌)

```
class Smiley : public Circle {
    // ...
private:
    vector<unique_ptr<Shape>> eyes; // 눈 두 개
    unique_ptr<Shape> mouth;
};
```

[U04-108] (4.5.3 자원 누수 피하기, 인쇄 p.116)

삭제 책임이 있다면 표준 **unique_ptr**로 소유권을 표현한다. [U04-109] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

Smiley의 eyes를 **vector<unique_ptr<Shape>>**, mouth를 **unique_ptr<Shape>**로 바꾸면 멤버들의 소멸이 **자원**을 반납해 별도 Smiley **소멸자**를 정의할 필요가 없다. [U04-110] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

원문은 이 방법의 효율이 올바른 벌거벗은 포인터 코드와 동등하다고 설명한다. [U04-111] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

read_shape()도 **unique_ptr<Shape>**를 반환하고 user()는 그 포인터들을 **vector**에 보관한다. [U04-112] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

unique_ptr가 스코프를 벗어날 때 소유 **객체**를 삭제한다. [U04-113] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

이 버전을 사용하려면 draw_all()·rot**at**e_all()도 **vector<unique_ptr<Shape>>&**를 받도록 바꿔야 한다는 원문의 조건을 함께 유지한다. [U04-114] (4.5.3 자원 누수 피하기, 인쇄 p.115-117)

4.6 조언

아이디어를 코드로 직접 표현하고, 간단한 개념에는 구체 클래스를 사용한다. [U04-115] (4.6 조언, 인쇄 p.117-118)

성능이 중요한 경우에도 **구체 타입**을 고려한다. [U04-116] (4.6 조언, 인쇄 p.117-118)

생성자가 초기화를 맡고 상태를 바꾸지 않는 멤버 함수는 **const**로 선언한다. [U04-117] (4.6 조언, 인쇄 p.117-118)

표현에 직접 접근해야 하는 함수는 멤버로, 대칭적 연산은 비멤버로 만든다. [U04-118] (4.6 조언, 인쇄 p.117-118)

자원 획득에는 해제가 짝을 이루게 하고 **RAII 자원 핸들**을 쓴다. [U04-119] (4.6 조언, 인쇄 p.117-118)

컨테이너에는 초깃값 목록 **생성자**를 제공한다. [U04-120] (4.6 조언, 인쇄 p.117-118)

표현과 인터페이스를 완전히 분리하려면 추상 클래스를 사용하고 다형 **객체**는 포인터·참조로 접근한다. [U04-121] (4.6 조언, 인쇄 p.117-118)

계층 관계를 표현할 때 **인터페이스 상속**과 **구현 상속**을 구별한다. [U04-122] (4.6 조언, 인쇄 p.117-118)

가상 함수가 있는 클래스에는 가상 **소멸자**를 두고 override로 의도를 드러낸다. [U04-123] (4.6 조언, 인쇄 p.117-118)

불가피한 계층 탐색에는 dynamic_cast를 사용한다. [U04-124] (4.6 조언, 인쇄 p.117-118)

찾지 못한 **타입**이 대안이면 포인터, 실패이면 참조를 사용한다. [U04-125] (4.6 조언, 인쇄 p.117-118)

소유권은 **unique_ptr·shared_ptr**로 표현해 해제 누락을 피한다. [U04-126] (4.6 조언, 인쇄 p.117-118)

주요 비교표

구체 타입과 추상 타입

기준	구체 타입	추상 타입
표현 [U04-127] (구체 타입과 추상 타입, 인쇄 p.93-106)	**타입** 정의에 존재 [U04-128] (구체 타입과 추상 타입, 인쇄 p.93-106)	사용자에게 구현을 완전히 감춤 [U04-129] (구체 타입과 추상 타입, 인쇄 p.93-106)
사용 [U04-130] (구체 타입과 추상 타입, 인쇄 p.93-106)	직접 **객체**·지역 **변수**·복사·이동 [U04-131] (구체 타입과 추상 타입, 인쇄 p.93-106)	인터페이스의 포인터·참조 [U04-132] (구체 타입과 추상 타입, 인쇄 p.93-106)
변경 영향 [U04-133] (구체 타입과 추상 타입, 인쇄 p.93-106)	표현 변경 시 다시 컴파일 [U04-134] (구체 타입과 추상 타입, 인쇄 p.93-106)	인터페이스가 유지되면 사용자 재컴파일 없이 구현 변경 [U04-135] (구체 타입과 추상 타입, 인쇄 p.93-106)
예 [U04-136] (구체 타입과 추상 타입, 인쇄 p.93-106)	complex, Vector [U04-137] (구체 타입과 추상 타입, 인쇄 p.93-106)	Container, Shape [U04-138] (구체 타입과 추상 타입, 인쇄 p.93-106)

상속의 두 역할

구분	원문의 의미	예
인터페이스 상속 [U04-139] (상속의 두 역할, 인쇄 p.112)	기반 **타입**을 쓰는 곳에 파생 **객체**를 사용 [U04-140] (상속의 두 역할, 인쇄 p.112)	Container, Shape [U04-141] (상속의 두 역할, 인쇄 p.112)
구현 상속 [U04-142] (상속의 두 역할, 인쇄 p.112)	기반의 데이터·함수로 파생 구현 단순화 [U04-143] (상속의 두 역할, 인쇄 p.112)	Smiley의 Circle **생성자**·draw 활용 [U04-144] (상속의 두 역할, 인쇄 p.112)

dynamic_cast의 두 실패 표현

대상	타입 불일치	선택 기준
포인터 [U04-145] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)	nullptr 반환 [U04-146] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)	다른 **타입**이 합법적인 대안 [U04-147] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)
참조 [U04-148] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)	bad_cast 예외 [U04-149] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)	기대 **타입**이 아니면 실패 [U04-150] (dynamic_cast의 두 실패 표현, 인쇄 p.114-115)

핵심 용어 표

용어	원문 정의·설명
구체 타입	**구체 타입**은 **타입** 정의의 일부로 메모리 표현을 포함하며, 내장 **타입**처럼 작동하는 것이 기본 아이디어다. [U04-001] (4.2 구체 타입, 인쇄 p.93)
컨테이너	**컨테이너**는 요소의 컬렉션을 저장하는 **객체**다. [U04-151] (4.2.2 컨테이너, 인쇄 p.97)
소멸자	**소멸자**는 클래스 이름 앞에 ~를 붙인 이름을 가지며, Vector의 **소멸자**는 delete[]로 할당한 배열을 해제한다. [U04-152] (4.2.2 컨테이너, 인쇄 p.98)
추상 타입	**추상 타입**은 사용자에게 상세 구현을 완전히 감추며, 인터페이스와 메모리 표현을 분리한다. [U04-153] (4.3 추상 타입, 인쇄 p.102)
가상 함수 테이블	**가상 함수 테이블**(vtbl)은 일반적인 구현 기법에서 가상 함수 이름을 인덱스로 바꿔 접근하는 함수 포인터 테이블이다. [U04-154] (4.4 가상 함수, 인쇄 p.107)
인터페이스 상속	**인터페이스 상속**은 기반 클래스 **객체**를 사용하는 곳에서 파생 클래스 **객체**를 대신 사용할 수 있게 한다. [U04-155] (4.5.1 계층 구조의 장점, 인쇄 p.112)
구현 상속	**구현 상속**은 기반 클래스의 함수와 데이터 멤버로 파생 클래스의 구현을 단순하게 한다. [U04-002] (4.5.1 계층 구조의 장점, 인쇄 p.112)

장 요약

요약 기준: 4.6 조언

구체 클래스로 간단한 개념을 표현하고, **객체** 초기화는 **생성자**가 맡는다. [U04-156] (4.6 조언 [3]·[5], 인쇄 p.117)

생성자에서 **자원**을 획득하는 클래스라면 그 **자원**을 해제하는 **소멸자**가 필요하다. [U04-157] (4.6 조언 [10], 인쇄 p.118)

인터페이스와 구현을 완전히 분리하려면 추상 클래스를 사용하고, 다형 **객체**에는 포인터와 참조로 접근한다. [U04-158] (4.6 조언 [14]~[15], 인쇄 p.118)

클래스 계층 구조를 설계할 때 **구현 상속**과 **인터페이스 상속**을 구별한다. [U04-159] (4.6 조언 [20], 인쇄 p.118)

제5장. 필수적인 연산

(제5장. 필수적인 연산, 인쇄 p.119-139)

학습 목표·범위

생성자와 **소멸자**, 복사와 이동은 논리적으로 서로 구분되지 않는 관련 연산이다. [U05-001] (5.1.1 필수적인 연산, 인쇄 p.120)

move()는 아무것도 이동시키지 않고 앞으로 이동시킬 인자의 **rvalue 참조**를 반환하는 일종의 **타입** 변환이다. [U05-002] (5.2.2 컨테이너 이동, 인쇄 p.130)

핵심 개념·동작 절차·원문 예제

5.1 소개

초기화·대입·복사·이동은 프로그램의 기초 동작이다. [U05-003] (5.1 소개, 인쇄 p.119)

원문은 이러한 연산에 관례적인 의미가 있으며 이를 임의로 무시해서는 안 된다고 설명한다. [U05-004] (5.1 소개, 인쇄 p.119)

타입을 정의할 때는 **객체** 생성에서 사용·전달·정리까지의 동작이 일관되게 연결되도록 설계한다. [U05-005] (5.1 소개, 인쇄 p.119)

5.1.1 필수적인 연산

생성자·**소멸자**·복사·이동은 서로 관련된 연산이다. [U05-006] (5.1.1 필수적인 연산, 인쇄 p.120-122)

소멸자가 동적 저장 공간이나 락을 해제하는 등 특별한 일을 한다면 복사·이동에도 별도 정의가 필요할 가능성이 크다. [U05-007] (5.1.1 필수적인 연산, 인쇄 p.120-122)

원문은 일반 **생성자**, 기본 **생성자**, 복사·이동 **생성자**, 복사·이동 대입, **소멸자**를 한 집합으로 제시한다. [U05-008] (5.1.1 필수적인 연산, 인쇄 p.120-122)

함께 설계할 생애 주기 연산 (원문 발췌)

```
class X {
public:
    X(Sometype); // "일반적인 생성자": 객체 생성
    X(); // 기본 생성자
    X(const X&); // 복사 생성자
    X(X&&); // 이동 생성자
    X& operator=(const X&); // 복사 대입: 대상을 정리하고 복사
    X& operator=(X&&); // 이동 대입: 대상을 정리하고 이동
    ~X(); // 소멸자: 정리 작업
    // ...
};
```

[U05-009] (5.1.1 필수적인 연산, 인쇄 p.120)

객체는 대입 원본, 초깃값, 함수 인자, 함수 반환 값, 예외로서 복사·이동될 수 있다. [U05-010] (5.1.1 필수적인 연산, 인쇄 p.120-122)

대입에는 대입 연산자를 사용하고 다른 경우에는 **생성자**가 관련되지만, 대상의 저장 공간에 직접 초기화해 **생성자** 호출을 생략할 수 있다. [U05-011] (5.1.1 필수적인 연산, 인쇄 p.120-122)

이 복사 생략은 **객체** 전달의 최적화다. [U05-012] (5.1.1 필수적인 연산, 인쇄 p.120-122)

컴파일러는 일반 **생성자**를 제외한 특수 멤버를 필요할 때 생성하며 **=default**로 기본 구현을 명시할 수 있다. [U05-013] (5.1.1 필수적인 연산, 인쇄 p.120-122)

원문은 일부 기본 함수를 명시하면 나머지 기본 함수가 생성되지 않는다고 설명하므로, 연산 집합 전체를 함께 고려한다. [U05-014] (5.1.1 필수적인 연산, 인쇄 p.120-122)

포인터가 delete할 **자원**을 가리킨다면 단순한 멤버별 복사는 적합하지 않다. [U05-015] (5.1.1 필수적인 연산, 인쇄 p.120-122)

반대로 비소유 포인터라면 그 사실도 독자가 알아야 한다. [U05-016] (5.1.1 필수적인 연산, 인쇄 p.120-122)

기반 클래스 복사의 사용 금지 (원문 발췌)

```
class Shape {
public:
    Shape(const Shape&) =delete; // 복사 연산 없음
    Shape& operator=(const Shape&) =delete;
    // ...
};

void copy(Shape& s1, const Shape& s2)
{
    s1 = s2; // 에러: Shape의 복사 연산이 제거됨
}
```

[U05-017] (5.1.1 필수적인 연산, 인쇄 p.122)

기본 의미가 충분한 클래스는 컴파일러가 멤버의 기본 생성·복사·이동·소멸을 조합하도록 두고, 특별한 의미가 필요하다면 필수 연산을 함께 정의한다. **=delete**는 어떤 연산의 사용을 금지하며 사용하면 컴파일 시간 에러가 난다. [U05-018] (5.1.1 필수적인 연산, 인쇄 p.120-122)

원문은 멤버별 복사가 적합하지 않은 Shape 기반 클래스를 예로 든다. [U05-019] (5.1.1 필수적인 연산, 인쇄 p.120-122)

특수 멤버 외의 함수에도 **=delete**를 사용할 수 있다. [U05-020] (5.1.1 필수적인 연산, 인쇄 p.120-122)

5.1.2 변환

인자 하나를 받는 **생성자**는 그 인자 **타입**에서 클래스 **타입**으로의 변환을 정의한다. [U05-021] (5.1.2 변환, 인쇄 p.122-123)

complex에 double을 주어 {값,0}을 만드는 암묵 변환은 유용할 수 있다. [U05-022] (5.1.2 변환, 인쇄 p.122-123)

반면 int 7을 Vector로 암묵 변환해 요소 7개짜리 **객체**를 만드는 것은 원하지 않는 결과일 수 있다. [U05-023] (5.1.2 변환, 인쇄 p.122-123)

explicit 생성자로 암묵 변환 제한 (원문 발췌)

```
class Vector {
public:
    explicit Vector(int s); // int에서 Vector로 암묵적인 변환 불가
    // ...
};

Vector v1(7); // OK: v1은 7개 요소를 포함
Vector v2 = 7; // 에러: int에서 Vector로 암묵적인 변환 불가
```

[U05-024] (5.1.2 변환, 인쇄 p.123)

explicit은 명시적 변환만 허용하게 한다. [U05-025] (5.1.2 변환, 인쇄 p.122-123)

이때 Vector v1(7)은 가능하지만 Vector v2 = 7은 금지된다. [U05-026] (5.1.2 변환, 인쇄 p.122-123)

원문은 특별한 이유가 없다면 한 인자를 받는 **생성자**를 explicit으로 선언해 Vector 쪽의 관례를 따르라고 권한다. [U05-027] (5.1.2 변환, 인쇄 p.122-123)

5.1.3 멤버 초깃값

데이터 멤버 선언에서 기본 멤버 초깃값을 지정할 수 있다. [U05-028] (5.1.3 멤버 초깃값, 인쇄 p.124)

원문의 complex는 re와 im을 각각 0으로 지정하고, 두 인자 **생성자**는 둘 다 대체하며 한 인자 **생성자**는 re만 지정한다. [U05-029] (5.1.3 멤버 초깃값, 인쇄 p.124)

생성자가 별도 값을 주지 않은 멤버에는 기본값을 사용한다. [U05-030] (5.1.3 멤버 초깃값, 인쇄 p.124)

이를 통해 **생성자** 코드를 줄이고 실수로 멤버를 초기화하지 않는 일을 막는다. [U05-031] (5.1.3 멤버 초깃값, 인쇄 p.124)

5.2 복사와 이동

원문은 사용자 정의 **타입**과 내장 **타입**의 기본 복사 의미를 멤버별 복사로 설명한다. [U05-032] (5.2 복사와 이동, 인쇄 p.124-125)

두 double로 표현되는 complex에서는 초기화와 대입 시 두 성분을 복사하면 독립된 같은 값이 되어 적합하다. [U05-033] (5.2 복사와 이동, 인쇄 p.124-125)

그러나 **자원 핸들**이거나 **추상 타입**이면 이런 의미가 적절하지 않은 경우가 많다. [U05-034] (5.2 복사와 이동, 인쇄 p.124-125)

클래스 설계 시 복사가 무엇을 뜻하는지 반드시 고려한다. [U05-035] (5.2 복사와 이동, 인쇄 p.124-125)

5.2.1 컨테이너 복사

Vector의 elem 포인터와 sz만 복사하면 두 Vector가 같은 요소를 가리키게 된다. [U05-036] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

한 벡터의 요소 변경이 다른 벡터에도 보이고, **자원**을 각 **객체**가 책임진다는 의미와 충돌한다. [U05-037] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

원문의 bad_copy()는 이 문제를 도식으로 보여 준다. [U05-038] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

소멸자가 있는 **자원** 소유 클래스라면 기본 복사가 적절한지 특히 확인해야 한다. [U05-039] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

자기 저장 공간을 갖는 Vector 복사 생성자 (원문 발췌)

```
Vector::Vector(const Vector& a) // 복사 생성자
    :elem{new double[a.sz]}, // 요소를 저장할 공간 할당
    sz{a.sz}
{
    for (int i=0; i!=sz; ++i) // 요소를 복사
        elem[i] = a.elem[i];
}
```

[U05-040] (5.2.1 컨테이너 복사, 인쇄 p.126)

기존 자원을 교체하는 복사 대입 (원문 발췌)

```
Vector& Vector::operator=(const Vector& a) // 복사 대입
{
    double* p = new double[a.sz];
    for (int i=0; i!=a.sz; ++i)
        p[i] = a.elem[i];
    delete[] elem; // 예전 요소 삭제
    elem = p;
    sz = a.sz;
    return *this;
}
```

[U05-041] (5.2.1 컨테이너 복사, 인쇄 p.127)

복사 **생성자**는 원본 크기의 공간을 새로 할당하고 각 요소를 복사해, 두 Vector가 각각 자기 요소의 복사본을 갖게 한다. [U05-042] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

복사 대입은 대상에 이미 있던 **자원**을 정리해야 하므로 새 배열 할당·요소 복사 후 이전 배열을 해제하고 포인터·크기를 교체한다. [U05-043] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

this는 멤버 함수를 호출한 **객체**를 가리키며 *this는 결과로 대상 **객체** 자체를 돌려준다. [U05-044] (5.2.1 컨테이너 복사, 인쇄 p.125-127)

5.2.2 컨테이너 이동

큰 **객체**의 복사 비용은 클 수 있다. [U05-045] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

참조 인자로 입력 복사는 피할 수 있지만 사라질 지역 **객체**를 참조로 반환할 수는 없다. [U05-046] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

원문의 벡터 덧셈은 지역 res를 만들어 반환한다. [U05-047] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

덧셈을 연속하면 더 이상 사용하지 않을 중간 결과를 복사하는 비용이 문제다. [U05-048] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

원하는 것은 원소의 또 다른 복사본이 아니라 계산 결과를 함수 밖으로 가져오는 것이다. [U05-049] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

지역 결과를 값으로 반환하는 벡터 덧셈 (원문 발췌)

```
Vector operator+(const Vector& a, const Vector& b)
{
    if (a.size()!=b.size())
        throw Vector_size_mismatch{};
    Vector res(a.size());

    for (int i=0; i!=a.size(); ++i)
        res[i]=a[i]+b[i];
    return res;
}
```

[U05-050] (5.2.2 컨테이너 이동, 인쇄 p.127-128)

배열 소유권을 가져오는 이동 생성자 (원문 발췌)

```
Vector::Vector(Vector&& a)
    :elem{a.elem}, // 인자로부터 요소를 가져옴
    sz{a.sz}
{
    a.elem = nullptr; // 이제 요소를 포함하지 않음
    a.sz = 0;
}
```

[U05-051] (5.2.2 컨테이너 이동, 인쇄 p.129)

이동 **생성자**는 원본 elem과 sz를 가져오고 원본을 nullptr·0 상태로 만든다. &&는 **rvalue 참조**를 나타내며, 원문은 함수 반환 결과처럼 이동 가능한 값을 표현하는 데 사용한다고 설명한다. [U05-052] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

이동은 원본에서 값을 제거하므로 이동 **생성자**는 **const** 인자를 받지 않는다. [U05-053] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

이동 대입도 비슷한 목적을 갖는다. [U05-054] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

이동 후 원본은 **소멸자**를 문제없이 호출할 수 있어야 하고, 일반적으로 다시 대입할 수도 있어야 한다. [U05-055] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

프로그래머가 앞으로 값을 사용하지 않을 것을 알면 **std::move**로 그 의도를 나타낼 수 있다. [U05-056] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

move() 자체는 아무것도 이동시키지 않고 이동 가능하게 만드는 **rvalue 참조**를 반환하는 일종의 **타입** 변환이다. [U05-057] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

원문은 이동 뒤의 값을 사용하지 않는 것이 좋다고 주석으로 명시한다. [U05-058] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

객체 초기화에서는 복사 생략으로 실제 이동 생성 호출도 줄어들 수 있다. [U05-059] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

반면 기존 **객체**에 대한 대입의 복사·이동은 암묵적으로 없애기 어려우므로 이동 대입은 성능 면에서 중요한 역할을 한다. [U05-060] (5.2.2 컨테이너 이동, 인쇄 p.127-131)

5.3 자원 관리

생성자·복사·이동·소멸을 정의하면 **객체**가 소유하는 **자원**의 생애를 제어할 수 있다. [U05-061] (5.3 자원 관리, 인쇄 p.131-133)

이동은 복사할 수 없거나 복사하기 싫은 **객체**를 낮은 비용으로 다른 스코프로 옮긴다. [U05-062] (5.3 자원 관리, 인쇄 p.131-133)

원문의 예에서 **thread**는 복사할 수 없고, double 100만 개의 Vector는 복사를 피하고 싶은 대상이다. [U05-063] (5.3 자원 관리, 인쇄 p.131-133)

스레드와 큰 Vector의 자원 이전 (원문 발췌)

```
std::vector<thread> my_threads;

Vector init(int n)
{
    thread t {heartbeat}; // heartbeat를 동시적으로(각자 다른 스레드로) 실행
    my_threads.push_back(std::move(t)); // t를 my_threads로 이동(13.2.2절)
    // ... 다른 초기화 수행...

    Vector vec(n);
    for (int i=0; i!=vec.size(); ++i)
        vec[i] = 777;
    return vec; // vec를 init() 밖으로 이동
}

auto v = init(1'000'000); // heartbeat 을 시작하고 v를 초기화
```

[U05-064] (5.3 자원 관리, 인쇄 p.131)

자원 핸들에는 포인터 자체도 감춰지게 할 수 있으며 추가 부담 없이 단순하고 유지 보수하기 쉬운 코드를 얻는다. [U05-065] (5.3 자원 관리, 인쇄 p.131-133)

강력한 **자원** 안전성은 일반적인 의미의 **자원** 모두에 대해 누수를 막는 것이다. [U05-066] (5.3 자원 관리, 인쇄 p.131-133)

메모리뿐 아니라 락·소켓·파일·스레드 **핸들**도 **자원**이다. [U05-067] (5.3 자원 관리, 인쇄 p.131-133)

오래 실행되는 시스템에서는 누수뿐 아니라 필요 이상 오래 보유하는 것도 문제다. [U05-068] (5.3 자원 관리, 인쇄 p.131-133)

원문은 가비지 컬렉션을 전역적 메모리 관리로 설명하고, 더 명확하고 일반적이며 지역적인 관리 방법이 없을 때 최후 수단으로 고려한다는 저자의 입장을 제시한다. [U05-069] (5.3 자원 관리, 인쇄 p.131-133)

먼저 각 **자원**의 소유자가 특정 스코프에 존재하고 종료 시 해제하도록 **RAII**를 체계적으로 사용한다. [U05-070] (5.3 자원 관리, 인쇄 p.131-133)

이동·스마트 포인터는 스코프 사이 **자원** 이전을, 공유 포인터는 공유 소유권을 표현한다. [U05-071] (5.3 자원 관리, 인쇄 p.131-133)

표준 라이브러리는 **string·vector·map**의 메모리, ifstream·ofstream의 파일, **thread**의 스레드, lock_guard·unique_lock의 락, **unique_ptr·shared_ptr**의 **객체**에 **RAII**를 적용한다. [U05-072] (5.3 자원 관리, 인쇄 p.131-133)

그 결과 보통의 코드에서 관리가 두드러지지 않으면서 소유 시간을 줄이는 암묵적 **자원** 관리가 가능하다. [U05-073] (5.3 자원 관리, 인쇄 p.131-133)

5.4 관례적인 연산

많은 프로그래머와 **표준 라이브러리**는 특정 연산의 관례적인 뜻을 가정한다. [U05-074] (5.4 관례적인 연산, 인쇄 p.133)

새로운 **타입**에서도 비교, 크기·범위 접근, 입출력, 리터럴, swap, hash의 의미가 관례를 따르도록 설계한다. [U05-075] (5.4 관례적인 연산, 인쇄 p.133)

5.4.1 비교

상등 비교 ==·!=는 복사 의미와 밀접하다. [U05-076] (5.4.1 비교, 인쇄 p.133-134)

복사한 뒤 원본과 복사본은 같아야 한다. ==를 정의하면 !=도 정의하고 a!=b가 !(a==b)를 뜻하게 한다. [U05-077] (5.4.1 비교, 인쇄 p.133-134)

순서 비교 연산도 서로 일관된 의미를 가져야 한다. [U05-078] (5.4.1 비교, 인쇄 p.133-134)

원문은 두 피연산자를 동등하게 취급하도록 이항 연산자를 해당 클래스와 같은 **네임스페이스**의 독립 함수로 정의하는 방식을 권한다. [U05-079] (5.4.1 비교, 인쇄 p.133-134)

[원문 확인 필요] 인쇄 134쪽 비교 규칙 목록은 첫 항목과 셋째 항목에 모두 !(b<a)를 인쇄한다. [U05-080] (5.4.1 비교, 인쇄 p.133-134)

이 요약에서는 외부 지식으로 그 식을 수정하거나 새로운 식으로 대체하지 않고, 비교 연산 간의 일관성이라는 원문 설명을 보존했다. [U05-081] (5.4.1 비교, 인쇄 p.133-134)

5.4.2 컨테이너 연산

특별한 이유가 없다면 **표준 라이브러리 컨테이너** 설계를 따른다. [U05-082] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

컨테이너를 필수 연산을 갖춘 **자원 핸들**로 구현하고 size()로 요소 개수를 알게 한다. [U05-083] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

알고리즘은 인덱스 대신 **반복자** 쌍으로 정의된 시퀀스를 사용한다. [U05-084] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

begin()은 첫 요소, end()는 마지막 요소의 다음을 가리킨다. [U05-085] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

반복자는 포인터처럼 ++로 다음으로 이동하고 *로 값을 읽고 쓴다. [U05-086] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

sort(v.begin(),v.end())처럼 **반복자** 쌍을 **표준 알고리즘**에 전달한다. [U05-087] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

구간 for도 같은 begin()·end() 범위를 사용하며 명시적 **반복자** 루프와 거의 같은 방식이다. [U05-088] (5.4.2 컨테이너 연산, 인쇄 p.134-135)

5.4.3 입력과 출력 연산

정수 쌍에 대한 <<·>>는 각각 왼쪽·오른쪽 시프트다. [U05-089] (5.4.3 입력과 출력 연산, 인쇄 p.135-136)

iostream에 대해서는 출력·입력 연산으로 사용한다. [U05-090] (5.4.3 입력과 출력 연산, 인쇄 p.135-136)

원문은 자세한 입출력 설명을 10장으로 연결하며, 이 절에서는 같은 기호의 관례적인 사용 맥락을 구별한다. [U05-091] (5.4.3 입력과 출력 연산, 인쇄 p.135-136)

5.4.4 사용자 정의 리터럴

사용자 정의 **타입**에도 내장 **타입**처럼 편리한 리터럴을 제공할 수 있다. [U05-092] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

적절한 접미사의 의미를 리터럴 연산자로 정의하면 접미사 앞 리터럴을 필요한 **타입**으로 변환한다. [U05-093] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

원문은 문자열 s, 시간 단위 s, 허수 i의 예를 들고, 사용하려면 적절한 **표준 헤더**와 **네임스페이스**를 가져와야 한다고 설명한다. [U05-094] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

허수 접미사를 정의하는 리터럴 연산자 (원문 발췌)

```
constexpr complex<double> operator""i(long double arg) // 허수
{
    return {0,arg};
}

complex<double> z = 2.7182818+6.283185i;
```

[U05-095] (5.4.4 사용자 정의 리터럴, 인쇄 p.137)

operator""는 리터럴 연산자를, 그 뒤 i는 접미사를 나타낸다. [U05-096] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

long double 인자는 이 접미사가 부동소수점 리터럴에 적용됨을, complex<double> 반환 **타입**은 만들어질 리터럴의 **타입**을 나타낸다. [U05-097] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

원문의 chrono 접미사는 h·min·s·ms·us·ns이고 complex에는 i·il·if가 열거된다. [U05-098] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

원문 표의 **string_view** 행은 **string**_literals 및 s로 인쇄되어 있다. [U05-099] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

[원문 확인 필요] 이를 임의의 다른 이름으로 고치지 않았다. [U05-100] (5.4.4 사용자 정의 리터럴, 인쇄 p.136-137)

5.4.5 swap()

sort()를 비롯한 많은 알고리즘은 두 **객체** 값을 교환하는 swap()을 사용하며 빠르고 예외를 던지지 않는다고 가정한다. [U05-101] (5.4.5 swap(), 인쇄 p.137-138)

std::swap(a,b)는 세 번의 이동으로 정의된다. [U05-102] (5.4.5 swap(), 인쇄 p.137-138)

복사 비용이 큰 **타입**을 정렬·교환해야 한다면 이동 연산 또는 swap(), 혹은 둘 다 제공하는 것이 좋다. [U05-103] (5.4.5 swap(), 인쇄 p.137-138)

표준 **컨테이너**와 **string**은 빠른 이동을 제공한다. [U05-104] (5.4.5 swap(), 인쇄 p.137-138)

5.4.6 hash<>

unordered_map<K,V>는 키 **타입** K와 값 **타입** V를 갖는 해시 테이블이다. [U05-105] (5.4.6 hash<>, 인쇄 p.138)

사용자 **타입** X를 키로 사용하려면 hash<X>를 정의해야 한다. [U05-106] (5.4.6 hash<>, 인쇄 p.138)

표준 라이브러리는 std::**string**을 포함한 일반적인 **타입**의 해시를 제공한다. [U05-107] (5.4.6 hash<>, 인쇄 p.138)

이 절은 해시 구현을 새로 전개하지 않고 11.5절로 연결한다. [U05-108] (5.4.6 hash<>, 인쇄 p.138)

5.5 조언

생성자·대입·**소멸자**를 관련 집합으로 설계하고 필수 연산은 모두 정의하거나 모두 기본에 맡긴다. [U05-109] (5.5 조언, 인쇄 p.138-139)

기본 구현이 충분하면 직접 다시 쓰지 않는다. [U05-110] (5.5 조언, 인쇄 p.138-139)

포인터를 가진 클래스나 **소멸자**가 있는 클래스는 사용자 정의 또는 제거된 복사·이동이 필요할 수 있다. [U05-111] (5.5 조언, 인쇄 p.138-139)

부적절한 기본 복사는 재정의하거나 막는다. [U05-112] (5.5 조언, 인쇄 p.138-139)

한 인자 **생성자**는 기본적으로 explicit으로 두고 합리적인 기본 멤버 값은 선언에 지정한다. [U05-113] (5.5 조언, 인쇄 p.138-139)

컨테이너는 값을 반환하되 이동으로 효율을 확보하고 큰 피연산자는 **const** 참조로 받는다. [U05-114] (5.5 조언, 인쇄 p.138-139)

어떤 **자원**도 누수되지 않도록 강력한 **자원** 안전성을 제공한다. [U05-115] (5.5 조언, 인쇄 p.138-139)

연산자는 관례에 맞게 오버로드하며 표준 **컨테이너** 설계를 따른다. [U05-116] (5.5 조언, 인쇄 p.138-139)

주요 비교표

복사와 이동

기준	Vector 복사	Vector 이동
저장 공간 [U05-117] (복사와 이동, 인쇄 p.125-131)	새 배열을 할당하고 요소 복사 [U05-118] (복사와 이동, 인쇄 p.125-131)	기존 배열 포인터를 가져옴 [U05-119] (복사와 이동, 인쇄 p.125-131)
원본 [U05-120] (복사와 이동, 인쇄 p.125-131)	자기 요소를 계속 가짐 [U05-121] (복사와 이동, 인쇄 p.125-131)	예제에서는 nullptr·크기 0 [U05-122] (복사와 이동, 인쇄 p.125-131)
입력 [U05-123] (복사와 이동, 인쇄 p.125-131)	**const** Vector& [U05-124] (복사와 이동, 인쇄 p.125-131)	Vector&& [U05-125] (복사와 이동, 인쇄 p.125-131)
결과 [U05-126] (복사와 이동, 인쇄 p.125-131)	독립된 같은 값 [U05-127] (복사와 이동, 인쇄 p.125-131)	**자원**의 소유권 이전 [U05-128] (복사와 이동, 인쇄 p.125-131)

생성자 제어 표기

표기	원문의 목적
=default [U05-129] (생성자 제어 표기, 인쇄 p.121-124)	컴파일러 기본 구현을 명시 [U05-130] (생성자 제어 표기, 인쇄 p.121-124)
=delete [U05-131] (생성자 제어 표기, 인쇄 p.121-124)	함수 사용을 컴파일 시간에 금지 [U05-132] (생성자 제어 표기, 인쇄 p.121-124)
explicit [U05-133] (생성자 제어 표기, 인쇄 p.121-124)	한 인자 **생성자**의 암묵 변환 제한 [U05-134] (생성자 제어 표기, 인쇄 p.121-124)
멤버 기본 초깃값 [U05-135] (생성자 제어 표기, 인쇄 p.121-124)	**생성자**가 값을 지정하지 않은 멤버 초기화 [U05-136] (생성자 제어 표기, 인쇄 p.121-124)

핵심 용어 표

용어	원문 정의·설명
관련된 필수 연산	**생성자**와 **소멸자**, 복사와 이동은 논리적으로 서로 구분되지 않는 관련 연산이다. [U05-001] (5.1.1 필수적인 연산, 인쇄 p.120)
=default	일반적인 **생성자**를 제외한 특수 멤버 함수의 기본 구현을 명시적으로 만들고 싶을 때 **=default**를 사용한다. [U05-137] (5.1.1 필수적인 연산, 인쇄 p.121)
=delete	**=delete**로 제거한 함수를 사용하면 컴파일 시간 에러가 발생한다. [U05-138] (5.1.1 필수적인 연산, 인쇄 p.122)
Vector 복사 생성자	원문의 **Vector 복사 생성자**는 새 저장 공간을 할당하고 요소를 복사하여 각 Vector가 자기 요소의 복사본을 갖게 한다. [U05-139] (5.2.1 컨테이너 복사, 인쇄 p.126)
복사 생성자·복사 대입	클래스 **객체**의 복사는 복사 **생성자**와 복사 대입이라는 두 멤버로 정의할 수 있다. [U05-140] (5.2.1 컨테이너 복사, 인쇄 p.126)
Vector 이동 생성자	원문의 **Vector 이동 생성자**는 원본의 elem과 sz를 가져오고 원본을 nullptr와 크기 0으로 만든다. [U05-141] (5.2.2 컨테이너 이동, 인쇄 p.129)
rvalue 참조	&&는 **rvalue 참조**를 나타내며, 누구도 다시는 대입할 수 없는 것의 참조로 값을 이동시킬 수 있다. [U05-142] (5.2.2 컨테이너 이동, 인쇄 p.129)
std::move	move()는 아무것도 이동시키지 않고 앞으로 이동시킬 인자의 **rvalue 참조**를 반환하는 일종의 **타입** 변환이다. [U05-002] (5.2.2 컨테이너 이동, 인쇄 p.130)

장 요약

요약 기준: 5.5 조언

생성자와 대입 연산자, **소멸자**를 상호 연관된 연산자의 집합으로 설계한다. [U05-143] (5.5 조언 [2], 인쇄 p.138)

기본 복사가 적합하지 않은 **타입**에는 복사를 재정의하거나 막는다. [U05-144] (5.5 조언 [9], 인쇄 p.139)

자원으로 취급할 수 있는 어떤 것도 누수되지 않게 강력한 **자원** 안전성을 제공한다. [U05-145] (5.5 조언 [12], 인쇄 p.139)

연산자를 오버로드할 때 관례적인 사용법을 흉내 내고 **표준 라이브러리 컨테이너**의 설계를 따른다. [U05-146] (5.5 조언 [14]~[15], 인쇄 p.139)

제6장. 템플릿

(제6장. 템플릿, 인쇄 p.141-162)

학습 목표·범위

템플릿은 **타입**이나 값의 집합을 파라미터화한 클래스나 함수다. [U06-001] (6.1 소개, 인쇄 p.141)

if constexpr에서는 선택된 경우만 인스턴스화되므로 지역적인 최적화가 가능하다. [U06-002] (6.4.3 컴파일 시간 if, 인쇄 p.160)

핵심 개념·동작 절차·원문 예제

6.1 소개

템플릿은 **타입**이나 값의 집합을 파라미터화한 클래스 또는 함수다. [U06-003] (6.1 소개, 인쇄 p.141)

특정 **타입**에 종속되지 않는 벡터와 같은 일반적인 개념을 표현하고, 인자를 지정하여 특정 **타입**이나 함수를 만들어 낸다. [U06-004] (6.1 소개, 인쇄 p.141)

요소를 저장한다는 벡터의 개념과 요소가 double이라는 사실을 분리하는 것이 출발점이다. [U06-005] (6.1 소개, 인쇄 p.141)

6.2 파라미터화된 타입

template<typename T>는 앞으로 선언할 클래스의 **타입** 파라미터를 T로 지정한다. [U06-006] (6.2 파라미터화된 타입, 인쇄 p.142-145)

이는 모든 **타입** T에 대해라는 뜻이다. [U06-007] (6.2 파라미터화된 타입, 인쇄 p.142-145)

타입 파라미터 자리에 **class**를 쓰는 오래된 표기도 같은 의미다. [U06-008] (6.2 파라미터화된 타입, 인쇄 p.142-145)

클래스의 멤버를 밖에서 정의할 때도 template 접두사와 Vector<T>를 이용한다. [U06-009] (6.2 파라미터화된 타입, 인쇄 p.142-145)

타입 파라미터로 일반화한 Vector (1/2) (원문 발췌)

```
template<typename T>
class Vector {
private:
    T* elem;    // elem은 T 타입 요소 sz개를 포함하는 배열을 가리킴
    int sz;
public:
    explicit Vector(int s);    // 생성자: 불변 조건을 조성하고, 자원을 획득
    ~Vector() { delete[] elem; } // 소멸자: 자원 해제

    // ... 복사와 이동 연산 ...

    T& operator[](int i);              // const가 아닌 Vector를 위한 연산자
    const T& operator[](int i) const;  // const Vector를 위한 연산자(4.2.1절)
    int size() const { return sz; }
};

template<typename T>
Vector<T>::Vector(int s)
{
    if (s<0)
        throw Negative_size{};
    elem = new T[s];
    sz = s;
}

template<typename T>
const T& Vector<T>::operator[](int i) const
{
    if (i<0 || size()<=i)
```

[U06-010] (6.2 파라미터화된 타입, 인쇄 p.142-143)

타입 파라미터로 일반화한 Vector (2/2) (원문 발췌)

```
        throw out_of_range{"Vector::operator[]"};
    return elem[i];
}
```

[U06-011] (6.2 파라미터화된 타입, 인쇄 p.142-143)

Vector<char>, Vector<**string**>, Vector<**list**<int>>는 서로 다른 요소 **타입**의 벡터다. [U06-012] (6.2 파라미터화된 타입, 인쇄 p.143-144)

중첩된 **템플릿** 인자 끝의 >>는 입력 연산자가 아니다. [U06-013] (6.2 파라미터화된 타입, 인쇄 p.143-144)

사용자 정의 Vector를 구간 for로 순회하려면 첫 요소를 나타내는 begin()과 마지막 요소 다음을 나타내는 end()를 알맞게 제공해야 한다. [U06-014] (6.2 파라미터화된 타입, 인쇄 p.143-144)

템플릿은 컴파일 시간 메커니즘이므로 같은 일을 손으로 작성한 코드에 비해 실행 시간의 추가 부담이 없다. [U06-015] (6.2 파라미터화된 타입, 인쇄 p.144-145)

템플릿과 **템플릿** 인자의 집합을 인스턴스화 또는 특수화라고 부른다. [U06-016] (6.2 파라미터화된 타입, 인쇄 p.144-145)

사용하는 각 인스턴스의 코드는 컴파일 후반의 인스턴스화 시간에 생성되며 **타입** 검사를 받는다. [U06-017] (6.2 파라미터화된 타입, 인쇄 p.144-145)

따라서 **타입** 안전성은 보장되지만 검사 시점이 늦다. [U06-018] (6.2 파라미터화된 타입, 인쇄 p.144-145)

6.2.1 제한된 템플릿 인자(C++20)

템플릿에 의미가 있으려면 인자가 특정 성질을 만족해야 한다. [U06-019] (6.2.1 제한된 템플릿 인자 (C++20), 인쇄 p.145)

Vector가 복사 연산을 제공하려면 요소도 복사할 수 있어야 한다. [U06-020] (6.2.1 제한된 템플릿 인자 (C++20), 인쇄 p.145)

Element와 같은 **컨셉**은 이 요구 사항을 확인하는 **술어**다. [U06-021] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)

컨셉을 지정한 인자는 제한된 인자, 이러한 인자를 사용하는 **템플릿**은 제한된 **템플릿**이다. [U06-022] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)

C++20 이전 코드에서는 필요한 조건을 문서화한다고 설명한다. [U06-023] (6.2.1 제한된 템플릿 인자 (C++20), 인쇄 p.145)

[원문 확인 필요] 원문 예는 Vector<int>를 OK, Vector<**thread**>를 표준 스레드 복사 불가에 의한 에러로 표시한다. [U06-024] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)

다만 int 예의 주석은 “int 는 복사할 수 없음”으로 인쇄되어 있어 본문의 복사 가능 조건 및 OK 표시와 함께 원문 확인이 필요하다. [U06-025] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)

이 주석을 반대로 고쳐 옮기지 않았다. [U06-026] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)

6.2.2 값 템플릿 인자

템플릿은 **타입**뿐 아니라 값도 인자로 받는다. [U06-027] (6.2.2 값 템플릿 인자, 인쇄 p.146)

Buffer의 T는 요소 **타입**, N은 크기를 나타낸다. [U06-028] (6.2.2 값 템플릿 인자, 인쇄 p.146)

이 방식으로 자유 저장 영역을 사용하지 않는 전역 버퍼와 스택의 지역 버퍼를 만들 수 있다. [U06-029] (6.2.2 값 템플릿 인자, 인쇄 p.146)

값 템플릿 인자는 반드시 상수 표현식이어야 한다. [U06-030] (6.2.2 값 템플릿 인자, 인쇄 p.146)

값 템플릿 인자를 받는 Buffer (원문 발췌)

```
template<typename T, int N>
struct Buffer {
    using value_type = T;
    constexpr int size() { return N; }
    T[N];
    // ...
};
```

[U06-031] (6.2.2 값 템플릿 인자, 인쇄 p.146)

6.2.3 템플릿 인자 추론

make_**pair**()는 전달받은 함수 인자로 반환할 **pair**의 **템플릿** 인자를 추론한다. [U06-032] (6.2.3 템플릿 인자 추론, 인쇄 p.146-149)

C++17에서는 **생성자** 인자를 통해 클래스 **템플릿** 인자도 추론할 수 있다. [U06-033] (6.2.3 템플릿 인자 추론, 인쇄 p.146-149)

원문 Vector에서는 초깃값 목록의 요소 **타입**이나 복사할 Vector의 **타입**에서 요소 **타입**을 알아내지만, 크기만 지정하는 생성에서는 요소 **타입**을 명시해야 한다. [U06-034] (6.2.3 템플릿 인자 추론, 인쇄 p.146-149)

추론은 중복을 줄이지만 항상 의도한 **타입**을 만들지는 않는다. [U06-035] (6.2.3 템플릿 인자 추론, 인쇄 p.148)

C 스타일 문자열 리터럴을 요소로 전달하면 **string**이 아닌 **const** char*가 추론될 수 있다. [U06-036] (6.2.3 템플릿 인자 추론, 인쇄 p.148)

string 리터럴을 원하면 접미사 s를 사용한다. [U06-037] (6.2.3 템플릿 인자 추론, 인쇄 p.148)

초깃값 목록의 요소 **타입**이 서로 일치하지 않으면 유일한 요소 **타입**을 추론하지 못한다. [U06-038] (6.2.3 템플릿 인자 추론, 인쇄 p.148)

생성자 인자 한 쌍이 **반복자**라는 사실과 **반복자**의 값 **타입**을 컴파일러가 추론하지 못하는 경우에는 추론 가이드를 지정할 수 있다. [U06-039] (6.2.3 템플릿 인자 추론, 인쇄 p.148-149)

원문 Vector2는 **반복자**의 value_type에서 요소 **타입**을 알아내게 한다. [U06-040] (6.2.3 템플릿 인자 추론, 인쇄 p.148-149)

다만 추론 가이드의 효과에는 미묘한 부분이 있어, 가능하다면 가이드가 필요하지 않게 클래스 **템플릿**을 설계하라고 조언한다. [U06-041] (6.2.3 템플릿 인자 추론, 인쇄 p.148-149)

6.3 파라미터화된 연산

타입이나 값에 따라 달라지는 연산은 함수 **템플릿**, **함수 객체**, **람다 표현식**으로 나타낼 수 있다. [U06-042] (6.3 파라미터화된 연산, 인쇄 p.149)

함수 객체는 데이터도 보유하면서 함수처럼 호출되는 **객체**이고, 람다는 이러한 **함수 객체**를 간결하게 표기하는 방법이다. [U06-043] (6.3 파라미터화된 연산, 인쇄 p.149)

표준 라이브러리는 **타입**과 알고리즘 모두를 파라미터화하는 데 이 수단들을 사용한다. [U06-044] (6.3 파라미터화된 연산, 인쇄 p.149)

6.3.1 함수 템플릿

sum()의 시퀀스 **타입**과 누산기 **타입**은 서로 독립적이다. [U06-045] (6.3.1 함수 템플릿, 인쇄 p.150)

호출자가 초기 누산기 값을 0으로 주는지 0.0으로 주는지에 따라 누산기의 **타입**이 달라지므로, 같은 int 벡터를 int 또는 double 덧셈으로 처리할 수 있다. [U06-046] (6.3.1 함수 템플릿, 인쇄 p.150)

원문은 double 누산기로 int의 최댓값보다 큰 합을 처리할 수 있다고 설명한다. [U06-047] (6.3.1 함수 템플릿, 인쇄 p.150)

이 sum()은 표준 accumul**at**e()의 단순화다. [U06-048] (6.3.1 함수 템플릿, 인쇄 p.150)

시퀀스와 누산기 타입을 분리한 sum (원문 발췌)

```
template<typename Sequence, typename Value>
Value sum(const Sequence& s, Value v)
{
    for (auto x : s)
        v+=x;
    return v;
}
```

[U06-049] (6.3.1 함수 템플릿, 인쇄 p.150)

함수 **템플릿**은 클래스의 멤버가 될 수 있지만 virtual 멤버는 될 수 없다. [U06-050] (6.3.1 함수 템플릿, 인쇄 p.150)

컴파일러가 프로그램에 존재하는 모든 인스턴스화를 알 수 없어 **가상 함수 테이블**을 만들 수 없기 때문이다. [U06-051] (6.3.1 함수 템플릿, 인쇄 p.150)

6.3.2 함수 객체

oper**at**or()를 정의한 **객체**는 함수처럼 호출할 수 있다. [U06-052] (6.3.2 함수 객체, 인쇄 p.151-152)

Less_than은 비교 기준을 **객체** 안에 저장하고 그 값보다 작은지를 판단한다. [U06-053] (6.3.2 함수 객체, 인쇄 p.151-152)

술어는 호출할 수 있고 결과로 true 또는 false를 반환하는 무엇이다. [U06-054] (6.3.2 함수 객체, 인쇄 p.151-152)

count()는 **컨테이너**의 각 요소에 **술어**를 적용해 참인 요소의 개수를 센다. [U06-055] (6.3.2 함수 객체, 인쇄 p.151-152)

데이터를 보유하는 함수 객체 (원문 발췌)

```
template<typename T>
class Less_than {
    const T val;    // 비교 대상
public:
    Less_than(const T& v) :val{v} { }
    bool operator()(const T& x) const { return x<val; }    // 연산자 호출
};
```

[U06-056] (6.3.2 함수 객체, 인쇄 p.151)

함수 객체는 서로 다른 비교 기준마다 별도 함수를 정의하거나 전역 **변수**에 기준을 저장하지 않아도 된다. [U06-057] (6.3.2 함수 객체, 인쇄 p.152)

간단한 **함수 객체**는 인라인으로 처리하기 쉬워 간접 함수 호출보다 효율적이라고 설명한다. [U06-058] (6.3.2 함수 객체, 인쇄 p.152)

일반 알고리즘에서 핵심 동작의 의미를 지정하는 이런 **객체**를 정책 **객체**라고도 부른다. [U06-059] (6.3.2 함수 객체, 인쇄 p.152)

6.3.3 람다 표현식

람다는 사용되는 위치에서 **함수 객체**를 암묵적으로 만든다. [U06-060] (6.3.3 람다 표현식, 인쇄 p.153)

캡처 리스트 [&]는 몸체에서 사용하는 지역 이름을 참조로 접근하고, [&x]는 x만 참조로 캡처한다. [U06-061] (6.3.3 람다 표현식, 인쇄 p.153)

원문의 복사 캡처 표기 설명에는 [=x]가 인쇄되어 있어 [원문 확인 필요]로 남긴다. [U06-062] (6.3.3 람다 표현식, 인쇄 p.153)

캡처할 지역 이름이 없을 때 빈 []를 사용할 수 있다. [U06-063] (6.3.3 람다 표현식, 인쇄 p.153)

같은 위치에서 정의하는 람다 (원문 발췌)

```
void f(const Vector<int>& vec, const list<string>& lst, int x, const string& s)
{
    cout << x
        << "보다 작은 값의 개수: " << count(vec, [&](int a){ return a<x; })
        << '\n';
    cout << s
        << "보다 작은 값의 개수: " << count(lst, [&](const string& a){ return a<s; })
        << '\n';
}
```

[U06-064] (6.3.3 람다 표현식, 인쇄 p.153)

람다는 간단한 동작을 한 곳에서 쓸 때 편리하다. [U06-065] (6.3.3 람다 표현식, 인쇄 p.153-154)

단순하지 않은 동작은 이름을 부여하여 목적을 명확히 하고 여러 곳에서 사용할 수 있게 한다. [U06-066] (6.3.3 람다 표현식, 인쇄 p.153-154)

for_all()과 람다를 결합하면 **컨테이너**를 순회하는 작업과 각 요소에서 수행할 작업을 분리할 수 있다. [U06-067] (6.3.3 람다 표현식, 인쇄 p.153-154)

원문은 **unique_ptr**<Shape>를 참조로 받으므로 호출이 Shape의 수명에 영향을 주지 않는다고 설명한다. [U06-068] (6.3.3 람다 표현식, 인쇄 p.153-154)

람다의 파라미터를 auto로 선언하면 다양한 **타입**을 받는 제네릭 람다가 된다. [U06-069] (6.3.3 람다 표현식, 인쇄 p.154-155)

원문 rot**ate**_and_draw()는 포인터 또는 **unique_ptr**로 저장한 Shape를 같은 방식으로 회전하고 그린다. [U06-070] (6.3.3 람다 표현식, 인쇄 p.154-155)

일반 함수 인자에 auto를 사용할 수 없다는 원문의 당시 설명은 현재 언어 지식으로 보충하거나 수정하지 않는다. [U06-071] (6.3.3 람다 표현식, 인쇄 p.154-155)

람다는 복잡한 초기화에도 쓰인다. [U06-072] (6.3.3 람다 표현식, 인쇄 p.155-157)

먼저 벡터를 선언하고 switch의 각 가지에서 대입하면, 의도한 값 전에 사용될 위험이 있고 초기화 코드가 다른 코드와 섞이거나 경우 하나를 놓치기 쉽다. [U06-073] (6.3.3 람다 표현식, 인쇄 p.155-157)

값을 반환하는 람다를 초깃값으로 사용하면 이 선택을 초기화 위치에 모을 수 있다. [U06-074] (6.3.3 람다 표현식, 인쇄 p.155-157)

원문은 개선 예에도 case 하나가 빠졌지만 이를 더 쉽게 알아볼 수 있다고 지적한다. [U06-075] (6.3.3 람다 표현식, 인쇄 p.155-157)

6.4 템플릿 메커니즘

좋은 **템플릿**을 만드는 도구에는 **타입**에 의존하는 값, **타입**·**템플릿**의 **별칭**, **if constexpr**에 의한 컴파일 시간 선택, **requires 표현식**에 의한 **타입**·표현식 속성 확인이 있다. [U06-076] (6.4 템플릿 메커니즘, 인쇄 p.157)

constexpr 함수와 st**at**ic_assert도 함께 사용한다. [U06-077] (6.4 템플릿 메커니즘, 인쇄 p.157)

이 기능들은 일반적인 추상화를 구성하는 기본 수단이다. [U06-078] (6.4 템플릿 메커니즘, 인쇄 p.157)

6.4.1 가변 템플릿

이 소절은 **타입** T에 따라 달라지는 상수나 **변수**를 정의하는 예를 다룬다. [U06-079] (6.4.1 가변 템플릿, 인쇄 p.157-158)

원문은 점성 계수 viscosity<T>, 3차원 벡터 external_accele**at**ion<T>와 대입 가능성을 나타내는 Assignable<T,T2>를 제시한다. [U06-080] (6.4.1 가변 템플릿, 인쇄 p.157-158)

값은 **constexpr** 표현식으로 정의할 수 있고 st**at**ic_assert로 해당 **타입**의 조건을 검사할 수 있다. [U06-081] (6.4.1 가변 템플릿, 인쇄 p.157-158)

이 아이디어는 7장의 **컨셉** 정의로 이어진다. [U06-082] (6.4.1 가변 템플릿, 인쇄 p.157-158)

원문 소절 제목을 유지하며 7.4의 가변 개수 인자 **템플릿**과 구별해 읽는다. [U06-083] (6.4.1 가변 템플릿, 인쇄 p.157-158)

6.4.2 별칭

using은 **타입**이나 **템플릿**의 다른 이름을 정의한다. [U06-084] (6.4.2 별칭, 인쇄 p.158-160)

size_t의 실제 **타입**은 구현마다 다르지만 같은 이름으로 접근하면 이식성을 유지할 수 있다. [U06-085] (6.4.2 별칭, 인쇄 p.158-160)

컨테이너가 요소 **타입**을 value_type으로 제공하는 관례를 따르면, 다른 코드가 특정 **컨테이너** 이름 없이 요소 **타입**을 얻을 수 있다. [U06-086] (6.4.2 별칭, 인쇄 p.158-160)

템플릿 인자의 일부를 고정하여 남은 인자만 받는 **별칭 템플릿**도 정의할 수 있다. [U06-087] (6.4.2 별칭, 인쇄 p.158-160)

일부 인자를 지정한 템플릿 별칭 (원문 발췌)

```
template<typename Value>
using String_map = Map<string,Value>;

String_map<int> m;    // m의 타입은 Map<string,int>
```

[U06-088] (6.4.2 별칭, 인쇄 p.160)

6.4.3 컴파일 시간 if

C++17의 **if constexpr**은 **타입**의 성질에 따라 필요한 구현을 선택한다. [U06-089] (6.4.3 컴파일 시간 if, 인쇄 p.160-161)

원문은 is_pod<T>를 바탕으로 단순하고 빠른 연산과 느리지만 안전한 연산을 구분한다. [U06-090] (6.4.3 컴파일 시간 if, 인쇄 p.160-161)

선택된 경우만 인스턴스화되므로 특정 **타입**에 대한 지역 최적화를 할 수 있다. [U06-091] (6.4.3 컴파일 시간 if, 인쇄 p.160-161)

컴파일 시간 if로 구현 선택 (원문 발췌)

```
template<typename T>
void update(T& target)
{
    // ...
    if constexpr(is_pod<T>::value)
        simple_and_fast(target);    // 단순한 데이터용
    else
        slow_and_safe(target);
    // ...
}
```

[U06-092] (6.4.3 컴파일 시간 if, 인쇄 p.160)

if constexpr은 단순한 텍스트 조작이 아니다. [U06-093] (6.4.3 컴파일 시간 if, 인쇄 p.160-161)

일반적인 문법·**타입**·스코프 규칙을 지켜야 하며, 조건부로 try의 시작과 끝을 서로 분리하는 원문 bad() 예는 문법 에러다. [U06-094] (6.4.3 컴파일 시간 if, 인쇄 p.160-161)

6.5 조언

여러 인자 **타입**에 적용할 알고리즘과 **컨테이너**를 **템플릿**으로 표현한다. [U06-095] (6.5 조언, 인쇄 p.161-162)

간단한 **함수 객체**를 한 곳에서만 사용한다면 람다를 이용한다. [U06-096] (6.5 조언, 인쇄 p.161-162)

표기를 단순하게 하고 구현을 숨길 때 **템플릿 별칭**을 사용한다. [U06-097] (6.5 조언, 인쇄 p.161-162)

템플릿의 **타입** 검사는 컴파일 시간에 수행되지만 인스턴스화까지 늦어질 수 있다. [U06-098] (6.5 조언, 인쇄 p.161-162)

사용되는 스코프에는 **템플릿**의 선언뿐 아니라 정의도 필요하다. [U06-099] (6.5 조언, 인쇄 p.161-162)

원문은 **템플릿** 정의를 사용하는 모든 변환 단위에서 포함하도록 조언한다. [U06-100] (6.5 조언, 인쇄 p.161-162)

주요 비교표

파라미터화된 연산의 표현

수단	원문에서의 역할
함수 **템플릿** [U06-101] (파라미터화된 연산의 표현, 인쇄 p.149-153)	**타입**·값이 달라져도 공통 연산 수행 [U06-102] (파라미터화된 연산의 표현, 인쇄 p.149-153)
함수 객체 [U06-103] (파라미터화된 연산의 표현, 인쇄 p.149-153)	호출 연산과 데이터를 함께 전달 [U06-104] (파라미터화된 연산의 표현, 인쇄 p.149-153)
람다 [U06-105] (파라미터화된 연산의 표현, 인쇄 p.149-153)	사용 위치에서 **함수 객체**를 간단히 생성 [U06-106] (파라미터화된 연산의 표현, 인쇄 p.149-153)

핵심 용어 표

용어	원문 정의·설명
템플릿	**템플릿**은 **타입**이나 값의 집합을 파라미터화한 클래스나 함수다. [U06-001] (6.1 소개, 인쇄 p.141)
제한된 인자·템플릿	**컨셉**이 적용된 **템플릿** 인자를 제한된 인자라고 하고, **컨셉**이 적용된 인자를 제한된 **템플릿**이라고 한다. [U06-107] (6.2.1 제한된 템플릿 인자(C++20), 인쇄 p.145)
값 템플릿 인자	**템플릿**의 값 인자는 반드시 상수 표현식이어야 한다. [U06-108] (6.2.2 값 템플릿 인자, 인쇄 p.146)
함수 객체	**함수 객체**는 함수처럼 호출할 수 있는 **객체**다. [U06-109] (6.3.2 함수 객체, 인쇄 p.151)
술어	**술어**는 호출 가능하며 그 결과로 true나 false를 반환하는 무언가를 말한다. [U06-110] (6.3.2 함수 객체, 인쇄 p.152)
람다 표현식	**람다 표현식**은 **함수 객체**를 암묵적으로 만들어내는 표기법이다. [U06-111] (6.3.3 람다 표현식, 인쇄 p.153)
별칭	**별칭**은 **타입**이나 **템플릿**의 동의어다. [U06-112] (6.4.2 별칭, 인쇄 p.158)
if constexpr	**if constexpr**에서는 선택된 경우만 인스턴스화되므로 지역적인 최적화가 가능하다. [U06-002] (6.4.3 컴파일 시간 if, 인쇄 p.160)

장 요약

요약 기준: 6.5 조언

다양한 인자 **타입**에 적용할 알고리즘과 **컨테이너**를 **템플릿**으로 표현한다. [U06-113] (6.5 조언 [1]~[2], 인쇄 p.161)

간단한 **함수 객체**를 한 곳에서만 사용한다면 람다를 이용한다. [U06-114] (6.5 조언 [7], 인쇄 p.161)

표기를 간단히 하고 상세 구현을 숨기기 위해 **템플릿 별칭**을 사용한다. [U06-115] (6.5 조언 [9], 인쇄 p.162)

템플릿을 사용할 때는 선언뿐 아니라 정의가 스코프에 있는지 확인한다. [U06-116] (6.5 조언 [10], 인쇄 p.162)

원문 확인 메모

원문 Buffer의 T[N]; 및 복사 캡처 설명의 [=x]는 의심되는 원문 표기를 보존하고 확인 필요로 표시했다.

Buffer의 T[N];와 복사 캡처 설명의 [=x]는 원문 표기를 임의 교정하지 않는다.

제7장. 컨셉과 제네릭 프로그래밍

(제7장. 컨셉과 제네릭 프로그래밍, 인쇄 p.163-182)

학습 목표·범위

컨셉은 하나 이상의 **타입**을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 **술어**다. [U07-001] (7.2.4 컨셉 정의, 인쇄 p.170)

접힘 표현식은 **파라미터 팩**의 요소를 연산으로 결합하며, 원문은 오른쪽 접힘과 왼쪽 접힘을 구분한다. [U07-002] (7.4.1 접힘 표현식, 인쇄 p.178)

핵심 개념·동작 절차·원문 예제

7.1 소개

템플릿은 **타입**·값·**템플릿** 자체를 정보 손실 없이 인자로 전달하고, 인스턴스화 때 알게 되는 정보로 최적화하며, 상수 인자로 컴파일 시간 계산도 수행하게 한다. [U07-003] (7.1 소개, 인쇄 p.163-164)

클래스는 코드와 값을 함께 담을 수 있으므로 **템플릿**의 파라미터화 대상은 단순한 요소 **타입**에 그치지 않는다. [U07-004] (7.1 소개, 인쇄 p.163-164)

제네릭 프로그래밍의 중심은 요구 사항을 만족하는 여러 **타입**에 공통으로 적용할 알고리즘을 설계·구현·사용하는 것이다. [U07-005] (7.1 소개, 인쇄 p.163-164)

템플릿은 컴파일 시간에 파라미터화할 수 있는 다형성을 제공하고, **컨셉**은 인자에 요구되는 기본 개념을 표현한다. [U07-006] (7.1 소개, 인쇄 p.163-164)

7.2 컨셉(C++20)

sum()은 서로 다른 시퀀스의 자료 구조와 서로 다른 요소 **타입**을 모두 받는다. [U07-007] (7.2 컨셉(C++20), 인쇄 p.164-165)

다만 시퀀스는 구간 for를 사용할 수 있는 begin()/end()를 제공해야 하고 요소는 누산할 값에 더할 수 있어야 한다. [U07-008] (7.2 컨셉(C++20), 인쇄 p.164-165)

이처럼 **템플릿** 인자가 만족해야 할 요구 사항을 **컨셉**이라고 한다. [U07-009] (7.2 컨셉(C++20), 인쇄 p.164-165)

원문은 **컨셉**의 언어 지원이 아직 ISO C++에 포함되지 않았고 ISO 기술 명세에 있으며, 구현체는 있지만 세부 사항의 변경이나 상용 사용까지의 시간이 위험 요소라고 설명한다. [U07-010] (7.2 컨셉(C++20), 인쇄 p.164-165)

소절 제목의 C++20 표기와 함께 이 당시 상태를 그대로 유지한다. [U07-011] (7.2 컨셉(C++20), 인쇄 p.164-165)

7.2.1 컨셉의 사용

typename은 인자가 **타입**이어야 한다는 최소 요구만 지정한다. [U07-012] (7.2.1 컨셉의 사용, 인쇄 p.165-167)

Sequence와 Number라는 의미 있는 제약을 쓰면 의도가 드러나고, 구현을 모두 보지 않아도 인터페이스에서 잘못된 호출을 막을 수 있어 에러 보고가 좋아진다. [U07-013] (7.2.1 컨셉의 사용, 인쇄 p.165-167)

그러나 시퀀스와 수라는 두 개별 조건만으로 요소와 누산기의 연산 관계까지 완전히 지정한 것은 아니다. [U07-014] (7.2.1 컨셉의 사용, 인쇄 p.165-167)

시퀀스와 수의 관계를 지정하는 요구 절 (원문 발췌)

```
template<Sequence Seq, Number Num>
    requires Arithmetic<Value_type<Seq>,Num>
Num sum(Seq s, Num n);
```

[U07-015] (7.2.1 컨셉의 사용, 인쇄 p.166)

Value_type은 시퀀스의 요소 **타입**이다. [U07-016] (7.2.1 컨셉의 사용, 인쇄 p.166)

Arithmetic<X,Y>는 두 수치 **타입** 사이의 산술 연산 가능성을 표현한다. [U07-017] (7.2.1 컨셉의 사용, 인쇄 p.166)

원문은 실제 구현에 +=만 등장해도 이를 지나치게 좁은 요구로 삼으면 유연성이 줄어든다고 설명한다. [U07-018] (7.2.1 컨셉의 사용, 인쇄 p.166)

나중에 +와 =를 이용하는 구현으로 바꿀 수도 있으므로 일반적인 의미를 가진 **컨셉**이 더 낫다. [U07-019] (7.2.1 컨셉의 사용, 인쇄 p.166)

부분적인 명세도 의도를 알리고 점진적으로 개발하는 데 유용하지만, 명시하지 못한 요구에 관한 에러는 인스턴스화 시간에 나타날 수 있다. [U07-020] (7.2.1 컨셉의 사용, 인쇄 p.166-167)

requires 절과 template<Sequence Seq>와 같은 축약 표기를 사용할 수 있다. [U07-021] (7.2.1 컨셉의 사용, 인쇄 p.166-167)

컨셉을 쓸 수 없는 환경에서는 명명 관례와 주석으로 요구를 나타낸다고 설명한다. [U07-022] (7.2.1 컨셉의 사용, 인쇄 p.166-167)

7.2.2 컨셉 기반 오버로딩

템플릿도 인자가 만족하는 속성에 따라 오버로딩할 수 있다. [U07-023] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

원문 advance()는 순방향 **반복자**에는 반복해서 ++를 수행하고, 임의 접근 **반복자**에는 +=를 수행하는 두 버전을 비교한다. [U07-024] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

컴파일러는 인자가 만족하는 가장 강한 요구 사항에 맞춰 선택한다. [U07-025] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

이 선택은 컴파일 시간에 이루어져 실행 시간의 부담을 추가하지 않는다. [U07-026] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

컨셉을 만족하지 못하는 인자가 있는 함수는 후보에서 제외한다. [U07-027] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

맞는 함수가 하나뿐이면 선택하고, 똑같이 맞는 두 함수를 구분하지 못하면 모호성 에러를 낸다. [U07-028] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

한 함수의 요구가 다른 함수의 모든 요구와 추가 요구를 포함한다면 더 엄격하게 맞는 함수를 선택한다. [U07-029] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

선택되는 함수는 모든 인자가 맞아야 하고, 다른 후보보다 각 인자가 적어도 같은 정도로 맞으며, 적어도 하나는 더 잘 맞아야 한다. [U07-030] (7.2.2 컨셉 기반 오버로딩, 인쇄 p.167-168)

7.2.3 유효한 코드

요구 사항의 바탕에는 어떤 표현식이 유효한가라는 문제가 있다. [U07-031] (7.2.3 유효한 코드, 인쇄 p.169-170)

requires 표현식은 주어진 표현식 집합이 유효한 코드인지 판단하는 **술어**다. [U07-032] (7.2.3 유효한 코드, 인쇄 p.169-170)

requires requires에서 첫 단어는 요구 절을, 두 번째 단어는 이 표현식을 시작하므로 중복 자체는 오타가 아니다. [U07-033] (7.2.3 유효한 코드, 인쇄 p.169-170)

표현식의 유효성 확인 (원문 발췌)

```
requires(Iter p, int i) { p[i]; p+i; }
```

[U07-034] (7.2.3 유효한 코드, 인쇄 p.169)

발췌 범위: advance() 예제에서 requires 표현식만 발췌했다. 함수 정의는 생략했다.

원문은 표현식을 직접 나열한 advance()에서도 +=와 반환 **타입** 요구를 빠뜨렸다고 지적한다. [U07-035] (7.2.3 유효한 코드, 인쇄 p.169-170)

이런 저수준 표현은 매우 유연하지만 의미를 전달하기 어렵다. [U07-036] (7.2.3 유효한 코드, 인쇄 p.169-170)

보통 코드에는 적절한 이름과 명시된 의미가 있는 **컨셉**을 사용하고, **requires 표현식**은 **컨셉** 정의에 사용하도록 조언한다. [U07-037] (7.2.3 유효한 코드, 인쇄 p.169-170)

7.2.4 컨셉 정의

컨셉은 하나 이상의 **타입**을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 **술어**이며 값은 항상 bool이다. [U07-038] (7.2.4 컨셉 정의, 인쇄 p.170-172)

원문 Equality_comparable은 같은 **타입**의 두 값을 ==와 !=로 비교할 수 있고 그 결과가 bool로 변환될 수 있어야 한다는 요구다. [U07-039] (7.2.4 컨셉 정의, 인쇄 p.170-172)

int에 대한 검사는 성공하지만, 구조체에 이러한 연산이 자동으로 정의되지 않으면 검사가 실패한다. [U07-040] (7.2.4 컨셉 정의, 인쇄 p.170-172)

두 **타입**을 비교하도록 확장할 때는 양방향의 ==와 !=를 요구한다. [U07-041] (7.2.4 컨셉 정의, 인쇄 p.171)

두 번째 **템플릿** 인자를 지정하지 않을 때 첫 **타입**을 쓰도록 기본 인자를 줄 수 있다. [U07-042] (7.2.4 컨셉 정의, 인쇄 p.171)

원문은 int와 double, 같은 int의 검사는 성공하고 int와 **string**의 검사는 실패하는 예를 제시한다. [U07-043] (7.2.4 컨셉 정의, 인쇄 p.171)

Sequence 정의는 값 **타입**과 **반복자 타입**을 제공하고, **컨테이너** 관례에 맞는 begin()/end()를 제공하며, **반복자**가 요소 **타입**에 맞는 입력 **반복자**여야 한다는 조건을 결합한다. [U07-044] (7.2.4 컨셉 정의, 인쇄 p.171-172)

기본 언어 개념을 정확히 표현하는 **컨셉**을 정의하는 일이 가장 어려우므로, 잘 정돈된 라이브러리의 **컨셉**을 이용하라고 설명한다. [U07-045] (7.2.4 컨셉 정의, 인쇄 p.171-172)

7.3 제네릭 프로그래밍

구체적이고 효율적인 알고리즘에서 자료 표현에 묶인 부분을 추상화하여 서로 다른 표현에도 적용하는 것이 **제네릭 프로그래밍**의 중심이다. [U07-046] (7.3 제네릭 프로그래밍, 인쇄 p.172)

컨셉은 기본 연산과 자료 구조에 대한 추상화이자 **템플릿** 파라미터가 만족해야 하는 요구 사항이다. [U07-047] (7.3 제네릭 프로그래밍, 인쇄 p.172)

7.3.1 컨셉의 사용

유용한 **컨셉**은 정수·부동소수점 수·시퀀스와 같은 해당 분야의 기본 개념을 표현한다. [U07-048] (7.3.1 컨셉의 사용, 인쇄 p.172-173)

Regular 타입은 기본 생성이 가능하고 생성·대입으로 독립적이면서 같은 값인 복사본을 만들 수 있으며 ==와 !=로 비교할 수 있다. [U07-049] (7.3.1 컨셉의 사용, 인쇄 p.172-173)

원문은 과도하게 영리한 프로그래밍 기법으로 기술적 문제를 겪지 않는다는 조건도 덧붙인다. [U07-050] (7.3.1 컨셉의 사용, 인쇄 p.172-173)

int, **vector**, **string**이 예이며, int와 **string**은 <·<=·>·>=로 비교 가능한 StrictTotallyOrdered의 예로도 설명한다. [U07-051] (7.3.1 컨셉의 사용, 인쇄 p.172-173)

컨셉은 단순한 구문보다 의미에 관한 개념이다. [U07-052] (7.3.1 컨셉의 사용, 인쇄 p.173)

예를 들어 +가 존재하기만 한다고 충분하지 않으며, +를 나눗셈으로 정의해서는 합리적인 수의 요구를 만족하지 않는다. [U07-053] (7.3.1 컨셉의 사용, 인쇄 p.173)

언어로 의미론 전체를 표현할 수 없으므로 분야의 지식과 상식이 필요하다. [U07-054] (7.3.1 컨셉의 사용, 인쇄 p.173)

원문은 의미 없는 Addable·Subtractable 같은 개념을 나열하기보다 분야의 기본 개념에 대응하도록 설계하라고 조언한다. [U07-055] (7.3.1 컨셉의 사용, 인쇄 p.173)

7.3.2 템플릿을 이용한 추상화

좋은 추상화는 실제 구체적 용례에서 시작한다. [U07-056] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.173-175)

모든 미래 필요를 예상하여 추상화하기보다는 하나 이상의 사례에서 불필요한 세부를 제거한다. [U07-057] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.173-175)

원문의 합산은 요소가 int인지, 저장소가 **vector**인지, 누산기가 double인지, 초깃값이 0인지에 대한 고정을 **템플릿** 인자와 함수 인자로 풀어낸다. [U07-058] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.173-175)

구체적인 합산을 일반화한 accumulate (원문 발췌)

```
template<typename Iter, typename Val>
Val accumulate(Iter first, Iter last, Val res)
{
    for (auto p = first; p!=last; ++p)
        res += *p;
    return res;
}
```

[U07-059] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.174)

반복자 한 쌍은 자료 구조를 추상화하고, Val은 누산기 **타입**을, res는 초깃값을 전달한다. [U07-060] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.174-175)

동일한 성능을 유지하며 구체적인 코드를 일반화하는 과정을 **리프팅**이라고 한다. [U07-061] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.174-175)

권장되는 개발 순서는 구체 버전을 만들고 디버그·테스트·측정한 후 **구체 타입**을 **템플릿** 인자로 교체하는 것이다. [U07-062] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.174-175)

begin()/end() 반복을 줄이려면 Range를 받는 인터페이스로 단순화할 수 있고, 더 일반화하려면 +=도 대체할 수 있다. [U07-063] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.174-175)

7.4 가변 템플릿

가변 **템플릿**은 다양한 **타입**의 인자를 가변 개수로 받는다. ...으로 선언하는 파라미터를 **파라미터 팩**이라고 하며, Tail은 **타입** 시퀀스, tail은 그 **타입**들에 해당하는 값의 시퀀스다. [U07-064] (7.4 가변 템플릿, 인쇄 p.176-177)

전통적인 구현은 첫 인자를 처리한 뒤 나머지 팩을 재귀 호출한다. [U07-065] (7.4 가변 템플릿, 인쇄 p.176-177)

파라미터 팩을 재귀적으로 처리 (원문 발췌)

```
void print()
{
    // 인자가 없으면 아무것도 하지 않음
}

template<typename T, typename ... Tail>
void print(T head, Tail... tail)
{
    // 각 인자에 대해
    cout << head << ' ';
    print(tail...);
}
```

[U07-066] (7.4 가변 템플릿, 인쇄 p.176)

마지막에 팩이 비면 인자 없는 print()가 호출되어 종료한다. [U07-067] (7.4 가변 템플릿, 인쇄 p.176-177)

빈 인자를 허용하고 싶지 않으면 **if constexpr**으로 실제 마지막 호출을 생성하지 않는 대안을 쓸 수 있다. [U07-068] (7.4 가변 템플릿, 인쇄 p.176-177)

빈 파라미터 팩 호출을 생성하지 않기 (원문 발췌)

```
template<typename T, typename ... Tail>
void print(T head, Tail... tail)
{
    cout << head << ' ';
    if constexpr(sizeof...(tail)> 0)
        print(tail...);
}
```

[U07-069] (7.4 가변 템플릿, 인쇄 p.177)

가변 **템플릿**의 장점은 원하는 여러 **타입**을 받을 수 있다는 유연성이다. [U07-070] (7.4 가변 템플릿, 인쇄 p.177)

반면 재귀 구현을 고치기 어렵고 컴파일 비용이 커질 수 있으며 인터페이스 **타입** 검사에 복잡한 **템플릿** 프로그램이 필요할 수 있다. [U07-071] (7.4 가변 템플릿, 인쇄 p.177)

표준 라이브러리의 중요한 수단이지만 남용할 수 있다고 경고한다. [U07-072] (7.4 가변 템플릿, 인쇄 p.177)

7.4.1 접힘 표현식

C++17의 **접힘 표현식**은 **파라미터 팩**의 요소에 연산을 적용하는 일부 가변 **템플릿** 구현을 간단히 만든다. (v + ... + 0)은 오른쪽의 0에서 시작하는 괄호 묶음으로, (0 + ... + v)는 왼쪽의 0에서 시작하는 괄호 묶음으로 설명된다. [U07-073] (7.4.1 접힘 표현식, 인쇄 p.177-180)

오른쪽 접힘과 원문 결과 (원문 발췌)

```
template<Number... T>
int sum(T... v)
{
    return (v + ... + 0);    // v의 모든 요소를 0부터 더함
}

int x = sum(1, 2, 3, 4, 5); // x는 15가 됨
int y = sum('a', 2.4, x);   // y는 114가 됨(2.4는 소수점 아래는 버려지고 'a'의 값은 97)
```

[U07-074] (7.4.1 접힘 표현식, 인쇄 p.178)

왼쪽 접힘 (원문 발췌)

```
template<typename ... T>
int sum2(T... v)
{
    return (0 + ... + v);    // v의 모든 요소를 0에 더함
}
```

[U07-075] (7.4.1 접힘 표현식, 인쇄 p.178)

접힘은 합산뿐 아니라 <<를 통한 모든 인자의 출력에도 쓸 수 있다. [U07-076] (7.4.1 접힘 표현식, 인쇄 p.179-180)

원문 to_**vector**는 여러 인자를 공통 요소 **타입**의 벡터에 담아 이후 다루기 쉽게 하는 예다. [U07-077] (7.4.1 접힘 표현식, 인쇄 p.179-180)

접힘의 일반적인 아이디어는 accumul**ate**에서도 사용되지만, 원문은 당시 C++의 **접힘 표현식**은 가변 **템플릿** 구현을 간단히 하는 용도로 제한된다고 설명한다. [U07-078] (7.4.1 접힘 표현식, 인쇄 p.179-180)

7.4.2 인자 포워딩

가변 **템플릿**의 중요한 용도는 인터페이스에서 받은 인자를 다른 인터페이스로 그대로 전달하는 것이다. [U07-079] (7.4.2 인자 포워딩, 인쇄 p.180)

원문 InputChannel은 실제 전송 방식을 Transport로 파라미터화하고 **생성자** 인자를 forward()로 Transport **생성자**에 전달한다. [U07-080] (7.4.2 인자 포워딩, 인쇄 p.180)

작성자가 특정 Transport에 필요한 인자의 종류를 몰라도 **객체**를 생성할 수 있다. [U07-081] (7.4.2 인자 포워딩, 인쇄 p.180)

일반성과 낮은 실행 부담이 중요한 기본 라이브러리에서 널리 쓴다고 설명한다. [U07-082] (7.4.2 인자 포워딩, 인쇄 p.180)

7.5 템플릿 컴파일 모델

컨셉을 사용하면 먼저 **템플릿** 인자가 **컨셉**을 만족하는지 확인한다. [U07-083] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

이때 확인할 수 없는 항목은 실제 코드를 생성하는 인스턴스화 시간까지 미룬다. [U07-084] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

컨셉이 없는 **템플릿**은 **타입** 검사를 인스턴스화 시점에 수행하므로 오류가 늦게 발견되고 여러 곳의 정보가 합쳐진 진단이 쓸모없어질 수 있다. [U07-085] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

원문은 이처럼 **템플릿** 인자에 필요한 연산이 있는지 뒤늦게 확인하는 특성을 컴파일 시간 덕 타이핑과 연결한다. [U07-086] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

일반적인 **객체**가 **타입**에 따라 연산을 갖는 것과 달리 **템플릿** 계산의 많은 부분은 **객체**보다 값에 관한 것이며, **constexpr** 함수의 지역 **변수**는 예외로 든다. [U07-087] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

제한되지 않은 **템플릿**은 사용하는 스코프에 선언뿐 아니라 정의가 있어야 한다. [U07-088] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

그래서 보통 정의는 .cpp보다 헤더에 놓인다. [U07-089] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

원문은 모듈을 사용하면 일반 함수와 비슷하게 코드를 정리할 수 있고 텍스트 포함 문제로부터 보호할 수 있다고 설명한다. [U07-090] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

7.6 조언

템플릿 인자의 요구 개념을 세심하게 고려하고 **컨셉**을 설계 도구로 사용한다. [U07-091] (7.6 조언, 인쇄 p.182)

의미 있는 동작을 내포하는 **컨셉**을 사용하고 요구 연산을 완전하게 명시한다. [U07-092] (7.6 조언, 인쇄 p.182)

구체 구현을 디버그하고 측정한 뒤 일반화한다. [U07-093] (7.6 조언, 인쇄 p.182)

같은 **타입**의 인자 목록에는 초깃값 목록을 사용하고 가변 **템플릿**을 남용하지 않는다. [U07-094] (7.6 조언, 인쇄 p.182)

템플릿의 정의가 사용되는 스코프에 있어야 한다. [U07-095] (7.6 조언, 인쇄 p.182)

주요 비교표

원문이 대비한 검사 시점

방식	먼저 확인하는 내용	남는 검사
컨셉 사용 [U07-096] (원문이 대비한 검사 시점, 인쇄 p.181)	**템플릿** 인자의 **컨셉** 만족 여부 [U07-097] (원문이 대비한 검사 시점, 인쇄 p.181)	그 시점에 확인할 수 없는 항목은 인스턴스화 때 검사 [U07-098] (원문이 대비한 검사 시점, 인쇄 p.181)
컨셉 미사용 [U07-099] (원문이 대비한 검사 시점, 인쇄 p.181)	**컨셉**에 의한 인터페이스 검사 없음 [U07-100] (원문이 대비한 검사 시점, 인쇄 p.181)	**타입** 검사가 인스턴스화 시간까지 지연 [U07-101] (원문이 대비한 검사 시점, 인쇄 p.181)

핵심 용어 표

용어	원문 정의·설명
컨셉	**컨셉**은 하나 이상의 **타입**을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 **술어**다. [U07-001] (7.2.4 컨셉 정의, 인쇄 p.170)
requires 표현식	**requires 표현식**은 주어진 문장이 유효한 코드면 true, 그렇지 않으면 false로 평가되는 **술어**다. [U07-102] (7.2.3 유효한 코드, 인쇄 p.169)
제네릭 프로그래밍	**제네릭 프로그래밍**은 서로 다른 데이터 표현에 적용할 제네릭 알고리즘을 만들기 위해 구체적이고 효율적인 알고리즘을 추상화하는 아이디어에 주안점을 둔다. [U07-103] (7.3 제네릭 프로그래밍, 인쇄 p.172)
Regular	**Regular 타입**은 기본 **생성자**로 생성하고 **생성자**·대입으로 독립적이고 같은 값인 복사본을 만들며 ==와 !=로 비교할 수 있다. [U07-104] (7.3.1 컨셉의 사용, 인쇄 p.173)
리프팅	**리프팅**은 동일한 성능을 유지하면서 구체적인 코드를 일반화하는 과정이다. [U07-105] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.175)
파라미터 팩	...으로 선언된 파라미터를 **파라미터 팩**이라고 한다. [U07-106] (7.4 가변 템플릿, 인쇄 p.176)
접힘 표현식	**접힘 표현식**은 **파라미터 팩**의 요소를 연산으로 결합하며, 원문은 오른쪽 접힘과 왼쪽 접힘을 구분한다. [U07-002] (7.4.1 접힘 표현식, 인쇄 p.178)

장 요약

요약 기준: 7.6 조언

템플릿 인자에 가정하는 개념을 세심히 고려하고 **컨셉**을 설계 도구로 사용한다. [U07-107] (7.6 조언 [2]·[4], 인쇄 p.182)

템플릿을 설계할 때 우선 **구체 타입**을 바탕으로 구현하고 디버깅과 측정을 거친다. [U07-108] (7.6 조언 [3], 인쇄 p.182)

의미 있는 동작을 내포하지 않는 **컨셉**은 피하고, **컨셉**에 요구되는 연산의 집합을 완벽히 명시한다. [U07-109] (7.6 조언 [10]~[11], 인쇄 p.182)

동일한 **타입**의 인자 목록에는 가변 **템플릿** 대신 초깃값 목록을 사용하고, 사용하는 곳에는 **템플릿** 정의가 존재해야 한다. [U07-110] (7.6 조언 [13]~[14], 인쇄 p.182)

원문 확인 메모

컨셉의 표준·기술 명세 상태와 코드 표기는 원문 당시의 설명을 유지했다.

컨셉의 표준·기술 명세 상태는 원문 당시의 설명을 유지한다.

제8장. 라이브러리의 개요

(제8장. 라이브러리의 개요, 인쇄 p.183-188)

학습 목표·범위

C++ **표준 라이브러리**는 가장 널리 사용하는 기본 자료 구조와 그 자료 구조에서 동작하는 알고리즘을 제공한다. [U08-001] (8.2 표준 라이브러리 구성 요소, 인쇄 p.186)

표준 라이브러리는 **std 네임스페이스**에 정의되므로, 기능을 사용할 때 std:: 접두사를 사용할 수 있다. [U08-002] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

핵심 개념·동작 절차·원문 예제

8.1 소개

실제 프로그램은 기본 언어 기능만으로 작성하기보다 라이브러리를 토대로 구성된다. [U08-003] (8.1 소개, 인쇄 p.183-184)

좋은 라이브러리를 이용하면 기본 언어만으로 작성하기 어려운 작업을 쉽게 만들 수 있다. [U08-004] (8.1 소개, 인쇄 p.183-184)

이 책은 모든 세부를 완전히 설명하기보다 **표준 라이브러리**의 유용한 기능과 일반적인 사용법을 익히는 데 초점을 둔다. [U08-005] (8.1 소개, 인쇄 p.183-184)

표준 라이브러리에는 설계·구현·유지 보수·확장을 위한 많은 노력이 들어간다. [U08-006] (8.1 소개, 인쇄 p.183-184)

원문은 직접 코드를 다시 만드는 대신 이 기능을 사용하라고 권한다. [U08-007] (8.1 소개, 인쇄 p.183-184)

한편 GUI·웹·데이터베이스 인터페이스나 산업별 기반 라이브러리도 있지만, 이 책은 표준에서 정의한 C++와 이식성 있는 예제에 집중하므로 이러한 외부 시스템은 다루지 않는다. [U08-008] (8.1 소개, 인쇄 p.183-184)

8.2 표준 라이브러리 구성 요소

표준 라이브러리는 널리 쓰는 기본 자료 구조와 그 위에서 동작하는 알고리즘을 일반적인 형태로 제공한다. [U08-009] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

기본적인 사용 예는 작업 자체의 복잡성에 비해 쉽게 배울 수 있고, 단순한 대안에 비해 큰 추가 부담이 없으며, 초보자와 전문가 모두에게 유용해야 한다는 선정 기준을 설명한다. [U08-010] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

언어 실행 지원과 C **표준 라이브러리**: 메모리 할당, 실행 시간 **타입** 정보, **타입** 시스템 위배를 줄이기 위한 일부 수정. [U08-011] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

문자열·문자열 뷰·**정규 표현식**: 여러 문자 세트, 지역화, 읽기 전용 부분 문자열, 패턴 매칭. [U08-012] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

입출력·파일 시스템: **타입**·스트림·버퍼링·로케일을 확장할 수 있는 입출력과 이식성 있는 파일 조작. [U08-013] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

컨테이너·알고리즘: **vector**·**map**과 **find**·sort·merge 등의 확장 가능한 **STL** 프레임워크. [U08-014] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

수치·**동시성**·병렬 계산: 수학 함수, 복소수, 수치 벡터, 난수와 **thread**·**mutex**, 일부 병렬 알고리즘. [U08-015] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

일반적 유틸리티: **타입** 특성, **pair**·**variant**·optional, 시간, 스마트 포인터와 **자원** 관리, array·bitset·**tuple**, 단위를 나타내는 접미사. [U08-016] (8.2 표준 라이브러리 구성 요소, 인쇄 p.184-186)

8.3 표준 라이브러리 헤더와 네임스페이스

표준 기능은 **표준 헤더**가 제공하며 **std 네임스페이스**에 정의된다. [U08-017] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)

사용하려는 기능의 헤더를 포함하고 std:: 등의 방법으로 이름에 접근해야 한다. [U08-018] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)

원문의 많은 예제는 단순하게 보이도록 헤더와 std::를 생략하므로, 그러한 코드 조각이 독립적인 완전한 프로그램이라는 뜻은 아니다. [U08-019] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)

기능을 제공하는 표준 헤더 (원문 발췌)

```
#include<string>
#include<list>
```

[U08-020] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

std 이름을 직접 한정하기 (원문 발췌)

```
std::string sheep {"Four legs Good; two legs Baaad!"};
```

[U08-021] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

발췌 범위: 같은 원문 영역의 첫 선언을 발췌했다. 이어지는 slogans 목록 선언은 생략했다.

using namespace std를 쓰면 std:: 없이 이름을 사용할 수 있다. [U08-022] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.187)

하지만 한 **네임스페이스**의 모든 이름을 전역으로 가져오는 것은 일반적으로 좋지 않다고 명시한다. [U08-023] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.187)

책에서는 **표준 라이브러리**의 사용법 학습을 위해 이 방식을 쓰기도 한다. [U08-024] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.187)

기능	원문에 나열된 헤더
알고리즘·**컨테이너** [U08-025] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)	<algorithm>, <array>, <**forward_list**>, <**list**>, <**map**>, <set>, <**unordered_map**>, <**vector**> [U08-026] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)
문자·입출력 [U08-027] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)	<**string**>, <regex>, <ios>, <iostream>, <fstream>, <sstream> [U08-028] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)
수치·시간·**동시성** [U08-029] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)	<cm**at**h>, <complex>, <random>, <chrono>, <**thread**>, <**future**> [U08-030] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)
자원·지원 [U08-031] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)	<memory>, <stdexcept>, <utility>, <**variant**>, <filesystem> [U08-032] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186-188)

C 라이브러리의 <stdlib.h> 같은 헤더도 제공한다. [U08-033] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.188)

이름 앞에 c를 붙이고 .h를 제거한 <cstdlib> 같은 버전은 이름을 **std 네임스페이스**에 선언한다. [U08-034] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.188)

헤더 목록은 선별된 일부이며 **표준 라이브러리** 전체 목록이 아니다. [U08-035] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.188)

8.4 조언

라이브러리를 활용하고, 선택할 수 있다면 **표준 라이브러리**를 사용한다. [U08-036] (8.4 조언, 인쇄 p.188)

표준 라이브러리가 모든 문제를 해결하는 것은 아니다. [U08-037] (8.4 조언, 인쇄 p.188)

필요한 기능을 제공하는 헤더를 포함한다. [U08-038] (8.4 조언, 인쇄 p.188)

표준 라이브러리 기능은 **std 네임스페이스**에 정의된다. [U08-039] (8.4 조언, 인쇄 p.188)

핵심 용어 표

용어	원문 정의·설명
표준 라이브러리	C++ **표준 라이브러리**는 가장 널리 사용하는 기본 자료 구조와 그 자료 구조에서 동작하는 알고리즘을 제공한다. [U08-001] (8.2 표준 라이브러리 구성 요소, 인쇄 p.186)
STL	**컨테이너**와 **find**(), sort(), merge() 등을 비롯한 알고리즘 프레임워크를 관례적으로 **STL**이라고 한다. [U08-040] (8.2 표준 라이브러리 구성 요소, 인쇄 p.185)
표준 헤더	**표준 라이브러리**의 모든 기능은 **표준 헤더**에서 제공된다. [U08-041] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)
std 네임스페이스	**표준 라이브러리**는 **std 네임스페이스**에 정의되므로, 기능을 사용할 때 std:: 접두사를 사용할 수 있다. [U08-002] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

장 요약

요약 기준: 8.4 조언

라이브러리를 활용하고 선택권이 있다면 **표준 라이브러리**를 사용한다. [U08-042] (8.4 조언 [1]~[2], 인쇄 p.188)

표준 라이브러리가 만병통치약은 아니다. [U08-043] (8.4 조언 [3], 인쇄 p.188)

원하는 기능을 제공하는 헤더를 #include해야 하며, **표준 라이브러리** 기능은 **std 네임스페이스**에 정의된다. [U08-044] (8.4 조언 [4]~[5], 인쇄 p.188)

제9장. 문자열과 정규 표현식

(제9장. 문자열과 정규 표현식, 인쇄 p.189-208)

학습 목표·범위

string은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. [U09-001] (9.2 문자열, 인쇄 p.190,192)

짧은 문자열 최적화는 짧은 값을 **객체** 자체에 두고 긴 값에만 자유 저장소를 이용한다. [U09-002] (9.2.1 string 구현, 인쇄 p.193-194)

핵심 개념·동작 절차·원문 예제

9.1 소개

텍스트 조작을 위해 **표준 라이브러리**는 문자 시퀀스를 저장·조작하는 **string**, 저장 방식에 독립적으로 문자를 읽는 **string_view**, 패턴을 기술하고 찾는 regex를 제공한다. [U09-003] (9.1 소개, 인쇄 p.189-190)

string과 regex는 여러 문자 **타입**에 적용할 수 있으며 원문은 유니코드를 그 예로 든다. [U09-004] (9.1 소개, 인쇄 p.189-190)

9.2 문자열

string은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. +는 문자열 연결을 뜻하며 **string**·문자열 리터럴·C 스타일 문자열·문자 하나를 함께 연결할 수 있다. [U09-005] (9.2 문자열, 인쇄 p.190-192)

끝에 추가하는 경우 +=는 의도를 명료하게 나타내고 간결하며 때로는 더 효율적이어서 원문이 권장한다. [U09-006] (9.2 문자열, 인쇄 p.190-192)

이동 **생성자**가 있으므로 긴 **string**도 값으로 효율적으로 반환할 수 있다. [U09-007] (9.2 문자열, 인쇄 p.190-192)

string은 변경할 수 있고 대입·인덱싱·부분 문자열 연산을 제공한다. [U09-008] (9.2 문자열, 인쇄 p.191)

substr()은 지정한 위치부터 길이만큼 별도 **string**을 복사한다. [U09-009] (9.2 문자열, 인쇄 p.191)

replace()는 지정한 부분을 교체하며, 원래 부분과 새 문자열의 길이가 같을 필요는 없다. [U09-010] (9.2 문자열, 인쇄 p.191)

부분 문자열 복사와 교체 (원문 발췌)

```
string name = "Niels Stroustrup";

void m3()
{
    string s = name.substr(6,10); // s = "Stroustrup"
    name.replace(0,5,"nicholas"); // name은 "nicholas Stroustrup"
    name[0] = toupper(name[0]);   // name은 "Nicholas Stroustrup"
}
```

[U09-011] (9.2 문자열, 인쇄 p.191)

비교에는 ==와 !=, 사전 순서 비교에는 <·<=·>·>=를 사용한다. [U09-012] (9.2 문자열, 인쇄 p.191-192)

[]와 **at**()을 이용한 접근, **반복자** 순회, 입력과 스트리밍도 제공한다. [U09-013] (9.2 문자열, 인쇄 p.191-192)

C 스타일의 0으로 끝나는 문자 배열이 필요할 때는 c_str()로 읽기 전용 문자 포인터를 얻는다. [U09-014] (9.2 문자열, 인쇄 p.191-192)

원문은 필요할 때에만 이 표현을 사용하라고 조언한다. [U09-015] (9.2 문자열, 인쇄 p.191-192)

문자열 리터럴을 std::**string**으로 사용하려면 접미사 s를 쓴다. [U09-016] (9.2 문자열, 인쇄 p.192)

접미사가 없는 리터럴과의 **타입** 차이를 주의한다. [U09-017] (9.2 문자열, 인쇄 p.192)

원문은 std::literals::**string**_literals **네임스페이스**가 필요하다고 설명한다. [U09-018] (9.2 문자열, 인쇄 p.192)

9.2.1 string 구현

일반적으로 사용하는 **string**은 직접 처음 만든 구현보다 표준 구현을 선택하라고 설명한다. [U09-019] (9.2.1 string 구현, 인쇄 p.193-194)

흔히 사용하는 **짧은 문자열 최적화**는 짧은 값을 **객체** 자체에 두고 긴 값에만 자유 저장소를 이용한다. [U09-020] (9.2.1 string 구현, 인쇄 p.193-194)

문자열 길이가 변하면 표현도 적절히 바뀌며, 짧다고 보는 길이는 구현에 따라 달라진다. [U09-021] (9.2.1 string 구현, 인쇄 p.193-194)

실제 성능은 실행 환경의 영향을 받는다. [U09-022] (9.2.1 string 구현, 인쇄 p.193-194)

특히 다중 스레드 환경의 메모리 할당 비용과 서로 다른 길이 문자열에 의한 메모리 단편화는 **짧은 문자열 최적화**를 사용하는 이유다. [U09-023] (9.2.1 string 구현, 인쇄 p.193-194)

문자 **타입**을 일반화한 basic_**string**<Char>가 바탕이며, **string**은 basic_**string**<char>의 **별칭**이다. [U09-024] (9.2.1 string 구현, 인쇄 p.193-194)

원문 Jchar 예처럼 다른 문자 **타입**에도 일반적인 문자열 연산을 적용할 수 있다. [U09-025] (9.2.1 string 구현, 인쇄 p.193-194)

9.3 문자열 뷰

string_view는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있다. [U09-026] (9.3 문자열 뷰, 인쇄 p.194-196)

문자는 **string**·C 스타일 문자열 등 서로 다른 방식으로 저장될 수 있다. [U09-027] (9.3 문자열 뷰, 인쇄 p.194-196)

뷰 자체는 문자를 소유하지 않으므로 포인터·참조 및 **STL 반복자**와 유사하다. [U09-028] (9.3 문자열 뷰, 인쇄 p.194-196)

서로 다른 저장 형태를 받는 문자열 뷰 (원문 발췌)

```
string cat(string_view sv1, string_view sv2)
{
    string res(sv1.length()+sv2.length());
    char* p = &res[0];
    for (char c : sv1)    // 복사를 수행하는 한 가지 방법
        *p++ = c;
    copy(sv2.begin(),sv2.end(),p); // 다른 방법
    return res;
}
```

[U09-029] (9.3 문자열 뷰, 인쇄 p.195)

const string&를 받는 compose()와 비교해 원문 c**at**()은 다양한 저장 방식을 받을 수 있고, C 스타일 문자열에서 임시 **string**을 만들지 않으며, 부분 문자열을 쉽게 전달할 수 있다. [U09-030] (9.3 문자열 뷰, 인쇄 p.195-196)

접미사 sv를 쓰는 문자열 뷰 리터럴은 길이를 컴파일 시간에 계산한다. [U09-031] (9.3 문자열 뷰, 인쇄 p.195-196)

원문은 std::literals::**string_view**_literals를 지정한다. [U09-032] (9.3 문자열 뷰, 인쇄 p.195-196)

지역 문자열을 가리키는 뷰 반환의 위험 (원문 발췌)

```
string_view bad()
{
    string s = "Once upon a time";
    return {&s[5],4};    // 위험: 지역 변수를 가리키는 포인터 반환
}
```

[U09-033] (9.3 문자열 뷰, 인쇄 p.196)

뷰를 반환할 때는 실제로 무엇을 가리키는지 수명을 확인해야 한다. [U09-034] (9.3 문자열 뷰, 인쇄 p.196)

string_view는 읽기 전용이므로 문자를 소문자로 바꾸는 함수의 쓰기 인터페이스로 사용할 수 없다. [U09-035] (9.3 문자열 뷰, 인쇄 p.196)

원문은 이런 용도에 **gsl::span** 또는 gsl::**string**_span을 연결한다. [U09-036] (9.3 문자열 뷰, 인쇄 p.196)

구간 밖 접근 결과를 예측해서는 안 되며, 검사가 필요하면 out_of_range를 던지는 **at**()이나 gsl::**string**_span을 사용하도록 설명한다. [U09-037] (9.3 문자열 뷰, 인쇄 p.196)

9.4 정규 표현식

정규 표현식은 텍스트 안의 패턴을 간단히 기술하고 찾는 도구다. [U09-038] (9.4 정규 표현식, 인쇄 p.197-198)

<regex>는 regex 클래스와 검색·매칭·치환·**반복자**를 제공한다. [U09-039] (9.4 정규 표현식, 인쇄 p.197-198)

로우 문자열 리터럴은 백슬래시와 큰따옴표를 직접 표현할 수 있어 **정규 표현식**을 적을 때 편리하다. [U09-040] (9.4 정규 표현식, 인쇄 p.197-198)

일반 문자열에서는 백슬래시를 두 번 적어야 한다. [U09-041] (9.4 정규 표현식, 인쇄 p.197-198)

로우 문자열로 기술한 원문 우편번호 패턴 (원문 발췌)

```
regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 패턴: XXddddd-dddd와 그 변종
```

[U09-042] (9.4 정규 표현식, 인쇄 p.197)

기능	원문에서의 역할
regex_match() [U09-043] (9.4 정규 표현식, 인쇄 p.197-198)	입력 전체를 **정규 표현식**과 매칭 [U09-044] (9.4 정규 표현식, 인쇄 p.197-198)
regex_search() [U09-045] (9.4 정규 표현식, 인쇄 p.197-198)	입력 안에서 매칭되는 부분 검색 [U09-046] (9.4 정규 표현식, 인쇄 p.197-198)
regex_replace() [U09-047] (9.4 정규 표현식, 인쇄 p.197-198)	매칭되는 부분을 찾아 교체 [U09-048] (9.4 정규 표현식, 인쇄 p.197-198)
regex_iterator [U09-049] (9.4 정규 표현식, 인쇄 p.197-198)	매치와 부분 매치를 찾으며 순회 [U09-050] (9.4 정규 표현식, 인쇄 p.197-198)
regex_token_iterator [U09-051] (9.4 정규 표현식, 인쇄 p.197-198)	매치되지 않는 부분 순회 [U09-052] (9.4 정규 표현식, 인쇄 p.197-198)

9.4.1 검색

regex_search(line,matches,pat)은 line에서 pat에 맞는 부분을 찾아 matches에 저장하고, 없으면 false를 반환한다. [U09-053] (9.4.1 검색, 인쇄 p.198-199)

smatch는 **string**의 매치 결과들을 저장하는 **컨테이너**다. [U09-054] (9.4.1 검색, 인쇄 p.198-199)

matches[0]은 전체 매치이고 이후 항목은 괄호로 지정한 부분 패턴에 대응한다. [U09-055] (9.4.1 검색, 인쇄 p.198-199)

선택적인 부분 패턴을 이용할 때는 항목의 존재와 matched 상태를 확인한다. [U09-056] (9.4.1 검색, 인쇄 p.198-199)

파일에서 전체 매치와 부분 매치 찾기 (원문 발췌)

```
void use()
{
    ifstream in("file.txt");    // 입력 파일
    if (!in)                    // 파일이 열렸는지 확인
        cerr << "파일 없음\n";

    regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 코드

    int lineno = 0;
    for (string line; getline(in,line); ) {
        ++lineno;
        smatch matches;         // 매치된 문자열 저장
        if (regex_search(line , matches, pat)) {
            cout << lineno << ": " << matches[0] << '\n'; // 전체 매치
            if (1<matches.size() && matches[1].matched) // 부분 패턴이 존재하고
                                                        // 그 부분 패턴이 매치됐으면
                cout << "\t: " << matches[1] << '\n';     // 부분 매치
        }
    }
}
```

[U09-057] (9.4.1 검색, 인쇄 p.198-199)

여러 행에 걸친 패턴에도 개행 문자를 포함할 수 있지만 그런 입력을 한 행씩만 읽어서는 안 된다. [U09-058] (9.4.1 검색, 인쇄 p.199)

regex는 패턴을 효율적으로 실행할 수 있는 상태 기계로 실행 시간에 컴파일한다고 설명한다. [U09-059] (9.4.1 검색, 인쇄 p.199)

9.4.2 정규 표현식 표기법

원문의 기본 표기법은 ECMAScript의 ECMA 표준을 따른다. [U09-060] (9.4.2 정규 표현식 표기법, 인쇄 p.199-200)

특수 문자는 보통 문자와 다른 역할을 갖는다. [U09-061] (9.4.2 정규 표현식 표기법, 인쇄 p.199-200)

괄호는 부분 패턴을 만들고, 대괄호는 문자 클래스를, 중괄호는 반복 횟수를 지정한다. [U09-062] (9.4.2 정규 표현식 표기법, 인쇄 p.199-200)

점은 문자 하나, 세로줄은 대안, ^와 $는 각각 행의 시작과 끝을 나타낸다. ^는 문자 클래스의 반전에도 쓰인다. [U09-063] (9.4.2 정규 표현식 표기법, 인쇄 p.199-200)

반복 표기	의미
{n} [U09-064] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	정확히 n번 [U09-065] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)
{n,} [U09-066] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	n번 이상 [U09-067] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)
{n,m} [U09-068] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	n번 이상 m번 이하 [U09-069] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)
* [U09-070] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	0번 이상 [U09-071] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)
+ [U09-072] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	1번 이상 [U09-073] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)
? [U09-074] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)	0번 또는 1번 [U09-075] (9.4.2 정규 표현식 표기법, 인쇄 p.199-206)

접미 반복이 없으면 패턴은 기본적으로 한 번 등장한다. [U09-076] (9.4.2 정규 표현식 표기법, 인쇄 p.201-202)

반복 표기 뒤의 ?는 가장 짧은 매치를 고르는 지연 또는 비욕심쟁이 매칭을 뜻한다. [U09-077] (9.4.2 정규 표현식 표기법, 인쇄 p.201-202)

기본의 욕심쟁이 매칭은 가장 긴 매치를 찾는다. [U09-078] (9.4.2 정규 표현식 표기법, 인쇄 p.201-202)

원문 abababe에서 (ab)+는 전체를, (ab)+?는 첫 ab를 매치한다고 설명한다. [U09-079] (9.4.2 정규 표현식 표기법, 인쇄 p.201-202)

문자 클래스에는 alnum·alpha·blank·cntrl·digit·graph·lower·print·punct·space·upper·xdigit 등 이름이 있다. [U09-080] (9.4.2 정규 표현식 표기법, 인쇄 p.202-203)

이름을 [: :]로 감싼 뒤 문자 클래스의 [ ] 안에서 사용한다. [U09-081] (9.4.2 정규 표현식 표기법, 인쇄 p.202-203)

숫자·공백·단어 문자의 축약어는 각각 \d·\s·\w이고, \D·\S·\W는 각각 그 여집합이다. [U09-082] (9.4.2 정규 표현식 표기법, 인쇄 p.202-203)

원문은 비표준인 \l·\u·\L·\U도 구분하고 이식성을 위해 이름을 사용하라고 조언한다. [U09-083] (9.4.2 정규 표현식 표기법, 인쇄 p.202-203)

식별자 예는 첫 문자가 밑줄 또는 알파벳이고, 이후가 밑줄·알파벳·숫자의 시퀀스라는 조건을 설명한다. [U09-084] (9.4.2 정규 표현식 표기법, 인쇄 p.203-204)

alpha와 alnum에 밑줄이 포함되지 않는다는 점, [:alpha:]를 문자 클래스 바깥에 적으면 의도가 달라진다는 점을 잘못된 패턴과 비교한다. [U09-085] (9.4.2 정규 표현식 표기법, 인쇄 p.203-204)

입력 전체를 검사하는 함수에는 regex_match()를 사용한다. [U09-086] (9.4.2 정규 표현식 표기법, 인쇄 p.203-204)

group은 괄호로 구분되어 sub_match에 저장될 수 있다. [U09-087] (9.4.2 정규 표현식 표기법, 인쇄 p.205-206)

그룹을 묶되 부분 매치를 저장할 필요가 없으면 (?: 형태를 사용한다. [U09-088] (9.4.2 정규 표현식 표기법, 인쇄 p.205-206)

원문은 그룹 개수·중첩·반복에 따른 저장과 \1로 첫 그룹을 다시 참조하는 태그 예를 비교한다. [U09-089] (9.4.2 정규 표현식 표기법, 인쇄 p.205-206)

태그 사이를 .*로 탐욕적으로 잡으면 첫 <부터 마지막 >까지 걸칠 수 있으므로, 의도한 짧은 부분을 찾는 데 .*?를 사용한다고 설명한다. [U09-090] (9.4.2 정규 표현식 표기법, 인쇄 p.205-206)

[원문 확인 필요] 일부 식별자 검사 코드와 패턴에서 본문 설명과 기호의 대응을 다시 확인해야 한다. [U09-091] (9.4.2 정규 표현식 표기법, 인쇄 p.200,204-205)

이 교재에서는 판독에 의심이 있는 코드의 기호를 임의로 고쳐 재작성하지 않고, 해당 페이지에서 명시된 규칙을 중심으로 정리했다. [U09-092] (9.4.2 정규 표현식 표기법, 인쇄 p.200,204-205)

9.4.3 반복자

regex_iterator는 시퀀스에서 패턴을 차례로 찾아 순회한다. [U09-093] (9.4.3 반복자, 인쇄 p.206-207)

string에 대해 사용하는 원문 s**regex_iterator** 예는 공백 뒤의 단어를 부분 매치로 출력한다. [U09-094] (9.4.3 반복자, 인쇄 p.206-207)

정규 표현식 반복자로 단어 찾기 (원문 발췌)

```
void test()
{
    string input = "aa as; asd ++e^asdf asdfg";
    regex pat {R"(\s+(\w+))"};
    for (sregex_iterator p(input.begin(),input.end(),pat);
         p!=sregex_iterator{}; ++p)
        cout << (*p)[1] << '\n';
}
```

[U09-095] (9.4.3 반복자, 인쇄 p.206)

원문의 첫 패턴 출력은 as·asd·asdfg다. [U09-096] (9.4.3 반복자, 인쇄 p.206-207)

앞 공백이 없는 aa는 빠진다. [U09-097] (9.4.3 반복자, 인쇄 p.206-207)

패턴을 R"((\w+))"로 단순화한 다음 페이지의 출력은 aa·as·asd·e·asdf·asdfg다. [U09-098] (9.4.3 반복자, 인쇄 p.206-207)

이 결과는 원문에 제시된 목록만 옮긴 것이다. [U09-099] (9.4.3 반복자, 인쇄 p.206-207)

regex_iterator는 양방향 **반복자**를 요구하므로 입력 **반복자**만 제공하는 istream을 직접 순회할 수 없다고 설명한다. [U09-100] (9.4.3 반복자, 인쇄 p.206-207)

쓰기는 지원하지 않고 끝 표시는 기본 생성한 **regex_iterator**{}를 쓴다. [U09-101] (9.4.3 반복자, 인쇄 p.206-207)

9.5 조언

문자 시퀀스를 저장할 때 **string**을 사용하고 C 스타일 함수 대신 문자열 연산을 사용한다. [U09-102] (9.5 조언, 인쇄 p.207-208)

부분 문자열의 복사는 substr(), 교체는 replace()로 표현한다. [U09-103] (9.5 조언, 인쇄 p.207-208)

구간 검사가 필요할 때 **at**()을 사용한다. [U09-104] (9.5 조언, 인쇄 p.207-208)

여러 저장 형태의 문자 시퀀스를 읽는 함수는 **string_view**를 사용하고 소유·수명 문제를 기억한다. [U09-105] (9.5 조언, 인쇄 p.207-208)

문자열 리터럴의 s와 문자열 뷰 리터럴의 sv를 구별한다. [U09-106] (9.5 조언, 인쇄 p.207-208)

패턴은 로우 문자열로 명료하게 표현하고 전체 매칭과 부분 검색을 구별한다. [U09-107] (9.5 조언, 인쇄 p.207-208)

필요한 부분 패턴만 저장하고 욕심쟁이·지연 매칭의 차이를 확인한다. [U09-108] (9.5 조언, 인쇄 p.207-208)

주요 비교표

문자열 저장과 문자열 뷰

기준	string	string_view
역할 [U09-109] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	문자 시퀀스 저장·조작 [U09-110] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	여러 저장 형태의 문자 시퀀스 읽기 [U09-111] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)
소유 [U09-112] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	문자를 직접 관리 [U09-113] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	문자를 소유하지 않음 [U09-114] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)
수정 [U09-115] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	인덱싱·replace 등으로 변경 [U09-116] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	읽기 전용 [U09-117] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)
수명 [U09-118] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	자기 **객체**의 문자열 [U09-119] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)	가리키는 문자 영역의 수명에 의존 [U09-120] (문자열 저장과 문자열 뷰, 인쇄 p.190-196)

핵심 용어 표

용어	원문 정의·설명
string	**string**은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. [U09-001] (9.2 문자열, 인쇄 p.190,192)
짧은 문자열 최적화	**짧은 문자열 최적화**는 짧은 값을 **객체** 자체에 두고 긴 값에만 자유 저장소를 이용한다. [U09-002] (9.2.1 string 구현, 인쇄 p.193-194)
string_view	**string_view**는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있으며, 문자를 소유하지 않는다. [U09-121] (9.3 문자열 뷰, 인쇄 p.194-195)
정규 표현식	**정규 표현식**은 텍스트 안의 패턴을 간단히 기술하고 찾는 도구다. [U09-038] (9.4 정규 표현식, 인쇄 p.197-198)
smatch	**smatch**는 **string**의 매치 결과들을 저장하는 **컨테이너**다. [U09-054] (9.4.1 검색, 인쇄 p.198-199)
regex_iterator	**regex_iterator**는 시퀀스에서 패턴을 차례로 찾아 순회한다. [U09-093] (9.4.3 반복자, 인쇄 p.206-207)

장 요약

요약 기준: 9.5 조언

문자 시퀀스를 저장할 때 **string**을 사용하고 C 스타일 함수 대신 문자열 연산을 사용한다. [U09-102] (9.5 조언, 인쇄 p.207-208)

부분 문자열의 복사는 substr(), 교체는 replace()로 표현한다. [U09-103] (9.5 조언, 인쇄 p.207-208)

구간 검사가 필요할 때 **at**()을 사용한다. [U09-104] (9.5 조언, 인쇄 p.207-208)

여러 저장 형태의 문자 시퀀스를 읽는 함수는 **string_view**를 사용하고 소유·수명 문제를 기억한다. [U09-105] (9.5 조언, 인쇄 p.207-208)

원문 확인 메모

cat의 문자열 생성 표기를 원문 그대로 보존했다. 일부 정규 표현식 코드 기호는 원문 확인 필요로 표시하고 임의로 수정하지 않았다.

제10장. 입력과 출력

(제10장. 입력과 출력, 인쇄 p.209-232)

학습 목표·범위

입출력 스트림 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. [U10-001] (10.1 소개, 인쇄 p.210-211)

cout은 표준 출력, cerr은 에러 보고 스트림이다. [U10-002] (10.2 출력, 인쇄 p.211,213)

핵심 개념·동작 절차·원문 예제

10.1 소개

입출력 스트림 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. [U10-001] (10.1 소개, 인쇄 p.210-211)

ostream은 **타입** 있는 값을 문자 또는 바이트의 스트림으로 바꾸고, istream은 반대 방향으로 문자 스트림을 **타입** 있는 **객체**로 변환한다. [U10-003] (10.1 소개, 인쇄 p.210-211)

이 연산은 **타입** 안전성과 **타입** 민감성을 갖고 사용자 정의 **타입**에도 확장할 수 있다. [U10-004] (10.1 소개, 인쇄 p.210-211)

스트림은 표준 입력·출력, 파일, **string** 버퍼에 적용된다. [U10-005] (10.1 소개, 인쇄 p.210-211)

소멸자가 자신이 소유한 버퍼와 파일 **핸들** 같은 **자원**을 해제하므로 **RAII**의 한 예다. [U10-006] (10.1 소개, 인쇄 p.210-211)

이진 입출력·여러 문자 **타입**·로케일·고급 버퍼링도 지원하지만 여기서는 상세히 다루지 않는다고 범위를 정한다. [U10-007] (10.1 소개, 인쇄 p.210-211)

10.2 출력

<<는 ostream에 값을 내보내는 연산이다. [U10-008] (10.2 출력, 인쇄 p.211-213)

cout은 표준 출력, cerr은 에러 보고 스트림이다. [U10-009] (10.2 출력, 인쇄 p.211-213)

cout에 정수 10을 쓰면 기본적으로 정수의 바이트 표현 대신 문자 1과 0으로 변환하여 출력한다. [U10-010] (10.2 출력, 인쇄 p.211-213)

한 출력 표현식의 결과를 다음 <<에 연결할 수 있어 관련 항목을 한 문장에 묶을 수 있다. [U10-011] (10.2 출력, 인쇄 p.211-213)

문자 상수를 출력하면 문자가 출력되지만 char 값을 int로 변환하여 출력하면 정수 값이 출력된다. [U10-012] (10.2 출력, 인쇄 p.212-213)

원문은 문자 a·정수에 저장한 b·문자 c를 출력하는 예에서, 사용한 ASCII 인코딩을 전제로 a98c가 나온다고 명시한다. [U10-013] (10.2 출력, 인쇄 p.212-213)

이 결과를 모든 인코딩에 관한 주장으로 넓히지 않는다. [U10-014] (10.2 출력, 인쇄 p.212-213)

10.3 입력

>>는 istream에서 값을 가져오는 연산이며 **cin**은 표준 입력 스트림이다. [U10-015] (10.3 입력, 인쇄 p.213-215)

오른쪽 피연산자의 **타입**이 어떤 입력을 받아 어디에 저장하는지를 결정한다. [U10-016] (10.3 입력, 인쇄 p.213-215)

정수와 부동소수점 입력을 연결해서 읽을 수 있다. [U10-017] (10.3 입력, 인쇄 p.213-215)

정수 입력은 숫자가 아닌 문자를 만나면 끝나고, >>는 기본적으로 앞 공백을 지나친다. [U10-018] (10.3 입력, 인쇄 p.213-215)

string에 대한 >>는 공백에서 읽기를 끝내므로 Eric Bloodaxe를 입력해도 첫 단어 Eric만 읽는다. [U10-019] (10.3 입력, 인쇄 p.214-215)

한 행 전체가 필요하면 getline()을 사용한다. [U10-020] (10.3 입력, 인쇄 p.214-215)

행을 마친 개행 문자는 버리고 다음 행을 읽을 수 있는 상태가 된다. [U10-021] (10.3 입력, 인쇄 p.214-215)

문자열은 입력에 따라 늘어나므로 최대 길이를 미리 계산할 필요가 없다고 설명한다. [U10-022] (10.3 입력, 인쇄 p.214-215)

한 행 전체를 읽는 getline (원문 발췌)

```
void hello_line()
{
    cout << "이름을 입력하세요\n";
    string str;
    getline(cin,str);
    cout << "Hello, " << str << "!\n";
}
```

[U10-023] (10.3 입력, 인쇄 p.215)

원문은 문자 단위 조작보다 형식화된 입출력이 대체로 오류 가능성이 낮고 효율적이며 코드도 짧다고 설명한다. [U10-024] (10.3 입력, 인쇄 p.215)

istream은 메모리 관리와 구간 검사를 맡고, 메모리 안에서 형식화를 수행할 때는 **문자열 스트림**을 사용한다. [U10-025] (10.3 입력, 인쇄 p.215)

10.4 입출력 상태

스트림은 연산의 성공 여부를 나타내는 상태를 보유한다. [U10-026] (10.4 입출력 상태, 인쇄 p.215-217)

is>>i는 is의 참조를 반환하고, 이를 조건으로 검사하면 다음 연산을 수행할 수 있는지를 확인한다. [U10-027] (10.4 입출력 상태, 인쇄 p.215-217)

정수가 아닌 입력이나 입력 끝에서 반복을 멈추는 구조다. [U10-028] (10.4 입출력 상태, 인쇄 p.215-217)

입력 상태를 루프 조건으로 사용 (원문 발췌)

```
vector<int> read_ints(istream& is)
{
    vector<int> res;
    for (int i; is>>i; )
        res.push_back(i);
    return res;
}
```

[U10-029] (10.4 입출력 상태, 인쇄 p.215-216)

입출력 상태에는 형식화 정보·에러 상태·버퍼링 종류 등 작업에 필요한 정보가 들어간다. [U10-030] (10.4 입출력 상태, 인쇄 p.216)

사용자는 에러를 알리거나 회복 가능한 실패를 무시하기 위해 상태를 설정할 수 있다. [U10-031] (10.4 입출력 상태, 인쇄 p.216)

원문의 종료 표식 버전은 eof()면 정상 종료하고, 정수 입력 fail()이면 clear()로 상태를 리셋하고 unget()으로 되돌린 뒤 문자열 표식인지 확인한다. [U10-032] (10.4 입출력 상태, 인쇄 p.216)

[원문 확인 필요] 종료 표식 버전은 입력 스트림 매개**변수** is를 받지만 뒷부분의 문자열 읽기와 실패 상태 설정에는 **cin**을 사용한다. [U10-033] (10.4 입출력 상태, 인쇄 p.216-217)

이 교재에서는 일반화된 다른 코드로 고치지 않고 원문 설명의 흐름과 해당 표기를 구분해 둔다. [U10-034] (10.4 입출력 상태, 인쇄 p.216-217)

종료 표식 호출 예는 read_ints(**cin**,"stop")이다. [U10-035] (10.4 입출력 상태, 인쇄 p.216-217)

10.5 사용자 정의 타입의 입출력

사용자 정의 **타입**에도 의미 있는 텍스트 표현이 있다면 <<와 >>를 정의할 수 있다. [U10-036] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

ostream이나 istream 자체를 수정할 필요는 없다. [U10-037] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

Entry의 출력은 {"이름", 번호} 형식을 사용하고 스트림 참조를 반환한다. [U10-038] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

입력은 형식 검사와 에러 처리가 필요해 더 복잡하다. [U10-039] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

사용자 정의 타입의 입출력 연산 (1/2) (원문 발췌)

```
struct Entry {
    string name;
    int number;
};

ostream& operator<<(ostream& os, const Entry& e)
{
    return os << "{\"" << e.name << "\", " << e.number << "}";
}

istream& operator>>(istream& is, Entry& e)
    // { "이름" ,번호 } 쌍 읽기. { " " , 와 } 로 형식화됨
{
    char c, c2;
    if (is>>c && c=='{' && is>>c2 && c2=='"') { // { "로 시작
        string name;                            // string의 기본 값은 빈 문자열: ""
        while (is.get(c) && c!='"')              // " 이전의 모든 문자는 이름으로 저장
            name+=c;

        if (is>>c && c==',') {
            int number = 0;
            if (is>>number>>c && c=='}') {        // 번호와 } 읽기
                e = {name, number};              // 항목에 대입
                return is;
            }
        }
    }
    is.setstate(ios_base::failbit);              // 스트림에 실패를 기록함
    return is;
```

[U10-040] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

사용자 정의 타입의 입출력 연산 (2/2) (원문 발췌)

```
}
```

[U10-041] (10.5 사용자 정의 타입의 입출력, 인쇄 p.217-219)

조건에서 is>>c를 검사하면 읽기의 성공 여부도 확인한다. >>는 앞 공백을 지나치지만 get(c)는 그렇지 않으므로, 이름 바깥 공백은 건너뛰고 따옴표 안 공백은 이름에 보존한다. [U10-042] (10.5 사용자 정의 타입의 입출력, 인쇄 p.218-219)

잘못된 형식이면 failbit를 설정하여 호출 측의 스트림 조건에 실패를 전달한다. [U10-043] (10.5 사용자 정의 타입의 입출력, 인쇄 p.218-219)

원문은 John Marwood Cleese와 Michael Edward Palin을 공백 포함 이름의 예로 제시한다. [U10-044] (10.5 사용자 정의 타입의 입출력, 인쇄 p.218-219)

10.6 형식화

조정자는 입력·출력 형식을 제어한다. [U10-045] (10.6 형식화, 인쇄 p.219-221)

표준 **조정자**는 <ios>·<istream>·<ostream> 등에 있고 인자를 받는 **조정자**는 <iomanip>에 있다. [U10-046] (10.6 형식화, 인쇄 p.219-221)

정수의 기본 10진 출력 외에 hex와 oct로 진법을 지정할 수 있다. [U10-047] (10.6 형식화, 인쇄 p.219-221)

정수 출력 진법을 바꾸는 조정자 (원문 발췌)

```
cout << 1234 << ',' << hex << 1234 << ',' << oct << 1234 << '\n'; // 1234,4d2,2322 출력
```

[U10-048] (10.6 형식화, 인쇄 p.219)

부동소수점 형식	원문의 표현 방식	정밀도의 의미
default**float** [U10-049] (10.6 형식화, 인쇄 p.219-221)	값을 잘 보존하도록 구현체가 스타일 선택 [U10-050] (10.6 형식화, 인쇄 p.219-221)	최대 자릿수 [U10-051] (10.6 형식화, 인쇄 p.219-221)
scientific [U10-052] (10.6 형식화, 인쇄 p.219-221)	소수점 앞 한 자리와 지수 [U10-053] (10.6 형식화, 인쇄 p.219-221)	소수점 다음 최대 자릿수 [U10-054] (10.6 형식화, 인쇄 p.219-221)
fixed [U10-055] (10.6 형식화, 인쇄 p.219-221)	정수부·소수점·소수 부분 [U10-056] (10.6 형식화, 인쇄 p.219-221)	소수점 다음 최대 자릿수 [U10-054] (10.6 형식화, 인쇄 p.219-221)

원문은 hex**float**를 이용한 16진 부동소수점 표기도 제시한다. [U10-057] (10.6 형식화, 인쇄 p.220-221)

부동소수점 값은 버림이 아닌 반올림으로 출력한다. [U10-058] (10.6 형식화, 인쇄 p.220-221)

precision()은 정수 값의 출력에 영향을 주지 않는다. [U10-059] (10.6 형식화, 인쇄 p.220-221)

precision과 부동소수점 형식 조정은 지속적이므로 이후 출력에도 영향을 주며, 다음 출력 한 번에만 적용된다고 가정해서는 안 된다. [U10-060] (10.6 형식화, 인쇄 p.220-221)

10.7 파일 스트림

<fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다. [U10-061] (10.7 파일 스트림, 인쇄 p.221-222)

파일을 연 다음에는 스트림 상태를 확인하여 실제로 연결되었는지 확인해야 한다. [U10-062] (10.7 파일 스트림, 인쇄 p.221-222)

성공한 **파일 스트림**은 일반적인 istream·ostream처럼 사용할 수 있다. [U10-063] (10.7 파일 스트림, 인쇄 p.221-222)

파일 스트림을 연 뒤 상태 확인 (원문 발췌)

```
ofstream ofs {"target"};    // "o"는 "출력(output)"을 의미
if (!ofs)
    error("'target'을 쓰기용으로 열 수 없음");

ifstream ifs {"source"};    // "i"는 "입력(input)"을 의미
if (!ifs)
    error("'source'를 읽기용으로 열 수 없음 ");
```

[U10-064] (10.7 파일 스트림, 인쇄 p.221)

파일 스트림은 소유한 **자원**을 **소멸자**로 정리한다. [U10-065] (10.7 파일 스트림, 인쇄 p.221-222,231)

원문의 조언은 **파일 스트림**을 복사하지 않는 것이다. [U10-066] (10.7 파일 스트림, 인쇄 p.221-222,231)

세밀한 위치 제어나 열기 모드는 이 책의 범위를 벗어나며, 파일 이름과 파일 시스템 조작은 10.10으로 연결한다. [U10-067] (10.7 파일 스트림, 인쇄 p.221-222,231)

10.8 문자열 스트림

<sstream>의 i**string**stream은 **string**에서 읽고 o**string**stream은 **string**에 쓰며 **string**stream은 둘 다 한다. [U10-068] (10.8 문자열 스트림, 인쇄 p.222-223)

o**string**stream에 형식화된 출력을 수행하고 str()로 저장된 문자열을 얻는 예는 GUI 등에 표시할 문자열을 만드는 용도다. [U10-069] (10.8 문자열 스트림, 인쇄 p.222-223)

반대로 입력 문자열을 i**string**stream에 넣어 형식화된 입력을 수행할 수 있다. [U10-070] (10.8 문자열 스트림, 인쇄 p.222-223)

문자열 스트림으로 일반적인 타입 변환 (원문 발췌)

```
template<typename Target =string, typename Source =string>
Target to(Source arg)    // Source를 Target으로 변환
{
    stringstream interpreter;
    Target result;

    if (!(interpreter << arg)                  // arg를 스트림에 출력
        || !(interpreter >> result)           // 스트림에서 result를 읽기
        || !(interpreter >> std::ws).eof())    // 스트림에 남은 것이 있는가?
        throw runtime_error{"to<>() failed"};

    return result;
}
```

[U10-071] (10.8 문자열 스트림, 인쇄 p.223)

to()는 Source를 스트림에 출력하고 Target을 읽는 두 작업을 결합하며, 변환 후 공백 이외의 내용이 남으면 실패로 처리한다. [U10-072] (10.8 문자열 스트림, 인쇄 p.223)

원문은 문자열로 표현 가능한 **타입** 사이의 일반적인 변환 사례로 설명한다. [U10-073] (10.8 문자열 스트림, 인쇄 p.223)

함수 **템플릿** 인자가 추론 가능하거나 기본 값이 있으면 생략할 수 있으므로, 기본 Target=**string**과 Source 추론을 통해 to(1.2)처럼 쓸 수 있다. [U10-074] (10.8 문자열 스트림, 인쇄 p.223)

10.9 C 스타일 입출력

C++ **표준 라이브러리**는 printf·scanf 계열도 지원한다. [U10-075] (10.9 C 스타일 입출력, 인쇄 p.223-224)

원문은 대개 **타입**과 보안 면에서 안전하지 않고 입력을 안전하고 편리하게 구현하기 어려우며 사용자 정의 **타입**을 지원하지 않아 권장하지 않는다. [U10-076] (10.9 C 스타일 입출력, 인쇄 p.223-224)

C 스타일 입출력을 사용하지 않으면서 성능을 고려할 때 다음 호출을 제시한다. [U10-077] (10.9 C 스타일 입출력, 인쇄 p.223-224)

> **C 스타일 입출력과의 동기화 설정** (원문 발췌)
>
> `ios_base::sync_with_stdio(false); // 큰 오버헤드를 피할 수 있음`
> [U10-078] (10.9 C 스타일 입출력, 인쇄 p.224)

10.10 파일 시스템

<filesystem>은 시스템마다 다른 경로·파일 종류·권한·탐색 기능에 일관된 인터페이스를 제공한다. [U10-079] (10.10 파일 시스템, 인쇄 p.224-225)

path는 운영체제의 기본 문자 집합과 관례를 다루는 경로 표현이다. [U10-080] (10.10 파일 시스템, 인쇄 p.224-225)

명령행의 이름이나 문자열에서 만들고 파일 열기에 사용할 수 있다. [U10-081] (10.10 파일 시스템, 인쇄 p.224-225)

실제 사용 전에는 유효성을 검사하지 않으며 무엇이 유효한지는 실행 시스템 관례에 달려 있다. [U10-082] (10.10 파일 시스템, 인쇄 p.224-225)

파일 시스템은 다른 프로그램과 함께 사용된다. [U10-083] (10.10 파일 시스템, 인쇄 p.225)

exists()로 파일 존재를 확인했어도 다음 is_regular_file() 호출 전에 파일이 없어질 수 있다. [U10-084] (10.10 파일 시스템, 인쇄 p.225)

이처럼 두 연산 사이의 외부 상태 변화 가능성을 원문이 경고한다. [U10-085] (10.10 파일 시스템, 인쇄 p.225)

디렉터리 순회와 파일 시스템 예외 (원문 발췌)

```
void print_directory(path p)
try
{
    if (is_directory(p)) {
        cout << p << ":\n";
        for (const directory_entry& x : directory_iterator{p})
            cout << " " << x.path() << '\n';
    }
}
catch (const filesystem_error& ex) {
    cerr << ex.what() << '\n';
}
```

[U10-086] (10.10 파일 시스템, 인쇄 p.226)

directory_entry는 한 항목이고 directory_iter**at**or는 디렉터리 항목을 순회한다. [U10-087] (10.10 파일 시스템, 인쇄 p.226-227)

하위 디렉터리까지 필요하면 recursive_directory_iter**at**or를 사용한다. [U10-088] (10.10 파일 시스템, 인쇄 p.226-227)

사전 순서 출력이 필요하면 원문은 **path**들을 **vector**에 담아 정렬하는 방법을 제시한다. [U10-089] (10.10 파일 시스템, 인쇄 p.226-227)

순회가 자동으로 정렬된다고 설명하지 않는다. [U10-090] (10.10 파일 시스템, 인쇄 p.226-227)

경로 기능	원문에서의 역할
/= 와 += [U10-091] (10.10 파일 시스템, 인쇄 p.224-230)	파일 이름 구분자를 넣어 연결 / 구분자 없이 연결 [U10-092] (10.10 파일 시스템, 인쇄 p.224-230)
n**at**ive(), **string**(), generic_**string**() [U10-093] (10.10 파일 시스템, 인쇄 p.224-230)	네이티브 표현 또는 제네릭 표현의 문자열 얻기 [U10-094] (10.10 파일 시스템, 인쇄 p.224-230)
filename(), stem(), extension() [U10-095] (10.10 파일 시스템, 인쇄 p.224-230)	파일 이름·꼬리·확장자 부분 다루기 [U10-096] (10.10 파일 시스템, 인쇄 p.224-230)
begin(), end() [U10-097] (10.10 파일 시스템, 인쇄 p.224-230)	경로 구성 요소 시퀀스의 시작·끝 [U10-098] (10.10 파일 시스템, 인쇄 p.224-230)
==, !=, 비교 연산·스트림 입출력 [U10-099] (10.10 파일 시스템, 인쇄 p.224-230)	상등·사전 순서 비교 및 경로 입출력 [U10-100] (10.10 파일 시스템, 인쇄 p.224-230)
u8**path**(s) [U10-101] (10.10 파일 시스템, 인쇄 p.224-230)	UTF-8로 인코딩된 s에서 경로 생성 [U10-102] (10.10 파일 시스템, 인쇄 p.224-230)

원문 표는 value_type이 POSIX에서 char, 윈도우에서 wchar_t라고 설명한다. [U10-103] (10.10 파일 시스템, 인쇄 p.227-228)

string_type은 해당 문자 **타입**의 basic_**string**이며 **const**_iter**at**or는 **const** 양방향 **반복자**, iter**at**or는 그 **별칭**이다. [U10-104] (10.10 파일 시스템, 인쇄 p.227-228)

[원문 확인 필요] extension()의 옮긴이 설명은 ".을 제외한 확장자"로 인쇄되어 있어 원문 표기를 그대로 알리고 임의로 수정하지 않는다. [U10-105] (10.10 파일 시스템, 인쇄 p.227-228)

파일 시스템 연산	원문의 기능 분류
exists, file_size [U10-106] (10.10 파일 시스템, 인쇄 p.224-230)	존재 여부·바이트 수 조회 [U10-107] (10.10 파일 시스템, 인쇄 p.224-230)
copy, copy_file [U10-108] (10.10 파일 시스템, 인쇄 p.224-230)	파일 또는 디렉터리 복사 / 파일 내용 복사 [U10-109] (10.10 파일 시스템, 인쇄 p.224-230)
cre**at**e_directory, cre**at**e_directories [U10-110] (10.10 파일 시스템, 인쇄 p.224-230)	상위 경로가 존재해야 하는 생성 / 중간 경로도 생성 [U10-111] (10.10 파일 시스템, 인쇄 p.224-230)
current_**path** 계열 [U10-112] (10.10 파일 시스템, 인쇄 p.224-230)	현재 작업 디렉터리 처리 [U10-113] (10.10 파일 시스템, 인쇄 p.224-230)
remove [U10-114] (10.10 파일 시스템, 인쇄 p.224-230)	파일 또는 빈 디렉터리 삭제 [U10-115] (10.10 파일 시스템, 인쇄 p.224-230)

원문은 대부분의 연산에 실패하면 filesystem_error를 던지는 기본 버전과 error_code를 추가 인자로 받아 결과를 확인하는 버전이 있다고 설명한다. [U10-116] (10.10 파일 시스템, 인쇄 p.229)

실패가 자주 일어나 예외를 던지는 일이 예외적 상황과 맞지 않는 경우에는 에러 코드를 사용하라고 조언한다. [U10-117] (10.10 파일 시스템, 인쇄 p.229)

권한 등을 받는 세부 오버로드는 여기서 다루지 않는다. [U10-118] (10.10 파일 시스템, 인쇄 p.229)

파일 종류는 is_block_file·is_character_file·is_directory·is_empty·is_fifo·is_other·is_regular_file·is_socket·is_symlink로 질의하고 st**at**us_known으로 상태를 아는지 확인한다. [U10-119] (10.10 파일 시스템, 인쇄 p.229-230)

알 수 없는 종류는 기타로 분류한다. [U10-120] (10.10 파일 시스템, 인쇄 p.229-230)

경로 이름·자연어·인코딩은 복잡하지만 이 추상화가 이식성과 단순함을 제공한다고 정리한다. [U10-121] (10.10 파일 시스템, 인쇄 p.229-230)

10.11 조언

일반 출력은 cout, 에러 보고는 cerr을 사용하고 <<와 >>를 연결해 의도를 명확하게 한다. [U10-122] (10.11 조언, 인쇄 p.230-232)

형식이 잘못된 입력을 고려하고 회복 가능한 입력 실패는 스트림 상태로 처리한다. [U10-123] (10.11 조언, 인쇄 p.230-232)

문자 수준 입력은 꼭 필요할 때만 사용한다. [U10-124] (10.11 조언, 인쇄 p.230-232)

원문은 endl을 피하라고 조언한다. [U10-125] (10.11 조언, 인쇄 p.230-232)

사용자 정의 **타입**에 의미 있는 텍스트 표현이 있으면 입출력 연산자를 제공한다. [U10-126] (10.11 조언, 인쇄 p.230-232)

정밀도와 형식 설정은 이후 출력에도 지속된다. [U10-127] (10.11 조언, 인쇄 p.230-232)

파일 사용 전에 연결 상태를 확인하고 메모리상 형식화에는 **문자열 스트림**을 쓴다. [U10-128] (10.11 조언, 인쇄 p.230-232)

특정 운영체제 인터페이스보다 <filesystem>을 사용한다. [U10-129] (10.11 조언, 인쇄 p.230-232)

주요 비교표

파일과 문자열 스트림의 대응

방향	파일	문자열
읽기 [U10-130] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	ifstream [U10-131] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	i**string**stream [U10-132] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)
쓰기 [U10-133] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	ofstream [U10-134] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	o**string**stream [U10-135] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)
읽기·쓰기 [U10-136] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	fstream [U10-137] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)	**string**stream [U10-138] (파일과 문자열 스트림의 대응, 인쇄 p.221-222)

파일 시스템 에러 보고

방식	보고·확인
기본 버전 [U10-139] (파일 시스템 에러 보고, 인쇄 p.229)	실패 시 filesystem_error 예외 [U10-140] (파일 시스템 에러 보고, 인쇄 p.229)
error_code 인자 버전 [U10-141] (파일 시스템 에러 보고, 인쇄 p.229)	인자로 전달한 에러 코드 확인 [U10-142] (파일 시스템 에러 보고, 인쇄 p.229)

핵심 용어 표

용어	원문 정의·설명
입출력 스트림	**입출력 스트림** 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. [U10-001] (10.1 소개, 인쇄 p.210-211)
cout·cerr	cout은 표준 출력, cerr은 에러 보고 스트림이다. [U10-002] (10.2 출력, 인쇄 p.211,213)
cin	**cin**은 표준 입력 스트림이며, >>는 istream에서 값을 가져오는 연산이다. [U10-143] (10.3 입력, 인쇄 p.213,215)
조정자	**조정자**는 입력·출력 형식을 제어한다. [U10-144] (10.6 형식화, 인쇄 p.219,221)
파일 스트림	<fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다. [U10-061] (10.7 파일 스트림, 인쇄 p.221-222)
문자열 스트림	<sstream>의 i**string**stream은 **string**에서 읽고 o**string**stream은 **string**에 쓰며 **string**stream은 둘 다 한다. [U10-068] (10.8 문자열 스트림, 인쇄 p.222-223)
path	**path**는 운영체제의 기본 문자 집합과 관례를 다루는 경로 표현이다. [U10-145] (10.10 파일 시스템, 인쇄 p.225,229)

장 요약

요약 기준: 10.11 조언

일반 출력은 cout, 에러 보고는 cerr을 사용하고 <<와 >>를 연결해 의도를 명확하게 한다. [U10-122] (10.11 조언, 인쇄 p.230-232)

형식이 잘못된 입력을 고려하고 회복 가능한 입력 실패는 스트림 상태로 처리한다. [U10-123] (10.11 조언, 인쇄 p.230-232)

문자 수준 입력은 꼭 필요할 때만 사용한다. 원문은 endl을 피하라고 조언한다. [U10-146] (10.11 조언, 인쇄 p.230-232)

사용자 정의 **타입**에 의미 있는 텍스트 표현이 있으면 입출력 연산자를 제공한다. [U10-126] (10.11 조언, 인쇄 p.230-232)

원문 확인 메모

종료 표식 예제의 is/cin 혼용과 extension 설명은 원문 확인 필요로 표시했다. 책에서 다루지 않는 상세 열기 모드·인코딩·권한 오버로드는 보충하지 않았다.

제11장. 컨테이너

(제11장. 컨테이너, 인쇄 p.233-252)

학습 목표·범위

컨테이너는 **객체**를 저장하는 일이 주목적인 클래스다. [U11-001] (11.1 소개, 인쇄 p.234)

vector<T>는 T **타입** 요소가 메모리에 연속적으로 저장되는 시퀀스다. [U11-002] (11.2 vector, 인쇄 p.234-235)

핵심 개념·동작 절차·원문 예제

11.1 소개

컨테이너는 **객체**를 저장하는 일이 주목적인 클래스다. [U11-001] (11.1 소개, 인쇄 p.234)

작업에 적합한 **컨테이너**와 기본 연산을 선택하는 일은 프로그램 구성의 중요한 단계다. [U11-003] (11.1 소개, 인쇄 p.234)

이 장은 이름과 전화번호를 담는 Entry의 전화번호부로 표준 **컨테이너**를 비교한다. [U11-004] (11.1 소개, 인쇄 p.234)

예제에서는 불필요한 복잡성을 피하려고 전화번호를 int로 표현한다. [U11-005] (11.1 소개, 인쇄 p.234)

11.2 vector

vector<T>는 T **타입** 요소가 메모리에 연속적으로 저장되는 시퀀스다. [U11-002] (11.2 vector, 인쇄 p.234-235)

전형적인 구현은 첫 요소, 사용 중인 요소의 끝, 할당 공간의 끝을 가리키는 정보와 **할당자**로 구성된다. [U11-006] (11.2 vector, 인쇄 p.234-235)

기본 **할당자**는 new와 delete로 메모리를 관리한다. [U11-007] (11.2 vector, 인쇄 p.234-235)

인덱스는 0에서 시작하며 size()는 실제 요소 수를 알려준다. [U11-008] (11.2 vector, 인쇄 p.235-236)

요소 목록을 주는 {} 초기화와 초기 크기를 지정하는 () 초기화를 구별한다. [U11-009] (11.2 vector, 인쇄 p.235-236)

크기만 지정하면 요소는 **타입**의 기본 값으로 초기화되고, 두 번째 인자로 별도의 초기 값을 지정할 수 있다. [U11-010] (11.2 vector, 인쇄 p.235-236)

구간 for는 **vector**가 제공하는 요소 구간 전체를 순회한다. [U11-011] (11.2 vector, 인쇄 p.235-236)

vector의 구간 for 순회 (원문 발췌)

```
void print_book(const vector<Entry>& book)
{
    for (const auto& x : book) // "auto"에 대해서는 1.4절 참고
        cout << x << '\n';
}
```

[U11-012] (11.2 vector, 인쇄 p.235)

size와 capacity는 다르다. [U11-013] (11.2 vector, 인쇄 p.236-237)

size는 실제 요소 수, capacity는 재할당 없이 저장할 수 있는 슬롯 수다. [U11-014] (11.2 vector, 인쇄 p.236-237)

reserve()는 요소를 위한 공간을 확보하며, 새 공간을 할당하면 기존 요소를 그곳으로 이동시킨다. [U11-015] (11.2 vector, 인쇄 p.236-237)

push_back()은 끝에 요소를 추가하고 size를 1 늘린다. [U11-016] (11.2 vector, 인쇄 p.236-237)

원문의 간략한 Vector 구현은 공간이 부족할 때 용량을 늘려 매번 재할당하는 일을 피한다. [U11-017] (11.2 vector, 인쇄 p.236-237)

용량을 확보한 뒤 요소를 생성하는 push_back (원문 발췌)

```
template<typename T>
void Vector<T>::push_back(const T& t)
{
    if (capacity()<size()+1)        // t를 저장할 공간이 있는지 확인
        reserve(size()==0?8:2*size()); // 용량을 2배로 증가
    new(space) T{t}; // *space 를 t로 초기화
    ++space;
}
```

[U11-018] (11.2 vector, 인쇄 p.237)

발췌 범위: Vector 클래스 선언은 생략하고, 원문의 push_back 함수 정의 전체를 발췌했습니다.

성능 향상을 기대한다는 이유만으로 reserve()를 호출할 필요는 없다. [U11-019] (11.2 vector, 인쇄 p.237-238)

원문은 **vector**의 증가 전략을 신뢰하고, 요소 포인터나 **반복자**의 유효성을 위해 재할당을 막아야 할 때 명시적으로 reserve()를 사용하도록 권한다. [U11-020] (11.2 vector, 인쇄 p.237-238)

대입과 초기화로 **vector**를 복사하면 각 요소도 복사되어 독립된 복사본이 생긴다. [U11-021] (11.2 vector, 인쇄 p.237-238)

요소가 많으면 비용이 클 수 있으므로 복사가 바람직하지 않을 때 참조·포인터 또는 이동 연산을 사용한다. [U11-022] (11.2 vector, 인쇄 p.237-238)

11.2.1 요소

표준 **컨테이너**에는 내장 **타입**, 사용자 정의 **타입**, 포인터 등 거의 모든 **타입**을 요소로 사용할 수 있다. [U11-023] (11.2.1 요소, 인쇄 p.238-239)

요소를 삽입하면 값이 **컨테이너** 안으로 복사된다. [U11-024] (11.2.1 요소, 인쇄 p.238-239)

정수 7을 넣으면 요소 자체가 7의 값을 가지며, 다른 곳의 7을 가리키는 참조나 포인터를 저장하는 것이 아니다. [U11-025] (11.2.1 요소, 인쇄 p.238-239)

이 방식은 간결하고 빠르게 접근할 수 있는 저장 구조를 만든다. [U11-026] (11.2.1 요소, 인쇄 p.238-239)

가상 함수에 의존하는 클래스 계층의 **다형적인 동작**을 유지하려면 **객체** 자체 대신 포인터나 스마트 포인터를 요소로 저장한다. [U11-027] (11.2.1 요소, 인쇄 p.238-239)

원문은 **vector**<Shape>로 Circle이나 Smiley를 저장하려는 방식을 피하고, **vector**<**unique_ptr**<Shape>>를 사용하는 형태를 제시한다. [U11-028] (11.2.1 요소, 인쇄 p.238-239)

11.2.2 구간 검사

표준 **vector**의 oper**at**or[]는 구간 검사를 보장하지 않는다. [U11-029] (11.2.2 구간 검사, 인쇄 p.239-240)

book[book.size()]처럼 마지막 요소 다음을 참조하는 코드는 유효한 접근이 아니다. [U11-030] (11.2.2 구간 검사, 인쇄 p.239-240)

at()은 구간 밖 인덱스에 out_of_range를 던진다. [U11-031] (11.2.2 구간 검사, 인쇄 p.239-240)

원문의 Vec는 **vector**의 **생성자**를 상속하고 oper**at**or[]를 **at**()에 연결한다. [U11-032] (11.2.2 구간 검사, 인쇄 p.239-240)

at()에 연결하여 구간 검사하는 Vec (원문 발췌)

```
template<typename T>
class Vec : public std::vector<T> {
public:
    using vector<T>::vector; // (Vec라는 이름으로) vector의 생성자를 사용

    T& operator[](int i) // 구간 검사
        { return vector<T>::at(i); }

    const T& operator[](int i) const // const 객체용 구간 검사; 4.2.1절
        { return vector<T>::at(i); }
};
```

[U11-033] (11.2.2 구간 검사, 인쇄 p.239)

Vec의 구간 밖 접근은 예외를 발생시켜 호출자가 잡을 수 있게 한다. [U11-034] (11.2.2 구간 검사, 인쇄 p.240-241)

잡지 않은 예외가 있으면 프로그램은 잘 정의된 방식으로 종료된다. [U11-035] (11.2.2 구간 검사, 인쇄 p.240-241)

원문은 main()의 try 블록과 기본 예외 **핸들**러에서 cerr로 진단을 출력하는 예도 제시한다. [U11-036] (11.2.2 구간 검사, 인쇄 p.240-241)

구간 검사의 비용은 하드웨어·최적화·접근 방식에 따라 다르므로, 표준은 모든 인덱싱에 검사를 강제하지 않는다. [U11-037] (11.2.2 구간 검사, 인쇄 p.240-241)

일부 구현은 디버그 검사 기능을 제공한다. [U11-038] (11.2.2 구간 검사, 인쇄 p.240-241)

구간 for나 begin()/end() 범위를 이용한 순회는 추가적인 구간 검사 비용 없이 구간 밖 접근을 피한다. [U11-039] (11.2.2 구간 검사, 인쇄 p.241)

코드에서 **at**()을 직접 사용할 수 있다면 Vec를 따로 만들 필요가 없다. [U11-040] (11.2.2 구간 검사, 인쇄 p.241)

11.3 list

list는 이중 연결 리스트다. [U11-041] (11.3 list, 인쇄 p.241-242)

다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다. [U11-042] (11.3 list, 인쇄 p.241-242)

vector처럼 인덱스로 접근하는 대신 처음부터 원하는 요소를 찾는다. [U11-043] (11.3 list, 인쇄 p.241-242)

begin()은 첫 요소를, end()는 마지막 요소 다음을 가리키는 **반복자**를 반환한다. [U11-044] (11.3 list, 인쇄 p.241-242)

반복자로 전화번호부 선형 탐색 (원문 발췌)

```
int get_number(const string& s)
{
    for (auto p = phone_book.begin(); p!=phone_book.end(); ++p)
        if (p->name==s)
            return p->number;
    return 0; // 0 은 "수를 찾지 못함"을 의미
}
```

[U11-045] (11.3 list, 인쇄 p.243)

반복자 p에 대해 *p는 요소를 참조하고 ++p는 다음 요소로 진행한다. [U11-046] (11.3 list, 인쇄 p.243)

p->m은 (*p).m과 같다. [U11-047] (11.3 list, 인쇄 p.243)

insert(p,elem)은 elem을 복사한 요소를 p 앞에 넣으며, p가 end()여도 된다. [U11-048] (11.3 list, 인쇄 p.243)

erase(p)는 p의 요소를 제거하고 소멸시킨다. [U11-049] (11.3 list, 인쇄 p.243)

원문의 전화번호부 예제는 **vector**로도 작성할 수 있고, 짧은 시퀀스에서는 삽입·삭제까지 **vector** 쪽이 더 효율적일 수 있으므로 직관만으로 **list**를 선택하지 않는다. [U11-050] (11.3 list, 인쇄 p.243)

forward_list는 순방향 순회만 제공하는 단일 연결 리스트다. [U11-051] (11.3 list, 인쇄 p.243-244)

이전 요소 포인터가 없어 공간을 아끼며 빈 **객체**의 크기는 포인터 하나다. [U11-052] (11.3 list, 인쇄 p.243-244)

요소 개수도 저장하지 않으므로 개수를 알려면 직접 세어야 한다. [U11-053] (11.3 list, 인쇄 p.243-244)

원문은 이 특성이 필요할 때 선택하도록 한다. [U11-054] (11.3 list, 인쇄 p.243-244)

11.4 map

map은 키와 값의 쌍을 담고 조회에 최적화된 연관 **컨테이너**다. [U11-055] (11.4 map, 인쇄 p.244-245)

원문은 일반적인 구현을 균형 이진 탐색 트리, 특히 레드-블랙 트리로 설명한다. [U11-056] (11.4 map, 인쇄 p.244-245)

연관 배열 또는 딕셔너리라고도 한다. [U11-057] (11.4 map, 인쇄 p.244-245)

키 **타입**의 값으로 인덱싱하면 대응하는 매핑된 **타입**의 값을 얻는다. [U11-058] (11.4 map, 인쇄 p.244-245)

map의 키로 전화번호 조회 (원문 발췌)

```
int get_number(const string& s)
{
    return phone_book[s];
}
```

[U11-059] (11.4 map, 인쇄 p.245)

발췌 범위: 동일 페이지의 map 초기화는 생략하고 get_number 함수 정의 전체를 발췌했습니다.

없는 키에 대한 []는 조회만 하는 연산이 아니다. [U11-060] (11.4 map, 인쇄 p.245)

키를 못 찾으면 값 **타입**의 기본 값을 가진 항목이 삽입된다. [U11-061] (11.4 map, 인쇄 p.245)

전화번호부에서는 int의 기본 값 0이 유효하지 않은 번호를 나타낸다. [U11-062] (11.4 map, 인쇄 p.245)

이런 항목이 들어가는 것을 원치 않으면 [] 대신 **find**()와 insert()를 사용한다. [U11-063] (11.4 map, 인쇄 p.245)

11.5 unordered_map

map의 조회 비용은 요소 수 n에 대해 O(log(n))이다. [U11-064] (11.5 unordered_map, 인쇄 p.245-246)

unordered_map은 해시 테이블을 이용하며 순서 비교 함수를 요구하지 않는다. [U11-065] (11.5 unordered_map, 인쇄 p.245-246)

map과 비슷하게 초기화하고 인덱싱할 수 있지만 요소 사이의 순서를 유지하지 않는다. [U11-066] (11.5 unordered_map, 인쇄 p.245-246)

표준 라이브러리는 **string**·내장 **타입**·여러 라이브러리 **타입**의 기본 해시 함수를 제공한다. [U11-067] (11.5 unordered_map, 인쇄 p.245-246)

사용자 정의 **타입**에는 해시 **함수 객체**를 전달하거나 std::hash를 특수화할 수 있다. [U11-068] (11.5 unordered_map, 인쇄 p.246-248)

원문의 Record 예제는 이름과 product_code의 표준 해시를 배타적 논리합(^)으로 결합한다. [U11-069] (11.5 unordered_map, 인쇄 p.246-248)

좋은 해시 함수에는 데이터에 대한 지식과 실험이 필요할 수 있다. [U11-070] (11.5 unordered_map, 인쇄 p.246-248)

좋은 함수를 쓸 때 큰 **unordered_map**은 **map**보다 훨씬 빠를 수 있지만, 잘못된 해시 함수에서는 최악의 경우가 **map**보다 훨씬 느릴 수 있다. [U11-071] (11.5 unordered_map, 인쇄 p.246-248)

11.6 컨테이너 개요

표준 컨테이너	원문에서 정리한 역할
vector<T> / **list**<T> / **forward_list**<T> [U11-072] (11.6 컨테이너 개요, 인쇄 p.248)	가변 크기 벡터 / 이중 연결 리스트 / 단일 연결 리스트 [U11-073] (11.6 컨테이너 개요, 인쇄 p.248)
deque<T> [U11-074] (11.6 컨테이너 개요, 인쇄 p.248)	양방향 큐 [U11-075] (11.6 컨테이너 개요, 인쇄 p.248)
set<T> / multiset<T> [U11-076] (11.6 컨테이너 개요, 인쇄 p.248)	값만 가진 집합 / 값의 중복을 허용하는 집합 [U11-077] (11.6 컨테이너 개요, 인쇄 p.248)
map<K,V> / multi**map**<K,V> [U11-078] (11.6 컨테이너 개요, 인쇄 p.248)	키와 값의 연관 배열 / 키의 중복을 허용하는 **map** [U11-079] (11.6 컨테이너 개요, 인쇄 p.248)
unordered_map / unordered_multi**map** [U11-080] (11.6 컨테이너 개요, 인쇄 p.248)	해시 조회를 사용하는 **map** / multi**map** [U11-081] (11.6 컨테이너 개요, 인쇄 p.248)
unordered_set / unordered_multiset [U11-082] (11.6 컨테이너 개요, 인쇄 p.248)	해시 조회를 사용하는 set / multiset [U11-083] (11.6 컨테이너 개요, 인쇄 p.248)

컨테이너는 std에 정의되고 <**vector**>, <**list**>, <**map**> 등에서 제공된다. [U11-084] (11.6 컨테이너 개요, 인쇄 p.249-250)

queue·stack·priority_queue는 **컨테이너** 어댑터이며 array와 bitset은 고정 크기 구조다. [U11-085] (11.6 컨테이너 개요, 인쇄 p.249-250)

여러 **컨테이너**는 일관된 표기와 의미를 갖도록 설계되어, 알고리즘을 개별 **컨테이너 타입**과 독립적으로 기술할 수 있다. [U11-086] (11.6 컨테이너 개요, 인쇄 p.249-250)

연산	의미와 주의
value_type [U11-087] (11.6 컨테이너 개요, 인쇄 p.249-250)	요소 **타입** [U11-088] (11.6 컨테이너 개요, 인쇄 p.249-250)
begin()/end(), cbegin()/cend() [U11-089] (11.6 컨테이너 개요, 인쇄 p.249-250)	첫 요소 / 마지막 요소 다음; **const 반복자**도 제공 [U11-090] (11.6 컨테이너 개요, 인쇄 p.249-250)
size()/empty() [U11-091] (11.6 컨테이너 개요, 인쇄 p.249-250)	요소 수 / 비어 있는지 검사 [U11-092] (11.6 컨테이너 개요, 인쇄 p.249-250)
capacity()/reserve(k) [U11-093] (11.6 컨테이너 개요, 인쇄 p.249-250)	새 할당 없는 저장 용량 / 용량 확보 [U11-094] (11.6 컨테이너 개요, 인쇄 p.249-250)
resize(k) [U11-095] (11.6 컨테이너 개요, 인쇄 p.249-250)	요소 수 변경; 추가 요소는 value_type{} [U11-096] (11.6 컨테이너 개요, 인쇄 p.249-250)
[k] / **at**(k) [U11-097] (11.6 컨테이너 개요, 인쇄 p.249-250)	구간 검사 없는 접근 / 구간 밖에서 예외 [U11-098] (11.6 컨테이너 개요, 인쇄 p.249-250)
push_back(x) / emplace_back(a) [U11-099] (11.6 컨테이너 개요, 인쇄 p.249-250)	끝에 x 추가 / 인자로 새 요소를 직접 생성 [U11-100] (11.6 컨테이너 개요, 인쇄 p.249-250)
insert(p,x) / erase(p) [U11-101] (11.6 컨테이너 개요, 인쇄 p.249-250)	p 앞에 삽입 / p 요소 제거 [U11-102] (11.6 컨테이너 개요, 인쇄 p.249-250)

연산	의미와 주의
= / == / < [U11-103] (11.6 컨테이너 개요, 인쇄 p.249-250)	대입 / 요소의 상등성 / 사전 순 비교 [U11-104] (11.6 컨테이너 개요, 인쇄 p.249-250)

각 **컨테이너**에 맞는 연산을 선택해야 한다. [U11-105] (11.6 컨테이너 개요, 인쇄 p.250)

vector는 연속 저장에 따른 저렴한 인덱싱·순회를 제공하지만 중간 삽입·삭제 때 요소 이동이 필요하다. [U11-106] (11.6 컨테이너 개요, 인쇄 p.250)

list는 그 반대의 성격을 갖는다. [U11-107] (11.6 컨테이너 개요, 인쇄 p.250)

빈 경우가 많다면 한 워드만 필요한 **forward_list**를 고려할 수 있다. [U11-108] (11.6 컨테이너 개요, 인쇄 p.250)

배치 연산은 **객체**를 먼저 만들어 복사하는 대신, 전달한 **생성자** 인자로 **컨테이너**의 새 저장 공간에 요소를 직접 만든다. [U11-109] (11.6 컨테이너 개요, 인쇄 p.250)

11.7 조언

표준 **컨테이너**를 익히고 기본 선택으로 **vector**를 고려한다. [U11-110] (11.7 조언, 인쇄 p.251-252)

성능은 직관이 아니라 측정으로 확인한다. [U11-111] (11.7 조언, 인쇄 p.251-252)

순회에는 구간 for나 시작/끝 **반복자**를 사용한다. [U11-112] (11.7 조언, 인쇄 p.251-252)

구간 검사가 필요하면 **at**()을 사용하며 []가 검사한다고 가정하지 않는다. [U11-113] (11.7 조언, 인쇄 p.251-252)

vector의 크기 변경 후 **반복자**를 그대로 사용하지 않는다. [U11-114] (11.7 조언, 인쇄 p.251-252)

포인터·**반복자** 유효성을 위한 재할당 방지와 단순 성능 추측을 구별한다. [U11-115] (11.7 조언, 인쇄 p.251-252)

컨테이너 요소는 값이다. [U11-116] (11.7 조언, 인쇄 p.251-252)

다형적인 동작이 필요하면 포인터를 저장한다. [U11-117] (11.7 조언, 인쇄 p.251-252)

컨테이너 전달은 참조로, 반환은 값으로 한다. [U11-118] (11.7 조언, 인쇄 p.251-252)

많은 데이터의 빠른 조회에는 순서 없는 **컨테이너**, 순서 있는 순회에는 **map**·set 같은 순서 있는 연관 **컨테이너**를 고려한다. [U11-119] (11.7 조언, 인쇄 p.251-252)

크기로 생성할 때 ()와 요소 목록으로 생성할 때 {}를 구별한다. [U11-120] (11.7 조언, 인쇄 p.251-252)

해시 함수의 적절성은 실험으로 확인한다. [U11-121] (11.7 조언, 인쇄 p.251-252)

주요 비교표

map과 unordered_map

비교 기준	map	unordered_map
조회 구조 [U11-122] (map과 unordered_map, 인쇄 p.245-248)	균형 이진 탐색 트리 [U11-123] (map과 unordered_map, 인쇄 p.245-248)	해시 테이블 [U11-124] (map과 unordered_map, 인쇄 p.245-248)
필요한 함수 [U11-125] (map과 unordered_map, 인쇄 p.245-248)	순서 비교 함수; 기본적으로 < [U11-126] (map과 unordered_map, 인쇄 p.245-248)	해시 함수 [U11-127] (map과 unordered_map, 인쇄 p.245-248)
요소 순서 [U11-128] (map과 unordered_map, 인쇄 p.245-248)	순서 있는 시퀀스 [U11-129] (map과 unordered_map, 인쇄 p.245-248)	순서 유지 안 함 [U11-130] (map과 unordered_map, 인쇄 p.245-248)
성능 조건 [U11-131] (map과 unordered_map, 인쇄 p.245-248)	조회 O(log(n)) [U11-132] (map과 unordered_map, 인쇄 p.245-248)	좋은 해시는 큰 데이터에서 빠름; 나쁜 해시는 최악의 경우 훨씬 느림 [U11-133] (map과 unordered_map, 인쇄 p.245-248)

핵심 용어 표

용어	원문 정의·설명
컨테이너	**컨테이너**는 **객체**를 저장하는 일이 주목적인 클래스다. [U11-001] (11.1 소개, 인쇄 p.234)
vector<T>	**vector<T>**는 T **타입** 요소가 메모리에 연속적으로 저장되는 시퀀스다. [U11-002] (11.2 vector, 인쇄 p.234-235)
list	**list**는 이중 연결 리스트이며, 다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다. [U11-134] (11.3 list, 인쇄 p.241,243)
map	**map**은 키와 값의 쌍을 담고 조회에 최적화된 연관 **컨테이너**다. [U11-055] (11.4 map, 인쇄 p.244-245)
unordered_map	**unordered_map**은 해시 테이블을 이용하며 요소 사이의 순서를 유지하지 않는다. [U11-135] (11.5 unordered_map, 인쇄 p.246,248)
at()	**at**()은 구간 밖 인덱스에 out_of_range를 던진다. [U11-031] (11.2.2 구간 검사, 인쇄 p.239-240)

장 요약

요약 기준: 11.7 조언

표준 **컨테이너**를 익히고 기본 선택으로 **vector**를 고려한다. 성능은 직관이 아니라 측정으로 확인한다. [U11-136] (11.7 조언, 인쇄 p.251-252)

순회에는 구간 for나 시작/끝 **반복자**를 사용한다. 구간 검사가 필요하면 **at**()을 사용하며 []가 검사한다고 가정하지 않는다. [U11-137] (11.7 조언, 인쇄 p.251-252)

vector의 크기 변경 후 **반복자**를 그대로 사용하지 않는다. 포인터·**반복자** 유효성을 위한 재할당 방지와 단순 성능 추측을 구별한다. [U11-138] (11.7 조언, 인쇄 p.251-252)

컨테이너 요소는 값이다. **다형적인 동작**이 필요하면 포인터를 저장한다. **컨테이너** 전달은 참조로, 반환은 값으로 한다. [U11-139] (11.7 조언, 인쇄 p.251-252)

제12장. 알고리즘

(제12장. 알고리즘, 인쇄 p.253-274)

학습 목표·범위

입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 **반복자** 쌍으로 나타낸다. [U12-001] (12.1 소개, 인쇄 p.254-255)

반복자는 특정 **타입**의 **객체**이며 저장 구조에 맞는 정보를 가진다. [U12-002] (12.3 반복자 타입, 인쇄 p.259-260)

핵심 개념·동작 절차·원문 예제

12.1 소개

표준 라이브러리 알고리즘은 **컨테이너**에 저장한 요소의 정렬·출력·검색·부분집합 추출 같은 공통 작업을 제공한다. [U12-003] (12.1 소개, 인쇄 p.254-255)

입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 **반복자** 쌍으로 나타낸다. [U12-001] (12.1 소개, 인쇄 p.254-255)

sort는 비교 연산으로 입력 구간을 정렬하고 unique_copy는 이웃한 중복을 복사하지 않는다. [U12-004] (12.1 소개, 인쇄 p.254-255)

기존 출력 위치를 지정하면 그 뒤 요소를 덮어쓰므로 충분한 출력 공간이 있어야 한다. [U12-005] (12.1 소개, 인쇄 p.254-255)

back_inserter는 **컨테이너** 뒤에 삽입하는 **반복자**를 만들어 필요에 따라 **컨테이너**를 확장한다. [U12-006] (12.1 소개, 인쇄 p.254-255)

이 방식은 미리 크기를 정하거나 C 스타일 realloc()으로 직접 관리할 필요를 줄인다. [U12-007] (12.1 소개, 인쇄 p.254-255)

정렬 후 중복 없이 새 list에 삽입 (원문 발췌)

```
list<Entry> f(vector<Entry>& vec)
{
    list<Entry> res;
    sort(vec.begin(),vec.end());
    unique_copy(vec.begin(),vec.end(),back_inserter(res)); // res 뒤에 붙이기
    return res;
}
```

[U12-008] (12.1 소개, 인쇄 p.255)

12.2 반복자의 사용

begin()과 end()는 **컨테이너**에서 **반복자**를 얻는 대표적인 함수다. [U12-009] (12.2 반복자의 사용, 인쇄 p.255-256)

find는 찾은 요소의 **반복자**를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다. [U12-010] (12.2 반복자의 사용, 인쇄 p.255-256)

그러므로 반환값을 끝 **반복자**와 비교하여 검색 성공을 판정한다. [U12-011] (12.2 반복자의 사용, 인쇄 p.255-256)

원문의 **find**_all은 일치하는 값 자체 대신 그 위치를 가리키는 **반복자**들의 **vector**를 반환한다. [U12-012] (12.2 반복자의 사용, 인쇄 p.256-258)

이 **반복자**로 원래 요소를 바꿀 수도 있으므로, 변경할 문자열이나 **컨테이너**는 **const**가 아닌 참조로 받는다. [U12-013] (12.2 반복자의 사용, 인쇄 p.256-258)

처음에는 **string** 전용으로 작성하고 이후 **컨테이너** C와 값 V의 **템플릿**으로 일반화한다. [U12-014] (12.2 반복자의 사용, 인쇄 p.256-258)

반복자 타입 별칭을 사용한 제네릭 find_all (원문 발췌)

```
template<typename T>
using Iterator = typename T::iterator; // T의 반복자

template<typename C, typename V>
vector<Iterator<C>> find_all(C& c, V v) // c에서 v가 나타나는 곳을 모두 찾기
{
    vector<Iterator<C>> res;
    for (auto p = c.begin(); p!=c.end(); ++p)
        if (*p==v)
            res.push_back(p);
    return res;
}
```

[U12-015] (12.2 반복자의 사용, 인쇄 p.258)

typename C::iter**at**or의 typename은 iter**at**or가 값이 아닌 **타입**임을 컴파일러에 알려준다. [U12-016] (12.2 반복자의 사용, 인쇄 p.258-259)

타입 별칭 Iter**at**or<T>는 이 세부사항을 숨긴다. [U12-017] (12.2 반복자의 사용, 인쇄 p.258-259)

알고리즘은 **반복자**로 데이터에 적용되므로 저장 **컨테이너**를 알 필요가 없고, **컨테이너** 역시 어떤 알고리즘이 쓰이는지 알 필요 없이 **반복자**를 제공한다. [U12-018] (12.2 반복자의 사용, 인쇄 p.258-259)

이 분리가 일반적이고 유연한 소프트웨어의 바탕이다. [U12-019] (12.2 반복자의 사용, 인쇄 p.258-259)

12.3 반복자 타입

반복자는 특정 **타입**의 **객체**이며 저장 구조에 맞는 정보를 가진다. [U12-002] (12.3 반복자 타입, 인쇄 p.259-260)

vector에서는 단순 포인터가 합리적인 구현일 수 있고, **vector** 포인터와 인덱스를 함께 저장하면 구간 검사를 할 수 있다. [U12-020] (12.3 반복자 타입, 인쇄 p.259-260)

list에서는 다음 링크를 따라가야 하므로 단순 요소 포인터가 아니라 링크를 가리키는 정보가 필요하다. [U12-021] (12.3 반복자 타입, 인쇄 p.259-260)

구현은 달라도 ++로 다음 요소를 가리키고 *로 요소를 참조하는 공통 의미와 명명 규칙을 따른다. [U12-022] (12.3 반복자 타입, 인쇄 p.259-260)

이러한 요구를 표현한 것이 Iterator **컨셉**이다. [U12-023] (12.3 반복자 타입, 인쇄 p.259-260)

사용자에게 내부 구현 전체가 필요한 것은 아니며 각 **컨테이너**의 iterator·**const**_iterator라는 관례적인 이름을 이용하면 된다. [U12-024] (12.3 반복자 타입, 인쇄 p.259-260)

12.4 스트림 반복자

요소 시퀀스는 **컨테이너**에만 있는 것이 아니다. [U12-025] (12.4 스트림 반복자, 인쇄 p.261)

입력 스트림도 값의 시퀀스를 만들고 출력 스트림도 시퀀스를 받는다. [U12-026] (12.4 스트림 반복자, 인쇄 p.261)

ostream_iterator는 스트림과 출력 **타입**을 지정해 만들며, 역참조 위치에 대입하면 스트림으로 출력한다. ++는 배열 포인터와 같은 사용 방식을 흉내 낸다. [U12-027] (12.4 스트림 반복자, 인쇄 p.261)

istream_iterator는 입력 스트림을 읽기 전용 **컨테이너**처럼 다루게 한다. [U12-028] (12.4 스트림 반복자, 인쇄 p.261)

파일 단어를 정렬하고 중복 없이 출력 (원문 발췌)

```
int main()
{
    string from, to;
    cin >> from >> to; // 원본과 출력 파일 이름 가져오기

    ifstream is {from}; // 파일 "from"의 입력 스트림
    istream_iterator<string> ii {is}; // 스트림에 사용할 입력 반복자
    istream_iterator<string> eos {}; // 입력(의 끝을 나타내는) 보초 값

    ofstream os {to}; // 파일 "to"의 출력 스트림
    ostream_iterator<string> oo {os,"\n"}; // 스트림의 출력 반복자

    vector<string> b {ii,eos}; // b는 입력으로부터 초기화된 벡터
    sort(b.begin(),b.end()); // 버퍼를 정렬

    unique_copy(b.begin(),b.end(),oo); // 버퍼를 출력에 복사, 중복된 값은 버림

    return !is.eof() || !os; // 에러 상태 반환(1.2.1절, 10.4절)
}
```

[U12-029] (12.4 스트림 반복자, 인쇄 p.262)

기본 생성한 istream_iterator는 입력 끝의 보초 값으로 쓰인다. [U12-030] (12.4 스트림 반복자, 인쇄 p.262-263)

ostream_iterator의 두 번째 인자는 출력 값 사이의 구분자다. [U12-031] (12.4 스트림 반복자, 인쇄 p.262-263)

예제는 파일의 단어를 **vector**로 읽고 정렬한 뒤 unique_copy로 중복을 제거해 출력한다. [U12-032] (12.4 스트림 반복자, 인쇄 p.262-263)

원문은 set을 사용해 입력 단계에서 중복을 없애고 순서를 유지하면 sort·unique_copy 대신 copy만 쓸 수 있다는 대안도 제시한다. [U12-033] (12.4 스트림 반복자, 인쇄 p.262-263)

코드 길이를 줄이는 선택은 취향과 가독성을 함께 고려한다. [U12-034] (12.4 스트림 반복자, 인쇄 p.262-263)

12.5 술어

술어는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다. [U12-035] (12.5 술어, 인쇄 p.263-264)

find가 지정 값과 같은 요소를 찾는 데 비해 **find_if**는 전달된 **술어**가 참이 되는 첫 요소를 찾는다. [U12-036] (12.5 술어, 인쇄 p.263-264)

원문의 **map** 예제는 값 부분이 42보다 큰 쌍을 찾는다. [U12-037] (12.5 술어, 인쇄 p.263-264)

비교 기준을 저장한 **함수 객체**와 **람다 표현식** 모두 사용할 수 있다. [U12-038] (12.5 술어, 인쇄 p.263-264)

람다 술어를 전달한 find_if (원문 발췌)

```
auto p = find_if(m.begin(), m.end(), [](const pair<string,int>& r) {
    return r.second>42;
});
```

[U12-039] (12.5 술어, 인쇄 p.264)

발췌 범위: 원문 map 검색 설명 중 람다 호출 블록 전체를 발췌했습니다. m의 정의와 Greater_than 대안은 생략했습니다.

술어는 적용 대상 요소를 변경하면 안 된다. [U12-040] (12.5 술어, 인쇄 p.264)

원문의 Gre**at**er_than은 비교값을 val에 저장하고 **pair**의 second를 이 값과 비교한다. [U12-041] (12.5 술어, 인쇄 p.264)

람다 버전도 같은 비교를 직접 표현한다. [U12-042] (12.5 술어, 인쇄 p.264)

12.6 알고리즘 요약

이 책에서 **표준 알고리즘**은 요소 시퀀스에 적용되는 함수 **템플릿** 연산이다. [U12-043] (12.6 알고리즘 요약, 인쇄 p.264-266)

std에 정의되고 <algorithm>에서 제공된다. [U12-044] (12.6 알고리즘 요약, 인쇄 p.264-266)

컨테이너뿐 아니라 **string**과 내장 배열에도 적용할 수 있다. [U12-045] (12.6 알고리즘 요약, 인쇄 p.264-266)

알고리즘	구간에 대한 역할
for_each [U12-046] (12.6 알고리즘 요약, 인쇄 p.265)	모든 요소에 함수 적용 [U12-047] (12.6 알고리즘 요약, 인쇄 p.265)
find / **find_if** [U12-048] (12.6 알고리즘 요약, 인쇄 p.265)	지정 값 / **술어**를 만족하는 첫 위치 [U12-049] (12.6 알고리즘 요약, 인쇄 p.265)
count / count_if [U12-050] (12.6 알고리즘 요약, 인쇄 p.265)	지정 값 / **술어**에 맞는 요소 개수 [U12-051] (12.6 알고리즘 요약, 인쇄 p.265)
replace / replace_if [U12-052] (12.6 알고리즘 요약, 인쇄 p.265)	지정 값 / **술어**에 맞는 요소를 새 값으로 대체 [U12-053] (12.6 알고리즘 요약, 인쇄 p.265)
copy / copy_if / move [U12-054] (12.6 알고리즘 요약, 인쇄 p.265)	전체 복사 / 조건에 맞는 요소 복사 / 이동 [U12-055] (12.6 알고리즘 요약, 인쇄 p.265)
unique_copy [U12-056] (12.6 알고리즘 요약, 인쇄 p.265)	이웃한 중복을 제외하고 복사 [U12-057] (12.6 알고리즘 요약, 인쇄 p.265)
sort [U12-058] (12.6 알고리즘 요약, 인쇄 p.265)	기본 < 또는 주어진 비교 연산으로 정렬 [U12-059] (12.6 알고리즘 요약, 인쇄 p.265)
equal_range [U12-060] (12.6 알고리즘 요약, 인쇄 p.265)	정렬된 구간에서 같은 값의 부분 구간을 찾음; 기본적으로 이진 탐색 [U12-061] (12.6 알고리즘 요약, 인쇄 p.265)

알고리즘	구간에 대한 역할
merge [U12-062] (12.6 알고리즘 요약, 인쇄 p.265)	정렬된 두 구간을 기본 또는 지정 비교 연산으로 병합 [U12-063] (12.6 알고리즘 요약, 인쇄 p.265)

일부 알고리즘은 요소 값을 변경하지만 입력 시퀀스 자체에 요소를 추가하거나 제거하지는 않는다. [U12-064] (12.6 알고리즘 요약, 인쇄 p.266)

반복자만으로는 소유 **컨테이너**를 알 수 없기 때문이다. [U12-065] (12.6 알고리즘 요약, 인쇄 p.266)

크기를 바꾸려면 back_inserter처럼 **컨테이너**를 아는 수단이나 push_back·erase 같은 **컨테이너** 연산이 필요하다. [U12-066] (12.6 알고리즘 요약, 인쇄 p.266)

원문은 직접 루프를 만들기 전에 잘 설계된 **표준 알고리즘**으로 표현할 수 있는지 살피도록 권한다. [U12-067] (12.6 알고리즘 요약, 인쇄 p.266)

12.7 컨셉(C++20)

이 절은 원문 집필 시점의 Ranges Technical Specification과 C++20에 대한 전망을 설명한다. [U12-068] (12.7 컨셉(C++20), 인쇄 p.266)

원문은 <experimental/ranges>의 **컨셉**을 소개하며 비슷한 기능이 C++20의 std에 들어가기를 기대한다고 쓴다. [U12-069] (12.7 컨셉(C++20), 인쇄 p.266)

이 노트는 그 시점의 명칭과 조건을 그대로 다룬다. [U12-070] (12.7 컨셉(C++20), 인쇄 p.266)

Range는 begin/end 시퀀스의 일반화다. {begin,end}, {begin,n} 또는 {begin,pred}로 구간을 정의할 수 있다. [U12-071] (12.7 컨셉(C++20), 인쇄 p.266-267)

n은 요소 수, pred는 **반복자**가 끝에 도달했는지 판정하는 **술어**다. [U12-072] (12.7 컨셉(C++20), 인쇄 p.266-267)

마지막 방식은 동적으로 정의되거나 무한한 시퀀스도 표현한다. [U12-073] (12.7 컨셉(C++20), 인쇄 p.266-267)

이 구간을 받는 알고리즘은 sort(v)와 같은 호출을 가능하게 한다. [U12-074] (12.7 컨셉(C++20), 인쇄 p.266-267)

핵심 컨셉	원문에서 요구하는 성질
Same / DerivedFrom / ConvertibleTo [U12-075] (12.7 컨셉(C++20), 인쇄 p.267-268)	같은 **타입** / 파생 관계 / 변환 가능 [U12-076] (12.7 컨셉(C++20), 인쇄 p.267-268)
CommonReference / Common [U12-077] (12.7 컨셉(C++20), 인쇄 p.267-268)	공통 참조 **타입** / 공통 **타입**을 가짐 [U12-078] (12.7 컨셉(C++20), 인쇄 p.267-268)
Integral / SignedIntegral / UnsignedIntegral [U12-079] (12.7 컨셉(C++20), 인쇄 p.267-268)	정수 / 부호 있는 정수 / 부호 없는 정수 [U12-080] (12.7 컨셉(C++20), 인쇄 p.267-268)
Assignable / SwappableWith / Swappable [U12-081] (12.7 컨셉(C++20), 인쇄 p.267-268)	대입 가능 / 두 **타입** 값의 교환 가능 / 같은 **타입** 사이의 교환 가능 [U12-082] (12.7 컨셉(C++20), 인쇄 p.267-268)

Common은 여러 **타입**을 공통 **타입** C로 변환한 뒤에도 비교가 합리적이어야 하는 알고리즘을 기술하는 데 쓰인다. [U12-083] (12.7 컨셉(C++20), 인쇄 p.268)

원문은 **string**과 char*, int와 double의 비교를 들고 **string**과 int는 비교하지 않는다고 구별한다. [U12-084] (12.7 컨셉(C++20), 인쇄 p.268)

라이브러리가 적당한 정의를 제공하지 않는 경우가 아니라면 common_type_t를 직접 특수화할 필요가 없다. [U12-085] (12.7 컨셉(C++20), 인쇄 p.268)

비교·객체 컨셉	**핵심 관계**
Boolean [U12-086] (12.7 컨셉(C++20), 인쇄 p.268-269)	불리언 값으로 사용 가능 [U12-087] (12.7 컨셉(C++20), 인쇄 p.268-269)
WeaklyEqualityComparableWith / WeaklyEqualityComparable [U12-088] (12.7 컨셉(C++20), 인쇄 p.268-269)	서로 다른 **타입** / 같은 **타입** 값의 ==·!= 비교 [U12-089] (12.7 컨셉(C++20), 인쇄 p.268-269)
EqualityComparableWith / EqualityComparable [U12-090] (12.7 컨셉(C++20), 인쇄 p.268-269)	상등성 비교; 같은 **타입** 버전은 T,T 관계 [U12-091] (12.7 컨셉(C++20), 인쇄 p.268-269)
StrictTotallyOrderedWith / StrictTotallyOrdered [U12-092] (12.7 컨셉(C++20), 인쇄 p.268-269)	<·<=·>·>=를 이용한 완전 순서; 같은 **타입** 버전은 T,T 관계 [U12-093] (12.7 컨셉(C++20), 인쇄 p.268-269)
De**struct**ible / Con**struct**ible / DefaultCon**struct**ible [U12-094] (12.7 컨셉(C++20), 인쇄 p.268-269)	소멸 및 주소 연산 / 주어진 인자로 생성 / 기본 생성 [U12-095] (12.7 컨셉(C++20), 인쇄 p.268-269)

비교·객체 컨셉	핵심 관계
MoveCon**struct**ible / CopyCon**struct**ible [U12-096] (12.7 컨셉(C++20), 인쇄 p.268-269)	이동 생성 / 복사와 이동 생성 [U12-097] (12.7 컨셉(C++20), 인쇄 p.268-269)
Movable / Copyable [U12-098] (12.7 컨셉(C++20), 인쇄 p.268-269)	이동 생성·대입·교환 / 복사 생성 및 복사 대입을 포함 [U12-099] (12.7 컨셉(C++20), 인쇄 p.268-269)
Semiregular / **Regular** [U12-100] (12.7 컨셉(C++20), 인쇄 p.268-269)	Copyable와 기본 생성 / 여기에 상등성 비교를 포함 [U12-101] (12.7 컨셉(C++20), 인쇄 p.268-269)

원문은 WeaklyEqualityComparableWith와 WeaklyEqualityComparable를 혼용하면 오버로드 기회가 줄 수 있다고 경고한다. [U12-102] (12.7 컨셉(C++20), 인쇄 p.269)

Regular 타입은 int처럼 동작해 사용이 단순해지는 이상적인 성격으로 설명하며, ==가 없는 기본적인 클래스는 Semiregular로 구분한다. [U12-103] (12.7 컨셉(C++20), 인쇄 p.269)

호출 가능 컨셉	원문에서 설명한 의미
Invocable<F,Args> [U12-104] (12.7 컨셉(C++20), 인쇄 p.270)	Args 인자 목록으로 F 호출 [U12-105] (12.7 컨셉(C++20), 인쇄 p.270)
Invocable**Regular**<F,Args> [U12-106] (12.7 컨셉(C++20), 인쇄 p.270)	호출 가능하고 상등성을 보존 [U12-107] (12.7 컨셉(C++20), 인쇄 p.270)
Predi**cat**e<F,Args> [U12-108] (12.7 컨셉(C++20), 인쇄 p.270)	호출하여 bool 반환 [U12-109] (12.7 컨셉(C++20), 인쇄 p.270)
Rel**at**ion<F,T,U> [U12-110] (12.7 컨셉(C++20), 인쇄 p.270)	Predi**cat**e<F,T,U> [U12-111] (12.7 컨셉(C++20), 인쇄 p.270)
StrictWeakOrder<F,T,U> [U12-112] (12.7 컨셉(C++20), 인쇄 p.270)	Rel**at**ion이 엄격하고 약한 순서를 제공 [U12-113] (12.7 컨셉(C++20), 인쇄 p.270)

상등성 보존은 x==y일 때 f(x)==f(y)가 되는 성질이다. [U12-114] (12.7 컨셉(C++20), 인쇄 p.270)

비교 연산에는 일반적으로 엄격하고 약한 순서를 가정한다. [U12-115] (12.7 컨셉(C++20), 인쇄 p.270)

원문은 Rel**at**ion과 StrictWeakOrder의 차이가 의미론적인 것이며, 이 차이를 이름으로 의도에 드러낸다고 설명한다. [U12-116] (12.7 컨셉(C++20), 인쇄 p.270)

반복자·구간 컨셉	원문에서의 조건과 관계
Iter**at**or / Sentinel / SizedSentinel [U12-117] (12.7 컨셉(C++20), 인쇄 p.270-271)	증가·역참조 / 구간 끝의 보초 / 차이 연산이 가능한 보초 [U12-118] (12.7 컨셉(C++20), 인쇄 p.270-271)
InputIter**at**or / OutputIter**at**or [U12-119] (12.7 컨셉(C++20), 인쇄 p.270-271)	입력의 읽기 / 출력의 쓰기 [U12-120] (12.7 컨셉(C++20), 인쇄 p.270-271)
ForwardIter**at**or / BidirectionalIter**at**or [U12-121] (12.7 컨셉(C++20), 인쇄 p.270-271)	여러 번 순방향 순회 / 여기에 -- 지원 [U12-122] (12.7 컨셉(C++20), 인쇄 p.270-271)
RandomAccessIter**at**or [U12-123] (12.7 컨셉(C++20), 인쇄 p.270-271)	양방향 **반복자**에 원문 표의 산술·인덱싱 연산을 추가 [U12-124] (12.7 컨셉(C++20), 인쇄 p.270-271)
Permutable / Mergeable / Sortable [U12-125] (12.7 컨셉(C++20), 인쇄 p.270-271)	이동·교환 / 정렬 구간 병합 / 지정 관계로 정렬 가능 [U12-126] (12.7 컨셉(C++20), 인쇄 p.270-271)

반복자·구간 컨셉	원문에서의 조건과 관계
Range / SizedRange [U12-127] (12.7 컨셉 (C++20), 인쇄 p.270-271)	시작 **반복자**와 보초 / 크기를 상수 시간에 알 수 있음 [U12-128] (12.7 컨셉 (C++20), 인쇄 p.270-271)
View / BoundedRange [U12-129] (12.7 컨셉 (C++20), 인쇄 p.270-271)	상수 시간 복사·이동·대입 / **반복자**와 보초의 **타입**이 같음 [U12-130] (12.7 컨셉 (C++20), 인쇄 p.270-271)
InputRange / OutputRange / ForwardRange / BidirectionalRange / RandomAccessRange [U12-131] (12.7 컨셉 (C++20), 인쇄 p.270-271)	각 이름에 대응하는 **반복자 컨셉**을 만족하는 구간 [U12-132] (12.7 컨셉 (C++20), 인쇄 p.270-271)

보초는 주어진 **반복자**에서 시작해 종료 **술어**를 만족할 때까지 순회하게 하는 개념이다. [U12-133] (12.7 컨셉 (C++20), 인쇄 p.271-272)

원문은 C 스타일 문자열에서 *p==0을 끝 조건으로 삼는 예를 든다. [U12-134] (12.7 컨셉(C++20), 인쇄 p.271-272)

반복자의 종류는 알고리즘을 더 적절하게 선택하는 데 활용한다. [U12-135] (12.7 컨셉(C++20), 인쇄 p.271-272)

12.8 컨테이너 알고리즘

원문은 Ranges를 기다리지 않고도 **컨테이너**를 받아 내부에서 begin()/end() 버전을 호출하는 간단한 sort 래퍼를 작성할 수 있다고 설명한다. [U12-136] (12.8 컨테이너 알고리즘, 인쇄 p.272)

기본 비교 버전과 **술어** 버전을 별도 E**std 네임스페이스**에 두면 std 사용 시 충돌을 피하고 나중에 Range로 교체하기도 쉽다. [U12-137] (12.8 컨테이너 알고리즘, 인쇄 p.272)

12.9 병렬 알고리즘

각 데이터 항목의 계산이 서로 독립적이면 같은 작업을 병렬로 수행할 수 있다. [U12-138] (12.9 병렬 알고리즘, 인쇄 p.273-274)

병렬 실행은 여러 스레드에서 작업하고 **벡터화된 실행(SIMD)**은 한 스레드에서 벡터화로 작업한다. [U12-139] (12.9 병렬 알고리즘, 인쇄 p.273-274)

<execution>은 seq·par·par_unseq 실행 정책을 제공한다. [U12-140] (12.9 병렬 알고리즘, 인쇄 p.273-274)

정렬의 실행 정책 (원문 발췌)

```
sort(v.begin(),v.end()); // 순차
sort(seq,v.begin(),v.end()); // 순차(기본과 동일)
sort(par,v.begin(),v.end()); // 병렬
sort(par_unseq,v.begin(),v.end()); // 병렬 혹은 벡터화
```

[U12-141] (12.9 병렬 알고리즘, 인쇄 p.273)

정책은 실행에 대한 힌트이며 실제 병렬성은 컴파일러와 런타임 스케줄러가 정한다. [U12-142] (12.9 병렬 알고리즘, 인쇄 p.273-274)

효과는 알고리즘·요소 수·하드웨어·사용률에 따라 달라지므로 반드시 실험으로 확인한다. [U12-143] (12.9 병렬 알고리즘, 인쇄 p.273-274)

원문은 12.6절 표의 알고리즘 가운데 equal_range를 병렬 정책 대상에서 제외해 설명한다. [U12-144] (12.9 병렬 알고리즘, 인쇄 p.273-274)

병렬 실행을 요청할 때 **데이터 경합**과 데드락을 피해야 한다. [U12-145] (12.9 병렬 알고리즘, 인쇄 p.273-274)

12.10 조언

알고리즘의 입력은 **반복자**로 정의한 **반개구간**이며, 검색 실패는 일반적으로 입력 끝으로 표시한다. [U12-146] (12.10 조언, 인쇄 p.274)

알고리즘은 전달된 시퀀스 자체에 요소를 추가·제거하지 않는다. [U12-147] (12.10 조언, 인쇄 p.274)

직접 루프를 만들기 전에 표준 제네릭 알고리즘을 고려한다. [U12-148] (12.10 조언, 인쇄 p.274)

함수 객체와 **술어**로 동작을 확장하되 **술어**가 인자를 바꾸지 않게 한다. [U12-149] (12.10 조언, 인쇄 p.274)

반복자 쌍 방식이 번거로우면 **컨테이너**/구간 알고리즘을 사용한다. [U12-150] (12.10 조언, 인쇄 p.274)

주요 비교표

병렬 실행과 벡터화된 실행

기준	병렬 실행	벡터화된 실행
실행 수단 [U12-151] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	여러 스레드; 여러 코어일 수도 있음 [U12-152] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	한 스레드에서 벡터화(SIMD) [U12-153] (병렬 실행과 벡터화된 실행, 인쇄 p.273)
정책 요청 [U12-154] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	par [U12-155] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	par_unseq가 병렬 또는 벡터화를 허용 [U12-156] (병렬 실행과 벡터화된 실행, 인쇄 p.273)
보장 조건 [U12-157] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	가능한 경우; 정책은 힌트 [U12-158] (병렬 실행과 벡터화된 실행, 인쇄 p.273)	가능한 경우; 정책은 힌트 [U12-158] (병렬 실행과 벡터화된 실행, 인쇄 p.273)

핵심 용어 표

용어	원문 정의·설명
반개구간	입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 **반복자** 쌍으로 나타낸다. [U12-001] (12.1 소개, 인쇄 p.254-255)
반복자	**반복자**는 특정 **타입**의 **객체**이며 저장 구조에 맞는 정보를 가진다. [U12-002] (12.3 반복자 타입, 인쇄 p.259-260)
술어	**술어**는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다. [U12-035] (12.5 술어, 인쇄 p.263-264)
ostream_iterator	**ostream_iterator**는 스트림과 출력 **타입**을 지정해 만들며, 역참조 위치에 대입하면 스트림으로 출력한다. [U12-159] (12.4 스트림 반복자, 인쇄 p.261,263)
표준 알고리즘	**표준 알고리즘**은 요소 시퀀스에 적용되는 함수 **템플릿** 연산이며, std에 정의되고 <algorithm>에서 제공된다. [U12-160] (12.6 알고리즘 요약, 인쇄 p.265-266)
병렬 실행·SIMD	**병렬 실행**은 여러 스레드에서 작업하고 **벡터화된 실행(SIMD)**은 한 스레드에서 벡터화로 작업한다. [U12-139] (12.9 병렬 알고리즘, 인쇄 p.273-274)
find	**find**는 찾은 요소의 **반복자**를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다. [U12-010] (12.2 반복자의 사용, 인쇄 p.255-256)
find_if	**find_if**는 전달된 **술어**가 참이 되는 첫 요소를 찾는다. [U12-161] (12.5 술어, 인쇄 p.263-264)

장 요약

요약 기준: 12.10 조언

알고리즘의 입력은 **반복자**로 정의한 **반개구간**이며, 검색 실패는 일반적으로 입력 끝으로 표시한다. [U12-146] (12.10 조언, 인쇄 p.274)

알고리즘은 전달된 시퀀스 자체에 요소를 추가·제거하지 않는다. [U12-147] (12.10 조언, 인쇄 p.274)

직접 루프를 만들기 전에 표준 제네릭 알고리즘을 고려한다. **함수 객체**와 **술어**로 동작을 확장하되 **술어**가 인자를 바꾸지 않게 한다. [U12-162] (12.10 조언, 인쇄 p.274)

반복자 쌍 방식이 번거로우면 **컨테이너**/구간 알고리즘을 사용한다. [U12-150] (12.10 조언, 인쇄 p.274)

원문 확인 메모

[원문 확인 필요] 책 270쪽 RandomAccessIterator 표의 일부 산술 기호는 제공 이미지에서 물음표 모양으로 보인다. 해당 기호를 임의로 복원하지 않고 연산 범주만 요약했다.

제13장. 유틸리티

(제13장. 유틸리티, 인쇄 p.275-287,290-314)

학습 목표·범위

자원은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 **핸들**·파일 **핸들**이 예다. [U13-001] (13.2 자원 관리, 인쇄 p.276-277)

unique_ptr은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. [U13-002] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277,281)

핵심 개념·동작 절차·원문 예제

13.1 소개

이 장은 **컨테이너**·입출력 같은 큰 분류에 딱 들어맞지 않지만 널리 쓰이는 **표준 라이브러리** 유틸리티를 다룬다. [U13-003] (13.1 소개, 인쇄 p.276)

설계와 프로그램을 표현하는 일반적인 용어로 쓰이는 **타입**과, 다른 라이브러리 기능을 구현하는 재료가 포함된다. [U13-004] (13.1 소개, 인쇄 p.276)

유틸리티는 지나치게 복잡하거나 다른 기능에 의존하지 않아야 한다. [U13-005] (13.1 소개, 인쇄 p.276)

13.2 자원 관리

자원은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 **핸들**·파일 **핸들**이 예다. [U13-001] (13.2 자원 관리, 인쇄 p.276-277)

누수는 장시간 프로그램의 성능 저하나 충돌뿐 아니라 짧은 프로그램의 **자원** 부족도 일으킬 수 있다. [U13-006] (13.2 자원 관리, 인쇄 p.276-277)

표준 라이브러리는 **자원**을 책임지는 **객체**의 생성·소멸과 **자원**의 생애주기를 연결해 누수를 막는다. [U13-007] (13.2 자원 관리, 인쇄 p.276-277)

RAII는 **생성자**에서 **자원**을 획득하고 **소멸자**에서 해제하는 관례적 관리 방식이다. [U13-008] (13.2 자원 관리, 인쇄 p.276-277)

원문의 scoped_lock은 생성 시 **mutex**를 얻고, return·함수 끝·예외 등 어떤 방식으로 스코프를 벗어나도 **소멸자**로 해제한다. [U13-009] (13.2 자원 관리, 인쇄 p.276-277)

vector·**map**·**string**·iostream의 저장 공간이나 파일 **핸들**도 같은 원리에 따라 관리된다. [U13-010] (13.2 자원 관리, 인쇄 p.276-277)

13.2.1 unique_ptr과 shared_ptr

<memory>의 **unique_ptr**은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. [U13-011] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277-278)

자유 저장소의 **객체**를 스마트 포인터에 맡기면 예외나 이른 반환에서 해제를 빠뜨리는 일을 막을 수 있다. [U13-012] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277-278)

다만 지역 **객체**나 **컨테이너** 같은 더 높은 수준의 **자원 핸들**로 표현할 수 있다면 그것을 먼저 사용한다. [U13-013] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277-278)

내장 포인터와 unique_ptr의 해제 비교 (원문 발췌)

```
void f(int i, int j) // X* vs. unique_ptr<X>
{
    X* p = new X; // 새로운 X 할당
    unique_ptr<X> sp {new X}; // 새로운 X를 할당하고, 그 포인터를 unique_ptr에 넘겨줌
    // ...

    if (i<99) throw Z{}; // 여기서 예외를 던지거나
    if (j<77) return; // "일찍" 반환될 수 있음
    // ... p와 sp 사용 ..
    delete p; // *p 소멸
}
```

[U13-014] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.278)

unique_ptr은 개별 **객체**나 배열의 **핸들**이며 소유권 전달에 이동을 사용한다. [U13-015] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.278-279)

shared_ptr은 복사하여 소유권을 나누고 마지막 소유자가 소멸할 때 **자원**을 해제한다. [U13-016] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.278-279)

함수에 **shared_ptr**를 넘기면 함수가 종료된 뒤에도 복사본이 남을 수 있으므로, **자원** 소멸 시점을 단순히 호출 함수의 끝으로 예상하면 안 된다. [U13-017] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.278-279)

공유에는 추가 비용과 생애주기 예측의 어려움이 있어 꼭 공유해야 할 때만 사용한다. [U13-018] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.278-279)

make_unique와 make_shared는 **객체** 생성과 포인터 인계를 함께 처리해 잘못된 포인터 전달이나 인계 누락을 줄인다. [U13-019] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280-281)

원문은 make_shared가 소유자 수를 세기 위한 별도 할당을 줄일 수 있다는 이점도 설명한다. [U13-020] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280-281)

포인터가 필요한 대표적인 경우는 다형적인 **객체**를 참조하거나 **객체**를 공유하는 경우다. [U13-021] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280-281)

객체 컬렉션 반환은 **컨테이너**로 처리할 수 있다. [U13-022] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280-281)

스마트 포인터는 여전히 포인터다. [U13-023] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280)

특히 **shared_ptr** 자체는 소유자들의 읽기·쓰기 규칙을 명시하지 않으므로, 수명 관리만으로 **자원** 경합 같은 문제까지 해결되지는 않는다. [U13-024] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.280)

13.2.2 move()와 forward()

이동과 복사의 선택은 대부분 암묵적이지만, 복사할 수 없는 **unique_ptr**을 다른 소유자로 넘길 때처럼 명시적인 이동이 필요할 수 있다. [U13-025] (13.2.2 move()와 forward(), 인쇄 p.281-282)

std::move() 자체가 자원을 이동시키는 것은 아니다. [U13-026] (13.2.2 move()와 forward(), 인쇄 p.281-282)

전달한 **객체**를 **rvalue 참조**로 변환해 이동 **생성자**나 이동 대입 연산이 선택되게 유도한다. [U13-027] (13.2.2 move()와 forward(), 인쇄 p.281-282)

이동을 이용한 swap (원문 발췌)

```
template <typename T>
void swap(T& a, T& b)
{
    T tmp {move(a)}; // T의 생성자에는 rvalue로 전달되므로 이동을 수행
    a = move(b); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
    b = move(tmp); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
}
```

[U13-028] (13.2.2 move()와 forward(), 인쇄 p.282)

이동 후에도 원본 **객체**는 남는다. [U13-029] (13.2.2 move()와 forward(), 인쇄 p.282-283)

원문은 move한 문자열을 나중에 다시 인덱싱하는 위험을 들어, 확실하고 큰 성능 개선이 있을 때만 명시적으로 move를 사용하도록 한다. [U13-030] (13.2.2 move()와 forward(), 인쇄 p.282-283)

이동 후 상태는 일반적으로 정해지지 않지만 **표준 라이브러리 타입**은 소멸·대입 가능한 상태로 남는다고 설명한다. [U13-031] (13.2.2 move()와 forward(), 인쇄 p.282-283)

이 원문은 **vector·string** 같은 **컨테이너**를 이동 후 비어 있는 상태로 설명한다. [U13-032] (13.2.2 move()와 forward(), 인쇄 p.282-283)

완벽한 포워딩은 인자의 성격을 바꾸지 않고 다른 함수로 넘기는 것이다. [U13-033] (13.2.2 move()와 forward(), 인쇄 p.283)

std::forward()는 lvalue와 rvalue의 차이를 처리한다는 점에서 move와 구별된다. [U13-034] (13.2.2 move()와 forward(), 인쇄 p.283)

포워딩할 때는 std::forward를 사용하고 같은 **객체**를 두 번 포워딩하지 않는다. [U13-035] (13.2.2 move()와 forward(), 인쇄 p.283)

포워딩 뒤에 그 **객체**를 다시 사용하지 않도록 주의한다. [U13-036] (13.2.2 move()와 forward(), 인쇄 p.283)

13.3 구간 검사: gsl::span

포인터와 요소 수를 따로 전달하는 인터페이스는 두 값이 실제 배열과 일치한다는 관례에 의존해 구간 에러를 일으키기 쉽다. [U13-037] (13.3 구간 검사: gsl::span, 인쇄 p.284-285)

gsl::span은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조다. [U13-038] (13.3 구간 검사: gsl::span, 인쇄 p.284-285)

vector나 내장 배열 등의 연속 요소를 가리키며, 그 요소를 소유하지 않는다. [U13-039] (13.3 구간 검사: gsl::span, 인쇄 p.284-285)

string_view나 **반복자** 쌍과 유사하지만 문자 읽기 전용에 한정되지 않는다. [U13-040] (13.3 구간 검사: gsl::span, 인쇄 p.284-285)

배열에서 span을 직접 만들면 컴파일러가 요소 수를 세므로 간단하고 안전하다. [U13-041] (13.3 구간 검사: gsl::span, 인쇄 p.285-286)

별도로 포인터·길이로 span을 만들 때는 그 선택이 명시적으로 드러난다. [U13-042] (13.3 구간 검사: gsl::span, 인쇄 p.285-286)

함수끼리 span을 넘기면 포인터와 개수를 각각 넘기는 것보다 단순하다. [U13-043] (13.3 구간 검사: gsl::span, 인쇄 p.285-286)

인덱싱의 구간 검사는 에러 시 gsl::fail_fast를 던진다고 원문이 설명하며, 성능이 중요한 코드에서는 검사를 막을 수 있다고 한다. [U13-044] (13.3 구간 검사: gsl::span, 인쇄 p.285-286)

이 절의 span은 원문 시점에 표준으로 제안된 GSL **타입**이다. [U13-045] (13.3 구간 검사: gsl::span, 인쇄 p.284-286)

원문은 향후 std::span의 구간 에러 처리에 계약을 사용하기를 기대한다고 쓴다. [U13-046] (13.3 구간 검사: gsl::span, 인쇄 p.284-286)

이 전망을 현재의 표준 사실로 바꾸어 보충하지 않는다. [U13-047] (13.3 구간 검사: gsl::span, 인쇄 p.284-286)

문자 구간에는 gsl::**string**_span을 제시한다. [U13-048] (13.3 구간 검사: gsl::span, 인쇄 p.284-286)

13.4 특수한 컨테이너

유사 컨테이너	원문에서 정리한 역할
T[N] / array<T,N> [U13-049] (13.4 특수한 컨테이너, 인쇄 p.287)	연속 저장된 T 요소 N개의 고정 크기 시퀀스; array는 내장 배열의 여러 문제를 줄임 [U13-050] (13.4 특수한 컨테이너, 인쇄 p.287)
bitset<N> / **vector**<bool> [U13-051] (13.4 특수한 컨테이너, 인쇄 p.287)	N비트의 고정 시퀀스 / **vector** 특수화로 저장한 비트 시퀀스 [U13-052] (13.4 특수한 컨테이너, 인쇄 p.287)
pair<T,U> / **tuple**<T...> [U13-053] (13.4 특수한 컨테이너, 인쇄 p.287)	두 요소 / 임의의 여러 **타입**을 가진 요소 시퀀스 [U13-054] (13.4 특수한 컨테이너, 인쇄 p.287)
basic_**string**<C> / **valarray**<T> [U13-055] (13.4 특수한 컨테이너, 인쇄 p.287)	문자열 연산이 있는 문자 시퀀스 / 수치 연산을 제공하는 수치 배열 [U13-056] (13.4 특수한 컨테이너, 인쇄 p.287)

여러 **컨테이너**는 서로 다르거나 겹치는 요구를 만족한다. [U13-057] (13.4 특수한 컨테이너, 인쇄 p.287)

pair와 **tuple**은 이형이며, 다른 **컨테이너**는 같은 **타입**의 요소를 담는 동형이다. [U13-058] (13.4 특수한 컨테이너, 인쇄 p.287)

array·**vector**·**tuple**의 요소는 연속 할당되고 **forward_list**·**map**은 연결 구조다. [U13-059] (13.4 특수한 컨테이너, 인쇄 p.287)

13.4.1 array

정리 범위 제한: 제공 자료에는 책 288~289쪽이 없다. [U13-060] (13.4.1 array, 인쇄 p.287,290)

array 설명의 앞부분을 확인할 수 없으므로 287쪽 표, 290쪽의 이어지는 설명, 313쪽 조언에서 확인되는 내용만 정리한다. [U13-061] (13.4.1 array, 인쇄 p.287,290)

array는 크기를 알고 있어 **표준 알고리즘**과 함께 쓸 수 있고 =로 복사할 수 있다. [U13-062] (13.4.1 array, 인쇄 p.290)

원문이 강조하는 장점은 내장 배열에서 생길 수 있는 예기치 않은 포인터 변환을 막는다는 것이다. [U13-063] (13.4.1 array, 인쇄 p.290)

Circle 배열을 Shape*로 받아 인덱싱하면 sizeof(Shape)<sizeof(Circle)이라는 원문의 전제하에 잘못된 오프셋이 생기지만 array는 그 포인터 변환을 허용하지 않는다. 290쪽의 이어지는 문장은 스택이 제한된 **자원**이며 스택 오버플로우가 생길 수 있다는 주의도 포함한다. [U13-064] (13.4.1 array, 인쇄 p.290)

13.4.2 bitset

bitset<N>은 컴파일 시간에 정한 N비트의 구간에 연산을 제공한다. [U13-065] (13.4.2 bitset, 인쇄 p.290-291)

참/거짓·켜짐/꺼짐 등의 플래그 집합을 나타내며, 내장 정수에 담기 어려운 비트 집합을 다루는 데 편리하다. [U13-066] (13.4.2 bitset, 인쇄 p.290-291)

비트에 번호 대신 이름을 붙여야 한다면 원문은 set이나 열거형을 권한다. [U13-067] (13.4.2 bitset, 인쇄 p.290-291)

정수나 문자열로 초기화할 수 있고 비트별 연산과 시프트를 제공한다. [U13-068] (13.4.2 bitset, 인쇄 p.291-292)

시프트는 0을 밀어 넣는다. [U13-069] (13.4.2 bitset, 인쇄 p.291-292)

to_ullong()과 to_**string**()은 **생성자**의 반대 방향으로 변환한다. [U13-070] (13.4.2 bitset, 인쇄 p.291-292)

출력은 높은 자릿수의 비트가 왼쪽에 오며 bitset 자체의 출력 연산자도 사용할 수 있다. [U13-071] (13.4.2 bitset, 인쇄 p.291-292)

정수를 bitset 문자열로 출력 (원문 발췌)

```
void binary(int i)
{
    bitset<8*sizeof(int)> b = i; // 8비트 바이트를 가정(14.7절 참고)
    cout << b.to_string() << '\n'; // i를 비트로 출력
}
```

[U13-072] (13.4.2 bitset, 인쇄 p.291)

13.4.3 pair와 tuple

값들의 모음 자체가 필요하면 간단한 **struct** 또는 **pair·tuple**을 사용할 수 있다. [U13-073] (13.4.3 pair와 tuple, 인쇄 p.292-293)

pair는 first와 second라는 일관된 멤버 이름을 가진 두 값의 묶음이다. [U13-074] (13.4.3 pair와 tuple, 인쇄 p.292-293)

equal_range는 정렬된 입력에서 같은 값에 해당하는 부분 구간을 두 **반복자**의 **pair**로 반환한다. [U13-075] (13.4.3 pair와 tuple, 인쇄 p.292-293)

요소가 연산을 지원하면 **pair**의 대입·비교도 가능하다. [U13-076] (13.4.3 pair와 tuple, 인쇄 p.292-293)

타입 추론이나 make_**pair**로 중복된 **타입** 명시를 줄일 수 있다. [U13-077] (13.4.3 pair와 tuple, 인쇄 p.293-294)

둘보다 많거나 적은 요소에는 **tuple**을 사용한다. [U13-078] (13.4.3 pair와 tuple, 인쇄 p.293-294)

원문은 **생성자** 인자에서의 **타입** 추론이 C++17부터 제공되며 이전 코드에서는 make_**tuple**을 사용할 수 있다고 설명한다. [U13-079] (13.4.3 pair와 tuple, 인쇄 p.293-294)

tuple 생성과 타입 추론 (원문 발췌)

```
tuple<string,int,double> t1 {"Shark",123,3.14}; // 타입을 명시적으로 지정
auto t2 = make_tuple(string{"Herring"},10,1.23); // 타입을
    // tuple<string,int,double>로 추론
tuple t3 {"Cod"s,20,9.99}; // 타입을 tuple<string,int,double>로 추론
```

[U13-080] (13.4.3 pair와 tuple, 인쇄 p.294)

get<0>()처럼 0부터 시작하는 상수 인덱스로 **tuple** 요소에 접근한다. [U13-081] (13.4.3 pair와 tuple, 인쇄 p.294-295)

그 **타입**이 **tuple** 안에서 유일할 때는 get<**string**>()처럼 **타입**으로도 지정할 수 있고 쓰기에도 get을 사용한다. [U13-082] (13.4.3 pair와 tuple, 인쇄 p.294-295)

pair도 get으로 접근 가능하고 **pair**·**tuple** 모두 구조화된 바인딩을 사용할 수 있다. [U13-083] (13.4.3 pair와 tuple, 인쇄 p.294-295)

제네릭할 필요가 없다면 이름 있는 멤버를 둔 **struct**가 더 읽기 쉽고 유지보수하기 좋을 수 있다. [U13-084] (13.4.3 pair와 tuple, 인쇄 p.294-295)

13.5 선택지

표준 라이브러리는 지정한 선택지 중 하나를 담는 **variant**, 특정 **타입**의 값 또는 값 없음을 표현하는 optional, **타입** 집합에 제한이 없는 **any**를 제공한다. [U13-085] (13.5 선택지, 인쇄 p.295)

각각 <**variant**>·<optional>·<**any**>에 정의되며, 관련 기능을 표현하지만 인터페이스가 통일되어 있지는 않다. [U13-086] (13.5 선택지, 인쇄 p.295)

13.5.1 variant

variant<A,B,C>는 **union**을 직접 사용하는 것보다 안전하고 편리한 선택지다. [U13-087] (13.5.1 variant, 인쇄 p.295-296)

대입·초기화한 값의 **타입**을 기억하며 사용자는 어떤 선택지가 저장되었는지 질의하고 값을 추출한다. [U13-088] (13.5.1 variant, 인쇄 p.295-296)

원문은 문자열 또는 에러 코드를 반환하는 경우와, 컴파일러 노드의 Expression·St**at**ement·Declar**at**ion·Type을 구분하는 경우를 든다. [U13-089] (13.5.1 variant, 인쇄 p.295-296)

저장 **타입**별로 여러 번 검사하는 대신 visit과 오버로드된 함수 집합으로 행동을 선택할 수 있다. [U13-090] (13.5.1 variant, 인쇄 p.297-298)

원문의 overloaded는 람다 등으로부터 오버로드 집합을 만드는 보조 **타입**이며 표준 기능 자체는 아니라고 명시한다. [U13-091] (13.5.1 variant, 인쇄 p.297-298)

추론 가이드도 이 구현에 사용된다. [U13-092] (13.5.1 variant, 인쇄 p.297-298)

visit 이후 오버로드 규칙으로 가장 적절한 람다가 호출된다. [U13-093] (13.5.1 variant, 인쇄 p.297-298)

잘못된 **타입**으로 접근하면 bad_**variant**_access를 던진다. [U13-094] (13.5.1 variant, 인쇄 p.297-298)

가상 호출보다 빠를 가능성은 실측해야 한다. [U13-095] (13.5.1 variant, 인쇄 p.297-298)

13.5.2 optional

optional<A>는 A 또는 아무것도 없음을 나타낸다. [U13-096] (13.5.2 optional, 인쇄 p.298-299)

객체를 반환하거나 반환하지 않는 함수에 유용하며 빈 {}가 값 없음에 해당한다. [U13-097] (13.5.2 optional, 인쇄 p.298-299)

조건 검사로 값 유무를 살피고 *로 저장 값에 접근하므로 포인터와 비슷한 사용 형태를 가진다. [U13-098] (13.5.2 optional, 인쇄 p.298-299)

값이 있는 optional만 합산 (원문 발췌)

```
int cat(optional<int> a, optional<int> b)
{
    int res = 0;
    if (a) res+=*a;
    if (b) res+=*b;
    return res;
}

int x = cat(17,19);
int y = cat(17,{});
int z = cat({},{});
```

[U13-099] (13.5.2 optional, 인쇄 p.299)

원문은 값이 없는 optional에 접근하면 동작을 예측할 수 없고 예외도 던지지 않으므로, 이 접근 방식에서 **타입** 안전성을 보장하지 않는다고 경고한다. [U13-100] (13.5.2 optional, 인쇄 p.299)

따라서 예제처럼 먼저 존재 여부를 검사하는 조건을 유지한다. [U13-101] (13.5.2 optional, 인쇄 p.299)

13.5.3 any

any는 임의의 **타입**을 저장하며 저장된 **객체**가 있으면 그 **타입**을 기억한다. [U13-102] (13.5.3 any, 인쇄 p.299-300)

제약을 제거한 **variant**처럼 설명할 수 있다. [U13-103] (13.5.3 any, 인쇄 p.299-300)

any_cast로 **타입**을 지정하여 값을 추출한다. [U13-104] (13.5.3 any, 인쇄 p.299-300)

원문은 저장된 **타입**과 다른 **타입**으로 접근하면 bad_**any**_access를 던진다고 표기하고, 예외에 의존하지 않는 접근 방법도 있다고 언급한다. [U13-105] (13.5.3 any, 인쇄 p.299-300)

[원문 확인 필요] 여기서 예외 이름 bad_**any**_access는 제공된 책 300쪽의 표기다. [U13-106] (13.5.3 any, 인쇄 p.300)

이 노트에서는 의심되는 명칭을 다른 이름으로 조용히 바꾸지 않았다. [U13-107] (13.5.3 any, 인쇄 p.300)

13.6 할당자

컨테이너의 기본 할당은 new·delete를 사용하지만, 특별한 성능·보안·스레드별 할당·메모리 구조 요구가 있으면 **할당자**를 지정할 수 있다. [U13-108] (13.6 할당자, 인쇄 p.300)

원문은 고급 **할당자** 전반 대신 이벤트 큐에서 풀 **할당자**로 단편화를 줄인 실제 사례를 제시한다. [U13-109] (13.6 할당자, 인쇄 p.300)

원래 시스템은 **vector**를 가진 이벤트를 **shared_ptr**로 전달하여 마지막 사용자가 자동 소멸시키는 단순한 구조였지만 단편화로 많은 메모리를 소비했다. [U13-110] (13.6 할당자, 인쇄 p.301-302)

풀 **할당자**는 같은 크기의 여러 **객체**를 한꺼번에 할당해 관리한다. [U13-111] (13.6 할당자, 인쇄 p.301-302)

C++17의 pmr 하위 **네임스페이스**에는 다형성 메모리 **자원**과 synchronized_pool_resource가 있어 **컨테이너**를 전면 교체하지 않고 **할당자**를 바꿀 수 있다. [U13-112] (13.6 할당자, 인쇄 p.301-302)

원문 사례에서 프로듀서 16개와 컨슈머 4개가 이벤트 100,000개를 전달한 뒤 사용 메모리가 6GB에서 3MB 미만으로 줄었다. [U13-113] (13.6 할당자, 인쇄 p.301-302)

이는 단편화 낭비를 줄인 결과이며 실제로 필요했던 메모리양이 바뀐 것은 아니다. [U13-114] (13.6 할당자, 인쇄 p.301-302)

이 수치는 해당 사례의 관측값으로만 이해한다. [U13-115] (13.6 할당자, 인쇄 p.301-302)

13.7 시간

<chrono>의 clock::now()는 시각인 time_point를 반환하고 두 시각의 차이는 시간 구간인 dur**at**ion이 된다. [U13-116] (13.7 시간, 인쇄 p.302-303)

시계별로 단위가 다르므로 dur**at**ion_cast로 적절한 단위에 맞춰 표현한다. [U13-117] (13.7 시간, 인쇄 p.302-303)

성능을 판단할 때는 추측 대신 실제 시간을 측정한다. [U13-118] (13.7 시간, 인쇄 p.302-303)

chrono로 작업 시간 측정 (원문 발췌)

```
using namespace std::chrono; // 하위 네임스페이스 std::chrono; 3.4절 참고

auto t0 = high_resolution_clock::now();
do_work();
auto t1 = high_resolution_clock::now();
cout << duration_cast<milliseconds>(t1-t0).count() << "msec\n";
```

[U13-119] (13.7 시간, 인쇄 p.302)

시간 단위 접미사는 std::chrono_literals에 정의되어 10ms·33us 같은 표현을 가능하게 한다. [U13-120] (13.7 시간, 인쇄 p.303)

원문은 년·월·달력·시간대·윤초를 다루는 확장 기능을 C++20 표준에 추가 중이라고 소개한다. [U13-121] (13.7 시간, 인쇄 p.303)

이는 원문 시점의 상태이며 현재 상황으로 보충하지 않는다. [U13-122] (13.7 시간, 인쇄 p.303)

13.8 함수 어댑션

함수를 다른 함수의 인자로 넘길 때 호출 형태나 **타입**이 정확히 맞지 않으면 람다, mem_fn, **function**을 이용하는 세 방법을 고려한다. [U13-123] (13.8 함수 어댑션, 인쇄 p.303-304)

13.8.1 람다를 어댑터로 사용

표준 알고리즘은 f(x) 형태로 인자를 호출하지만 멤버 함수는 x->f() 형태를 사용한다. [U13-124] (13.8.1 람다를 어댑터로 사용, 인쇄 p.304)

람다는 이 호출 방식의 차이를 연결할 수 있다. [U13-125] (13.8.1 람다를 어댑터로 사용, 인쇄 p.304)

람다로 멤버 함수 호출 연결 (원문 발췌)

```
void draw_all(vector<Shape*>& v)
{
    for_each(v.begin(),v.end(),[](Shape* p) { p->draw(); });
}
```

[U13-126] (13.8.1 람다를 어댑터로 사용, 인쇄 p.304)

13.8.2 mem_fn()

mem_fn(mf)는 멤버 함수 mf를 비멤버 함수처럼 호출할 수 있는 **함수 객체**로 바꾼다. [U13-127] (13.8.2 mem_fn(), 인쇄 p.304-305)

원문은 mem_fn(&Shape::draw)를 for_each에 넘겨 람다와 같은 연결을 수행한다. [U13-128] (13.8.2 mem_fn(), 인쇄 p.304-305)

람다 도입 이전에 **객체**지향 호출과 함수 스타일 호출을 연결하던 대표 수단이다. [U13-129] (13.8.2 mem_fn(), 인쇄 p.304-305)

13.8.3 function()

function 객체는 ()로 호출할 수 있는 대상을 저장한다. [U13-130] (13.8.3 function(), 인쇄 p.305)

함수·람다를 넣고 그 호출 **타입**을 명시하거나 초기 값에서 추론할 수 있다. [U13-131] (13.8.3 function(), 인쇄 p.305)

콜백이나 연산을 인자로 전달하는 데 유용하지만 직접 함수 호출보다 실행 비용이 있고, **function**은 **객체**이므로 자체에 오버로딩을 적용할 수 없다. [U13-132] (13.8.3 function(), 인쇄 p.305)

함수 객체의 오버로딩에는 앞선 overloaded를 고려한다. [U13-133] (13.8.3 function(), 인쇄 p.305)

13.9 타입 함수

타입 함수는 **타입** 자체를 인자로 받거나 **타입**을 반환하며 컴파일 시간에 평가된다. [U13-134] (13.9 타입 함수, 인쇄 p.306)

numeric_limits는 수치 **타입** 정보를, sizeof는 **객체**의 바이트 수를 알려준다. [U13-135] (13.9 타입 함수, 인쇄 p.306)

이런 기능은 컴파일 시간 계산을 통한 **타입** 검사와 성능 향상의 재료다. [U13-136] (13.9 타입 함수, 인쇄 p.306)

관련 기법을 메타프로그래밍, **템플릿**과 관련되면 **템플릿** 메타프로그래밍이라고 한다. [U13-137] (13.9 타입 함수, 인쇄 p.306)

원문은 아직 **컨셉**이 범용 표준이 아닌 시점이어서 **타입 함수**가 널리 쓰인다고 설명한다. [U13-138] (13.9 타입 함수, 인쇄 p.306)

13.9.1 iterator_traits

표준 sort는 임의 접근 **반복자**를 요구하지만 **forward_list**는 순방향 **반복자**만 제공한다. [U13-139] (13.9.1 iterator_traits, 인쇄 p.306-309)

원문은 임의 접근 구간에는 바로 sort를, 순방향 구간에는 **vector**로 복사·정렬·원래 구간으로 재복사를 선택하는 sort_helper 두 버전을 제시한다. [U13-140] (13.9.1 iterator_traits, 인쇄 p.306-309)

반복자 종류를 나타내는 태그로 적절한 오버로드를 컴파일 시간에 선택하는 기법이 **태그 디스패치**다. [U13-141] (13.9.1 iterator_traits, 인쇄 p.306-309)

포인터용 iterator_traits (원문 발췌)

```
template<class T>
struct iterator_traits<T*> {
    using difference_type = ptrdiff_t;
    using value_type = T;
    using pointer = T*;
    using reference = T&;
    using iterator_category = random_access_iterator_tag;
};
```

[U13-142] (13.9.1 iterator_traits, 인쇄 p.309)

발췌 범위: 원문의 iterator_traits<T*> 특수화 블록 전체를 발췌했습니다. 앞뒤 sort_helper와 타입 별칭 코드는 생략했습니다.

컨테이너의 iterator·value_type 멤버는 **타입 별칭**으로 표현할 수 있다. [U13-143] (13.9.1 iterator_traits, 인쇄 p.308-310)

포인터처럼 이런 멤버 **타입**이 없는 경우에는 <iterator>의 iterator_traits 특수화를 통해 값 **타입**·참조·포인터·**반복자** 종류를 얻는다. [U13-144] (13.9.1 iterator_traits, 인쇄 p.308-310)

따라서 int*도 random_access_iterator_tag를 가진 **반복자**로 다룰 수 있다. [U13-145] (13.9.1 iterator_traits, 인쇄 p.308-310)

원문은 같은 구분이 **컨셉**으로 더 직접 표현될 수 있고 표준화가 진행 중이라고 설명한다. [U13-146] (13.9.1 iterator_traits, 인쇄 p.308-310)

13.9.2 타입 술어

<type_traits>는 is_arithmetic·is_**class**·is_pod·is_literal_type·has_virtual_de**struct**or·is_base_of 같은 **타입 술어**를 제공한다. [U13-147] (13.9.2 타입 술어, 인쇄 p.310-311)

템플릿은 이 값으로 **타입**의 성질을 검사할 수 있다. [U13-148] (13.9.2 타입 술어, 인쇄 p.310-311)

원문은 산술 **타입**만 받는 complex의 st**at**ic_assert를 예로 들고, std::is_arithmetic<T>::value를 반환하는 Is_arithmetic() 래퍼로 표기를 간단히 한다. [U13-149] (13.9.2 타입 술어, 인쇄 p.310-311)

타입 술어의 표기를 단순화한 래퍼 (원문 발췌)

```
template<typename T>
constexpr bool Is_arithmetic()
{
    return std::is_arithmetic<T>::value;
}
```

[U13-150] (13.9.2 타입 술어, 인쇄 p.311)

13.9.3 enable_if

타입 술어는 st**at**ic_assert·컴파일 시간 if·**enable_if**의 조건으로 사용된다. [U13-151] (13.9.3 enable_if, 인쇄 p.311-312)

enable_if는 조건에 따라 정의를 선택하는 데 널리 쓰인다. [U13-152] (13.9.3 enable_if, 인쇄 p.311-312)

원문의 스마트 포인터 설명은 T가 클래스일 때에만 화살표 연산을 제공하려는 의도를 든다. [U13-153] (13.9.3 enable_if, 인쇄 p.311-312)

함수 바깥에서는 컴파일 시간 if를 쓸 수 없다는 문맥에서 **enable_if**를 소개한다. [U13-154] (13.9.3 enable_if, 인쇄 p.311-312)

enable_if는 표기가 낯설고 많은 경우 **컨셉**으로 대체될 수 있지만, 원문 시점의 **템플릿** 메타프로그래밍과 여러 **표준 라이브러리** 구성 요소의 기반이다. [U13-155] (13.9.3 enable_if, 인쇄 p.312)

이 기법은 **SFINAE(Substitution Failure Is Not An Error)** 언어 기능에 의존한다. [U13-156] (13.9.3 enable_if, 인쇄 p.312)

13.10 조언

자원에는 **RAII 자원 핸들**을 사용한다. [U13-157] (13.10 조언, 인쇄 p.312-314)

포인터가 필요하면 **unique_ptr**를 우선하고, 소유권을 공유해야 할 때만 **shared_ptr**를 쓴다. [U13-158] (13.10 조언, 인쇄 p.312-314)

생성에는 make_unique·make_shared를 사용한다. [U13-159] (13.10 조언, 인쇄 p.312-314)

move를 남용하지 않으며 forward는 포워딩에만 사용한다. [U13-160] (13.10 조언, 인쇄 p.312-314)

이동·포워딩한 **객체**를 나중에 읽지 않는다. [U13-161] (13.10 조언, 인쇄 p.312-314)

포인터와 개수를 별도로 전달하기보다 span을 사용한다. [U13-162] (13.10 조언, 인쇄 p.312-314)

고정 크기에는 array, N비트에는 bitset을 고려한다. [U13-163] (13.10 조언, 인쇄 p.312-314)

pair·tuple보다 이름 있는 **struct**가 읽기 쉬운 경우를 구별한다. [U13-164] (13.10 조언, 인쇄 p.312-314)

union을 직접 쓰기보다 **variant**를 고려한다. [U13-165] (13.10 조언, 인쇄 p.312-314)

단편화에는 **할당자**를 고려하고, 효율성은 실제 시간 측정으로 판단한다. [U13-166] (13.10 조언, 인쇄 p.312-314)

dur**at**ion은 단위를 명시하고 필요하면 dur**at**ion_cast로 바꾼다. [U13-167] (13.10 조언, 인쇄 p.312-314)

호출을 연결할 때 람다·mem_fn을, 호출 대상을 저장할 때 **function**을 쓴다. [U13-168] (13.10 조언, 인쇄 p.312-314)

타입 특성·**enable_if**·**컨셉**과 **타입 별칭**의 역할을 구분한다. [U13-169] (13.10 조언, 인쇄 p.312-314)

주요 비교표

스마트 포인터의 소유권

기준	unique_ptr	shared_ptr
소유권 [U13-170] (스마트 포인터의 소유권, 인쇄 p.277-281)	유일한 소유자 [U13-171] (스마트 포인터의 소유권, 인쇄 p.277-281)	여러 소유자가 공유 [U13-172] (스마트 포인터의 소유권, 인쇄 p.277-281)
전달 [U13-173] (스마트 포인터의 소유권, 인쇄 p.277-281)	이동; 복사할 수 없음 [U13-174] (스마트 포인터의 소유권, 인쇄 p.277-281)	복사하여 공유 [U13-175] (스마트 포인터의 소유권, 인쇄 p.277-281)
자원 해제 [U13-176] (스마트 포인터의 소유권, 인쇄 p.277-281)	소유 **핸들**의 수명에 연결 [U13-177] (스마트 포인터의 소유권, 인쇄 p.277-281)	마지막 소유자가 소멸할 때 [U13-178] (스마트 포인터의 소유권, 인쇄 p.277-281)
선택 [U13-179] (스마트 포인터의 소유권, 인쇄 p.277-281)	포인터가 필요하고 소유자가 하나일 때 우선 [U13-180] (스마트 포인터의 소유권, 인쇄 p.277-281)	공유 소유권이 실제로 필요할 때 [U13-181] (스마트 포인터의 소유권, 인쇄 p.277-281)

세 가지 선택지 타입

타입	표현하는 선택지	원문의 접근 방식
variant [U13-182] (세 가지 선택지 타입, 인쇄 p.295-300)	미리 지정한 여러 **타입** 중 하나 [U13-183] (세 가지 선택지 타입, 인쇄 p.295-300)	**타입** 질의·get·visit [U13-184] (세 가지 선택지 타입, 인쇄 p.295-300)
optional [U13-185] (세 가지 선택지 타입, 인쇄 p.295-300)	한 **타입**의 값 또는 없음 [U13-186] (세 가지 선택지 타입, 인쇄 p.295-300)	존재 검사 후 * [U13-187] (세 가지 선택지 타입, 인쇄 p.295-300)
any [U13-188] (세 가지 선택지 타입, 인쇄 p.295-300)	미리 제한하지 않은 **타입** [U13-189] (세 가지 선택지 타입, 인쇄 p.295-300)	**any**_cast로 지정 **타입** 추출 [U13-190] (세 가지 선택지 타입, 인쇄 p.295-300)

핵심 용어 표

용어	원문 정의·설명
자원	**자원**은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 **핸들**·파일 **핸들**이 예다. [U13-001] (13.2 자원 관리, 인쇄 p.276-277)
unique_ptr·shared_ptr	**unique_ptr**은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. [U13-002] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277,281)
gsl::span	**gsl::span**은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조이며, 그 요소를 소유하지 않는다. [U13-191] (13.3 구간 검사: gsl::span, 인쇄 p.284,286)
optional<A>	**optional<A>**는 A 또는 아무것도 없음을 나타낸다. [U13-096] (13.5.2 optional, 인쇄 p.298-299)
any	**any**는 임의의 **타입**을 저장하며 저장된 **객체**가 있으면 그 **타입**을 기억한다. [U13-102] (13.5.3 any, 인쇄 p.299-300)
function	**function 객체**는 ()로 호출할 수 있는 대상을 저장한다. [U13-130] (13.8.3 function(), 인쇄 p.305)
타입 함수	**타입 함수**는 **타입** 자체를 인자로 받거나 **타입**을 반환하며 컴파일 시간에 평가된다. [U13-134] (13.9 타입 함수, 인쇄 p.306)

장 요약

요약 기준: 13.10 조언

자원에는 **RAII 자원 핸들**을 사용한다. 포인터가 필요하면 **unique_ptr**를 우선하고, 소유권을 공유해야 할 때만 **shared_ptr**를 쓴다. 생성에는 make_unique·make_shared를 사용한다. [U13-192] (13.10 조언, 인쇄 p.312-314)

move를 남용하지 않으며 forward는 포워딩에만 사용한다. 이동·포워딩한 **객체**를 나중에 읽지 않는다. [U13-193] (13.10 조언, 인쇄 p.312-314)

포인터와 개수를 별도로 전달하기보다 span을 사용한다. 고정 크기에는 array, N비트에는 bitset을 고려한다. [U13-194] (13.10 조언, 인쇄 p.312-314)

pair·tuple보다 이름 있는 **struct**가 읽기 쉬운 경우를 구별한다. **union**을 직접 쓰기보다 **variant**를 고려한다. [U13-195] (13.10 조언, 인쇄 p.312-314)

원문 확인 메모

책 288~289쪽이 제공되지 않아 13.4.1 array의 앞부분은 확인할 수 없다. 확인된 앞뒤 페이지·표·조언만 정리했다.

[원문 확인 필요] 책 300쪽의 any 예외 이름 bad_any_access를 원문대로 보존했다. 의심되는 표기를 외부 지식으로 수정하지 않았다.

제14장. 수치

(제14장. 수치, 인쇄 p.315-325)

학습 목표·범위

<numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. [U14-001] (14.3 수치 알고리즘, 인쇄 p.318-319)

reduce는 accumul**at**e와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

핵심 개념·동작 절차·원문 예제

14.1 소개

수치 계산은 데이터베이스·네트워킹·장비 제어·그래픽스·시뮬레이션·금융 분석 등 큰 시스템의 일부로 쓰인다. [U14-003] (14.1 소개, 인쇄 p.316)

C++는 계산 과정에 복잡한 자료 구조가 함께 필요할 때 장점을 가진다. [U14-004] (14.1 소개, 인쇄 p.316)

이 장은 그런 작업을 지원하는 **표준 라이브러리** 수치 기능을 정리한다. [U14-005] (14.1 소개, 인쇄 p.316)

14.2 수학 함수

<cm**at**h>는 flo**at**·double·long double에 대한 표준 수학 함수를 제공한다. [U14-006] (14.2 수학 함수, 인쇄 p.316-317)

대응하는 복소수 버전은 <complex>에서 찾을 수 있고 반환 **타입**은 인자 **타입**과 같다. [U14-007] (14.2 수학 함수, 인쇄 p.316-317)

함수	원문에서 설명한 역할·조건
abs / ceil / floor [U14-008] (14.2 수학 함수, 인쇄 p.316-317)	절댓값 / x 이상인 가장 작은 정수 / x 이하인 가장 큰 정수 [U14-009] (14.2 수학 함수, 인쇄 p.316-317)
sqrt [U14-010] (14.2 수학 함수, 인쇄 p.316-317)	제곱근; x는 음수가 아니어야 함 [U14-011] (14.2 수학 함수, 인쇄 p.316-317)
cos / sin / tan [U14-012] (14.2 수학 함수, 인쇄 p.316-317)	코사인 / 사인 / 탄젠트 [U14-013] (14.2 수학 함수, 인쇄 p.316-317)
acos / asin / **at**an [U14-014] (14.2 수학 함수, 인쇄 p.316-317)	역코사인 / 역사인 / 역탄젠트; acos는 음이 아닌 결과, asin은 0에서 가장 가까운 결과 [U14-015] (14.2 수학 함수, 인쇄 p.316-317)
sinh / cosh / tanh [U14-016] (14.2 수학 함수, 인쇄 p.316-317)	하이퍼볼릭 사인 / 코사인 / 탄젠트 [U14-017] (14.2 수학 함수, 인쇄 p.316-317)
exp / log / log10 [U14-018] (14.2 수학 함수, 인쇄 p.316-317)	밑 e의 지수 / 자연 로그 / 밑 10의 로그; log의 x는 양수 [U14-019] (14.2 수학 함수, 인쇄 p.316-317)

원문은 <cerrno>의 errno로 에러를 나타내며 EDOM은 정의역 에러, ERANGE는 구간 에러라고 설명한다. [U14-020] (14.2 수학 함수, 인쇄 p.317)

예제는 먼저 errno를 0으로 지운 뒤 음수의 sqrt와 너무 큰 pow 결과를 검사한다. [U14-021] (14.2 수학 함수, 인쇄 p.317)

<cstdlib>에도 몇 가지 수학 함수가 있고, <cm**at**h>에는 beta·riemann_zeta·sph_bessel 같은 특수 수학 함수도 있다. [U14-022] (14.2 수학 함수, 인쇄 p.317)

14.3 수치 알고리즘

<numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. [U14-001] (14.3 수치 알고리즘, 인쇄 p.318-319)

각 알고리즘에는 흔히 쓰는 기본 연산 버전과, 적용 연산을 인자로 받는 일반화 버전이 있다. [U14-023] (14.3 수치 알고리즘, 인쇄 p.318-319)

모든 **표준 라이브러리** 시퀀스에 적용할 수 있다. [U14-024] (14.3 수치 알고리즘, 인쇄 p.318-319)

알고리즘	원문에서 정리한 역할
accumul**at**e(b,e,i) / accumul**at**e(b,e,i,f) [U14-025] (14.3 수치 알고리즘, 인쇄 p.318)	초기 값 i를 바탕으로 합산 / + 대신 f 사용 [U14-026] (14.3 수치 알고리즘, 인쇄 p.318)
inner_product [U14-027] (14.3 수치 알고리즘, 인쇄 p.318)	두 시퀀스의 대응 요소 곱을 합산하고 초기 값에 더함; +·*를 별도 연산으로 바꾸는 버전 제공 [U14-028] (14.3 수치 알고리즘, 인쇄 p.318)
partial_sum [U14-029] (14.3 수치 알고리즘, 인쇄 p.318)	각 위치까지의 부분합; + 대신 f를 지정할 수 있음 [U14-030] (14.3 수치 알고리즘, 인쇄 p.318)
adjacent_difference [U14-031] (14.3 수치 알고리즘, 인쇄 p.318)	이웃한 요소의 차이; 비어 있지 않으면 첫 출력은 첫 입력 값; - 대신 f 지정 가능 [U14-032] (14.3 수치 알고리즘, 인쇄 p.318)
iota [U14-033] (14.3 수치 알고리즘, 인쇄 p.318)	연속적인 값으로 구간을 채우는 기능; 원문 표는 각 요소에 ++v를 대입한다고 표기 [U14-034] (14.3 수치 알고리즘, 인쇄 p.318)
gcd / lcm [U14-035] (14.3 수치 알고리즘, 인쇄 p.318)	정수의 최대공약수 / 최소공배수 [U14-036] (14.3 수치 알고리즘, 인쇄 p.318)

accumulate의 초기 값과 합산 (원문 발췌)

```
list<double> lst {1, 2, 3, 4, 5, 9999.99999};
auto s = accumulate(lst.begin(),lst.end(),0.0); // 합 계산: 10014.9999
```

[U14-037] (14.3 수치 알고리즘, 인쇄 p.318)

[원문 확인 필요] 318쪽 iota 표는 ++v와 v+1, v+2, ...로 설명한다. [U14-038] (14.3 수치 알고리즘, 인쇄 p.318)

이 노트는 의심되는 시작 값 표기를 임의로 고치거나 다른 실행 결과를 만들지 않았다. [U14-039] (14.3 수치 알고리즘, 인쇄 p.318)

14.3.1 병렬 알고리즘

<numeric>의 병렬 **수치 알고리즘**은 기존 함수와 비슷하지만 일부 의미가 다르다. [U14-040] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

reduce는 accumul**at**e와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

초기 값과 실행 정책을 주는 형태, 기본 값을 사용하는 형태가 제공된다. [U14-041] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

알고리즘	차이와 역할
reduce [U14-042] (14.3.1 병렬 알고리즘, 인쇄 p.319)	순서가 정해지지 않는 누적 [U14-043] (14.3.1 병렬 알고리즘, 인쇄 p.319)
exclusive_scan / inclusive_scan [U14-044] (14.3.1 병렬 알고리즘, 인쇄 p.319)	i번째 합에서 i번째 입력 제외 / 포함 [U14-045] (14.3.1 병렬 알고리즘, 인쇄 p.319)
transform_**reduce** [U14-046] (14.3.1 병렬 알고리즘, 인쇄 p.319)	각 요소에 f를 적용한 뒤 **reduce** [U14-047] (14.3.1 병렬 알고리즘, 인쇄 p.319)
transform_exclusive_scan / transform_inclusive_scan [U14-048] (14.3.1 병렬 알고리즘, 인쇄 p.319)	각 요소에 f를 적용한 뒤 각각의 scan 실행 [U14-049] (14.3.1 병렬 알고리즘, 인쇄 p.319)

reduce와 병렬 실행 정책 (원문 발췌)

```
vector<double> v {1, 2, 3, 4, 5, 9999.99999};
auto s = reduce(v.begin(),v.end()); // double을 누적자로 사용해 합을 계산

vector<double> large;
// ... 많은 값들로 large를 채운다 ...
auto s2 = reduce(par_unseq, large.begin(),large.end()); // 가능한 병렬화 정도로 합을 계산
```

[U14-050] (14.3.1 병렬 알고리즘, 인쇄 p.320)

일반 병렬 알고리즘과 마찬가지로 실행 정책을 지정할 수 있다. [U14-051] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

기본 연산 대신 별도의 연산을 받거나 기본 정책·기본 값을 쓰는 형태도 있다고 원문은 설명하며, 표에서는 편의상 일부를 생략했다. [U14-052] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

14.4 복소수

<complex>의 **complex<Scalar>**는 실수부와 허수부 **타입**을 매개**변수**로 받는 복소수 **템플릿**이다. [U14-053] (14.4 복소수, 인쇄 p.320-321)

원문은 flo**at**·double·long double에 대한 복소수 사용을 보여 준다. [U14-054] (14.4 복소수, 인쇄 p.320-321)

일반적인 산술 연산과 sqrt·pow 같은 수학 함수를 사용할 수 있고 **생성자**의 실수부·허수부에는 기본 인자가 제공된다. [U14-055] (14.4 복소수, 인쇄 p.320-321)

14.5 난수

<random>의 난수 생성기는 **엔진**과 **분포**를 결합한다. [U14-056] (14.5 난수, 인쇄 p.321)

엔진은 난수 또는 의사 난수 시퀀스를 만들고, **분포**는 그 값을 수학적 **분포**의 특정 구간으로 매핑한다. [U14-057] (14.5 난수, 인쇄 p.321)

원문은 같은 확률로 정수를 만드는 uniform_int_distribution, 종형 곡선의 normal_distribution, exponential_distribution을 예로 든다. [U14-058] (14.5 난수, 인쇄 p.321)

엔진과 **분포**를 직접 조합한 주사위 예제에 이어 원문은 두 구성 요소를 Rand_int 클래스에 묶어 간단한 사용 인터페이스를 제시한다. [U14-059] (14.5 난수, 인쇄 p.321-322)

이 클래스는 기본 난수 **엔진**, 정수 **분포**, 호출 연산자, 시드 설정 연산을 가진다. [U14-060] (14.5 난수, 인쇄 p.321-322)

엔진과 분포를 묶은 Rand_int (원문 발췌)

```
class Rand_int {
public:
    Rand_int(int low, int high) :dist{low,high} { }
    int operator()() { return dist(re); } // int 생성
    void seed(int s) { re.seed(s); } // 난수 엔진에 새로운 시드 선택
private:
    default_random_engine re;
    uniform_int_distribution<> dist;
};
```

[U14-061] (14.5 난수, 인쇄 p.322)

원문의 히스토그램 예는 0부터 9까지의 값을 200번 생성하고 각 값의 빈도만큼 별표를 출력한다. [U14-062] (14.5 난수, 인쇄 p.322-323)

결과는 통계적 변동이 있는 **분포**로 제시되므로 이 노트에서 새로운 난수값이나 고정 출력표를 만들지 않는다. [U14-063] (14.5 난수, 인쇄 p.322-323)

조언에서는 충분한 무작위성을 확인하고 C의 rand()를 사용하지 않도록 권한다. [U14-064] (14.5 난수, 인쇄 p.322-323)

14.6 벡터 산술

일반 **컨테이너 vector**는 값을 저장하고 **반복자**·알고리즘 구조와 결합하는 데 초점을 두며 수학적인 벡터 연산을 직접 제공하지 않는다. [U14-065] (14.6 벡터 산술, 인쇄 p.323-324)

<**valarray**>의 **valarray**는 일반성이 다소 낮은 대신 수치 계산 최적화에 더 적합하도록 제공되는 배열 **템플릿**이다. [U14-066] (14.6 벡터 산술, 인쇄 p.323-324)

valarray의 배열 수준 산술 (원문 발췌)

```
void f(valarray<double>& a1, valarray<double>& a2)
{
    valarray<double> a = a1*3.14+a2/a1; // 수치 배열 연산자 *, +, /, =
    a2 += a1*3.14;
    a = abs(a);
    double d = a2[7];
    // ...
}
```

[U14-067] (14.6 벡터 산술, 인쇄 p.324)

valarray는 배열 수준의 산술 연산과 일반적인 수학 함수, 요소 인덱싱을 제공한다. [U14-068] (14.6 벡터 산술, 인쇄 p.324)

다차원 계산을 구현하기 위한 건너뛰기 접근(stride access)도 지원한다. [U14-069] (14.6 벡터 산술, 인쇄 p.324)

원문은 연산·요소 **타입**의 유연성보다 실행 효율이 중요할 때의 선택으로 설명한다. [U14-070] (14.6 벡터 산술, 인쇄 p.324)

14.7 수치 한계

<limits>의 **numeric_limits**는 내장 수치 **타입**의 속성을 기술한다. [U14-071] (14.7 수치 한계, 인쇄 p.324)

원문은 char의 부호 유무와 int가 필요한 큰 값을 표현할 수 있는지 st**at**ic_assert로 검사한다. [U14-072] (14.7 수치 한계, 인쇄 p.324)

max()가 **constexpr** 함수이므로 컴파일 시간 조건으로 사용할 수 있다. [U14-073] (14.7 수치 한계, 인쇄 p.324)

수치 타입의 성질을 컴파일 시간에 검사 (원문 발췌)

```
static_assert(numeric_limits<char>::is_signed,"부호 없는 문자!");
static_assert(100000<numeric_limits<int>::max(),"작은 int!");
```

[U14-074] (14.7 수치 한계, 인쇄 p.324)

14.8 조언

수치 문제를 수학적으로 확신할 수 없다면 전문적인 조언과 실험을 활용한다. [U14-075] (14.8 조언, 인쇄 p.325)

기본 언어 기능만으로 직접 만들기보다 라이브러리를 사용한다. [U14-076] (14.8 조언, 인쇄 p.325)

시퀀스를 계산하는 루프 전에 accumul**at**e·inner_product·partial_sum·adjacent_difference를 고려한다. [U14-077] (14.8 조언, 인쇄 p.325)

복소수에는 std::complex를 사용한다. [U14-078] (14.8 조언, 인쇄 p.325)

난수 생성은 **엔진**과 **분포**를 결합하고 무작위성에 주의를 기울인다. [U14-079] (14.8 조언, 인쇄 p.325)

수치 효율에 우선순위가 있으면 **valarray**를 고려한다. [U14-080] (14.8 조언, 인쇄 p.325)

numeric_limits로 수치 **타입**이 용도에 맞는지 검사한다. [U14-081] (14.8 조언, 인쇄 p.325)

주요 비교표

vector와 valarray

기준	vector	valarray
목적 [U14-082] (vector와 valarray, 인쇄 p.323-325)	일반적인 값 저장·**컨테이너** 구조 [U14-083] (vector와 valarray, 인쇄 p.323-325)	수치 계산 최적화 [U14-084] (vector와 valarray, 인쇄 p.323-325)
일반성 [U14-085] (vector와 valarray, 인쇄 p.323-325)	유연한 일반 **컨테이너** [U14-086] (vector와 valarray, 인쇄 p.323-325)	일반성은 상대적으로 낮음 [U14-087] (vector와 valarray, 인쇄 p.323-325)
수치 배열 연산 [U14-088] (vector와 valarray, 인쇄 p.323-325)	수학 벡터 연산을 직접 제공하지 않음 [U14-089] (vector와 valarray, 인쇄 p.323-325)	배열 산술·수학 함수·stride access [U14-090] (vector와 valarray, 인쇄 p.323-325)

accumulate와 reduce

기준	accumulate	reduce
구분 [U14-091] (accumulate와 reduce, 인쇄 p.318-320)	순차 **수치 알고리즘** [U14-092] (accumulate와 reduce, 인쇄 p.318-320)	병렬 **수치 알고리즘** [U14-093] (accumulate와 reduce, 인쇄 p.318-320)
적용 순서 [U14-094] (accumulate와 reduce, 인쇄 p.318-320)	순차 처리의 누적 [U14-095] (accumulate와 reduce, 인쇄 p.318-320)	요소 연산 순서가 정해지지 않음 [U14-096] (accumulate와 reduce, 인쇄 p.318-320)
정책 [U14-097] (accumulate와 reduce, 인쇄 p.318-320)	원문의 순차 예 [U14-098] (accumulate와 reduce, 인쇄 p.318-320)	실행 정책 지정 가능 [U14-099] (accumulate와 reduce, 인쇄 p.318-320)

핵심 용어 표

용어	원문 정의·설명
수치 알고리즘	<numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. [U14-001] (14.3 수치 알고리즘, 인쇄 p.318-319)
reduce	**reduce**는 accumul**at**e와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)
complex<Scalar>	<complex>의 **complex<Scalar>**는 실수부와 허수부 **타입**을 매개**변수**로 받는 복소수 **템플릿**이다. [U14-053] (14.4 복소수, 인쇄 p.320-321)
난수 엔진·분포	**엔진**은 난수 또는 의사 난수 시퀀스를 만들고, **분포**는 그 값을 수학적 **분포**의 특정 구간으로 매핑한다. [U14-100] (14.5 난수, 인쇄 p.321,323)
valarray	<**valarray**>의 **valarray**는 일반성이 다소 낮은 대신 수치 계산 최적화에 더 적합하도록 제공되는 배열 **템플릿**이다. [U14-066] (14.6 벡터 산술, 인쇄 p.323-324)
numeric_limits	<limits>의 **numeric_limits**는 내장 수치 **타입**의 속성을 기술한다. [U14-071] (14.7 수치 한계, 인쇄 p.324)

장 요약

요약 기준: 14.8 조언

수치 문제를 수학적으로 확신할 수 없다면 전문적인 조언과 실험을 활용한다. 기본 언어 기능만으로 직접 만들기보다 라이브러리를 사용한다. [U14-101] (14.8 조언, 인쇄 p.325)

시퀀스를 계산하는 루프 전에 accumul**at**e·inner_product·partial_sum·adjacent_difference를 고려한다. [U14-077] (14.8 조언, 인쇄 p.325)

복소수에는 std::complex를 사용한다. 난수 생성은 **엔진**과 **분포**를 결합하고 무작위성에 주의를 기울인다. [U14-102] (14.8 조언, 인쇄 p.325)

수치 효율에 우선순위가 있으면 **valarray**를 고려한다. **numeric_limits**로 수치 **타입**이 용도에 맞는지 검사한다. [U14-103] (14.8 조언, 인쇄 p.325)

원문 확인 메모

[원문 확인 필요] 318쪽 iota 표의 ++v 및 시작 값, accumulate 예제의 합 주석 자릿수는 원문 표기를 유지했다.

제15장. 동시성

(제15장. 동시성, 인쇄 p.327-345)

학습 목표·범위

동시성은 여러 **태스크**를 동시에 실행하는 것이다. [U15-001] (15.1 소개, 인쇄 p.327-328)

태스크는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. [U15-002] (15.2 태스크와 thread, 인쇄 p.328-329)

핵심 개념·동작 절차·원문 예제

15.1 소개

동시성은 여러 **태스크**를 동시에 실행하는 것으로, 여러 프로세서로 처리량을 늘리거나 대기 중에도 다른 일을 진행하여 반응성을 높이는 데 사용한다. [U15-003] (15.1 소개, 인쇄 p.327-328)

원문의 **표준 라이브러리**는 정교한 고수준 모델보다는 시스템 수준 **동시성**을 이식성과 **타입** 안전성을 고려해 제공한다. [U15-004] (15.1 소개, 인쇄 p.327-328)

C++는 한 주소 공간의 여러 스레드를 지원하며 메모리 모델과 원자적 연산을 제공한다. [U15-005] (15.1 소개, 인쇄 p.327-328)

데이터 경합은 변경 가능한 데이터에 대한 무질서한 동시 접근이다. [U15-006] (15.1 소개, 인쇄 p.327-328)

원자적 연산은 락 없는 프로그래밍의 기반이지만 원문은 이를 전문가에게 맡기도록 조언한다. [U15-007] (15.1 소개, 인쇄 p.327-328)

동시성 자체가 해답은 아니며, 순차 해결이 더 간단하고 빠를 수 있다. [U15-008] (15.1 소개, 인쇄 p.327-328)

직접 스레드를 관리하는 대신 병렬 알고리즘을 이용하는 방법도 있다. [U15-009] (15.1 소개, 인쇄 p.327-328)

15.2 태스크와 thread

태스크는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. [U15-002] (15.2 태스크와 thread, 인쇄 p.328-329)

<**thread**>의 std::**thread**에 함수 또는 **함수 객체**를 넘겨 실행을 시작한다. [U15-010] (15.2 태스크와 thread, 인쇄 p.328-329)

join()은 해당 스레드의 종료를 기다린다. [U15-011] (15.2 태스크와 thread, 인쇄 p.328-329)

함수와 함수 객체를 스레드에서 실행 (원문 발췌)

```
void f(); // 함수

struct F { // 함수 객체
    void operator()(); // F의 호출 연산자(6.3.2절)
};

void user()
{
    thread t1 {f}; // 별도 스레드에서 f() 실행
    thread t2 {F()}; // 별도 스레드에서 F()() 실행

    t1.join(); // t1을 기다림
    t2.join(); // t2를 기다림
}
```

[U15-012] (15.2 태스크와 thread, 인쇄 p.329)

한 프로그램의 스레드는 주소 공간을 공유하므로 **객체**를 통한 통신이 가능하지만 락 등으로 접근을 제어해야 한다. [U15-013] (15.2 태스크와 thread, 인쇄 p.329-330)

원문은 두 **태스크**가 동기화 없이 cout을 쓰면 실행 순서가 정해지지 않아 출력이 뒤섞일 수 있음을 보여 준다. [U15-014] (15.2 태스크와 thread, 인쇄 p.329-330)

명확한 통신 외에는 **태스크**를 분리하고, 인자를 받아 결과를 반환하는 함수처럼 다루는 것이 가장 단순한 접근이다. [U15-015] (15.2 태스크와 thread, 인쇄 p.329-330)

15.3 인자 전달

태스크에는 값·포인터·참조로 데이터를 전달할 수 있다. [U15-016] (15.3 인자 전달, 인쇄 p.330-331)

thread의 가변 **템플릿 생성자**는 실행할 함수와 그 인자들을 받는다. [U15-017] (15.3 인자 전달, 인쇄 p.330-331)

<**function**al>의 ref()를 쓰면 **객체**를 값으로 전달하는 대신 참조로 취급한다. [U15-018] (15.3 인자 전달, 인쇄 p.330-331)

함수 객체는 생성 시 받은 참조를 멤버에 저장할 수도 있다. [U15-019] (15.3 인자 전달, 인쇄 p.330-331)

참조 인자를 전달하는 두 방식 (원문 발췌)

```
void f(vector<double>& v); // v로 무언가를 수행하는 함수

struct F { // 함수 객체: v로 무언가를 수행
    vector<double>& v;
    F(vector<double>& vv) :v{vv} { }
    void operator()(); // 적용 연산자; 6.3.2절
};

int main()
{
    vector<double> some_vec {1,2,3,4,5,6,7,8,9};
    vector<double> vec2 {10,11,12,13,14};

    thread t1 {f,ref(some_vec)}; // f(some_vec)을 별도의 스레드에서 실행
    thread t2 {F{vec2}}; // F(vec2)()를 별도의 스레드에서 실행

    t1.join();
    t2.join();
}
```

[U15-020] (15.3 인자 전달, 인쇄 p.330-331)

원문의 F{vec2}가 vec2를 참조하는 동안 다른 **태스크**가 vec2에 접근해서는 안 된다. [U15-021] (15.3 인자 전달, 인쇄 p.330-331)

vec2를 값으로 넘기면 이 공유 위험을 피할 수 있다. [U15-022] (15.3 인자 전달, 인쇄 p.330-331)

컴파일러는 함수가 주어진 인자로 호출 가능한지 검사하고 스레드가 실행할 **함수 객체**를 구성한다. [U15-023] (15.3 인자 전달, 인쇄 p.330-331)

15.4 결과 반환

비**const** 참조로 입력을 받아 그것을 변경하는 방식으로 결과를 돌려줄 수도 있다. [U15-024] (15.4 결과 반환, 인쇄 p.331-332)

원문은 입력을 **const** 참조로 받고 결과 저장 위치를 별도 인자로 넘기는 편이 더 명확하다고 설명한다. [U15-025] (15.4 결과 반환, 인쇄 p.331-332)

함수 포인터 인자·**함수 객체** 멤버·참조 캡처 람다의 세 방식을 나란히 보여 준다. [U15-026] (15.4 결과 반환, 인쇄 p.331-332)

별도 출력 위치로 결과를 돌려받기 (원문 발췌)

```
void f(const vector<double>& v, double* res); // v에서 입력을 받아 *res에 결과를 저장

class F {
public:
    F(const vector<double>& vv, double* p) :v{vv}, res{p} { }
    void operator()(); // *res에 결과 저장
private:
    const vector<double>& v; // 입력
    double* res; // 출력을 저장할 곳
};

double g(const vector<double>&); // 반환 값 사용

void user(vector<double>& vec1, vector<double> vec2, vector<double> vec3)
{
    double res1;
    double res2;
    double res3;

    thread t1 {f,cref(vec1),&res1}; // f(vec1,&res1)을 별도 스레드로 실행
    thread t2 {F{vec2,&res2}}; // F{vec2,&res2}()를 별도 스레드로 실행
    thread t3 { [&](){ res3 = g(vec3); }}; // 지역 변수를 참조로 전달

    t1.join();
    t2.join();
    t3.join();

    cout << res1 << ' ' << res2 << ' ' << res3 << '\n';
}
```

[U15-027] (15.4 결과 반환, 인쇄 p.332)

모든 join() 뒤에 결과를 사용하므로 각 **태스크**의 완료를 기다린다. [U15-028] (15.4 결과 반환, 인쇄 p.331-332)

원문은 이런 참조 기반 결과 전달이 흔하고 동작하지만 덜 우아하다고 평가하고, 이후 **future·promise**로 더 높은 수준의 결과 전달을 설명한다. [U15-029] (15.4 결과 반환, 인쇄 p.331-332)

15.5 데이터 공유

공유 데이터를 동시에 변경할 때는 한 번에 한 **태스크**만 해당 데이터에 접근하도록 동기화한다. [U15-030] (15.5 데이터 공유, 인쇄 p.333)

변경 불가능한 데이터를 여러 **태스크**가 읽는 경우는 구별한다. [U15-031] (15.5 데이터 공유, 인쇄 p.333)

<**mutex**>의 **mutex**는 상호 배제 **객체**이며 lock()으로 획득하고 unlock()으로 해제한다. [U15-032] (15.5 데이터 공유, 인쇄 p.333)

scoped_lock은 이 획득·해제를 **RAII**로 묶는다. [U15-033] (15.5 데이터 공유, 인쇄 p.333)

scoped_lock으로 공유 데이터 보호 (원문 발췌)

```
mutex m; // 제어용 mutex
int sh; // 공유 데이터

void f()
{
    scoped_lock lck {m}; // mutex 획득
    sh += 7; // 공유 데이터 조작
} // 암묵적인 mutex 해제
```

[U15-034] (15.5 데이터 공유, 인쇄 p.333)

다른 스레드가 **mutex**를 가진 동안 획득 시도는 대기하며, 해제되면 기다리던 스레드가 진행한다. [U15-035] (15.5 데이터 공유, 인쇄 p.333-334)

mutex와 보호 데이터의 관계는 관례적이므로 같은 Record 안에 두거나 이름·주석으로 관계를 명확히 해야 한다. [U15-036] (15.5 데이터 공유, 인쇄 p.333-334)

명시적인 lock/unlock보다 **자원 핸들**이 간단하고 안전하다. [U15-037] (15.5 데이터 공유, 인쇄 p.333-334)

서로 다른 순서로 여러 **mutex**를 획득하면 상대가 가진 **mutex**를 기다리느라 모두 멈추는 **교착 상태(데드락)**가 생길 수 있다. [U15-038] (15.5 데이터 공유, 인쇄 p.334)

원문의 scoped_lock은 여러 **mutex**를 한 번에 받고 모두 획득한 뒤 진행하며, 일부만 획득한 채로 블록되지 않는다. [U15-039] (15.5 데이터 공유, 인쇄 p.334)

소멸 시에는 모두 해제한다. [U15-040] (15.5 데이터 공유, 인쇄 p.334)

공유 데이터는 호출·반환보다 낮은 수준의 통신이며 락 비용도 있다. [U15-041] (15.5 데이터 공유, 인쇄 p.334-335)

데이터 복사보다 더 빠르다고 측정 없이 가정하지 않는다. [U15-042] (15.5 데이터 공유, 인쇄 p.334-335)

shared_mutex는 읽기 스레드의 shared_lock을 함께 허용하고, 쓰기 스레드의 unique_lock에는 배타적 접근을 제공하여 읽기/쓰기 락을 구현한다. [U15-043] (15.5 데이터 공유, 인쇄 p.334-335)

15.6 이벤트 대기

시간 경과는 가장 간단한 이벤트다. [U15-044] (15.6 이벤트 대기, 인쇄 p.335-336)

this_**thread**는 현재 스레드를 가리키며 sleep_for로 기다릴 수 있다. [U15-045] (15.6 이벤트 대기, 인쇄 p.335-336)

chrono의 시각 차이와 dur**at**ion_cast로 경과 시간을 표시할 수 있고 별도 스레드를 만들 필요는 없다. [U15-046] (15.6 이벤트 대기, 인쇄 p.335-336)

<**condition_variable**>의 **condition_variable**은 다른 **태스크**의 행동으로 조건이나 이벤트가 생기기를 기다리는 수단이다. [U15-047] (15.6 이벤트 대기, 인쇄 p.336-337)

원문의 메시지 큐는 queue·조건 **변수**·**mutex**를 함께 사용한다. [U15-048] (15.6 이벤트 대기, 인쇄 p.336-337)

소비자는 unique_lock을 획득한 뒤 큐가 비어 있지 않다는 **술어**로 기다린다. [U15-049] (15.6 이벤트 대기, 인쇄 p.336-337)

wait는 대기 중 락을 해제하고 깨어날 때 다시 획득한다. [U15-050] (15.6 이벤트 대기, 인쇄 p.336-337)

조건 변수로 메시지 큐를 기다리는 소비자 (원문 발췌)

```
void consumer()
{
    while(true) {
        unique_lock lck {mutex}; // mutex 획득
        mcond.wait(lck,[] { return !mqueue.empty(); }); // lck을 해제하고 대기;
            // 깨어날 때 lck을 다시 획득
            // mqueue가 비어있는 동안 깨어나지 않음
        auto m = mqueue.front(); // 메시지 가져오기
        mqueue.pop();
        lck.unlock(); // lck 해제
        // ... m을 처리 ...
    }
}
```

[U15-051] (15.6 이벤트 대기, 인쇄 p.337)

발췌 범위: 소비자 함수 전체를 발췌했습니다. 336쪽의 전역 선언과 338쪽의 producer 함수는 생략했습니다. 원문의 메시지 처리 생략 주석을 유지했습니다.

깨어났다는 이유만으로 필요한 조건이 성립하는 것은 아니다. [U15-052] (15.6 이벤트 대기, 인쇄 p.337-338)

다른 **태스크**가 먼저 큐에 접근했을 수 있으므로 !mqueue.empty()를 다시 확인한다. [U15-053] (15.6 이벤트 대기, 인쇄 p.337-338)

메시지를 가져와 제거한 뒤에는 락을 해제하고 메시지를 처리해 임계 영역 시간을 줄인다. [U15-054] (15.6 이벤트 대기, 인쇄 p.337-338)

생산자는 락으로 큐 삽입을 보호하고 notify_one()으로 통지한다. [U15-055] (15.6 이벤트 대기, 인쇄 p.337-338)

unique_lock은 lock()/unlock()으로 낮은 수준의 제어가 가능하며 원문은 하나의 **mutex**만 다룬다고 설명한다. [U15-056] (15.6 이벤트 대기, 인쇄 p.337)

[원문 확인 필요] 337쪽은 scoped_lock과 비교하면서 unique_lock은 복사가 가능하다고 서술한다. [U15-057] (15.6 이벤트 대기, 인쇄 p.337)

이 문장은 의심되는 원문 표기로 남겨 두며, 이 노트에서 다른 규칙으로 조용히 교체하지 않는다. [U15-058] (15.6 이벤트 대기, 인쇄 p.337)

15.7 태스크 간 통신

<future>는 직접 **thread**·락을 다루는 것보다 높은 수준의 통신 수단을 제공한다. [U15-059] (15.7 태스크 간 통신, 인쇄 p.338)

future·promise는 **태스크** 사이의 결과 전달, **packaged_task**는 **태스크** 실행과 결과의 연결, **async**는 함수 호출과 비슷한 **태스크** 시작에 사용한다. [U15-060] (15.7 태스크 간 통신, 인쇄 p.338)

15.7.1 future와 promise

promise에 값을 넣으면 연결된 **future**에서 그 값을 볼 수 있다. [U15-061] (15.7.1 future와 promise, 인쇄 p.338-339)

future<X>::get()은 필요한 경우 값 계산이 끝날 때까지 기다리고 X를 얻는다. [U15-062] (15.7.1 future와 promise, 인쇄 p.338-339)

값 대신 예외가 전달되거나 값을 계산할 수 없으면 get()에서 예외를 던질 수 있다. [U15-063] (15.7.1 future와 promise, 인쇄 p.338-339)

promise의 set_value()와 set_exception()은 **future**의 get()에 대응한다. [U15-064] (15.7.1 future와 promise, 인쇄 p.339-340)

태스크는 결과 또는 예외 중 하나를 전달한다. [U15-065] (15.7.1 future와 promise, 인쇄 p.339-340)

current_exception()은 현재 잡힌 예외를 참조하며, 전달된 예외는 get()의 호출 경로 어딘가에서 처리해야 한다. [U15-066] (15.7.1 future와 promise, 인쇄 p.339-340)

락을 명시적으로 조작하지 않고도 시스템이 전달을 수행한다. [U15-067] (15.7.1 future와 promise, 인쇄 p.339-340)

promise로 값 또는 예외 전달 (원문 발췌)

```
void f(promise<X>& px) // 태스크: 결과를 px에 넣기
{
    // ...
    try {
        X res;
        // ... res에 값을 계산 ...
        px.set_value(res);
    }
    catch (...) { // res를 계산할 수 없는 경우
        px.set_exception(current_exception()); // future의 스레드로 예외 전달
    }
}
```

[U15-068] (15.7.1 future와 promise, 인쇄 p.339-340)

15.7.2 packaged_task

packaged_task는 **태스크**의 반환값이나 예외를 **promise**에 넣는 래퍼를 제공하고, get_**future**()로 대응 **future**를 돌려준다. [U15-069] (15.7.2 packaged_task, 인쇄 p.340-341)

템플릿 인자는 **태스크**의 호출 **타입**이며 **생성자** 인자는 수행할 함수다. [U15-070] (15.7.2 packaged_task, 인쇄 p.340-341)

원문의 accum 예제는 **vector** 구간의 두 절반을 별도 **태스크**로 합산한다. [U15-071] (15.7.2 packaged_task, 인쇄 p.340-341)

packaged_task로 두 구간의 계산과 결과 연결 (원문 발췌)

```
double accum(double* beg, double* end, double init)
    // 초깃값 init부터 시작해 [beg:end)를 모두 더함
{
    return accumulate(beg,end,init);
}

double comp2(vector<double>& v)
{
    using Task_type = double(double*,double*,double); // 태스크의 타입

    packaged_task<Task_type> pt0 {accum}; // 태스크(즉, accum)를 패키징
    packaged_task<Task_type> pt1 {accum};

    future<double> f0 {pt0.get_future()}; // pt0의 future를 저장
    future<double> f1 {pt1.get_future()}; // pt1의 future를 저장

    double* first = &v[0];
    thread t1 {move(pt0),first,first+v.size()/2,0}; // pt0를 위한 스레드 시작
    thread t2 {move(pt1),first+v.size()/2,first+v.size(),0}; // pt1을 위한 스레드 시작

    // ...

    return f0.get()+f1.get(); // 결과 가져오기
}
```

[U15-072] (15.7.2 packaged_task, 인쇄 p.341)

packaged_task는 **promise**를 소유하는 **자원 핸들**이므로 복사할 수 없고 **thread**로 전달할 때 move가 필요하다. [U15-073] (15.7.2 packaged_task, 인쇄 p.341-342)

명시적인 락 없이 **태스크**와 결과에 집중할 수 있으며 별도 스레드의 두 작업은 잠재적으로 **병렬 실행**된다. [U15-074] (15.7.2 packaged_task, 인쇄 p.341-342)

15.7.3 async()

async()는 잠재적으로 비동기 실행할 **태스크**를 함수 호출과 유사하게 시작한다. [U15-075] (15.7.3 async(), 인쇄 p.342)

호출 시작·결과 가져오기·실제 실행을 분리하여 **thread**나 락보다 **태스크** 자체에 집중하게 한다. [U15-076] (15.7.3 async(), 인쇄 p.342)

원문은 충분히 큰 **vector**를 네 구간으로 나누고 **future**의 결과를 모으는 예를 든다. [U15-077] (15.7.3 async(), 인쇄 p.342)

async로 네 구간의 결과를 모으기 (원문 발췌)

```
double comp4(vector<double>& v)
    // v가 충분히 크면 많은 태스크를 시작
{
    if (v.size()<10000) // 동시성을 활용할 만한가?
        return accum(v.begin(),v.end(),0.0);

    auto v0 = &v[0];
    auto sz = v.size();

    auto f0 = async(accum,v0,v0+sz/4,0.0); // 첫 번째 1/4
    auto f1 = async(accum,v0+sz/4,v0+sz/2,0.0); // 두 번째 1/4
    auto f2 = async(accum,v0+sz/2,v0+sz*3/4,0.0); // 세 번째 1/4
    auto f3 = async(accum,v0+sz*3/4,v0+sz,0.0); // 네 번째 1/4

    return f0.get()+f1.get()+f2.get()+f3.get(); // 결과를 모아서 병합
}
```

[U15-078] (15.7.3 async(), 인쇄 p.342)

원문은 락을 이용해 데이터를 공유하는 **태스크**에 **async**를 사용할 수 없다는 한계로 서술하며, 스레드 수는 호출 시점의 시스템 **자원**을 고려해 결정되므로 알 수 없다고 설명한다. [U15-079] (15.7.3 async(), 인쇄 p.343)

v.size()<10000 같은 기준은 스레드 시작 비용과 계산 비용에 대한 원시적인 가정으로, 성능 판단을 잘못할 수 있음을 경고한다. [U15-080] (15.7.3 async(), 인쇄 p.343)

accumul**at**e 같은 표준 연산을 직접 병렬화하기보다 **reduce**(par_unseq, ...) 같은 병렬 알고리즘이 더 잘 동작할 수 있다고 원문은 권한다. [U15-081] (15.7.3 async(), 인쇄 p.343)

async는 성능을 위한 병렬 계산만을 위한 것이 아니며 메인 작업 중 사용자 입력을 받는 **태스크** 같은 반응성 용도도 있다. [U15-082] (15.7.3 async(), 인쇄 p.343)

15.8 조언

처리량·반응성에 **동시성**이 필요한지 살피고 가능한 높은 추상화 수준에서 작업한다. [U15-083] (15.8 조언, 인쇄 p.343-345)

순차 해법과 병렬 알고리즘을 먼저 고려한다. [U15-084] (15.8 조언, 인쇄 p.343-345)

공유 데이터와 **데이터 경합**을 가능한 피한다. [U15-085] (15.8 조언, 인쇄 p.343-345)

thread의 종료는 join으로 기다린다. [U15-086] (15.8 조언, 인쇄 p.343-345)

mutex는 보호하는 데이터와 가까이 두고 **RAII 핸들**로 관리한다. [U15-087] (15.8 조언, 인쇄 p.343-345)

여러 락은 scoped_lock으로 함께 획득한다. [U15-088] (15.8 조언, 인쇄 p.343-345)

이벤트 통신에는 **condition_variable**과 unique_lock을 쓰고 조건 없이 기다리지 않는다. [U15-089] (15.8 조언, 인쇄 p.343-345)

임계 영역 시간을 최소화한다. [U15-090] (15.8 조언, 인쇄 p.343-345)

태스크를 동시에 실행할 수 있는 함수로 생각한다. [U15-091] (15.8 조언, 인쇄 p.343-345)

promise로 결과를 제공하고 **future**로 얻으며 **packaged_task**로 값·**예외 전달**을 묶는다. [U15-092] (15.8 조언, 인쇄 p.343-345)

간단한 **태스크** 시작에는 **async**를 사용하고 **thread**·**mutex**를 직접 다루는 일을 줄인다. [U15-093] (15.8 조언, 인쇄 p.343-345)

주요 비교표

태스크 통신 수단의 역할

수단	원문에서의 역할
promise [U15-094] (태스크 통신 수단의 역할, 인쇄 p.338-343)	값이나 예외를 넣는 쪽 [U15-095] (태스크 통신 수단의 역할, 인쇄 p.338-343)
future [U15-096] (태스크 통신 수단의 역할, 인쇄 p.338-343)	값이 준비될 때까지 기다리고 결과·예외를 받는 쪽 [U15-097] (태스크 통신 수단의 역할, 인쇄 p.338-343)
packaged_task [U15-098] (태스크 통신 수단의 역할, 인쇄 p.338-343)	**태스크**와 **promise**를 묶고 대응 **future**를 제공 [U15-099] (태스크 통신 수단의 역할, 인쇄 p.338-343)
async [U15-100] (태스크 통신 수단의 역할, 인쇄 p.338-343)	함수 호출처럼 **태스크**를 시작하고 결과 획득을 분리 [U15-101] (태스크 통신 수단의 역할, 인쇄 p.338-343)

원문에서 구별하는 락 핸들

핸들	용도와 원문 조건
scoped_lock [U15-102] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)	스코프 기반 **mutex** 관리; 여러 **mutex**를 함께 획득 [U15-103] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)
unique_lock [U15-104] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)	조건 **변수** wait와 명시적인 lock/unlock; 한 **mutex** 관리 [U15-105] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)
shared_lock [U15-106] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)	**shared_mutex**에서 여러 읽기 접근을 공유 [U15-107] (원문에서 구별하는 락 핸들, 인쇄 p.333-337)

핵심 용어 표

용어	원문 정의·설명
동시성	**동시성**은 여러 **태스크**를 동시에 실행하는 것이다. [U15-001] (15.1 소개, 인쇄 p.327-328)
태스크·thread	**태스크**는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. [U15-002] (15.2 태스크와 thread, 인쇄 p.328-329)
mutex	**mutex**는 상호 배제 **객체**이며 lock()으로 획득하고 unlock()으로 해제한다. [U15-108] (15.5 데이터 공유, 인쇄 p.333,335)
condition_variable	**condition_variable**은 다른 **태스크**의 행동으로 조건이나 이벤트가 생기기를 기다리는 수단이다. [U15-109] (15.6 이벤트 대기, 인쇄 p.336-337)
future·promise	**promise**에 값을 넣으면 연결된 **future**에서 그 값을 볼 수 있으며, **future**<X>::get()은 필요한 경우 계산이 끝날 때까지 기다린다. [U15-110] (15.7.1 future와 promise, 인쇄 p.339-340)
packaged_task	**packaged_task**는 **태스크**의 반환값이나 예외를 **promise**에 넣는 래퍼를 제공하고, get_**future**()로 대응 **future**를 돌려준다. [U15-069] (15.7.2 packaged_task, 인쇄 p.340-341)
async()	**async()**는 잠재적으로 비동기 실행할 **태스크**를 함수 호출과 유사하게 시작한다. [U15-111] (15.7.3 async(), 인쇄 p.342-343)

장 요약

요약 기준: 15.8 조언

처리량·반응성에 **동시성**이 필요한지 살피고 가능한 높은 추상화 수준에서 작업한다. 순차 해법과 병렬 알고리즘을 먼저 고려한다. [U15-112] (15.8 조언, 인쇄 p.343-345)

공유 데이터와 **데이터 경합**을 가능한 피한다. **thread**의 종료는 join으로 기다린다. [U15-113] (15.8 조언, 인쇄 p.343-345)

mutex는 보호하는 데이터와 가까이 두고 **RAII 핸들**로 관리한다. 여러 락은 scoped_lock으로 함께 획득한다. [U15-114] (15.8 조언, 인쇄 p.343-345)

이벤트 통신에는 **condition_variable**과 unique_lock을 쓰고 조건 없이 기다리지 않는다. 임계 영역 시간을 최소화한다. [U15-115] (15.8 조언, 인쇄 p.343-345)

원문 확인 메모

[원문 확인 필요] 337쪽은 unique_lock이 복사 가능하다고 서술한다. 원문 표기를 보존하고 다른 규칙을 보충하지 않았다.

[원문 확인 필요] 메시지 큐 예제는 336쪽 선언에서 mmutex, 337~338쪽 사용에서 mutex로 표기한다. 코드를 조용히 수정하지 않았다.

제16장. 역사와 호환성

(제16장. 역사와 호환성, 인쇄 p.347-370)

학습 목표·범위

ISO C++ 위원회는 **WG21**로 불린다. [U16-001] (16.1.3 ISO C++ 표준, 인쇄 p.355,357)

두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-002] (16.1.3 ISO C++ 표준, 인쇄 p.354-355)

핵심 개념·동작 절차·원문 예제

16.1 역사

C++는 시뮬러의 프로그램 조직화 기능과 C의 시스템 프로그래밍 효율성·유연성을 함께 제공하도록 설계됐다. [U16-003] (16.1 역사, 인쇄 p.347-348)

클래스와 파생 클래스, 가상 함수는 시뮬러에서 영향을 받았고, **템플릿**과 예외는 이후에 다른 아이디어를 바탕으로 추가됐다. [U16-004] (16.1 역사, 인쇄 p.347-348)

원문은 언어의 진화를 실제 활용 사례와 연결한다. [U16-005] (16.1 역사, 인쇄 p.347-348)

사용자와 동료의 의견을 듣고 당면한 문제를 해결하는 과정에서 언어와 라이브러리가 함께 발전했다. [U16-006] (16.1 역사, 인쇄 p.347-348)

아래 연혁과 기능 상태는 원문이 서술하던 시점을 그대로 따른다. [U16-007] (16.1 역사, 인쇄 p.347-348)

16.1.1 연혁

시기	원문에서 설명한 변화
1979 [U16-008] (16.1.1 연혁, 인쇄 p.348-350)	C with Classes 작업 시작. 클래스·파생 클래스, public/private, **생성자·소멸자**, **타입** 검사되는 함수 선언을 도입했다. [U16-009] (16.1.1 연혁, 인쇄 p.348-350)
1984~1985 [U16-010] (16.1.1 연혁, 인쇄 p.348-350)	이름을 C++로 바꾸고 가상 함수, 함수·연산자 오버로딩, 참조를 포함했다. 1985년 10월 14일 첫 상용 릴리스가 나왔다. [U16-011] (16.1.1 연혁, 인쇄 p.348-350)
1989~1991 [U16-012] (16.1.1 연혁, 인쇄 p.348-350)	ARM과 두 번째 The C++ Programming Language가 등장했다. **템플릿** 기반 **제네릭 프로그래밍**, 예외 기반 에러 처리, 일반적인 **자원** 관리 관례인 **RAII**가 소개됐다. [U16-013] (16.1.1 연혁, 인쇄 p.348-350)
1997~1998 [U16-014] (16.1.1 연혁, 인쇄 p.348-350)	**네임스페이스**·dynamic_cast·**템플릿** 개선과 **STL**을 포함한 ISO C++가 소개되고, 1998년 첫 ISO 표준이 제정됐다. [U16-015] (16.1.1 연혁, 인쇄 p.348-350)
2002~2003 [U16-016] (16.1.1 연혁, 인쇄 p.348-350)	C++0x 개정 작업을 시작했다. 2003년 표준 버그 수정과 Technical Report의 새 라이브러리 구성 요소는 이후 **C++11**로 이어졌다. [U16-017] (16.1.1 연혁, 인쇄 p.348-350)
2011~2013 [U16-018] (16.1.1 연혁, 인쇄 p.348-350)	**C++11**은 초기화, 이동, auto, 구간 for, 가변 **템플릿**, 람다, **타입 별칭**, **동시성** 메모리 모델 등을 포함했다. 원문은 완전한 구현체가 2013년에 등장했다고 서술한다. [U16-019] (16.1.1 연혁, 인쇄 p.348-350)
2014~2015 [U16-020] (16.1.1 연혁, 인쇄 p.348-350)	C++14의 가변 **템플릿**·자릿수 구분자·제네릭 람다·라이브러리 개선, Core Guidelines 프로젝트 시작, **컨셉**트 TS 승인을 기록한다. [U16-021] (16.1.1 연혁, 인쇄 p.348-350)
2017 [U16-022] (16.1.1 연혁, 인쇄 p.348-350)	C++17과 첫 구현체, 모듈 TS 및 Ranges TS 승인을 기록한다. [U16-023] (16.1.1 연혁, 인쇄 p.348-350)

시기	원문에서 설명한 변화
2020 [U16-024] (16.1.1 연혁, 인쇄 p.348-350)	원문에서는 ISO C++20 표준을 예정 사항으로 적었다. 현재 상태로 바꾸지 않는다. [U16-025] (16.1.1 연혁, 인쇄 p.348-350)

C++11은 개발 중 C++0x로 불렸다. [U16-026] (16.1.1 연혁, 인쇄 p.348-350)

원문은 완료 시점에 대한 낙관과 긴 지연을 설명하면서, C++14·C++17은 정해진 시점에 완료됐다고 대조한다. [U16-027] (16.1.1 연혁, 인쇄 p.348-350)

16.1.2 초창기

처음 해결하려던 문제는 멀티프로세서와 로컬 네트워크에 유닉스 커널 서비스를 배포하는 일이었다. [U16-028] (16.1.2 초창기, 인쇄 p.350-354)

시스템 부분과 상호작용을 정확히 명시하면서 하드웨어를 직접 다루고 높은 성능을 얻어야 했다. [U16-029] (16.1.2 초창기, 인쇄 p.350-354)

시뮬러는 조직화에 적합했으나 성능이 문제였고, C는 하드웨어와 성능에 적합했으나 모듈화가 어려웠다. [U16-030] (16.1.2 초창기, 인쇄 p.350-354)

이 두 요구를 결합한 것이 C with Classes였다. [U16-031] (16.1.2 초창기, 인쇄 p.350-354)

초기에는 이후의 연산자 오버로딩·참조·가상 함수·**템플릿**·예외 등이 없었다. [U16-032] (16.1.2 초창기, 인쇄 p.350-354)

C++라는 이름은 릭 매시티가 1983년 여름 제안했고, C의 증가 연산자 ++를 사용해 C의 확장을 표현했다. [U16-033] (16.1.2 초창기, 인쇄 p.350-354)

설계·문서화·구현은 함께 진행됐으며, 초창기의 진화는 공식 위원회보다 사용자 문제에 대한 토론에 기초했다. [U16-034] (16.1.2 초창기, 인쇄 p.350-354)

초기 설계의 중심에는 함수 인자 **타입** 검사, 클래스의 public/private에 의한 인터페이스와 구현 분리, **생성자**와 **소멸자**가 있었다. [U16-035] (16.1.2 초창기, 인쇄 p.350-354)

원문은 **생성자·소멸자** 도입을 특히 중요하게 평가한다. [U16-036] (16.1.2 초창기, 인쇄 p.350-354)

객체의 실행 환경을 만들고 되돌리는 기능이 **자원** 관리 전략과 간결한 표현 기법의 출발점이 됐기 때문이다. [U16-037] (16.1.2 초창기, 인쇄 p.350-354)

가상 함수는 실행 시간의 선택을 표현하는 다형성을 제공했지만, 시스템 코드에 충분히 빠른지에 대한 논쟁이 있었다. [U16-038] (16.1.2 초창기, 인쇄 p.350-354)

원문은 근본적으로 복잡한 시스템을 만들 때 언어와 도구의 도움을 활용해야 한다고 설명한다. [U16-039] (16.1.2 초창기, 인쇄 p.350-354)

C++를 범용 언어, 더 나은 C, 데이터 추상화를 제공하며 **객체**지향 프로그래밍을 지원하는 언어로 서술한다. [U16-040] (16.1.2 초창기, 인쇄 p.350-354)

데이터 추상화에는 정보 은닉뿐 아니라 클래스 계층에 속하지 않는 클래스와 **제네릭 프로그래밍**도 포함한다. [U16-041] (16.1.2 초창기, 인쇄 p.350-354)

템플릿을 설계할 때 유연성·효율성·이른 **타입** 검사를 동시에 얻기 어려웠다. [U16-042] (16.1.2 초창기, 인쇄 p.350-354)

C 스타일 시스템 코드와 경쟁하기 위해 유연성과 효율성을 선택했고, **타입** 확인 개선은 이어졌다. [U16-043] (16.1.2 초창기, 인쇄 p.350-354)

예외 설계는 여러 호출 수준으로의 전파, **핸들**러에 임의 정보 전달, 로컬 **객체**의 **소멸자**에 기반한 **자원** 관리와의 통합에 초점을 뒀다. [U16-044] (16.1.2 초창기, 인쇄 p.350-354)

이 **자원** 관리 기법을 Resource Acquisition Is Initialization, **RAII**라 부른다. [U16-045] (16.1.2 초창기, 인쇄 p.350-354)

추상 클래스의 다중 상속을 포함한 상속 일반화도 이루어졌지만, 원문은 **템플릿**과 예외보다 중요도가 낮다고 평가한다. [U16-046] (16.1.2 초창기, 인쇄 p.350-354)

복소수·벡터·스택·**입출력 스트림**을 설계하면서 연산자 오버로딩도 설계했고, 문자열·리스트와 **템플릿** 라이브러리 경험이 언어에 영향을 주었다. 1998년 **표준 라이브러리**의 주요 혁신은 제네릭 **컨테이너**와 알고리즘의 프레임워크인 **STL**이었다. [U16-047] (16.1.2 초창기, 인쇄 p.350-354)

초기의 **태스크** 라이브러리와 달리 표준 **동시성** 기능이 널리 사용되기까지는 2011년까지 기다려야 했다고 서술한다. [U16-048] (16.1.2 초창기, 인쇄 p.350-354)

코루틴의 C++20 포함은 원문에서 예정으로 표현된다. [U16-049] (16.1.2 초창기, 인쇄 p.350-354)

여러 언어의 영향보다 당면한 응용 분야의 요구를 결정적인 기준으로 삼은 문제 지향적 개발이 핵심이다. [U16-050] (16.1.2 초창기, 인쇄 p.350-354)

16.1.3 ISO C++ 표준

사용이 늘어나면서 구현자와 사용자 사이의 일관된 약속이 필요해졌다. 1987년 무렵 표준화의 필요성이 분명해졌고, 공유·검토된 레퍼런스 매뉴얼이 ANSI 표준의 기초가 됐다. [U16-051] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

ANSI X3J16 위원회는 1989년 12월 소집됐고, 1991년에는 ISO 작업에 포함됐다. [U16-052] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

ISO C++ 위원회는 **WG21**로 불린다. [U16-053] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

첫 ISO C++ 표준은 1998년 22:0으로 승인됐다. 2003년 버그 수정판은 원문에서 기본적으로 C++98과 같은 언어라고 설명한다. [U16-054] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-055] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

까다로워진 작업 절차는 명세의 엄격성을 높이는 동시에 혁신을 제한하기도 했다. [U16-056] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

표준 문서가 커진 것은 새 기능뿐 아니라 더 자세한 명세 때문이었다. [U16-057] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

기존 수십억 줄의 코드를 망가뜨리지 않도록 장기간 안정성을 유지하는 것도 필수 요구였다. [U16-058] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

C++11의 전반적인 목표는 시스템 프로그래밍과 라이브러리 제작에 더 적합한 언어를 만들고, 가르치고 배우기 쉽게 하는 것이었다. [U16-059] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

타입 안전성과 이식성을 갖춘 **동시성**, 적합한 메모리 모델과 스레드 라이브러리는 이 목표의 일부다. [U16-060] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

C++11 이후에는 13년 간격이 너무 길다는 공감 아래 정해진 시점에 표준을 내는 기차 모델과 3년 주기를 채택했다. [U16-061] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

기능을 병렬로 개발하기 위해 기술 명세 TS를 사용했지만, 추가 단계와 지연이 생기기도 했다. [U16-062] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

원문은 C++17이 계획된 대규모 변화에 이르지 못했고 **컨셉**·모듈·레인지가 준비되지 않았다고 평가한다. [U16-063] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

이 평가는 원문 시점의 평가로 보존한다. [U16-064] (16.1.3 ISO C++ 표준, 인쇄 p.354-357)

16.1.4 표준과 스타일

표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. [U16-065] (16.1.4 표준과 스타일, 인쇄 p.357-358)

기능 자체의 기술적 이해와, 기능·라이브러리·도구를 함께 써서 관리하기 쉽고 에러가 적으며 빠른 소프트웨어를 만드는 일은 구별된다. [U16-066] (16.1.4 표준과 스타일, 인쇄 p.357-358)

일관된 프로그래밍 스타일을 개발·지원하고 기존 코드도 그 스타일로 발전시키는 일이 필요하다. [U16-067] (16.1.4 표준과 스타일, 인쇄 p.357-358)

2015년에 시작한 C++ Core Guidelines는 원문에서 **타입** 안전성과 **자원** 안전성에 초점을 둔 노력으로 소개된다. [U16-068] (16.1.4 표준과 스타일, 인쇄 p.357-358)

정적 분석 도구와 이를 지원하는 라이브러리의 도움으로 효과적인 스타일을 지원하려는 것이다. [U16-069] (16.1.4 표준과 스타일, 인쇄 p.357-358)

이 교재는 원문이 언급한 목적만 정리하며 외부 지침의 세부 내용을 덧붙이지 않는다. [U16-070] (16.1.4 표준과 스타일, 인쇄 p.357-358)

16.1.5 C++의 사용

원문은 C++를 산업계의 요구 속에서 발전한 언어로 설명한다. [U16-071] (16.1.5 C++의 사용, 인쇄 p.358-359)

장치 드라이버·네트워킹·임베디드 시스템을 비롯해 운영체제, 웹 기반 시설, 금융, 의료, 자동차, 항공우주, 과학, 그래픽스·게임 등 여러 응용 분야를 든다. [U16-072] (16.1.5 C++의 사용, 인쇄 p.358-359)

공통점은 하드웨어를 효과적으로 사용하면서 복잡도를 낮추는 능력이 필요하다는 것이다. [U16-073] (16.1.5 C++의 사용, 인쇄 p.358-359)

사용 분야와 사용자 수에 관한 서술은 원문의 역사적 관점에 속한다. [U16-074] (16.1.5 C++의 사용, 인쇄 p.358-359)

16.2 C++ 기능의 진화

이 절은 **C++11**·C++14·C++17에 추가된 언어 기능과 **표준 라이브러리**를 구별해 나열한다. [U16-075] (16.2 C++ 기능의 진화, 인쇄 p.359)

기능은 따로 고립돼 쓰이지 않으며, 새 기능 상당수는 기존 기능이 제공하는 틀 위에서 의미를 갖는다. [U16-076] (16.2 C++ 기능의 진화, 인쇄 p.359)

아래 목록은 원문의 기능 이름을 학습 항목별로 묶은 것이며, 이 장에서 설명하지 않은 세부 규칙은 새로 보충하지 않는다. [U16-077] (16.2 C++ 기능의 진화, 인쇄 p.359)

16.2.1 C++11 언어 기능

관련 항목	원문 목록의 주요 기능
초기화·상수·**타입** [U16-078] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	{} 범용 초기화, auto **타입** 추론, **타입** 축소 방지, **constexpr**, **enum class**, st**at**ic_assert, initializer_**list** 매핑 [U16-079] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
순회·값·함수 [U16-080] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	구간 for, nullptr, **rvalue 참조**와 이동, 람다, 가변 **템플릿** 인자, **타입·템플릿 별칭** [U16-081] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
문자·표현·선언 [U16-082] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	유니코드 문자, long long, alignas/alignof, decltype, 로우 문자열, 일반화된 POD·**union**, 로컬 클래스의 **템플릿** 인자 사용, 접미사 반환 **타입** 구문 [U16-083] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
예외·속성 [U16-084] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	[[carries_dependency]], [[noreturn]], **noexcept** 지정자와 **noexcept** 연산자 [U16-085] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
매크로·이름 공간 [U16-086] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	C99 관련 확장 정수·문자열·매크로 기능, __func__, inline **네임스페이스** [U16-087] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
생성·변환·**템플릿** 제어 [U16-088] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	**생성자** 위임, 클래스 내 멤버 초기화, default/delete, 명시적 변환 연산자, 사용자 정의 리터럴, extern templ**at**e, 함수 **템플릿** 기본 인자, **생성자** 상속 [U16-089] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)
언어 규칙·**동시성** [U16-090] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)	중첩 **템플릿** 인자의 >> 표기, override/final, SFINAE 규칙, 메모리 모델, **thread**_local [U16-091] (16.2.1 C++11 언어 기능, 인쇄 p.359-361)

16.2.2 C++14 언어 기능

함수 반환 **타입** 추론과 개선된 **constexpr** 함수(예: for 루프 허용). [U16-092] (16.2.2 C++14 언어 기능, 인쇄 p.361)

가변 **템플릿**, 바이너리 리터럴과 자릿수 구분자. [U16-093] (16.2.2 C++14 언어 기능, 인쇄 p.361)

제네릭 람다, 더 일반적인 람다 캡처, [[deprec**at**ed]] 속성과 작은 확장들. [U16-094] (16.2.2 C++14 언어 기능, 인쇄 p.361)

16.2.3 C++17 언어 기능

안전성이 보장된 복사 생략, 과정렬된 **타입**의 동적 할당, 더 엄격한 평가 순서. [U16-095] (16.2.3 C++17 언어 기능, 인쇄 p.361-362)

UTF-8 리터럴(u8), 16진 부동소수점 리터럴, **접힘 표현식**. [U16-096] (16.2.3 C++17 언어 기능, 인쇄 p.361-362)

auto **템플릿** 파라미터, 클래스 **템플릿** 인자 **타입** 추론, 컴파일 시간 if. [U16-097] (16.2.3 C++17 언어 기능, 인쇄 p.361-362)

초깃값을 포함한 선택 구문, **constexpr** 람다, inline **변수**, 구조화된 바인딩. [U16-098] (16.2.3 C++17 언어 기능, 인쇄 p.361-362)

[[fallthrough]], [[nodiscard]], [[maybe_unused]], std::byte, 기저 **타입** 값으로 enum 초기화와 작은 확장들. [U16-099] (16.2.3 C++17 언어 기능, 인쇄 p.361-362)

16.2.4 C++11 표준 라이브러리 구성 요소

C++11 라이브러리의 확장은 새 구성 요소의 추가와 기존 구성 요소의 개선을 모두 포함한다. [U16-100] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

예를 들어 **정규 표현식**은 새 구성 요소이고, **컨테이너**의 이동 **생성자** 지원은 기존 구성 요소의 확장이다. [U16-101] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

묶음	원문 목록
컨테이너·자원 [U16-102] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)	initializer_**list 생성자**, **컨테이너** 이동, **forward_list**, **unordered_map**/unordered_multi**map**/unordered_set/unordered_multiset, **unique_ptr**/**shared_ptr**/weak_ptr, array, emplace(), 스코프 **할당자** [U16-103] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)
동시성·예외 [U16-104] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)	**thread**, 뮤텍스·락·조건 **변수**, **future**·**promise**·**async()**, 예외 복사와 다시 던짐, system_error, **at**omic [U16-105] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)
범용 라이브러리 [U16-106] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)	**tuple**, regex, 난수 **분포**·**엔진**, 정수 **타입** 이름, **constexpr**/**noexcept** 사용, **function**·bind(), 문자열 수치 변환 [U16-107] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)
특성·시간·알고리즘 [U16-108] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)	**타입** 특성, dur**at**ion/time_point, r**at**io, quick_exit, move()/copy_if()/is_sorted(), 가비지 컬렉션 ABI [U16-109] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

[원문 확인 필요] 원문 363쪽의 높은 수준 **동시성** 목록에는 packaged_**thread**라고 인쇄돼 있다. [U16-110] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

이 장에서는 원문 표기를 보존하며, 다른 식별자로 조용히 고치지 않는다. [U16-111] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

해당 기능의 구체적 코드는 15장 원문과 구별해 확인한다. [U16-112] (16.2.4 C++11 표준 라이브러리 구성 요소, 인쇄 p.362-363)

16.2.5 C++14 표준 라이브러리 구성 요소

shared_mutex와 사용자 정의 리터럴. [U16-113] (16.2.5 C++14 표준 라이브러리 구성 요소, 인쇄 p.363-364)

타입을 이용한 튜플 접근, 연관 **컨테이너**의 이형 조회, 작은 기능 추가. [U16-114] (16.2.5 C++14 표준 라이브러리 구성 요소, 인쇄 p.363-364)

16.2.6 C++17 표준 라이브러리 구성 요소

파일 시스템, 병렬 알고리즘, 특수 수학 함수. [U16-115] (16.2.6 C++17 표준 라이브러리 구성 요소, 인쇄 p.364)

string_view, **any**, **variant**, optional, invoke(). [U16-116] (16.2.6 C++17 표준 라이브러리 구성 요소, 인쇄 p.364)

기초 문자열 변환 to_chars/from_chars, 다형적 할당과 작은 확장. [U16-117] (16.2.6 C++17 표준 라이브러리 구성 요소, 인쇄 p.364)

16.2.7 제거되거나 지원 중단된 기능

광범위한 기존 코드 때문에 기능 제거는 오랜 경고 뒤에 신중히 이뤄진다. [U16-118] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

원문은 C++17에서 예외 명세와 그 지원 기능인 unexcepted_handler·set_unexpected()·get_unexpected()·unexpected()가 제거됐다고 설명하고 **noexcept** 사용을 권한다. [U16-119] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

[원문 확인 필요] unexcepted_handler는 원문 365쪽의 표기를 그대로 옮겼다. [U16-120] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

> **원문에서 제거 사례로 든 예외 명세** (원문 발췌)
>
> `void f() throw(X,Y); // C++98 스타일, 이제부턴 에러`
> [U16-121] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364)

문자 세 개의 예약어(trigraphs), 저장 영역 명시자 register, bool에 대한 ++가 제거되거나 허용되지 않는다. [U16-122] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

auto_ptr 대신 **unique_ptr**을 사용한다. [U16-123] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

C++98의 export 기능은 제거됐고, 원문은 모듈 키워드의 export를 따로 언급한다. [U16-124] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

소멸자가 있는 클래스에서 복사 연산을 자동 생성하는 기능은 **지원 중단**으로 설명된다. [U16-125] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

문자열을 char*에 대입하는 기능은 제거됐으며 **const** char* 또는 auto를 사용하도록 한다. [U16-126] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

기존 일부 **함수 객체**와 관련 함수는 람다와 **function**으로 대체하도록 권한다. [U16-127] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

지원 중단은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. [U16-128] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

컴파일러는 경고를 낼 수 있고, 호환성을 위해 **지원 중단** 기능이 오래 유지될 수도 있다. [U16-129] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

16.3 C/C++ 호환성

원문은 사소한 예외를 제외하면 C++가 C11의 대부분을 포함하며, 차이의 상당 부분이 C++의 엄격한 **타입** 검사에서 생긴다고 설명한다. [U16-130] (16.3 C/C++ 호환성, 인쇄 p.365)

이 설명은 두 언어의 모든 프로그램과 기능이 완전히 호환된다는 뜻으로 확대하지 않는다. [U16-131] (16.3 C/C++ 호환성, 인쇄 p.365)

차이는 컴파일러 진단으로 확인할 수 있다고 서술한다. [U16-132] (16.3 C/C++ 호환성, 인쇄 p.365)

16.3.1 C와 C++는 형제

ISO C와 ISO C++는 Classic C에서 갈라져 서로 다른 속도와 방향으로 발전한 형제 언어로 설명된다. [U16-133] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

둘 다 전통적인 C 스타일을 제공하지만 방식이 약간 다르고, 둘 모두 Classic C와도 100% 호환되지는 않는다. [U16-134] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

가계도는 기능을 직접 계승한 관계와 다른 언어에서 차용한 관계를 구별한다. [U16-135] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

호환성 차이의 원인이 언제나 C++의 변경인 것은 아니다. [U16-136] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

분리 후 C에 추가된 기능에서 생긴 차이도 있다. [U16-137] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

원문은 T*와 **void*** 대입, 전역 **const**의 링크, inline의 세부 의미를 예로 든다. [U16-138] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

한 언어의 라이브러리를 다른 언어에서 쓰거나 공통 도구를 만드는 경우 이 차이를 고려해야 한다. [U16-139] (16.3.1 C와 C++는 형제, 인쇄 p.366-367)

16.3.2 호환성 문제

설계와 프로그래밍 스타일이 C++의 기능을 활용하지 못할 수 있다. [U16-140] (16.3.2 호환성 문제, 인쇄 p.367)

void*를 캐스트 없이 T*로 바꾸는 코드는 문제가 될 수 있다. [U16-141] (16.3.2 호환성 문제, 인쇄 p.367)

class·priv**at**e 등 C++ 키워드를 C 코드의 식별자로 쓸 수 있다. [U16-142] (16.3.2 호환성 문제, 인쇄 p.367)

C와 C++로 각각 컴파일한 코드 사이의 링크 호환성을 고려해야 한다. [U16-143] (16.3.2 호환성 문제, 인쇄 p.367)

다른 문제가 없다면 C 코드를 C로 컴파일한 뒤 extern "C"를 이용해 C++에 링크할 수 있다. [U16-144] (16.3.2 호환성 문제, 인쇄 p.367)

따라서 문법 변환, 설계 개선, 링크 문제는 각각 구별해서 다뤄야 한다. [U16-145] (16.3.2 호환성 문제, 인쇄 p.367)

16.3.2.1 스타일 문제

C 프로그램을 한 줄씩 C++로 바꾸는 것만으로는 기본 구조와 에러의 근본 원인이 사라지지 않는다. [U16-146] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

원문은 불완전한 에러 처리, **자원** 누수, 버퍼 오버플로우가 번역 뒤에도 남을 수 있다고 설명한다. [U16-147] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

혜택을 얻으려면 설계와 구현 스타일도 바꿔야 한다. [U16-148] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

표준 라이브러리를 새로운 스타일의 학습 자료로 사용하고 C 라이브러리와의 차이를 확인한다. [U16-149] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

매크로 대신 원문에 제시된 **const**·**constexpr**·enum/**enum class**·inline·templ**at**e·namespace를 목적에 맞게 사용한다. [U16-150] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

변수를 필요 지점에서 선언하고 초기화한다. [U16-151] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

원문은 구문 위치, for 초기화 절, 조건 절에서의 선언을 언급한다. [U16-152] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

malloc 대신 new, realloc 대신 **vector**를 권하지만 malloc/free를 무조건 new/delete로 맞바꾸는 방식은 피하도록 한다. [U16-153] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

매우 낮은 수준의 구현 외에는 **void***·공용체·캐스트를 줄인다. [U16-154] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

명시 변환이 불가피하면 의도가 드러나는 명명된 캐스트를 사용한다. [U16-155] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

배열·C 스타일 문자열 대신 **string**·array·**vector**를 활용한다. [U16-156] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

특화 코드나 단순 배열 순회 외에는 포인터 산술을 사용하지 않도록 한다. [U16-157] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

장황한 C 스타일 코드가 더 짧은 **표준 라이브러리** 사용 코드보다 반드시 효율적이라고 가정하지 않는다. [U16-158] (16.3.2.1 스타일 문제, 인쇄 p.367-369)

16.3.2.2 void*

C에서 **void***를 다른 포인터 **타입**의 대입 우측이나 초깃값으로 사용할 수 있는 경우에도 C++에서는 항상 허용되지 않는다. [U16-159] (16.3.2.2 void*, 인쇄 p.369-370)

원문은 malloc의 결과를 int*로 바로 초기화하는 코드를 이 차이의 예로 제시한다. [U16-160] (16.3.2.2 void*, 인쇄 p.369-370)

malloc 결과의 암묵적 포인터 변환 (원문 발췌)

```
void f(int n)
{
    int* p = malloc(n*sizeof(int));  /* C++에서는 불허. C++의 "new"로 할당하자. */
    // ...
}
```

[U16-161] (16.3.2.2 void*, 인쇄 p.369)

이 제한은 단순한 불편만을 의미하지 않는다. [U16-162] (16.3.2.2 void*, 인쇄 p.369-370)

void*를 임의의 포인터 **타입**으로 암묵적으로 바꿀 때 잘못된 메모리 접근이 생길 수 있음을 다음 예가 보여 준다. [U16-163] (16.3.2.2 void*, 인쇄 p.369-370)

결과를 수치나 출력으로 새로 예측하지 않는다. [U16-164] (16.3.2.2 void*, 인쇄 p.369-370)

void* 변환이 초래할 수 있는 문제 (원문 발췌)

```
char ch;
void* pv = &ch;
int* pi = pv;     // C++에서는 허용하지 않음
*pi = 666;       // ch와 근처의 바이트를 덮어씀
```

[U16-165] (16.3.2.2 void*, 인쇄 p.369)

원문은 두 언어에서 모두 사용할 malloc 코드라면 결과를 적절한 **타입**으로 캐스트해야 한다고 설명한다. [U16-166] (16.3.2.2 void*, 인쇄 p.369-370)

C++만 사용할 때는 malloc을 사용하지 않도록 권한다. [U16-167] (16.3.2.2 void*, 인쇄 p.369-370)

16.3.2.3 링크

C와 C++는 서로 다른 링크 방식으로 구현될 수 있다. [U16-168] (16.3.2.3 링크, 인쇄 p.370)

원문은 **타입** 검사와 함수 오버로딩을 그 이유로 든다. [U16-169] (16.3.2.3 링크, 인쇄 p.370)

C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 **C 링크**를 명시한다. [U16-170] (16.3.2.3 링크, 인쇄 p.370)

C 링크를 지정하는 선언 (원문 발췌)

```
extern "C" double sqrt(double);
```

[U16-171] (16.3.2.3 링크, 인쇄 p.370)

이 선언을 통해 C 또는 C++ 코드 조각에서 sqrt(double)을 호출할 수 있으며, 정의는 C나 C++ 함수로 컴파일될 수 있다. [U16-172] (16.3.2.3 링크, 인쇄 p.370)

C는 오버로딩을 지원하지 않으므로 한 스코프에서 특정 이름으로 C에 링크되는 함수는 하나뿐이다. [U16-173] (16.3.2.3 링크, 인쇄 p.370)

링크 명세는 **타입** 검사를 바꾸지 않으며, C++ 함수 호출·인자 검사 규칙이 그대로 적용된다. [U16-174] (16.3.2.3 링크, 인쇄 p.370)

16.4 참고 자료

제공 파일에서 확인되는 참고 자료는 Boost 라이브러리 항목과 1990년 C 표준 항목까지다. [U16-175] (16.4 참고 자료, 인쇄 p.370)

여기서 인용된 자료는 추가 지식의 출처로 사용하지 않았다. [U16-176] (16.4 참고 자료, 인쇄 p.370)

이어지는 인쇄 371쪽 이후의 목록은 원문에서 확인 불가. [U16-177] (16.4 참고 자료, 인쇄 p.370)

16.5 조언

원문에서 확인 불가 목차에는 16.5 조언이 인쇄 376쪽부터 시작한다고 나오지만 해당 본문은 제공되지 않았다. [U16-178] (16.5 조언, PDF p.17)

목차의 위치만 확인했으며 조언의 내용을 만들어 넣지 않았다. [U16-179] (16.5 조언, PDF p.17)

주요 비교표

표준과 스타일

기준	표준	스타일
주요 역할 [U16-180] (표준과 스타일, 인쇄 p.357-358)	무엇이 어떻게 동작하는지 규정 [U16-181] (표준과 스타일, 인쇄 p.357-358)	기능·라이브러리·도구를 함께 효과적으로 사용 [U16-182] (표준과 스타일, 인쇄 p.357-358)
원문의 관심 [U16-183] (표준과 스타일, 인쇄 p.357-358)	기능의 정확한 명세 [U16-184] (표준과 스타일, 인쇄 p.357-358)	관리하기 쉽고 에러가 적으며 빠른 소프트웨어 [U16-185] (표준과 스타일, 인쇄 p.357-358)

지원 중단과 제거

구분	원문의 설명
지원 중단 [U16-186] (지원 중단과 제거, 인쇄 p.364-365)	피해야 한다는 강한 신호이며 경고를 낼 수 있다. 호환성 때문에 유지될 수도 있다. [U16-187] (지원 중단과 제거, 인쇄 p.364-365)
제거 [U16-188] (지원 중단과 제거, 인쇄 p.364-365)	표준에서 기능을 없앤다. 원문은 대규모 기존 코드 때문에 오랜 경고와 신중함이 필요하다고 설명한다. [U16-189] (지원 중단과 제거, 인쇄 p.364-365)

C/C++ 호환성의 서로 다른 층위

층위	원문에서 든 쟁점
설계와 스타일 [U16-190] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)	문법만 옮겨도 **자원** 누수·불완전한 에러 처리의 근본 원인은 남을 수 있다. [U16-191] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)
타입 검사 [U16-192] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)	**void***의 암묵적 변환처럼 C++에서 더 엄격한 제한이 있다. [U16-193] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)
링크 [U16-194] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)	각 언어로 컴파일한 코드를 연결할 때 extern "C"로 **C 링크**를 명시한다. [U16-195] (C/C++ 호환성의 서로 다른 층위, 인쇄 p.367-370)

핵심 용어 표

용어	원문 정의·설명
WG21	ISO C++ 위원회는 **WG21**로 불린다. [U16-001] (16.1.3 ISO C++ 표준, 인쇄 p.355,357)
C++11	두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-002] (16.1.3 ISO C++ 표준, 인쇄 p.354-355)
표준과 스타일	표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. [U16-065] (16.1.4 표준과 스타일, 인쇄 p.357-358)
지원 중단	**지원 중단**은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. [U16-128] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)
void*	C에서 **void***를 다른 포인터 **타입**의 대입 우측이나 초깃값으로 사용할 수 있는 경우에도 C++에서는 항상 허용되지 않는다. [U16-159] (16.3.2.2 void*, 인쇄 p.369-370)
C 링크	C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 **C 링크**를 명시한다. [U16-170] (16.3.2.3 링크, 인쇄 p.370)

장 요약

요약 기준: 편집 요약: 16.1~16.3의 수록 본문. 16.5 조언 본문은 원문에서 확인 불가.

두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-002] (16.1.3 ISO C++ 표준, 인쇄 p.354-355)

표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다. [U16-065] (16.1.4 표준과 스타일, 인쇄 p.357-358)

지원 중단은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. [U16-128] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 **C 링크**를 명시한다. [U16-170] (16.3.2.3 링크, 인쇄 p.370)

원문 확인 메모

인쇄 371쪽 이후의 참고 자료 나머지와 16.5 조언 본문은 제공되지 않았다. 16.5는 목차에서 존재와 시작 번호만 확인했다.

제공 PDF의 1~12쪽에 놓인 인쇄 359~370쪽을 347~358쪽 뒤로 돌려 원문 장 순서로 편집했다.

연혁과 기능 지원·예정 상태를 현재 사실로 갱신하지 않았다.

363쪽 packaged_thread 표기는 원문 확인이 필요한 표기로 보존했다.

원문 기반 복습 질문과 정답

제1장. 기초

01-1. 타입·객체·값·변수를 원문은 각각 어떻게 정의하는가? (1.4 타입과 변수, 산술 연산, 인쇄 p.26)

01-2. 지역 이름은 어디부터 어디까지 유효한가? (1.5 스코프와 수명, 인쇄 p.31)

01-3. const 값과 constexpr 값은 언제 계산될 수 있는가? (1.6 상수, 인쇄 p.33)

01-4. new로 생성한 객체는 언제까지 유효한가? (1.5 스코프와 수명, 인쇄 p.32)

정답과 원문 근거

1. 타입은 값과 연산의 집합을 정의하고, **객체**는 **타입**의 값을 갖는 메모리이며, 값은 **타입**에 따라 해석되는 비트의 집합이고 **변수**는 명명된 **객체**다. [U01-168] (1.4 타입과 변수, 산술 연산, 인쇄 p.26)

2. 지역 이름의 유효 스코프는 선언 지점부터 선언을 포함하는 블록의 끝까지다. [U01-161] (1.5 스코프와 수명, 인쇄 p.31)

3. const 값은 실행 시간에 계산할 수 있지만 **constexpr** 값은 반드시 컴파일 시간에 계산된다. [U01-169] (1.6 상수, 인쇄 p.33)

4. new로 생성한 **객체**는 delete로 소멸할 때까지 유효하다. [U01-162] (1.5 스코프와 수명, 인쇄 p.32)

제2장. 사용자 정의 타입

02-1. 클래스 인터페이스는 어떤 멤버로 정의되는가? (2.3 클래스, 인쇄 p.53)

02-2. 생성자란 무엇이며 언제 사용하는가? (2.3 클래스, 인쇄 p.54)

02-3. struct와 class 사이에 원문이 설명하는 차이는 무엇인가? (2.3 클래스, 인쇄 p.55)

02-4. enum class는 열거자의 스코프와 타입을 어떻게 다루는가? (2.5 열거형, 인쇄 p.58)

정답과 원문 근거

1. 클래스는 데이터·함수·**타입**을 멤버로 가질 수 있으며, 인터페이스는 public 멤버로 정의된다. [U02-001] (2.3 클래스, 인쇄 p.53)

2. 해당 클래스와 이름이 동일한 멤버 함수를 **생성자**라고 하며, 클래스를 **객체**화할 때 사용한다. [U02-075] (2.3 클래스, 인쇄 p.54)

3. struct는 멤버가 기본적으로 public인 **class**이며, **struct**에도 **생성자**를 포함한 멤버 함수를 정의할 수 있다. [U02-077] (2.3 클래스, 인쇄 p.55)

4. enum class는 독립된 **타입**이며 열거자의 스코프를 해당 **enum class**로 제한한다. [U02-002] (2.5 열거형, 인쇄 p.58)

제3장. 모듈화

03-1. 네임스페이스의 목적은 무엇인가? (3.4 네임스페이스, 인쇄 p.68)

03-2. 사전 조건과 클래스 불변 조건을 원문은 어떻게 구별하는가? (3.5.2 불변 조건, 인쇄 p.73-74)

03-3. noexcept 함수에서 예외를 던지면 어떻게 되는가? (3.5.1 예외, 인쇄 p.73)

03-4. static_assert(A,S)에서 A가 true가 아니면 어떻게 되는가? (3.5.5 정적 어써션, 인쇄 p.80)

정답과 원문 근거

1. 네임스페이스는 일부 선언을 함께 묶고 다른 이름과 충돌하지 않게 하는 메커니즘이다. [U03-001] (3.4 네임스페이스, 인쇄 p.68)

2. 사전 조건은 함수 작동의 기본 가정이고 **클래스 불변 조건**은 클래스 수준에서 보장돼야 할 조건이다. [U03-157] (3.5.2 불변 조건, 인쇄 p.73-74)

3. 절대로 예외를 던지지 않는 함수는 **noexcept**로 선언할 수 있으며, 그래도 예외를 던지면 std::termin**at**e()가 호출된다. [U03-150] (3.5.1 예외, 인쇄 p.73)

4. s**tat**ic_assert(A,S)는 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다. [U03-158] (3.5.5 정적 어써션, 인쇄 p.80)

제4장. 클래스

04-1. 구체 타입과 추상 타입은 표현과 인터페이스를 어떻게 다루는가? (4.2 구체 타입·4.3 추상 타입, 인쇄 p.93,102)

04-2. 컨테이너는 무엇을 저장하는 객체인가? (4.2.2 컨테이너, 인쇄 p.97)

04-3. Vector의 소멸자는 어떤 자원을 해제하는가? (4.2.2 컨테이너, 인쇄 p.98)

04-4. 인터페이스 상속과 구현 상속의 역할은 무엇인가? (4.5.1 계층 구조의 장점, 인쇄 p.112)

정답과 원문 근거

1. 구체 타입은 **타입** 정의 안에 메모리 표현을 포함하고, **추상 타입**은 상세 구현을 감추며 인터페이스와 메모리 표현을 분리한다. [U04-160] (4.2 구체 타입·4.3 추상 타입, 인쇄 p.93,102)

2. 컨테이너는 요소의 컬렉션을 저장하는 **객체**다. [U04-151] (4.2.2 컨테이너, 인쇄 p.97)

3. 소멸자는 클래스 이름 앞에 ~를 붙인 이름을 가지며, Vector의 **소멸자**는 delete[]로 할당한 배열을 해제한다. [U04-152] (4.2.2 컨테이너, 인쇄 p.98)

4. 인터페이스 상속은 기반 **객체** 대신 파생 **객체**를 사용하게 하고, **구현 상속**은 기반의 함수·데이터 멤버로 파생 구현을 단순하게 한다. [U04-161] (4.5.1 계층 구조의 장점, 인쇄 p.112)

제5장. 필수적인 연산

05-1. =default와 =delete는 어떤 목적으로 사용하는가? (5.1.1 필수적인 연산, 인쇄 p.121-122)

05-2. 원문의 Vector 복사 생성자는 두 객체의 요소를 어떻게 유지하는가? (5.2.1 컨테이너 복사, 인쇄 p.126)

05-3. 원문의 Vector 이동 생성자는 원본을 어떤 상태로 만드는가? (5.2.2 컨테이너 이동, 인쇄 p.129)

05-4. move() 함수 자체는 값을 이동시키는가? (5.2.2 컨테이너 이동, 인쇄 p.130)

정답과 원문 근거

1. =default는 기본 구현을 명시적으로 만들게 하고, **=delete**는 해당 함수의 사용을 컴파일 시간에 금지한다. [U05-147] (5.1.1 필수적인 연산, 인쇄 p.121-122)

2. 원문의 **Vector 복사 생성자**는 새 저장 공간을 할당하고 요소를 복사하여 각 Vector가 자기 요소의 복사본을 갖게 한다. [U05-139] (5.2.1 컨테이너 복사, 인쇄 p.126)

3. 원문의 **Vector 이동 생성자**는 원본의 elem과 sz를 가져오고 원본을 nullptr와 크기 0으로 만든다. [U05-141] (5.2.2 컨테이너 이동, 인쇄 p.129)

4. move()는 아무것도 이동시키지 않고 앞으로 이동시킬 인자의 **rvalue 참조**를 반환하는 일종의 **타입** 변환이다. [U05-002] (5.2.2 컨테이너 이동, 인쇄 p.130)

제6장. 템플릿

06-1. 템플릿이란 무엇인가? (6.1 소개, 인쇄 p.141)

06-2. 템플릿의 값 인자가 만족해야 할 조건은 무엇인가? (6.2.2 값 템플릿 인자, 인쇄 p.146)

06-3. 함수 객체와 술어는 각각 무엇인가? (6.3.2 함수 객체, 인쇄 p.151-152)

06-4. if constexpr에서 인스턴스화되는 경우는 무엇인가? (6.4.3 컴파일 시간 if, 인쇄 p.160)

정답과 원문 근거

1. 템플릿은 **타입**이나 값의 집합을 파라미터화한 클래스나 함수다. [U06-001] (6.1 소개, 인쇄 p.141)

2. 템플릿의 값 인자는 반드시 상수 표현식이어야 한다. [U06-108] (6.2.2 값 템플릿 인자, 인쇄 p.146)

3. 함수 객체는 함수처럼 호출할 수 있는 **객체**이며, **술어**는 호출 가능하고 true나 false를 반환하는 무언가다. [U06-117] (6.3.2 함수 객체, 인쇄 p.151-152)

4. if constexpr에서는 선택된 경우만 인스턴스화되므로 지역적인 최적화가 가능하다. [U06-002] (6.4.3 컴파일 시간 if, 인쇄 p.160)

제7장. 컨셉과 제네릭 프로그래밍

07-1. 컨셉은 어떤 술어인가? (7.2.4 컨셉 정의, 인쇄 p.170)

07-2. requires 표현식은 언제 true나 false가 되는가? (7.2.3 유효한 코드, 인쇄 p.169)

07-3. 리프팅은 무엇을 유지하면서 무엇을 일반화하는가? (7.3.2 템플릿을 이용한 추상화, 인쇄 p.175)

07-4. 컨셉을 쓸 때 바로 확인할 수 없는 사항은 언제 확인하는가? (7.5 템플릿 컴파일 모델, 인쇄 p.181)

정답과 원문 근거

1. 컨셉은 하나 이상의 **타입**을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 **술어**다. [U07-001] (7.2.4 컨셉 정의, 인쇄 p.170)

2. requires 표현식은 주어진 문장이 유효한 코드면 true, 그렇지 않으면 false로 평가되는 **술어**다. [U07-102] (7.2.3 유효한 코드, 인쇄 p.169)

3. 리프팅은 동일한 성능을 유지하면서 구체적인 코드를 일반화하는 과정이다. [U07-105] (7.3.2 템플릿을 이용한 추상화, 인쇄 p.175)

4. 컨셉을 사용하면 먼저 **템플릿** 인자의 **컨셉** 만족 여부를 검사하고, 그 시점에 확인할 수 없는 것은 인스턴스화 시간까지 미룬다. [U07-111] (7.5 템플릿 컴파일 모델, 인쇄 p.181)

제8장. 라이브러리의 개요

08-1. 표준 라이브러리는 어떤 자료구조와 연산을 제공하는가? (8.2 표준 라이브러리 구성 요소, 인쇄 p.186)

08-2. 원문이 STL이라 부르는 프레임워크는 무엇인가? (8.2 표준 라이브러리 구성 요소, 인쇄 p.185)

08-3. 표준 기능을 사용하려면 무엇을 포함해야 하는가? (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

08-4. 표준 라이브러리의 이름은 어느 네임스페이스에 정의되는가? (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

정답과 원문 근거

1. C++ **표준 라이브러리**는 가장 널리 사용하는 기본 자료 구조와 그 자료 구조에서 동작하는 알고리즘을 제공한다. [U08-001] (8.2 표준 라이브러리 구성 요소, 인쇄 p.186)

2. 컨테이너와 **find**(), sort(), merge() 등을 비롯한 알고리즘 프레임워크를 관례적으로 **STL**이라고 한다. [U08-040] (8.2 표준 라이브러리 구성 요소, 인쇄 p.185)

3. 표준 라이브러리의 모든 기능은 **표준 헤더**에서 제공된다. [U08-041] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

4. 표준 라이브러리는 **std 네임스페이스**에 정의되므로, 기능을 사용할 때 std:: 접두사를 사용할 수 있다. [U08-002] (8.3 표준 라이브러리 헤더와 네임스페이스, 인쇄 p.186)

제9장. 문자열과 정규 표현식

09-1. string의 주된 역할은? (9.2 문자열, 인쇄 p.190,192)

09-2. string_view가 문자를 소유하는가? (9.3 문자열 뷰, 인쇄 p.194-195)

09-3. smatch는 무엇을 저장하는가? (9.4.1 검색, 인쇄 p.198-199)

09-4. regex_iterator의 역할은? (9.4.3 반복자, 인쇄 p.206-207)

정답과 원문 근거

1. string은 문자 시퀀스를 저장하고 조작하는 **Regular 타입**이다. [U09-001] (9.2 문자열, 인쇄 p.190,192)

2. string_view는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있으며, 문자를 소유하지 않는다. [U09-121] (9.3 문자열 뷰, 인쇄 p.194-195)

3. smatch는 **string**의 매치 결과들을 저장하는 **컨테이너**다. [U09-054] (9.4.1 검색, 인쇄 p.198-199)

4. regex_iterator는 시퀀스에서 패턴을 차례로 찾아 순회한다. [U09-093] (9.4.3 반복자, 인쇄 p.206-207)

제10장. 입력과 출력

10-1. 입출력 스트림 라이브러리가 지원하는 작업은? (10.1 소개, 인쇄 p.210-211)

10-2. cout과 cerr은 각각 어떤 스트림인가? (10.2 출력, 인쇄 p.211,213)

10-3. 파일 스트림 세 종류의 용도는? (10.7 파일 스트림, 인쇄 p.221-222)

10-4. 문자열 스트림 세 종류의 용도는? (10.8 문자열 스트림, 인쇄 p.222-223)

정답과 원문 근거

1. 입출력 스트림 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다. [U10-001] (10.1 소개, 인쇄 p.210-211)

2. cout은 표준 출력, cerr은 에러 보고 스트림이다. [U10-002] (10.2 출력, 인쇄 p.211,213)

3. <fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다. [U10-061] (10.7 파일 스트림, 인쇄 p.221-222)

4. <sstream>의 i**string**stream은 **string**에서 읽고 o**string**stream은 **string**에 쓰며 **string**stream은 둘 다 한다. [U10-068] (10.8 문자열 스트림, 인쇄 p.222-223)

제11장. 컨테이너

11-1. 컨테이너란 무엇인가? (11.1 소개, 인쇄 p.234)

11-2. vector<T>의 요소 저장 방식은? (11.2 vector, 인쇄 p.234-235)

11-3. 없는 구간을 at()으로 접근하면? (11.2.2 구간 검사, 인쇄 p.239-240)

11-4. unordered_map은 어떤 저장 방식을 사용하며 요소 순서를 유지하는가? (11.5 unordered_map, 인쇄 p.246,248)

정답과 원문 근거

1. 컨테이너는 **객체**를 저장하는 일이 주목적인 클래스다. [U11-001] (11.1 소개, 인쇄 p.234)

2. vector<T>는 T **타입** 요소가 메모리에 연속적으로 저장되는 시퀀스다. [U11-002] (11.2 vector, 인쇄 p.234-235)

3. at()은 구간 밖 인덱스에 out_of_range를 던진다. [U11-031] (11.2.2 구간 검사, 인쇄 p.239-240)

4. unordered_map은 해시 테이블을 이용하며 요소 사이의 순서를 유지하지 않는다. [U11-135] (11.5 unordered_map, 인쇄 p.246,248)

제12장. 알고리즘

12-1. 알고리즘 입력의 반개구간은 어떻게 표현하는가? (12.1 소개, 인쇄 p.254-255)

12-2. 술어의 역할은? (12.5 술어, 인쇄 p.263-264)

12-3. find가 검색에 실패하면 무엇을 반환하는가? (12.2 반복자의 사용, 인쇄 p.255-256)

12-4. find_if는 어떤 요소를 찾는가? (12.5 술어, 인쇄 p.263-264)

정답과 원문 근거

1. 입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 **반개구간 [begin:end)**이며, 첫 요소와 마지막 요소 다음을 가리키는 **반복자** 쌍으로 나타낸다. [U12-001] (12.1 소개, 인쇄 p.254-255)

2. 술어는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다. [U12-035] (12.5 술어, 인쇄 p.263-264)

3. find는 찾은 요소의 **반복자**를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다. [U12-010] (12.2 반복자의 사용, 인쇄 p.255-256)

4. find_if는 전달된 **술어**가 참이 되는 첫 요소를 찾는다. [U12-161] (12.5 술어, 인쇄 p.263-264)

제13장. 유틸리티

13-1. 책에서 자원은 무엇을 뜻하는가? (13.2 자원 관리, 인쇄 p.276-277)

13-2. unique_ptr과 shared_ptr의 소유권 차이는? (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277,281)

13-3. optional<A>가 나타내는 두 상태는? (13.5.2 optional, 인쇄 p.298-299)

13-4. function 객체는 무엇을 저장하는가? (13.8.3 function(), 인쇄 p.305)

정답과 원문 근거

1. 자원은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 **핸들**·파일 **핸들**이 예다. [U13-001] (13.2 자원 관리, 인쇄 p.276-277)

2. unique_ptr은 유일한 소유권을, **shared_ptr**은 공유된 소유권을 표현한다. [U13-002] (13.2.1 unique_ptr과 shared_ptr, 인쇄 p.277,281)

3. optional<A>는 A 또는 아무것도 없음을 나타낸다. [U13-096] (13.5.2 optional, 인쇄 p.298-299)

4. function 객체는 ()로 호출할 수 있는 대상을 저장한다. [U13-130] (13.8.3 function(), 인쇄 p.305)

제14장. 수치

14-1. <numeric>이 일반화하는 작업은? (14.3 수치 알고리즘, 인쇄 p.318-319)

14-2. reduce의 연산 순서는 accumulate와 어떻게 다른가? (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

14-3. 난수 엔진과 분포는 각각 무엇을 하는가? (14.5 난수, 인쇄 p.321,323)

14-4. numeric_limits는 무엇을 기술하는가? (14.7 수치 한계, 인쇄 p.324)

정답과 원문 근거

1. <numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다. [U14-001] (14.3 수치 알고리즘, 인쇄 p.318-319)

2. reduce는 accumul**ate**와 달리 요소 연산의 순서가 정해지지 않는다. [U14-002] (14.3.1 병렬 알고리즘, 인쇄 p.319-320)

3. 엔진은 난수 또는 의사 난수 시퀀스를 만들고, **분포**는 그 값을 수학적 **분포**의 특정 구간으로 매핑한다. [U14-100] (14.5 난수, 인쇄 p.321,323)

4. <limits>의 **numeric_limits**는 내장 수치 **타입**의 속성을 기술한다. [U14-071] (14.7 수치 한계, 인쇄 p.324)

제15장. 동시성

15-1. 태스크와 thread의 관계는? (15.2 태스크와 thread, 인쇄 p.328-329)

15-2. mutex의 획득·해제 연산은? (15.5 데이터 공유, 인쇄 p.333,335)

15-3. promise와 future는 결과를 어떻게 전달하는가? (15.7.1 future와 promise, 인쇄 p.339-340)

15-4. packaged_task는 어떤 래퍼를 제공하는가? (15.7.2 packaged_task, 인쇄 p.340-341)

정답과 원문 근거

1. 태스크는 다른 계산과 동시에 실행할 수 있는 작업이며, **thread**는 이를 표현하는 시스템 수준 개념이다. [U15-002] (15.2 태스크와 thread, 인쇄 p.328-329)

2. mutex는 상호 배제 **객체**이며 lock()으로 획득하고 unlock()으로 해제한다. [U15-108] (15.5 데이터 공유, 인쇄 p.333,335)

3. promise에 값을 넣으면 연결된 **future**에서 그 값을 볼 수 있으며, **future**<X>::get()은 필요한 경우 계산이 끝날 때까지 기다린다. [U15-110] (15.7.1 future와 promise, 인쇄 p.339-340)

4. packaged_task는 **태스크**의 반환값이나 예외를 **promise**에 넣는 래퍼를 제공하고, get_**future**()로 대응 **future**를 돌려준다. [U15-069] (15.7.2 packaged_task, 인쇄 p.340-341)

제16장. 역사와 호환성

16-1. ISO C++ 위원회의 이름은? (16.1.3 ISO C++ 표준, 인쇄 p.355,357)

16-2. 원문에서 C++11의 승인 시점과 표결은? (16.1.3 ISO C++ 표준, 인쇄 p.354-355)

16-3. 지원 중단은 즉시 제거와 같은가? (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

16-4. C와 C++ 코드 사이의 호출을 위해 무엇을 명시하는가? (16.3.2.3 링크, 인쇄 p.370)

정답과 원문 근거

1. ISO C++ 위원회는 **WG21**로 불린다. [U16-001] (16.1.3 ISO C++ 표준, 인쇄 p.355,357)

2. 두 번째 표준 **C++11**은 2011년 8월 21:0으로 승인됐다. [U16-002] (16.1.3 ISO C++ 표준, 인쇄 p.354-355)

3. 지원 중단은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다. [U16-128] (16.2.7 제거되거나 지원 중단된 기능, 인쇄 p.364-365)

4. C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 **C 링크**를 명시한다. [U16-170] (16.3.2.3 링크, 인쇄 p.370)